

Chapter 7

Deep Learning and Other AI Applications

Recap: Machine Learning

Where do we see machine learning
being used in the real world?

Unlocking Our Phones



Detecting Road Signs



Recommending Movies

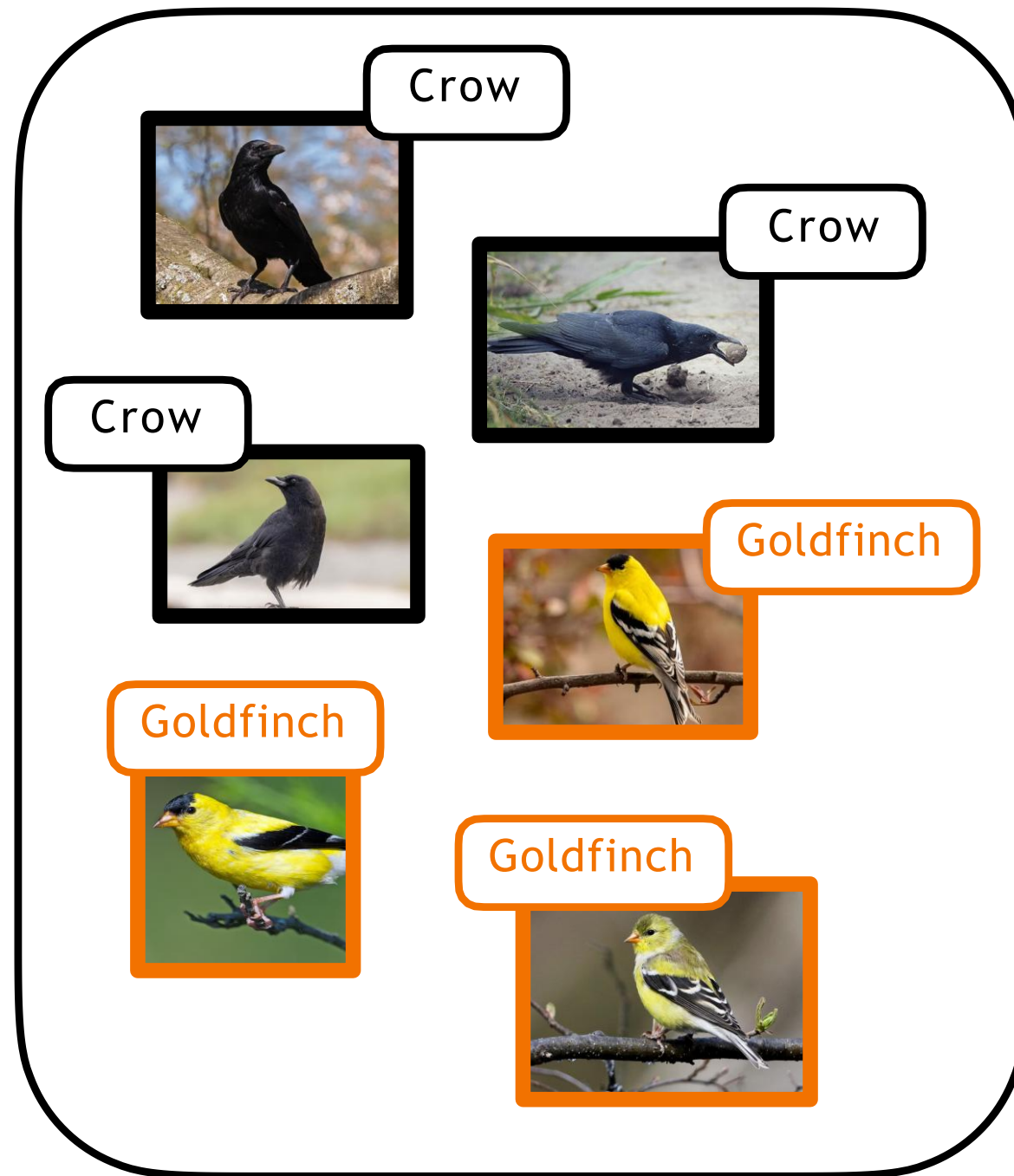


Machine learning is almost **everywhere!**

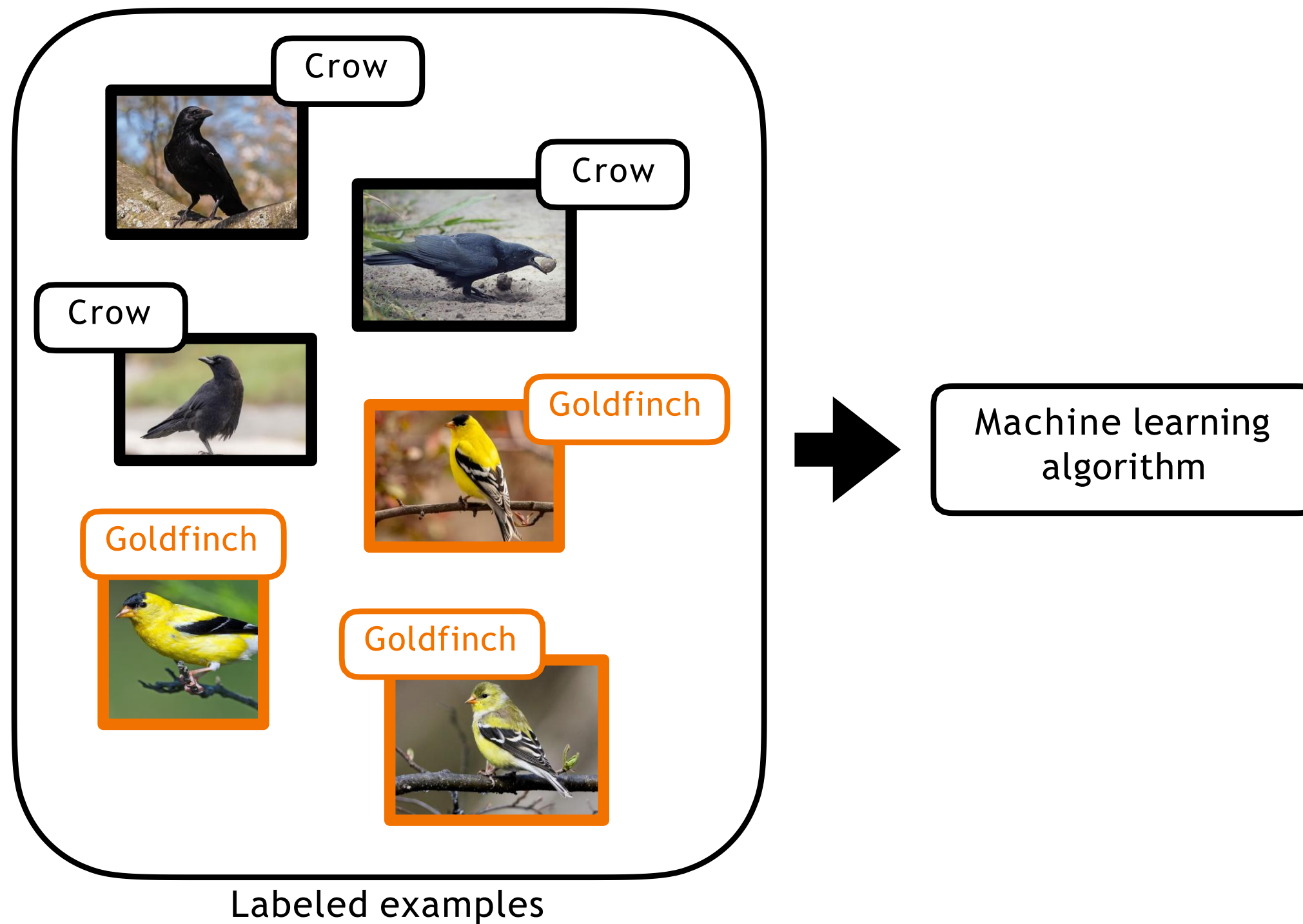
So, **how** does it work?

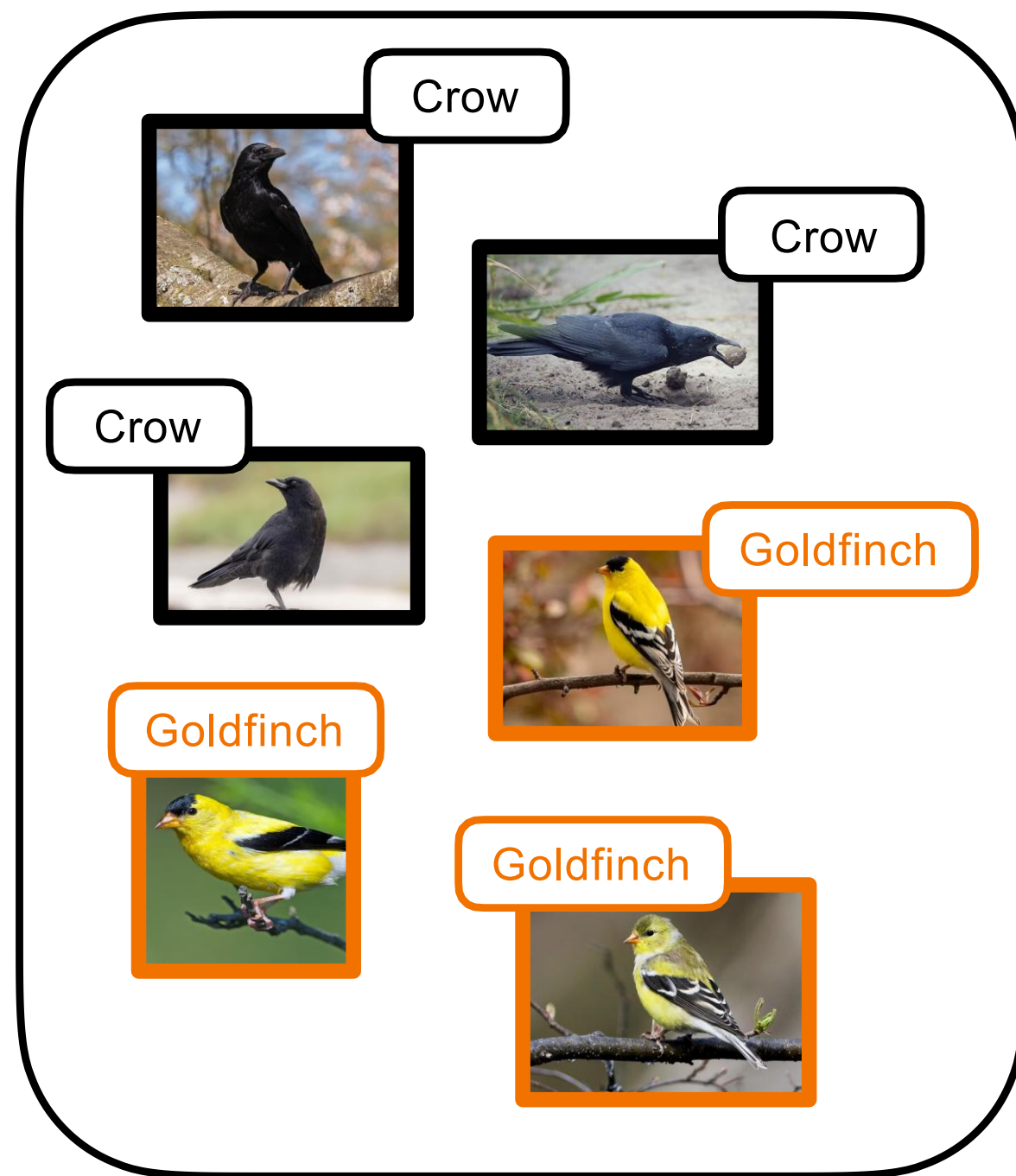
Machine learning lets computers learn
from data

Learn how?

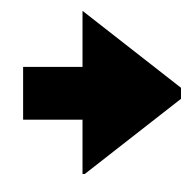


Labeled examples

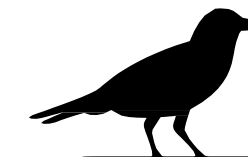
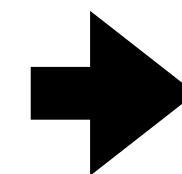




Labeled examples



Machine learning
algorithm



Bird classifier

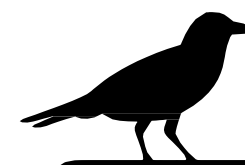
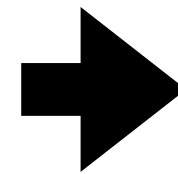


New bird picture

Goal



New bird picture

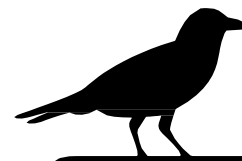
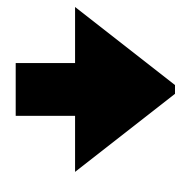


Bird classifier

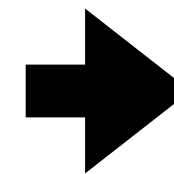
Goal



New bird picture



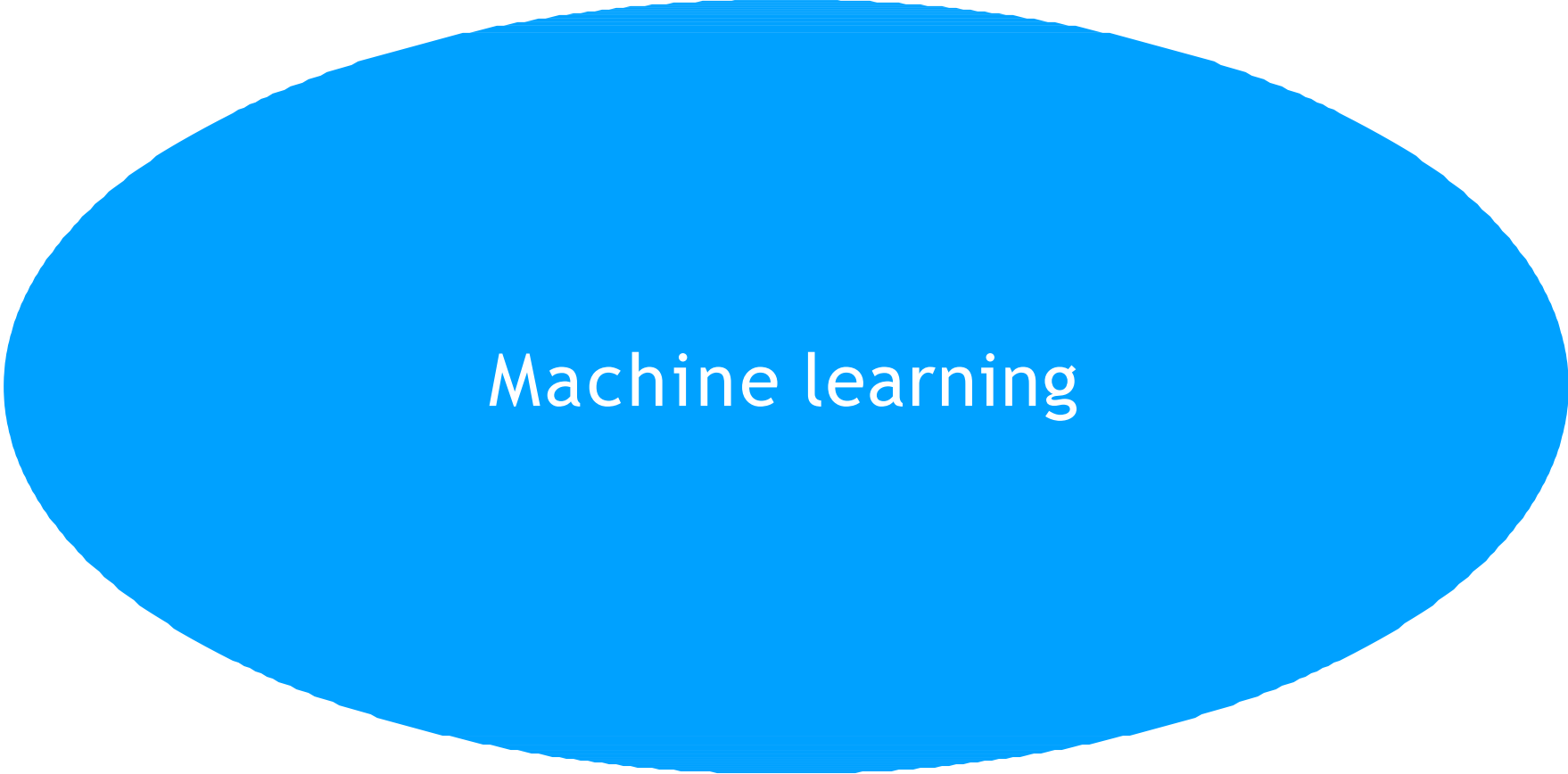
Bird classifier



Goldfinch

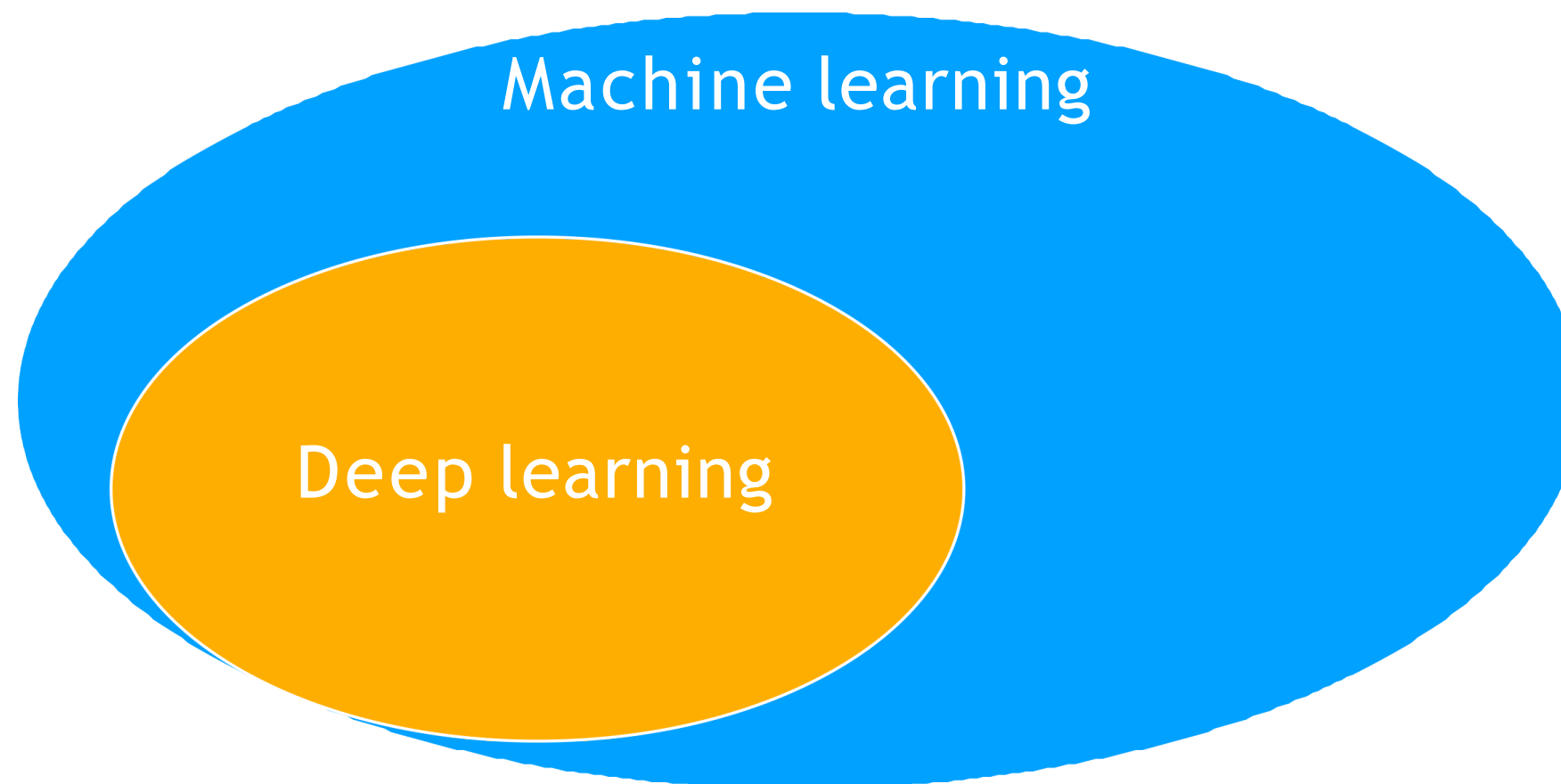
What does this have to do with **deep learning**?

Deep Learning is a Subset of Machine Learning

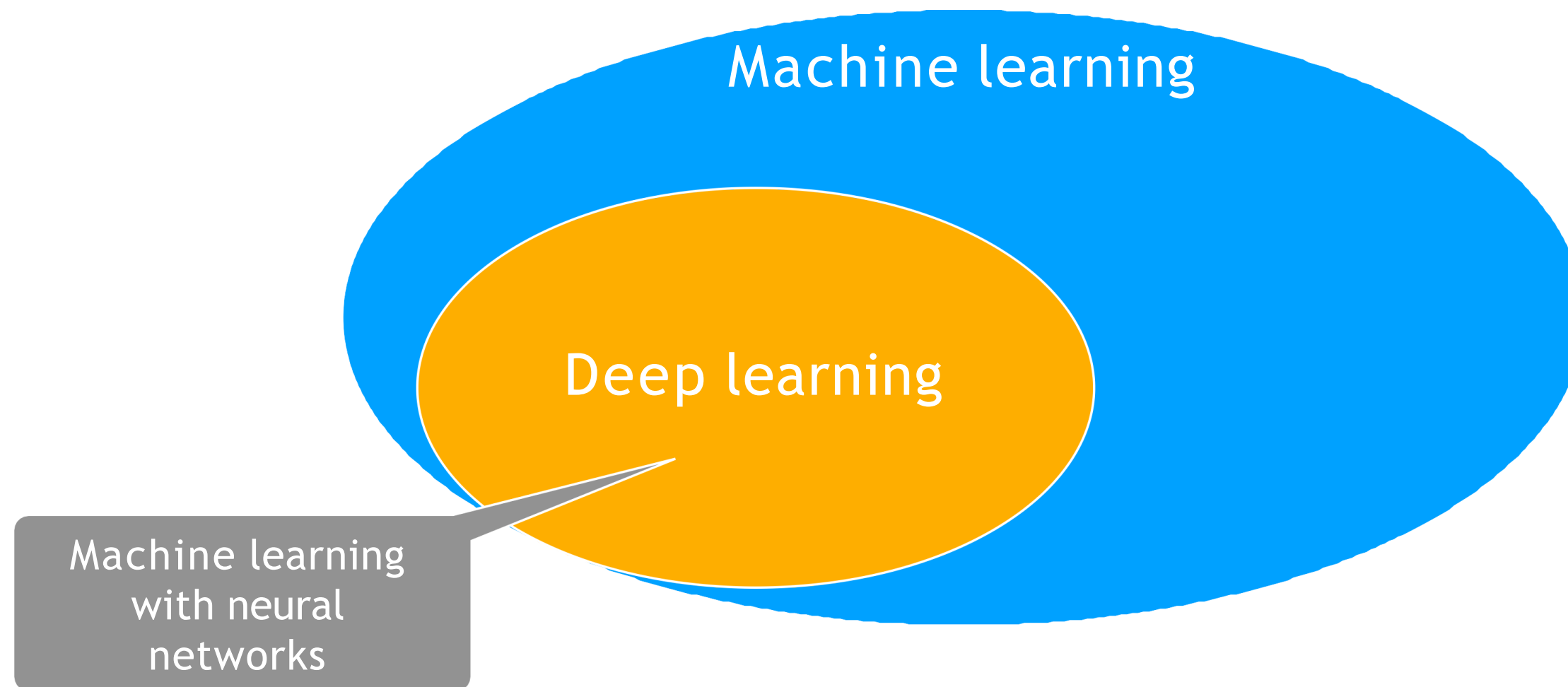


Machine learning

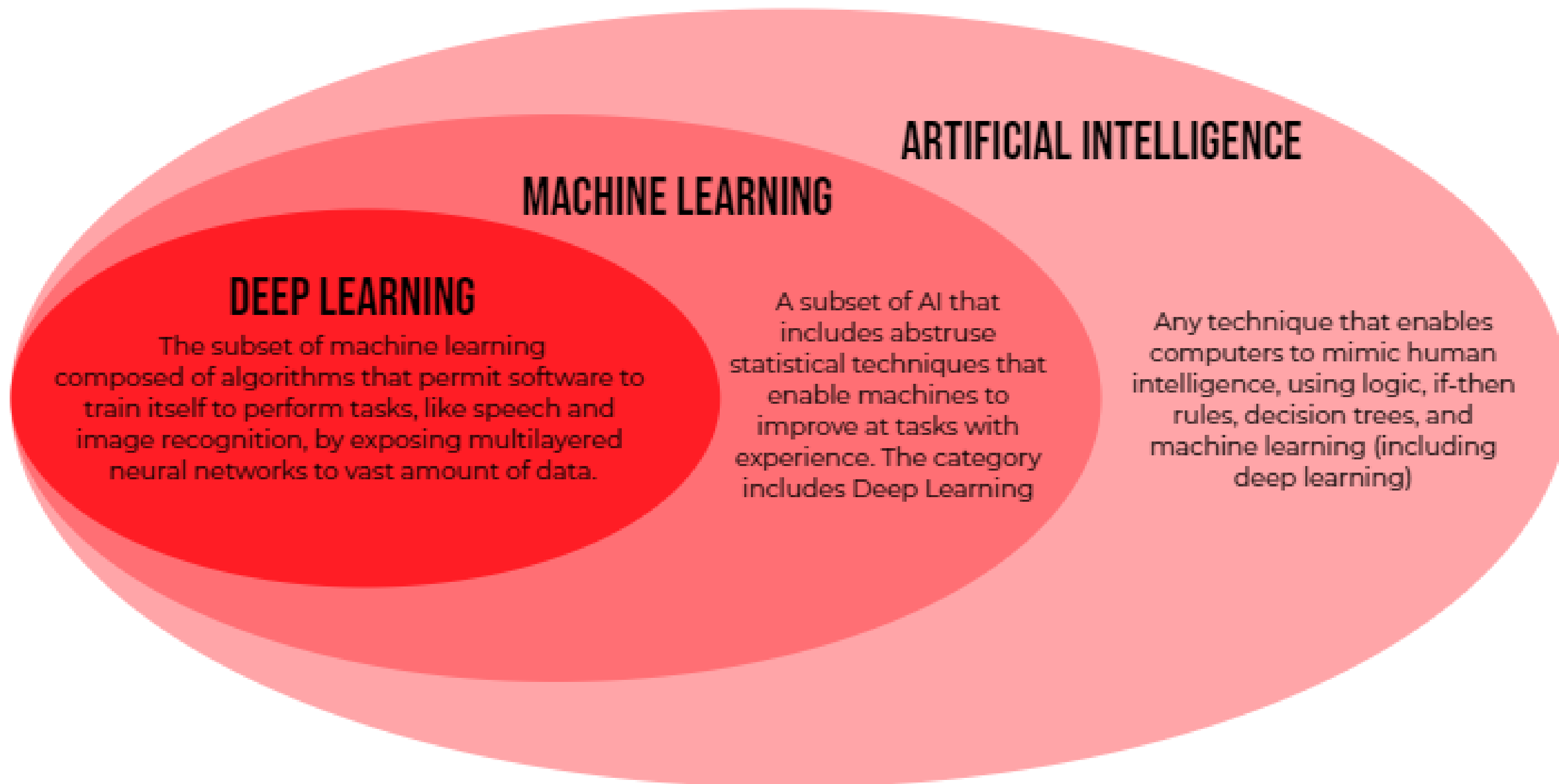
Deep Learning is a Subset of Machine Learning



Deep Learning is a Subset of Machine Learning



Machine Learning Started as a Subfield of AI



Deep Learning

- Deep Learning is a subset of machine learning, focused on neural networks with many layers (hence "deep").
- It is inspired by the human brain's structure, with neurons and synapses represented as artificial neurons and connections.

Deep Learning

Key Characteristics of Deep Learning:

- Multiple Layers: The depth of layers allows **neural networks** to learn complex patterns and features.
- End-to-End Learning: The model automatically learns features from raw data, minimizing the need for manual feature extraction.
- Representation Learning: As data passes through the network, the system refines its representation, from low-level features (edges, colors) to high-level abstractions (objects, shapes).

When do we use deep learning vs regular machine learning?

Deep learning

large datasets
in “unstructured” form
(e.g., images and text)



Deep learning

large datasets
in “unstructured” form
(e.g., images and text)



Regular machine learning

Datasets in “structured”
form like tables

	Beak length	Wing span	Primary color	...
Bird 1	3 cm	43 cm	black	...
Bird 2	0.5 cm	19 cm	yellow	...
Bird 3	5 cm	48 cm	black	...
Bird 4	4 cm	45 cm	black	...
...	...	...	...	...

Regular machine learning

“Unstructured” data



Manual feature extraction



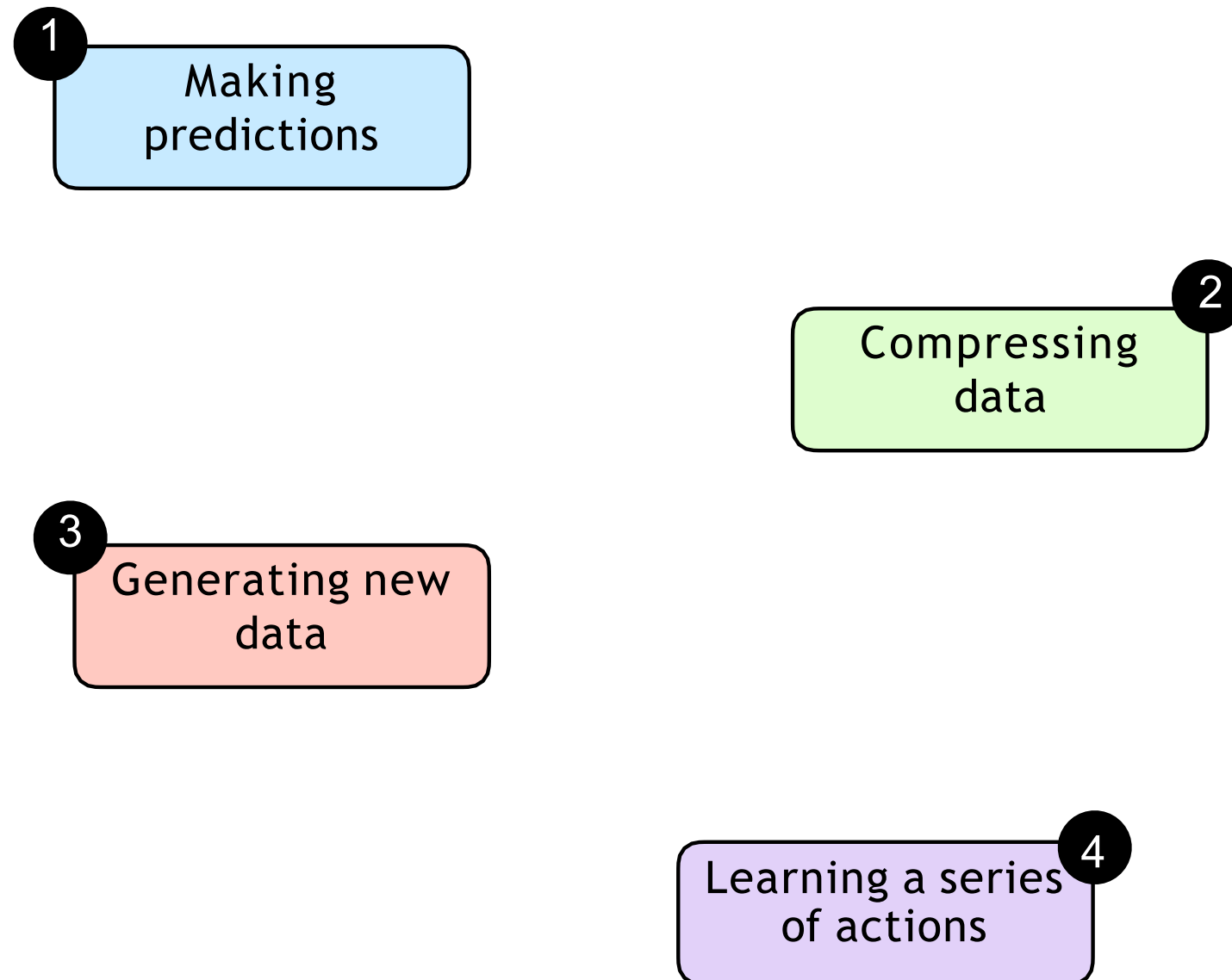
Datasets in “structured”
form like tables

	Beak length	Wing span	Primary color	...
Bird 1	3 cm	43 cm	black	...
Bird 2	0.5 cm	19 cm	yellow	...
Bird 3	5 cm	48 cm	black	...
Bird 4	4 cm	45 cm	black	...
...	...	...	...	...

Deep Learning vs Traditional Machine Learning

- **Feature Engineering:** Traditional ML requires domain-specific feature extraction, while deep learning learns features directly from the data.
- **Performance:** Deep learning outperforms traditional methods when provided with large datasets, making it effective for tasks like image recognition, speech processing, and NLP.

Common Applications Areas



1

Making
predictions

Is this email **spam** or not?

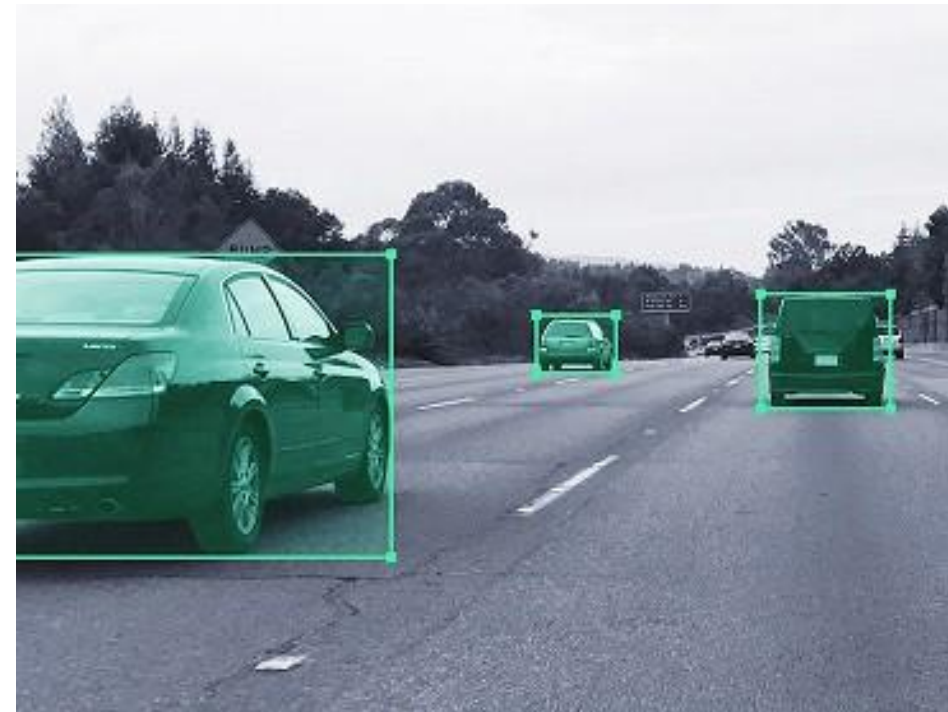


A label prediction task

1

Making
predictions

Where are the **cars** located?

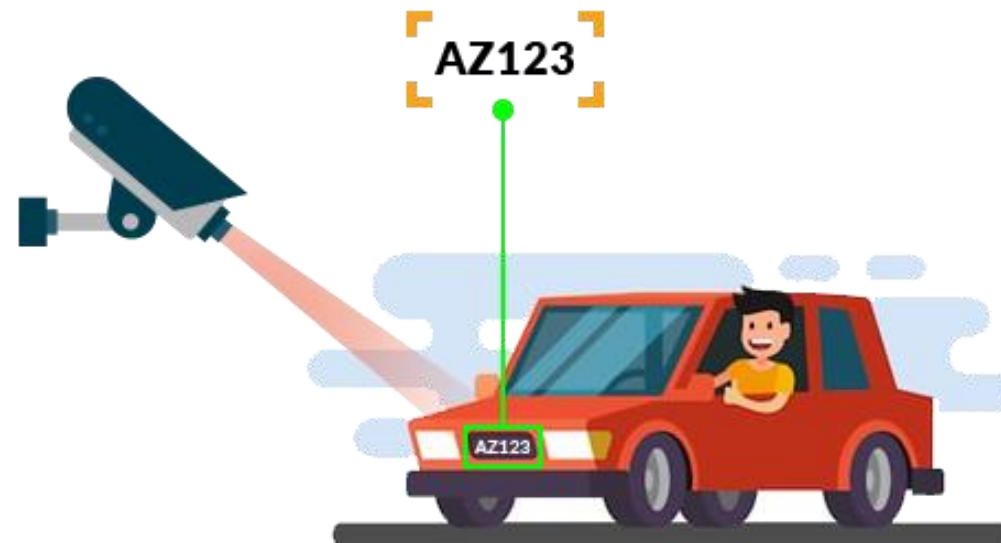


Predicting the bounding boxes

1

Making
predictions

What is the **license plate** code?



Recognizing or predicting letters and digits

Compressing
data

2

Compressing large datasets

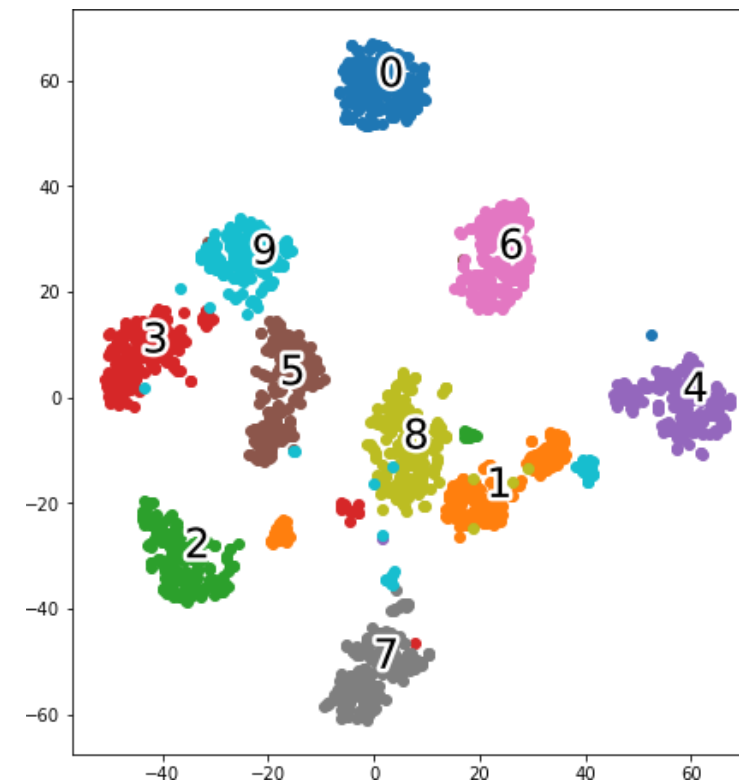


Extracting interesting data for storage

Compressing
data

2

Exploring datasets

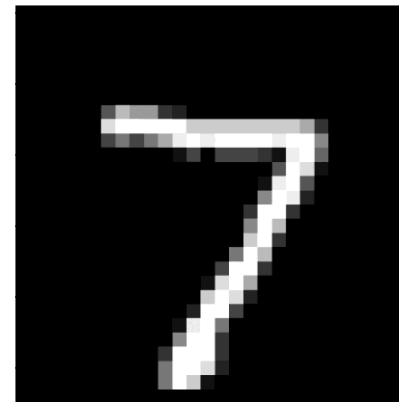


Reducing dimensionality for visualization & clustering

Compressing
data

2

Exploring datasets

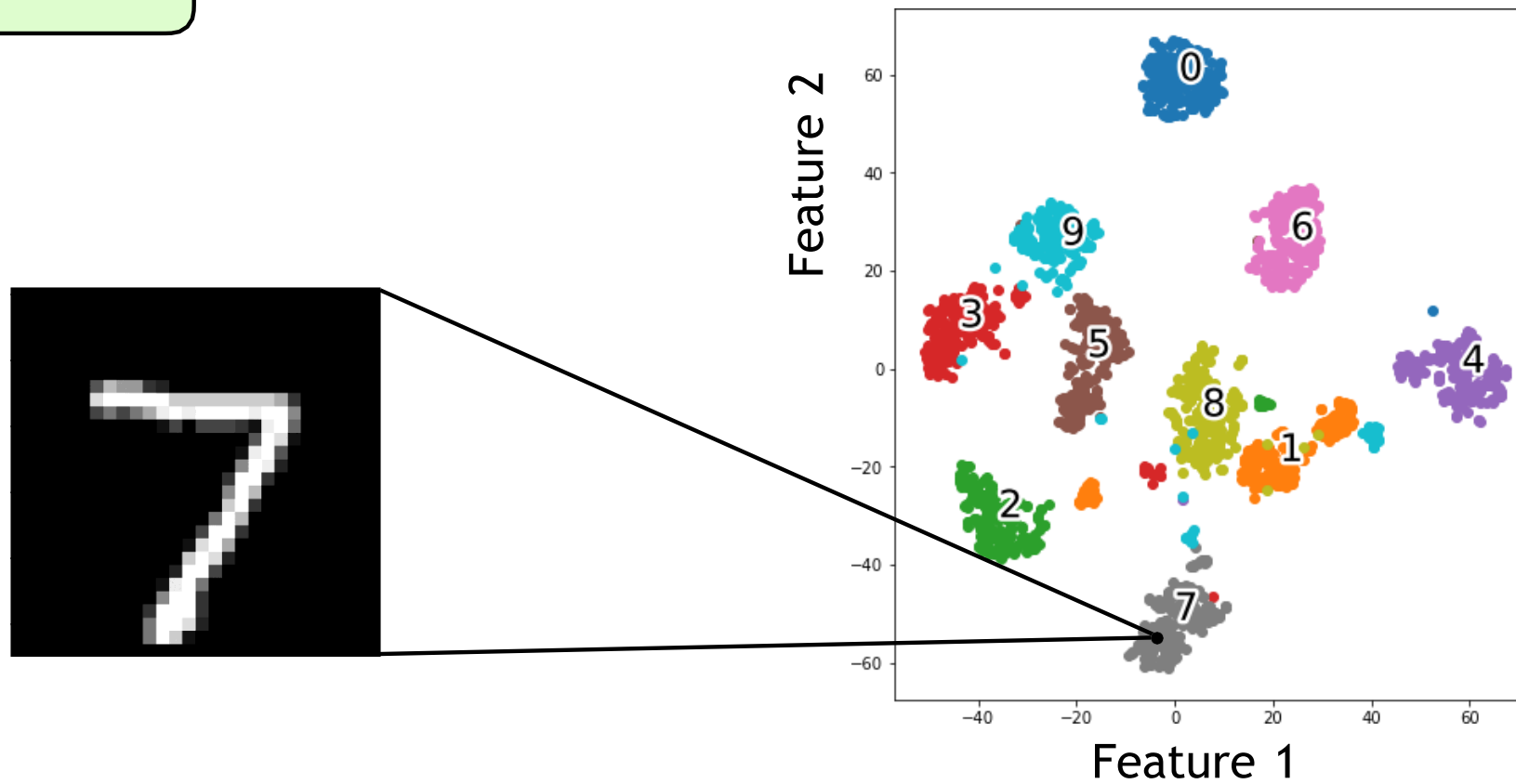


A 28x28 (=784) dimensional image

Exploring datasets

Compressing
data

2



Compressing 28x28 (=784) dimensional images to 2 dimensions

3

Generating new
data



This Person Does Not Exist



This Cat Does Not Exist



This Rental Does Not Exist

DALL-E, Parti, Imagen, and Others

TEXT PROMPT

an armchair in the shape of an avocado. . . .

AI-GENERATED IMAGES



[Edit prompt or view more images](#) ↓

Source: <https://openai.com/blog/dall-e/>

Learning a series
of actions

4

Performing moves to **win** a game



Learning a series
of actions

4

Moving a robot



Some more examples of Deep Learning Applications

- Image Classification: Identifying objects in an image (e.g., recognizing a cat in a photo).
- Speech Recognition: Translating spoken words into text (e.g., Google Assistant, Siri).
- Autonomous Driving: Detecting and recognizing objects in real-time for navigation.

The 3 Classic Categories of Machine Learning

The 3 Classic Categories of Machine Learning

Supervised
learning

The 3 Classic Categories of Machine Learning

Supervised
learning

Data with
answers (targets, labels)
available

The 3 Classic Categories of Machine Learning

Supervised
learning

Data with
answers (targets, labels)
available

Prediction tasks

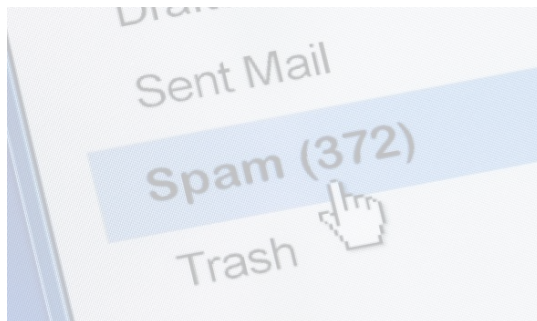


The 3 Classic Categories of Machine Learning

Supervised
learning

Data with
answers (targets, labels)
available

Prediction tasks



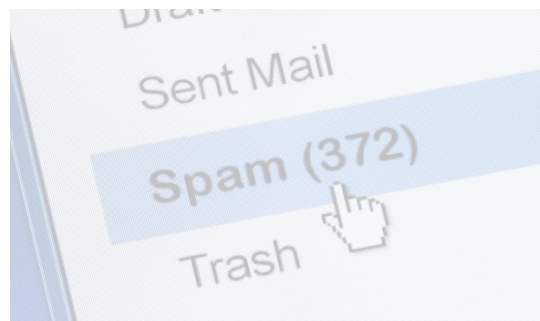
Unsupervised
learning

The 3 Classic Categories of Machine Learning

Supervised
learning

Data with
answers (targets, labels)
available

Prediction tasks



Unsupervised
learning

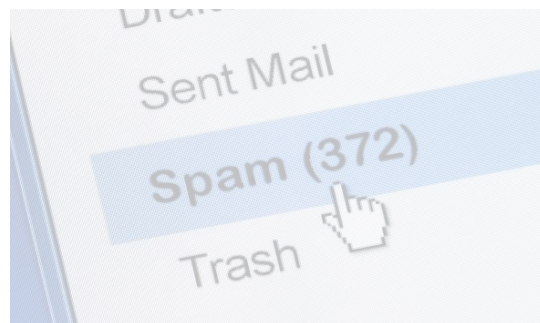
Data without
available
answers

The 3 Classic Categories of Machine Learning

Supervised
learning

Data with
answers (targets, labels)
available

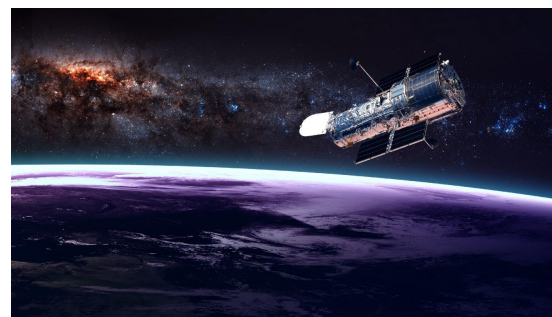
Prediction tasks



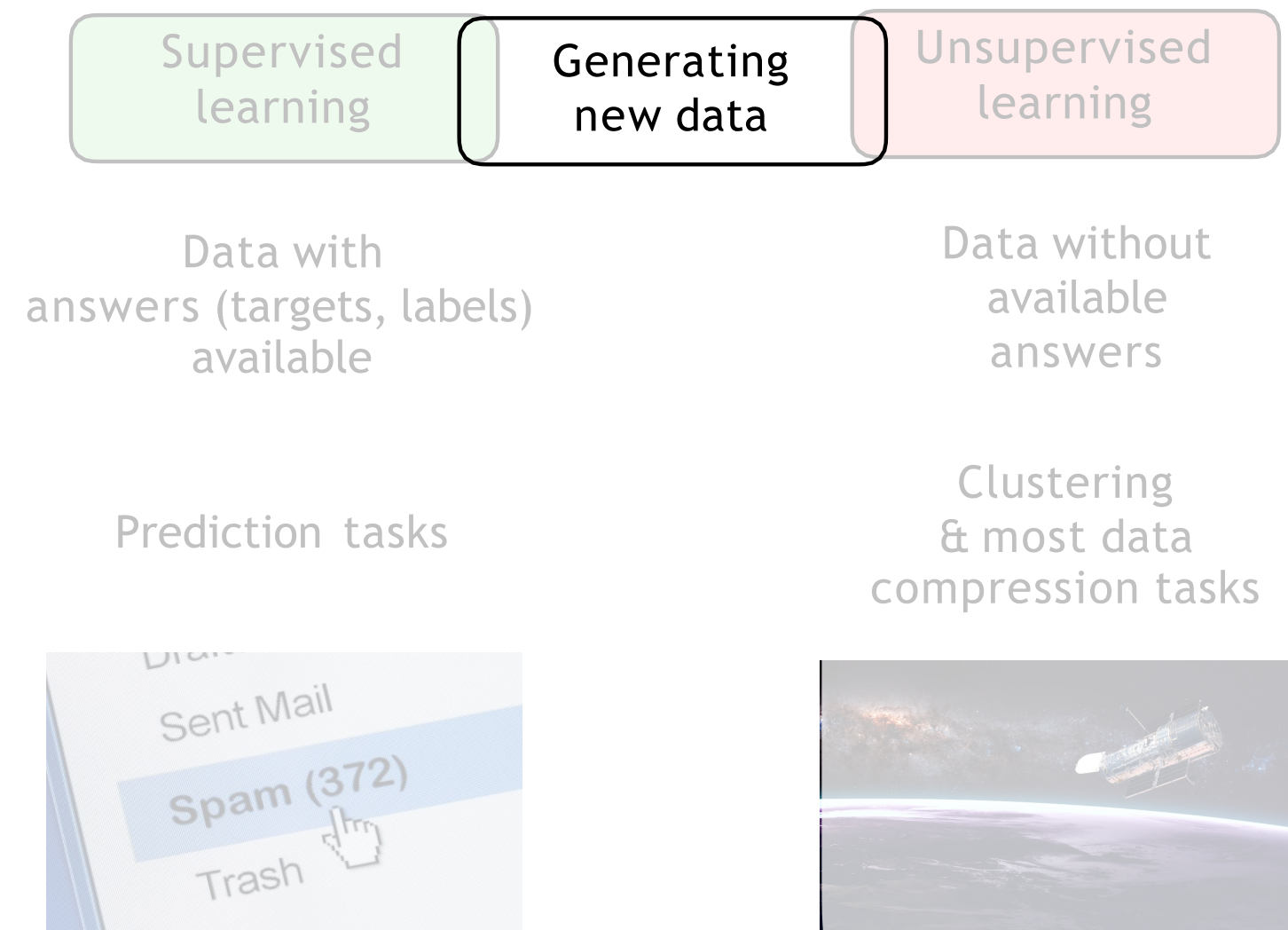
Unsupervised
learning

Data without
available
answers

Clustering
& most data
compression tasks



The 3 Classic Categories of Machine Learning

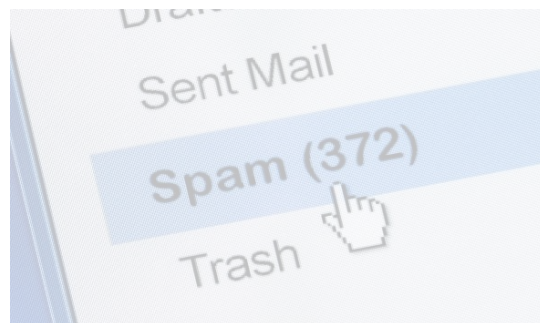


The 3 Classic Categories of Machine Learning

Supervised
learning

Data with
answers (targets, labels)
available

Prediction tasks

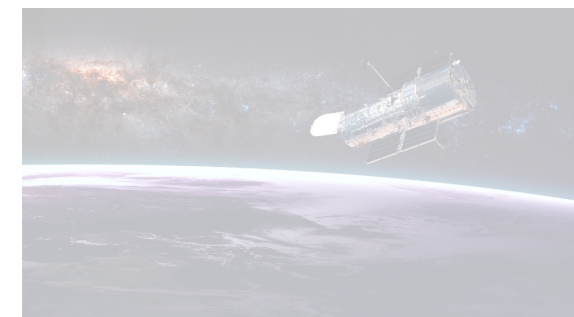


Reinforcement
learning

Unsupervised
learning

Data without
available
answers

Clustering
& most data
compression tasks

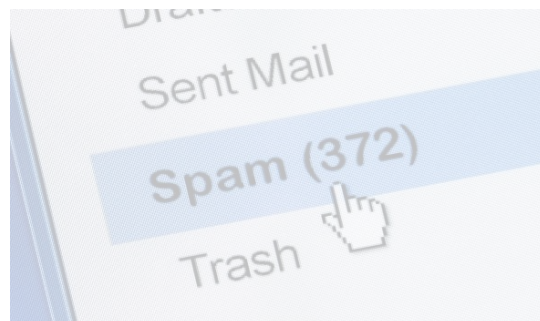


The 3 Classic Categories of Machine Learning

Supervised
learning

Data with
answers (targets, labels)
available

Prediction tasks



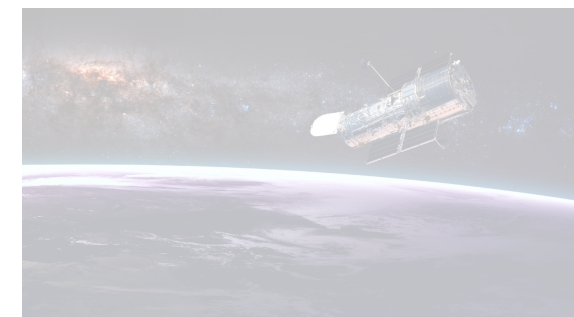
Reinforcement
learning

Series of steps
& a goal

Unsupervised
learning

Data without
available
answers

Clustering
& most data
compression tasks

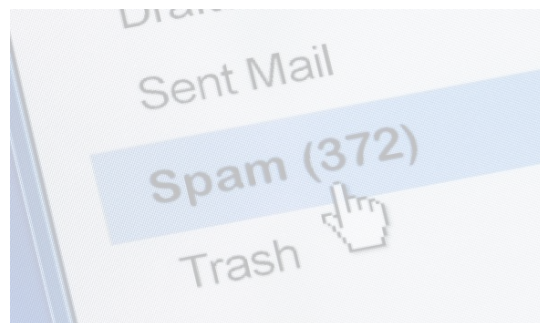


The 3 Classic Categories of Machine Learning

Supervised
learning

Data with
answers (targets, labels)
available

Prediction tasks



Reinforcement
learning

Series of steps
& a goal

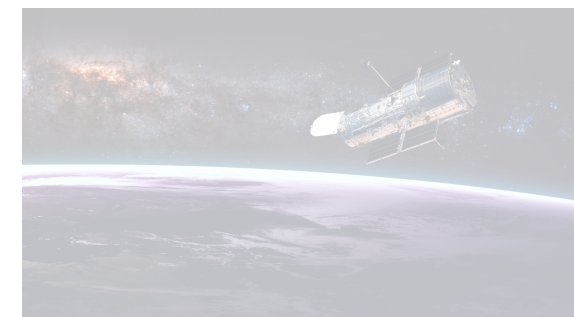
Learning a series
of actions



Unsupervised
learning

Data without
available
answers

Clustering
& most data
compression tasks



How does the supervised learning workflow look like?

An Example of A Classification Task

Is this email **spam** or not?



Supervised Learning

Label: poisonous



Example

Label: poisonous



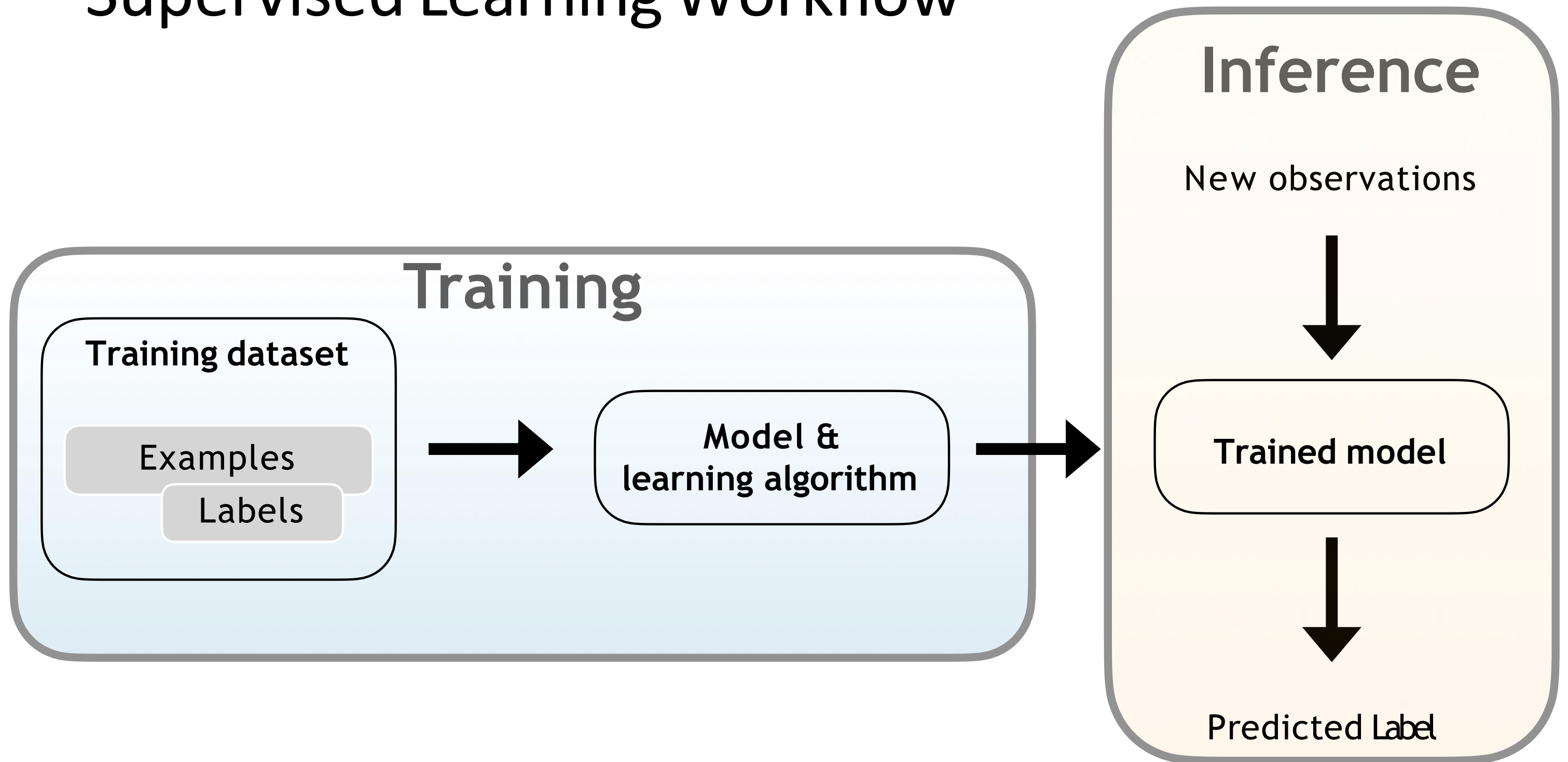
Example

Label: edible



Example

Supervised Learning Workflow



Common Types of Supervised Learning

1) Classification

Predicting a class label

Common Types of Supervised Learning

1) Classification

Predicting a class label

Which
animal is
this?



Common Types of Supervised Learning

1) Classification

Predicting a class label

Which
animal is
this?



Cat



Cat



Dog

Common Types of Supervised Learning

2) Regression

Predicting a continuous value

Common Types of Supervised Learning

2) Regression

Predicting a continuous value

How expensive is this?



Common Types of Supervised Learning

2) Regression

Predicting a continuous value

How
expensive is
this?



\$200,100



\$475,100



\$235,400

Supervised Learning Dataset

Input:



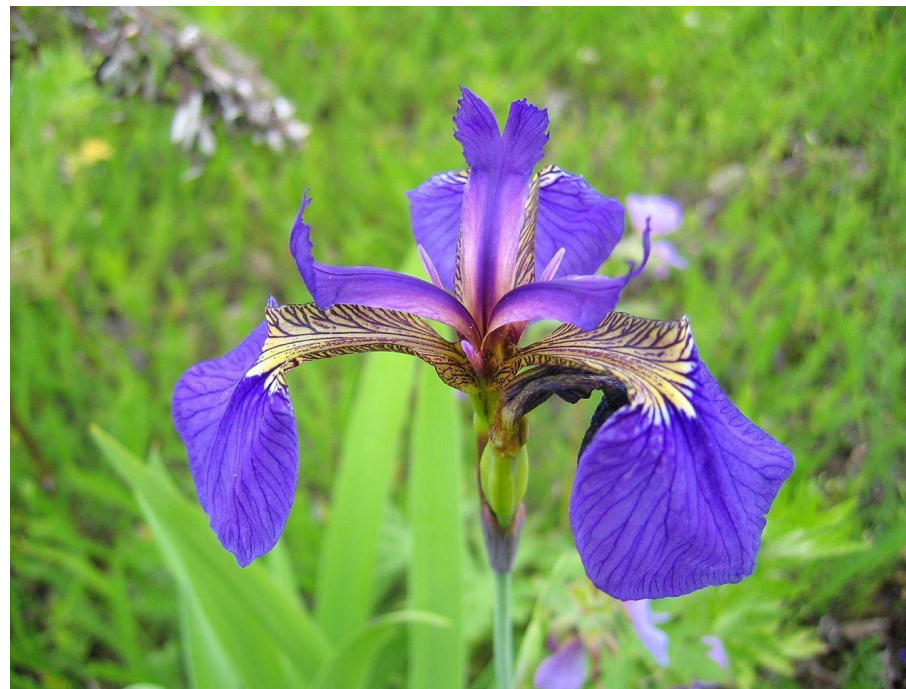
Label:

Iris-setosa

Supervised Learning Dataset

Unstructured data

Input:



Label:

Iris-setosa

Supervised Learning Dataset

Structured data

Unstructured data



Inputs:

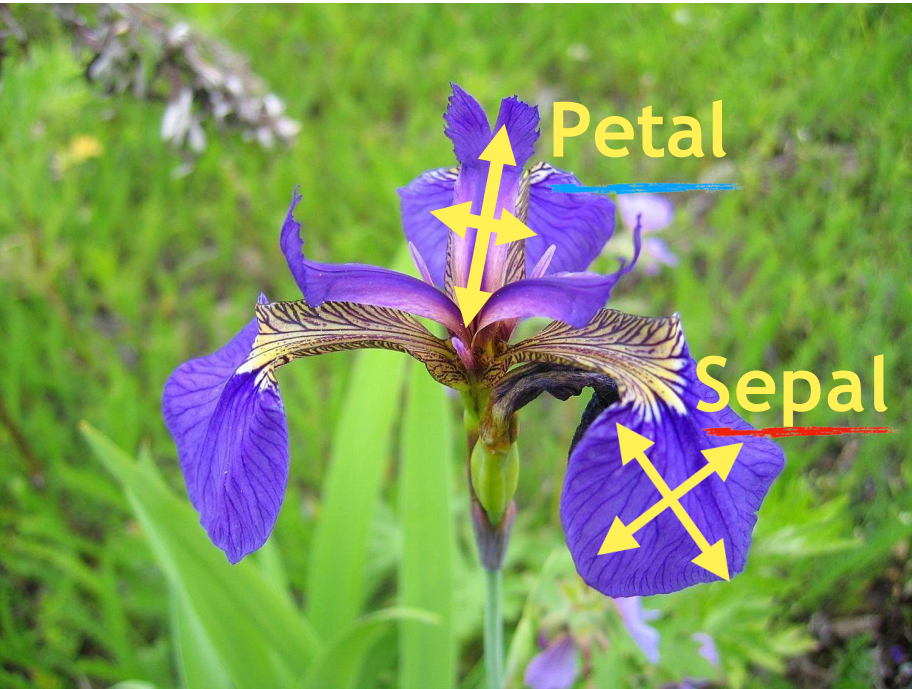
Sepal length	Sepal width	Petal length	Petal length
5.1	3.5	1.4	0.3
4.9	3	1.4	0.2
5.9	3	5.1	1.8
...	...	...	...

Labels:

Label
Iris-setosa
Iris-setosa
Iris-virginica
...

Supervised Learning Dataset

Unstructured data



Structured data

<u>Sepal length</u>	<u>Sepal width</u>	<u>Petal length</u>	<u>Petal length</u>
5.1	3.5	1.4	0.3
4.9	3	1.4	0.2
5.9	3	5.1	1.8
...	...	...	...

Supervised Learning Dataset

“Manual” feature engineering



Sepal length	Sepal width	Petal length	Petal length
5.1	3.5	1.4	0.3
4.9	3	1.4	0.2
5.9	3	5.1	1.8
...	...	...	...

Supervised Learning Dataset

Deep learning



“Traditional” machine learning

Sepal length	Sepal width	Petal length	Petal length
5.1	3.5	1.4	0.3
4.9	3	1.4	0.2
5.9	3	5.1	1.8
...	...	...	...

Let's explore how
machine learning classifiers
work!

Consider the simplest case: **binary classification**

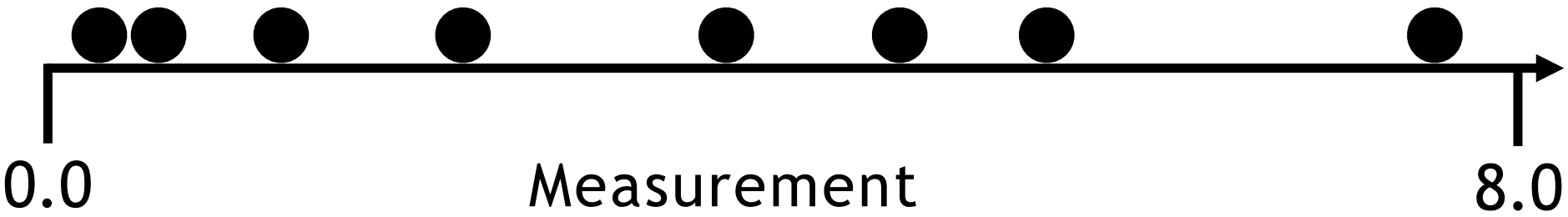
Examples of Binary Classification

- Spam / not-Spam
- Cat or dog
- Loan approved or not

Keeping things simple: A dataset with only
1 feature variable (measurement)

Inputs

Measurement
1.4
1.1
5.1
6.2
9.9
2.2
3.3
7.2

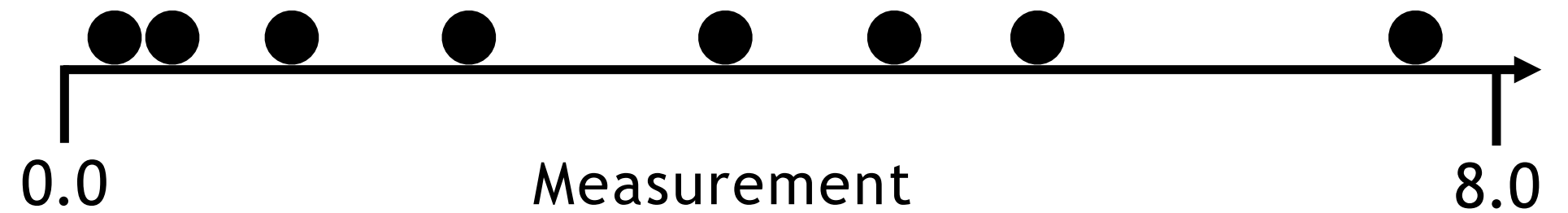


Inputs

Measurement
1.4
1.1
5.1
6.2
9.9
2.2
3.3
7.2

Labels

0
0
1
1
1
0
0
1

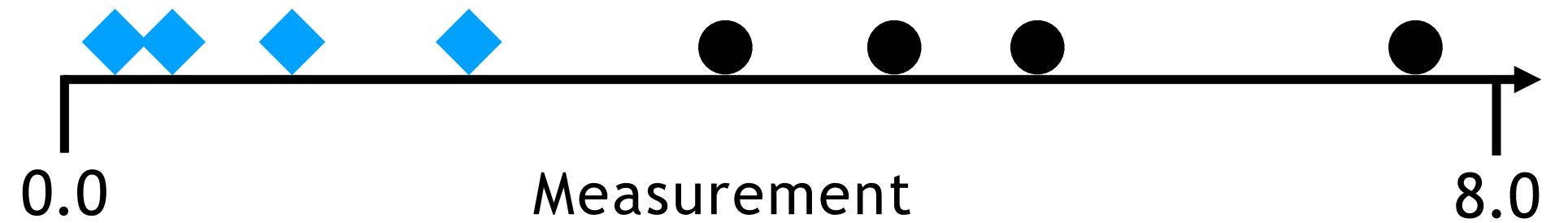


Inputs

Measurement
1.4
1.1
5.1
6.2
9.9
2.2
3.3
7.2

Labels

0
0
1
1
1
0
0
1

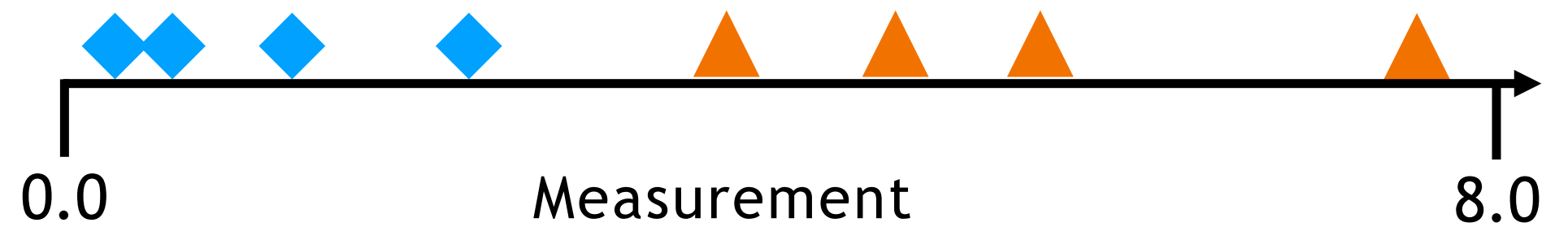


Inputs

Measurement
1.4
1.1
5.1
6.2
9.9
2.2
3.3
7.2

Labels

0
0
1
1
1
0
0
1

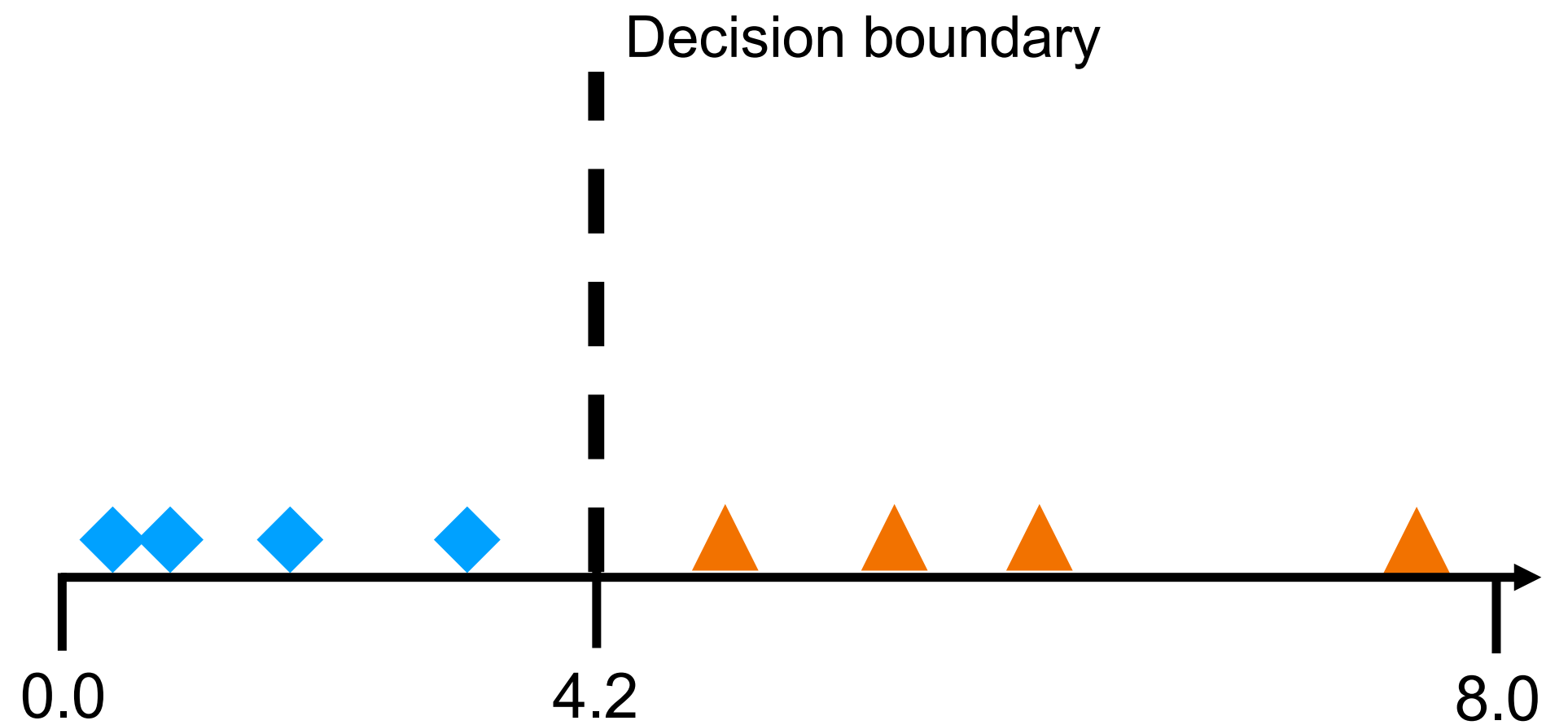


Inputs

Measurement
1.4
1.1
5.1
6.2
9.9
2.2
3.3
7.2

Labels

0
0
1
1
1
0
0
1

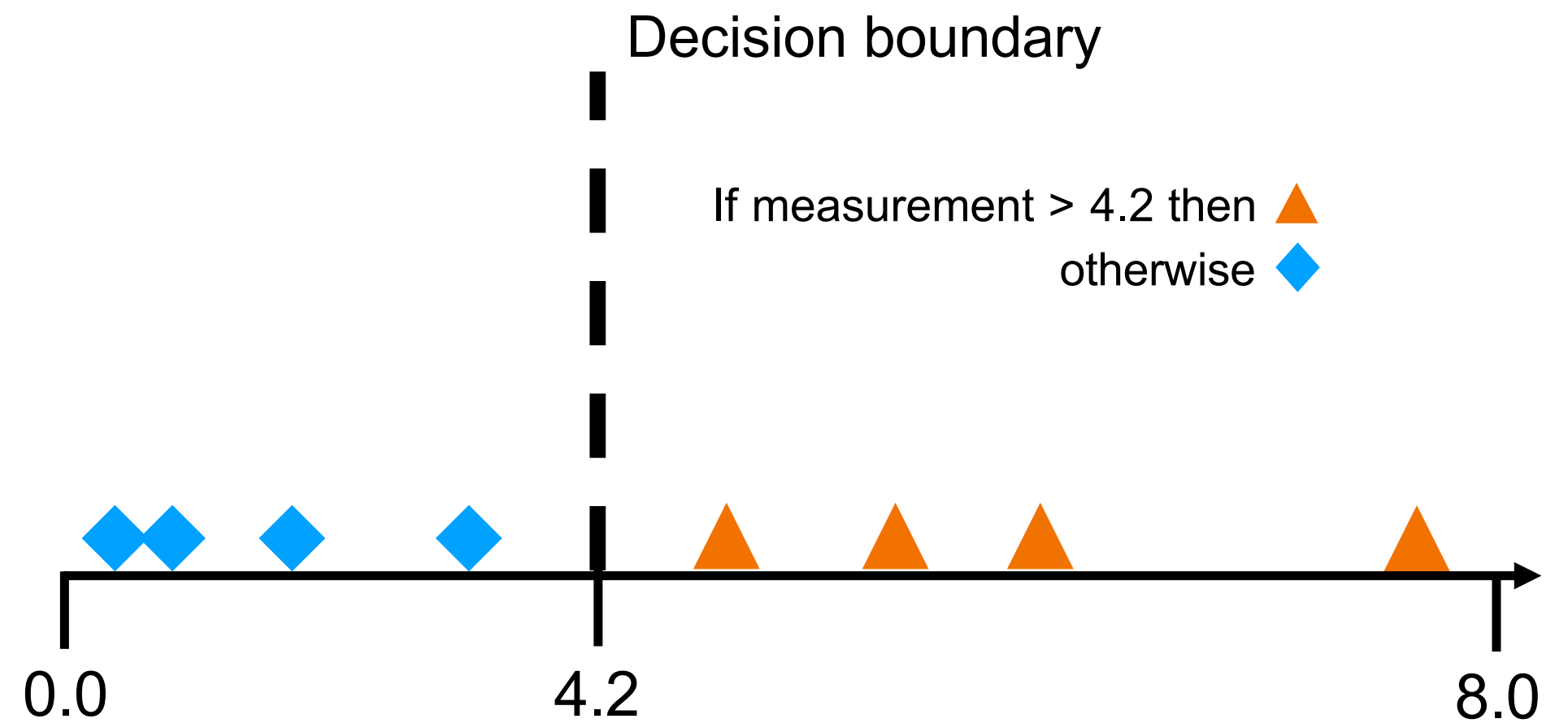


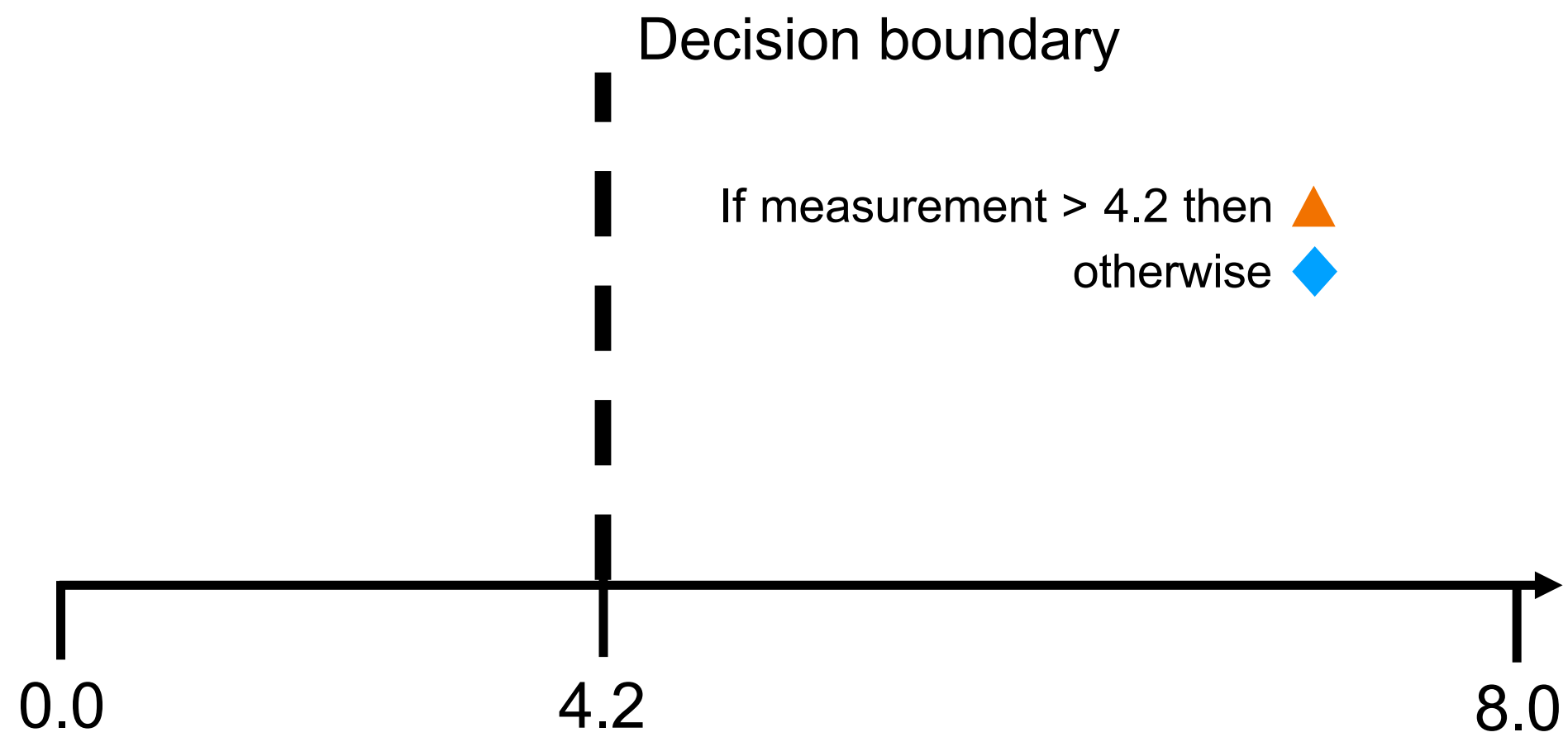
Inputs

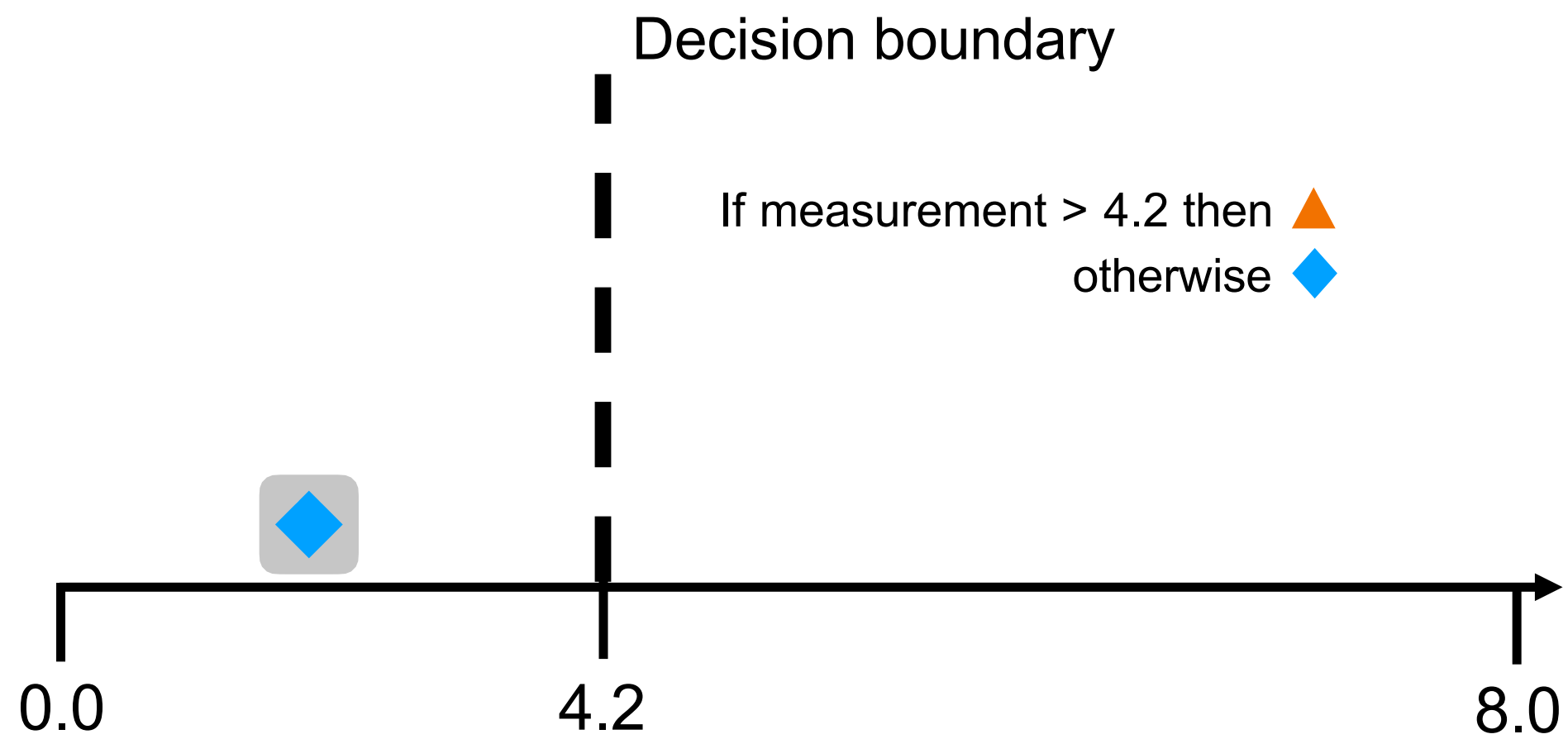
Measurement
1.4
1.1
5.1
6.2
9.9
2.2
3.3
7.2

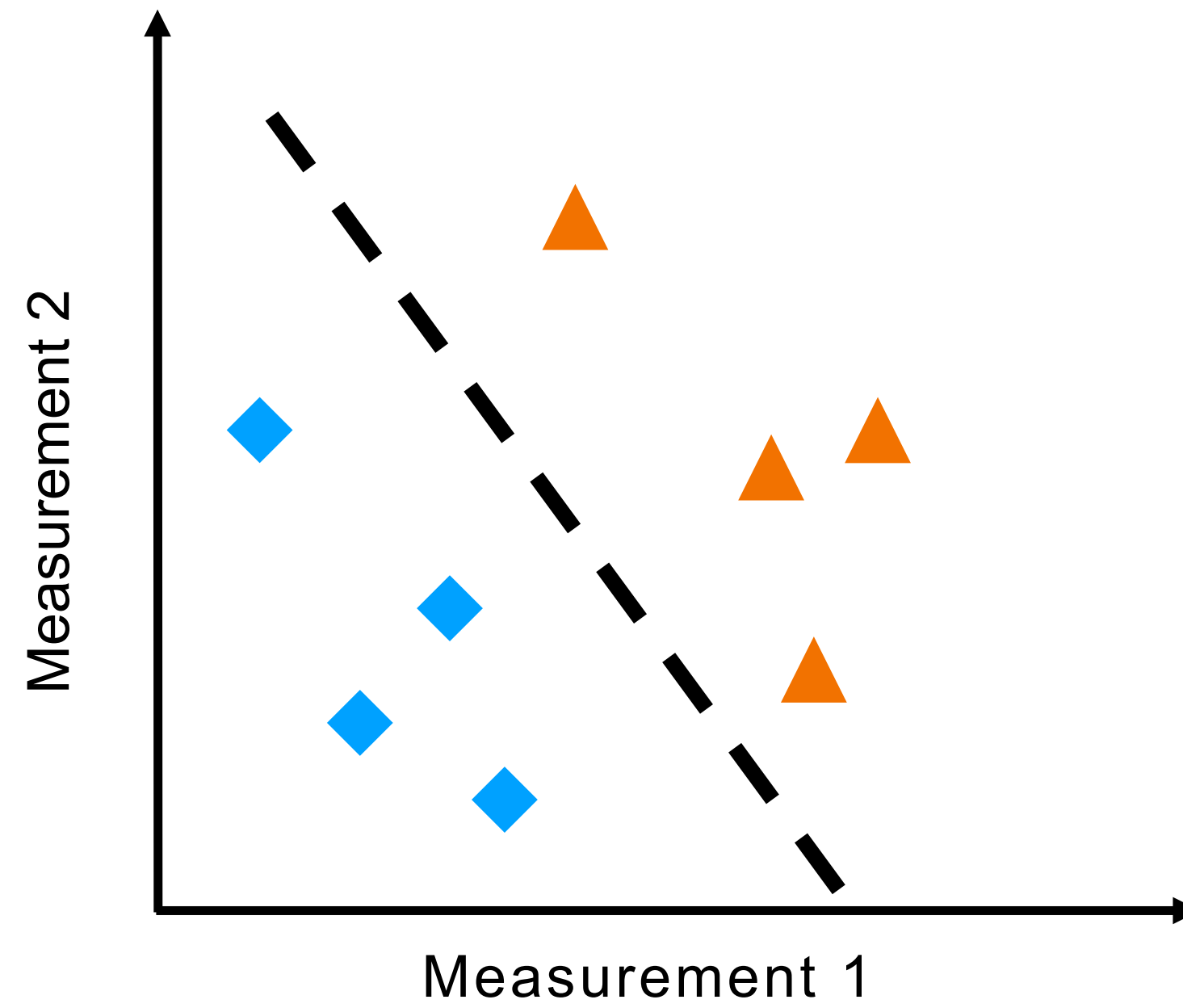
Labels

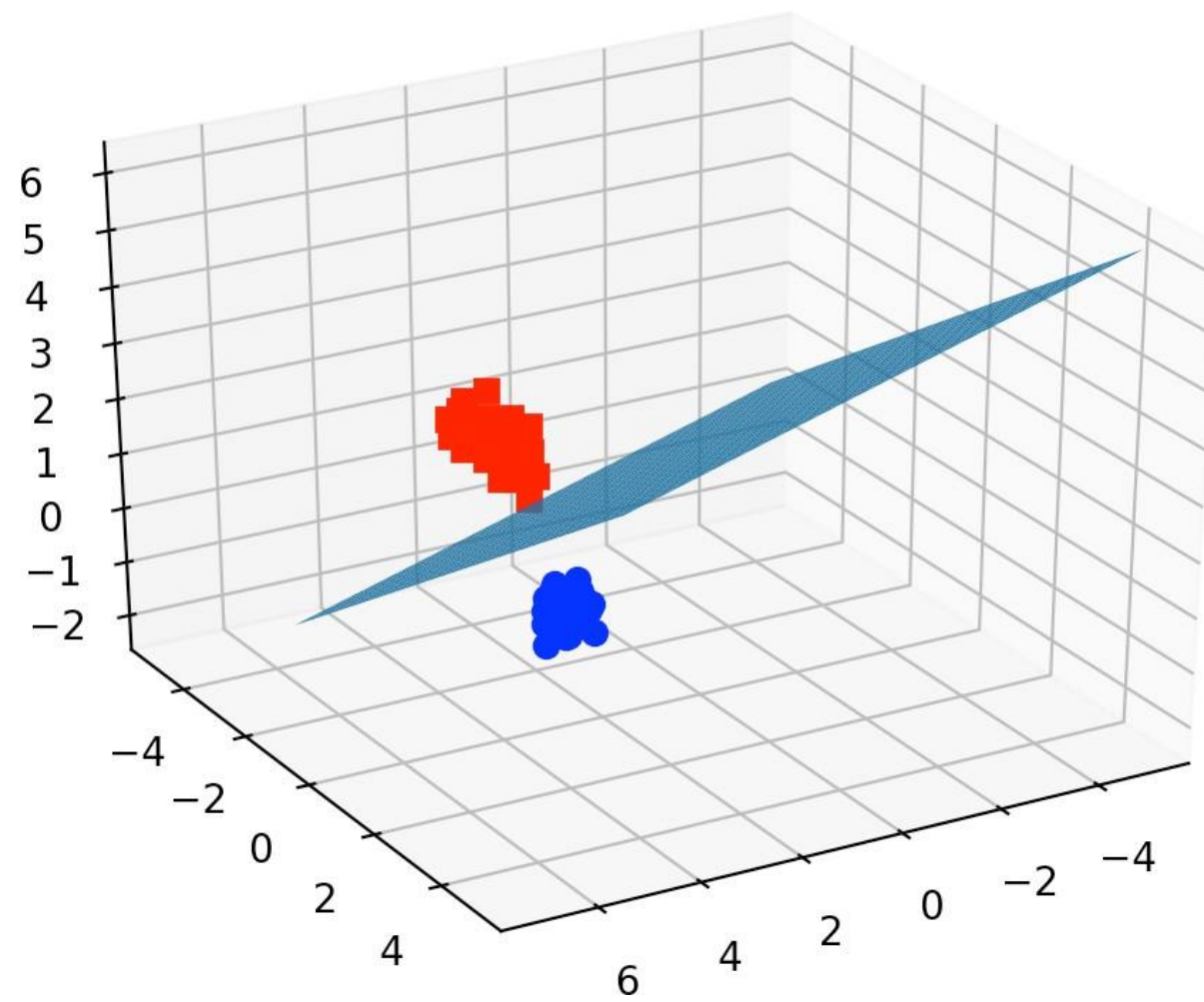
0
0
1
1
1
0
0
1











Perceptron

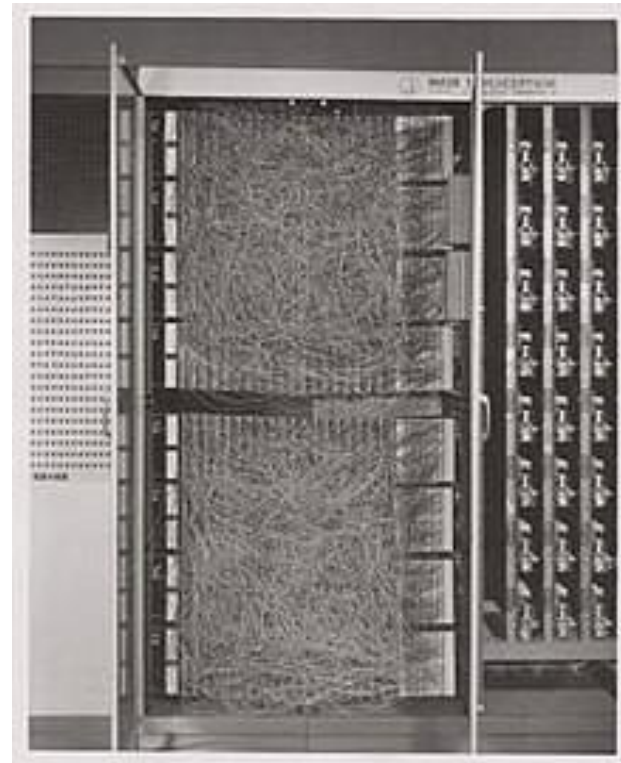
The Perceptron

One of the earliest machine learning algorithms

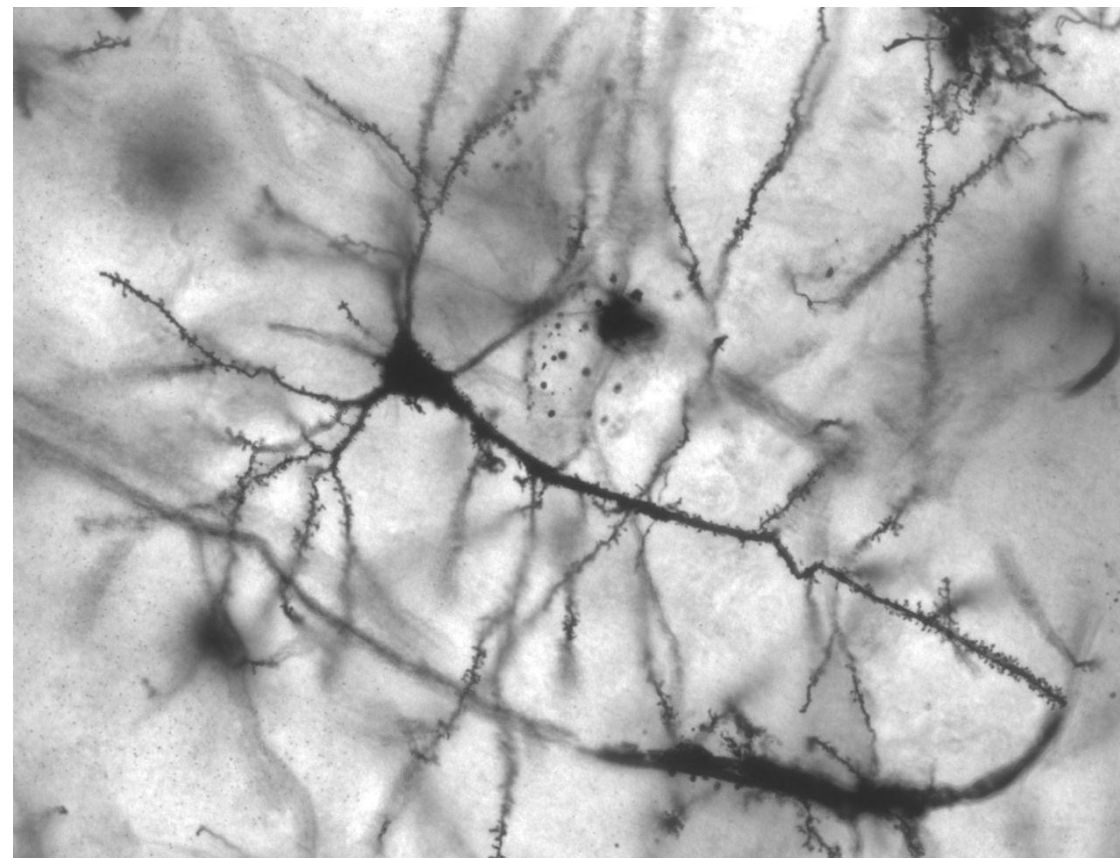
The structural building block of deep learning

The Perceptron

One of the earliest machine learning algorithms



Inspired by Neurons in the Human Brain



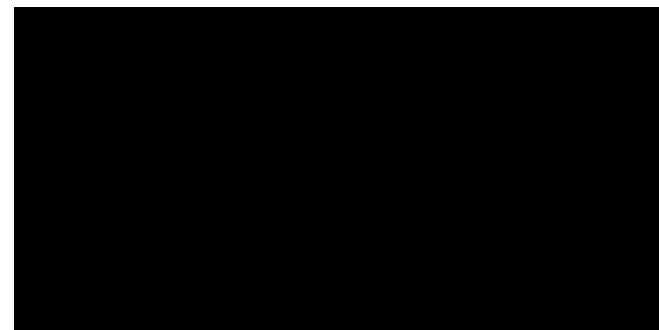
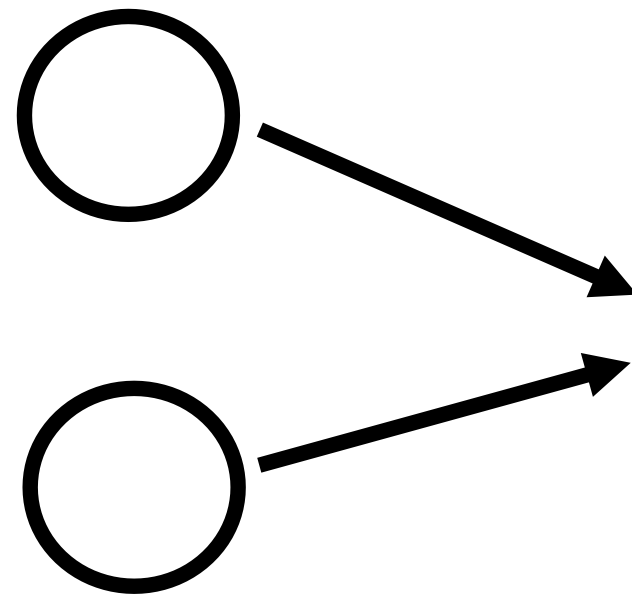
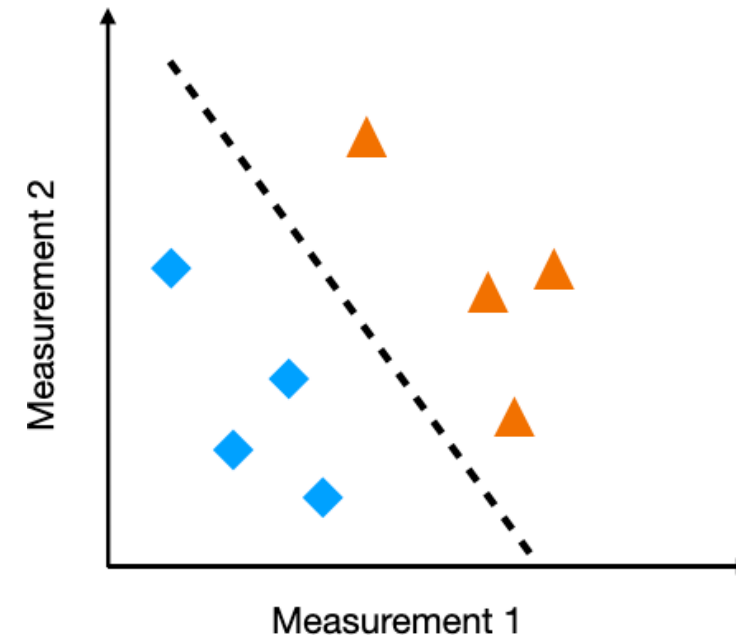
But it is not actually how the human brain works

Perceptron Structure

Inputs

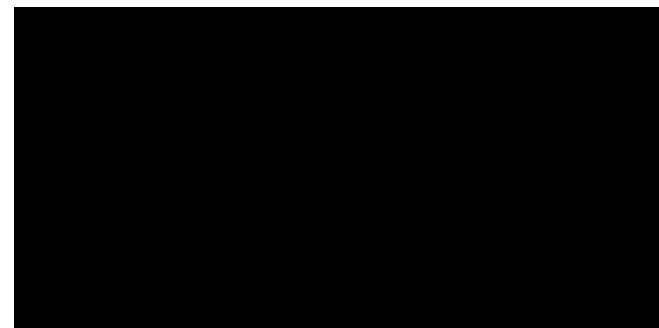
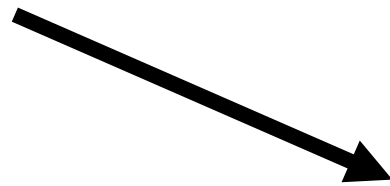
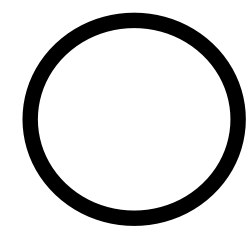
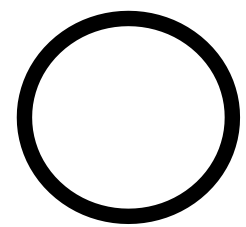


Inputs

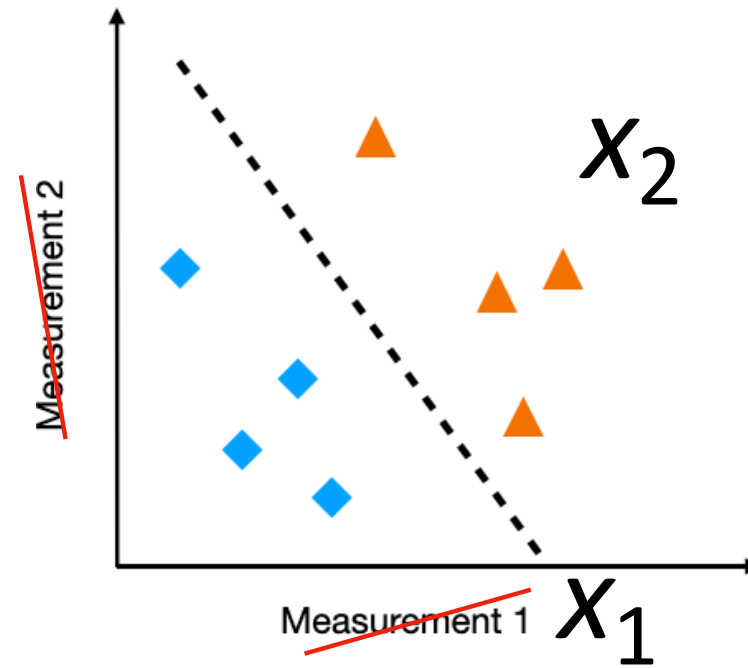


Outputs
(predictions)

Inputs



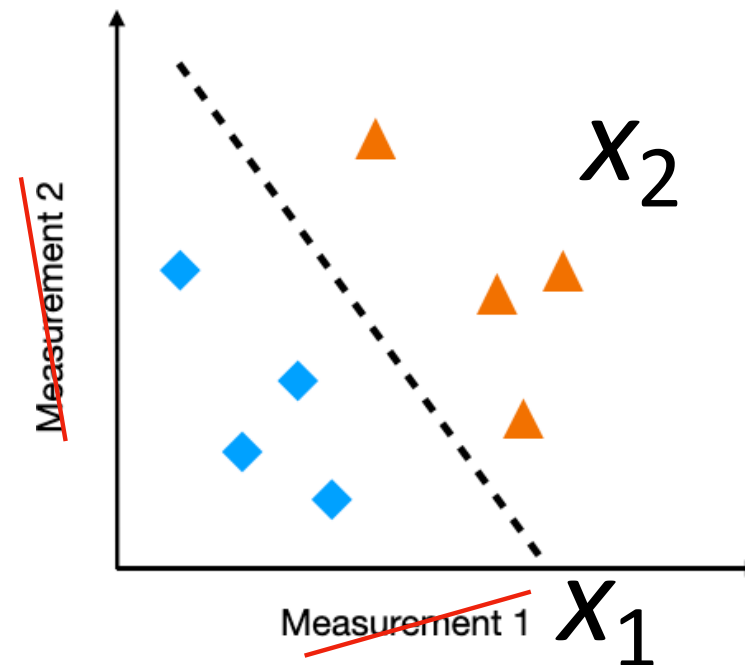
Outputs
(predictions)



Inputs

x_1

x_2

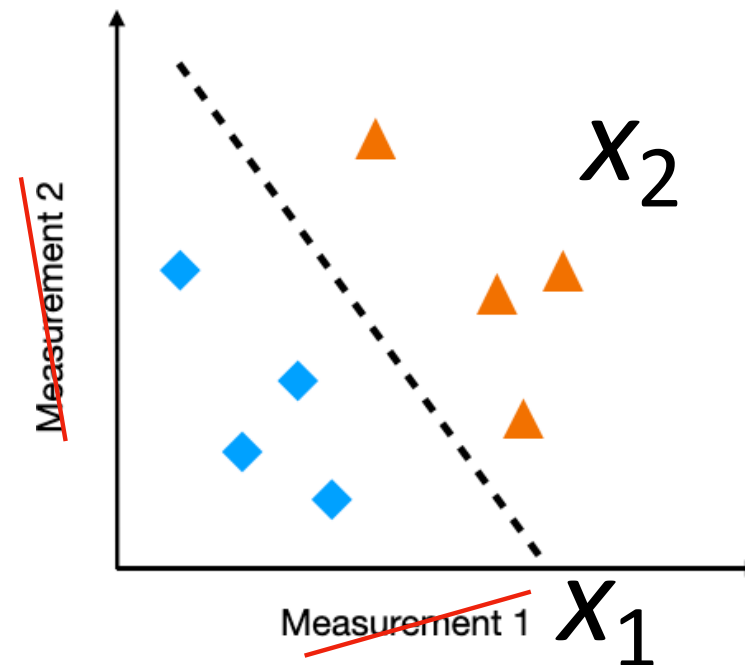


Outputs
(predictions)

Inputs

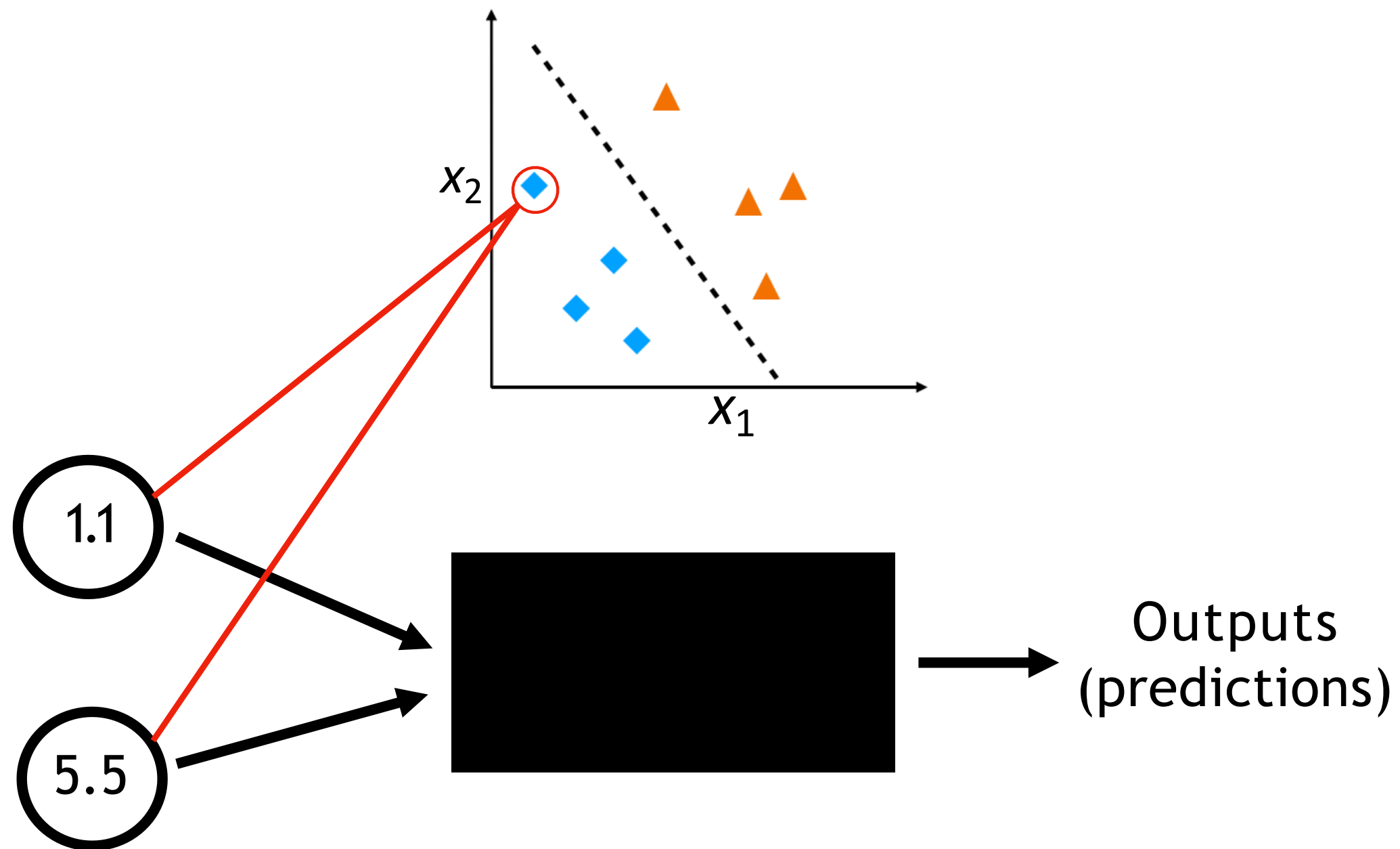
x_1

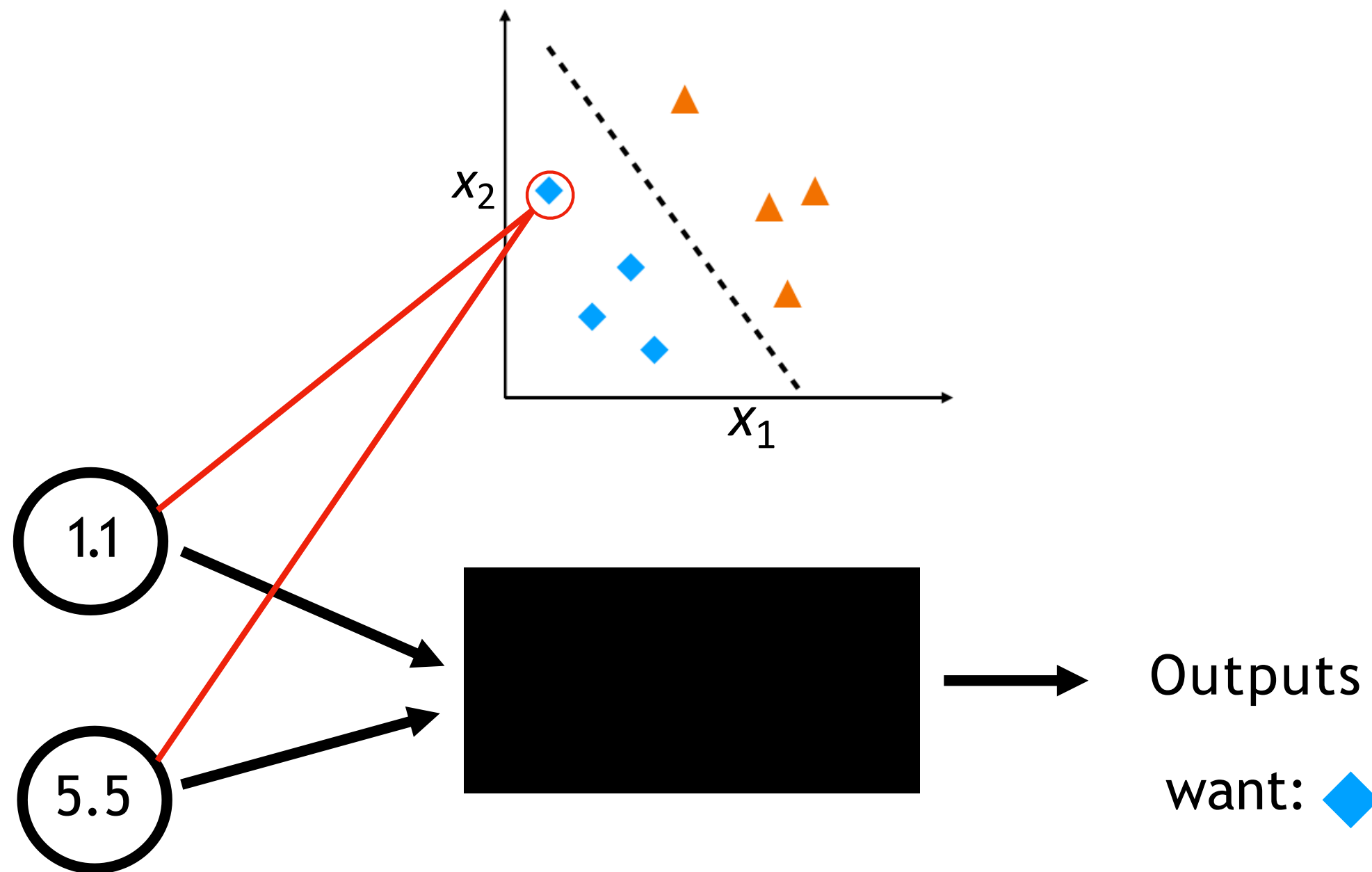
x_2

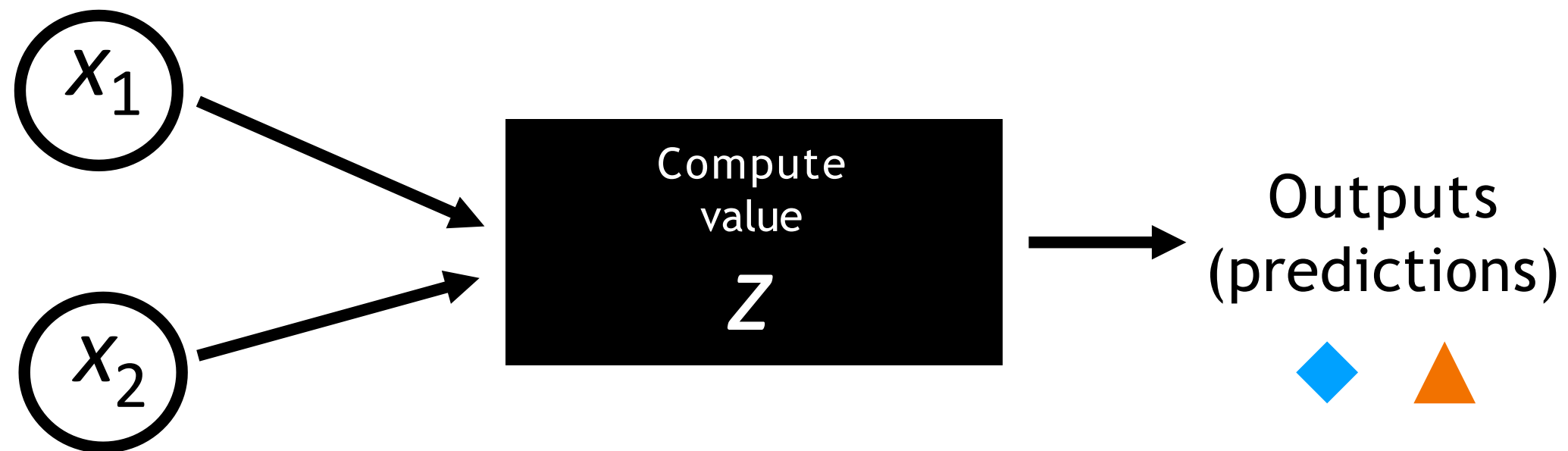
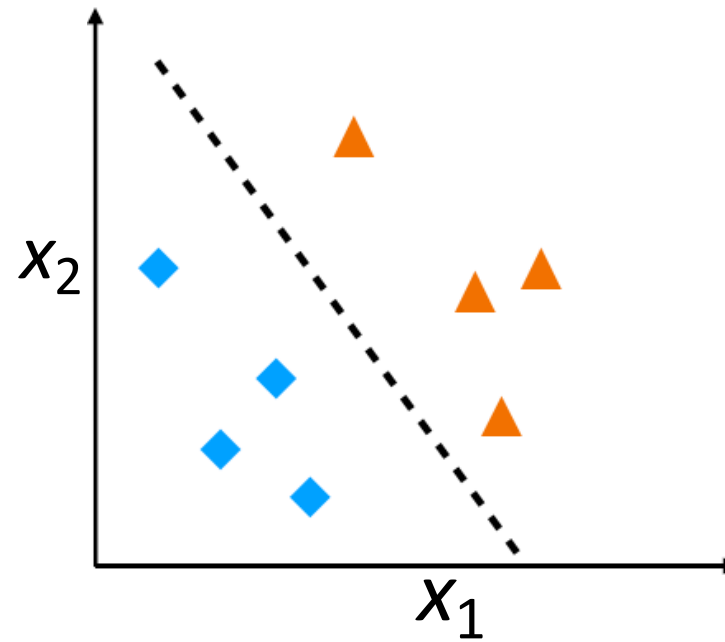


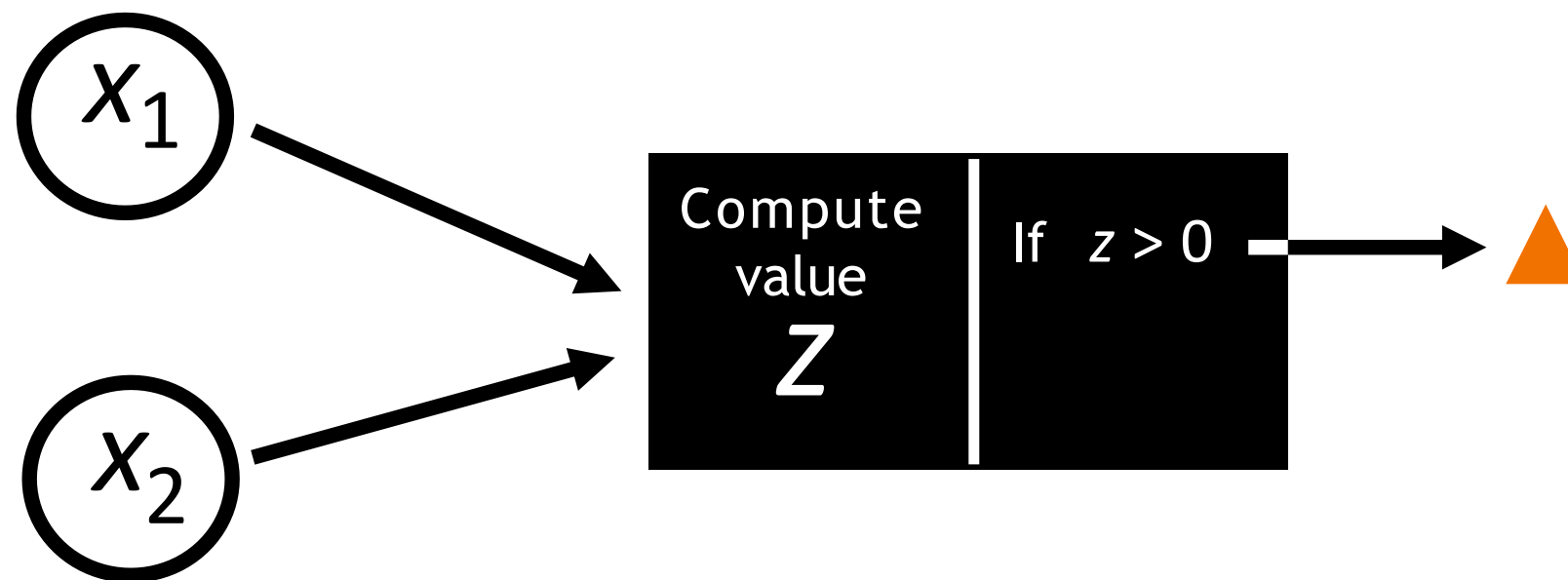
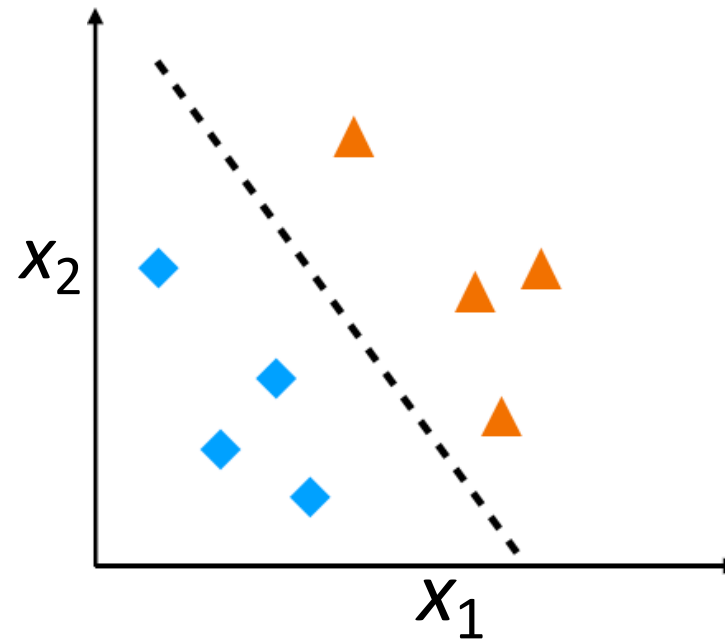
Outputs
(predictions)

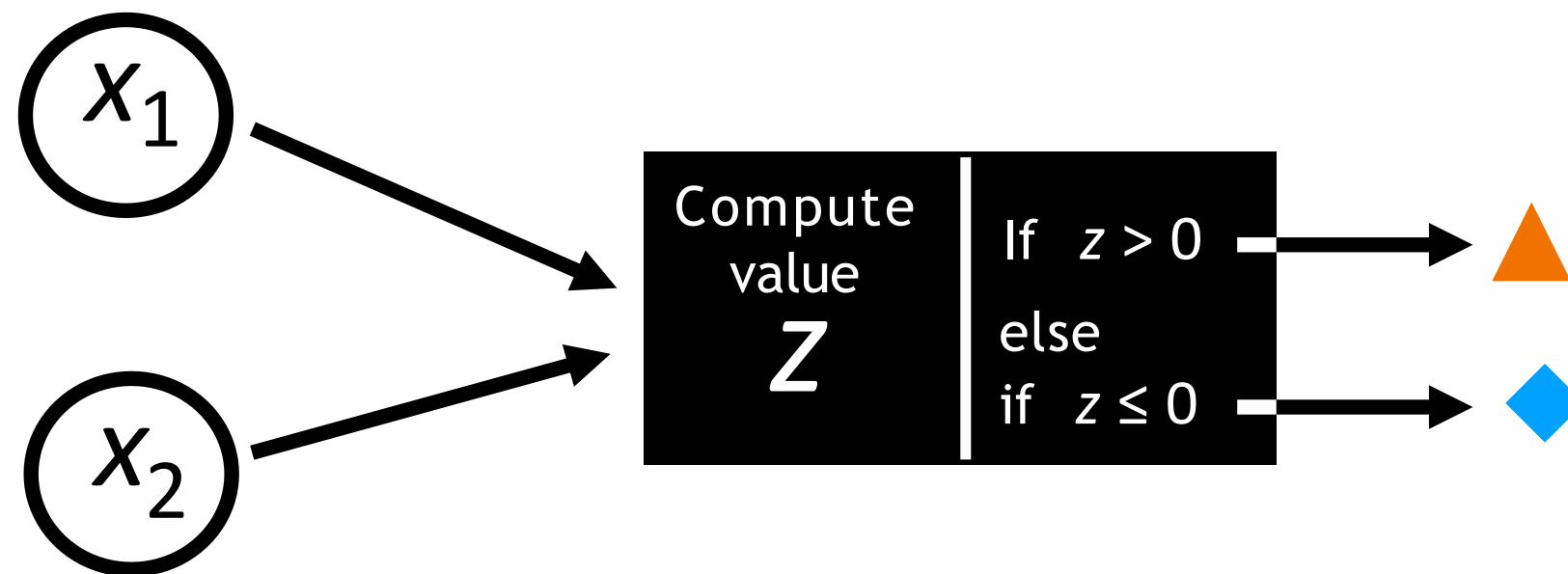
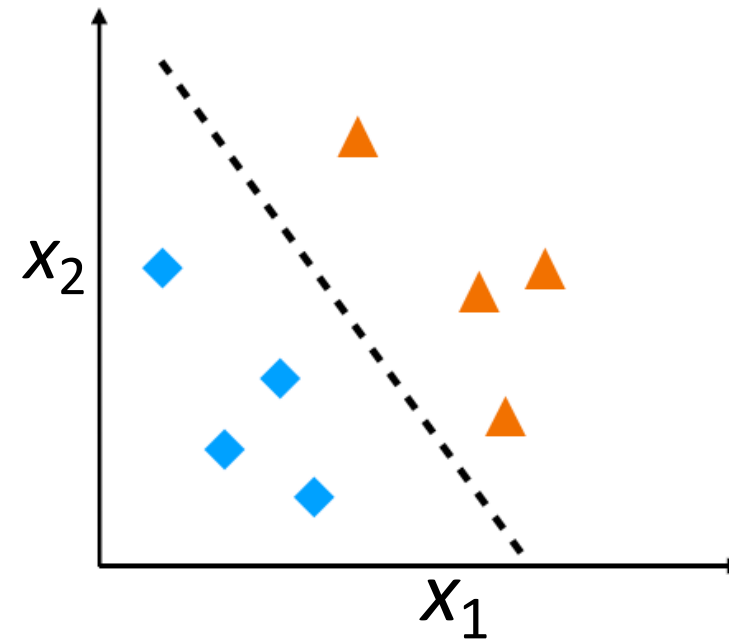




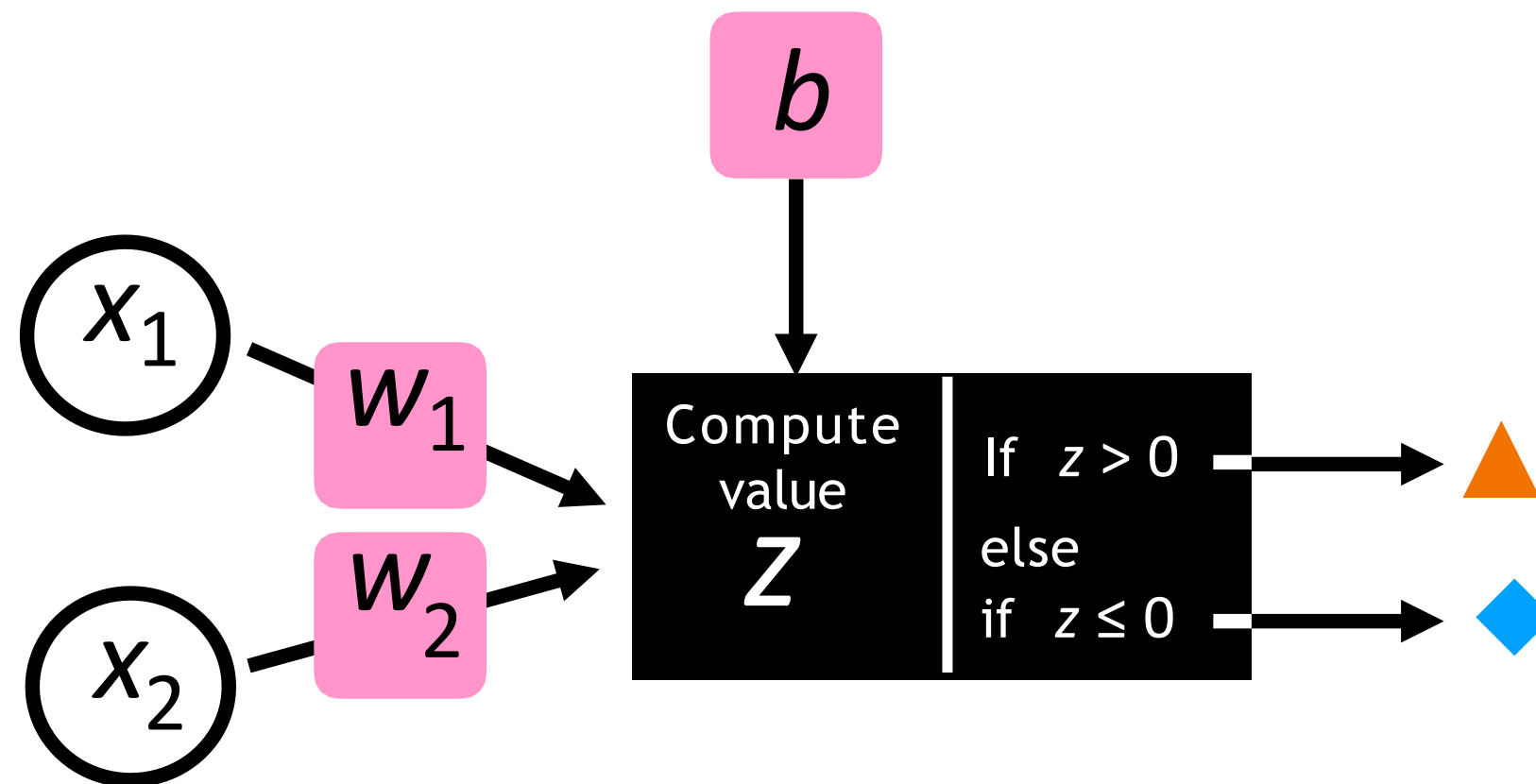






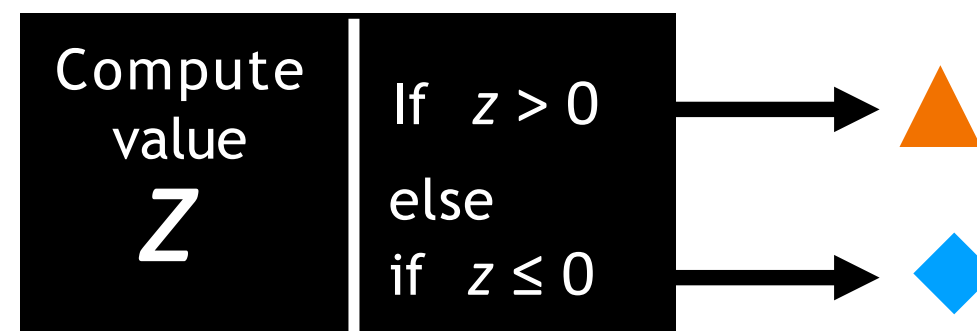


Model parameters to compute value



Model parameters to compute value

$$\textcircled{x_1} \times \boxed{w_1} + \textcircled{x_2} \times \boxed{w_2} + \boxed{b} = z$$



Model parameters to compute value

$$\textcircled{x_1} \times \boxed{w_1} + \textcircled{x_2} \times \boxed{w_2} + \boxed{b} = z$$

$$x_1 \times w_1 + x_2 \times w_2 + \dots + x_m \times w_m + b = z$$

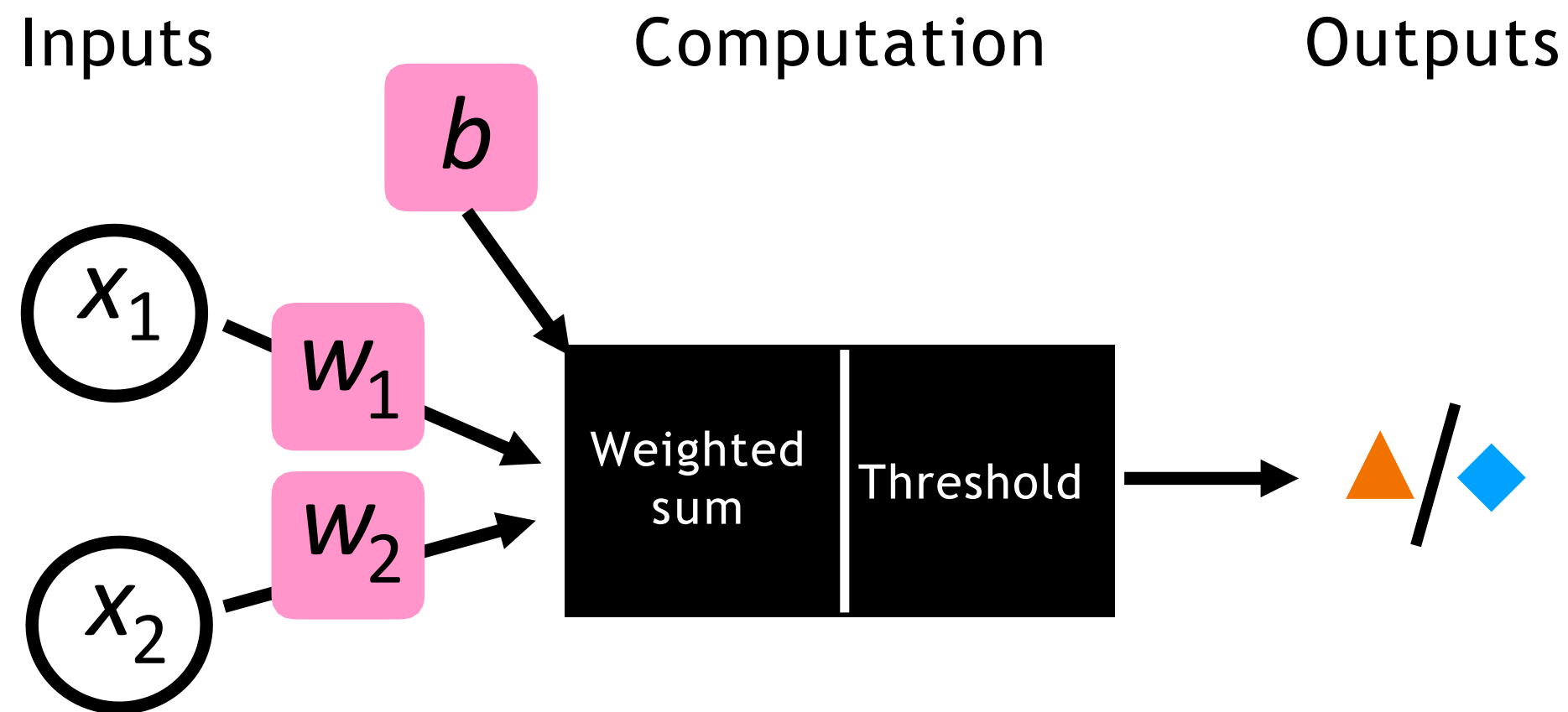
Model parameters to compute value

$$\textcircled{x_1} \times \boxed{w_1} + \textcircled{x_2} \times \boxed{w_2} + \boxed{b} = z$$

$$x_1 \times w_1 + x_2 \times w_2 + \dots + x_m \times w_m + b = z$$

$$= b + \sum_{i=1}^m x_i \times w_i$$

Perceptron Summary for Making Predictions



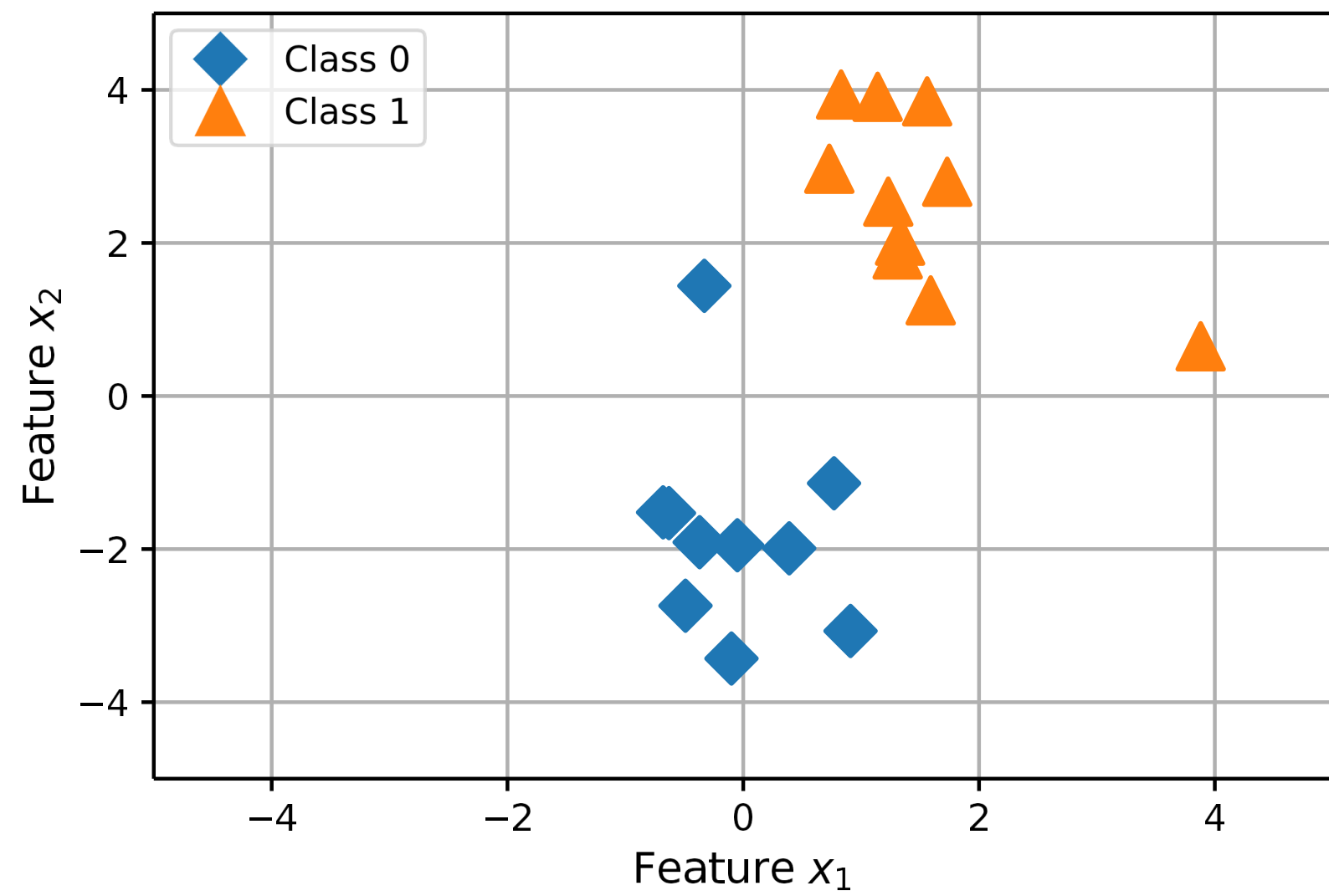
How does the perceptron **learn**
the model parameters?

w_1

w_2

b

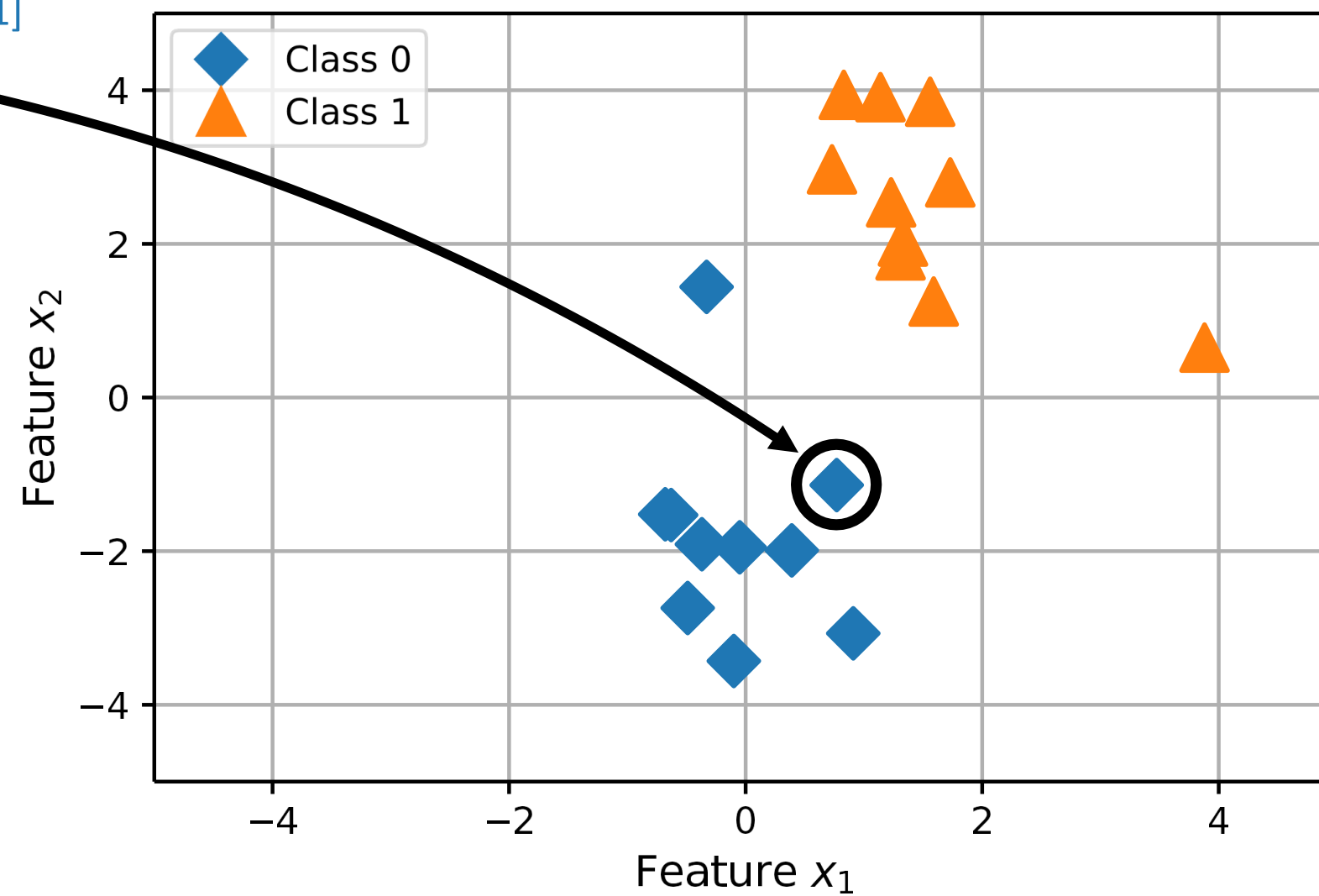
x_1	x_2	y
0.77	1.14	0
-0.33	1.44	0
0.91	-3.07	0
-0.37	-1.91	0
-0.63	-1.53	0
0.39	-1.99	0
...	...	...
1.33	2.03	1



x_1	x_2
0.77	-1.14
-0.33	1.44
0.91	-3.07
-0.37	-1.91
-0.63	-1.53
0.39	-1.99
...	...
1.33	2.03

y
0
0
0
0
0
0
...
1

Training
example
 $x^{[1]}$

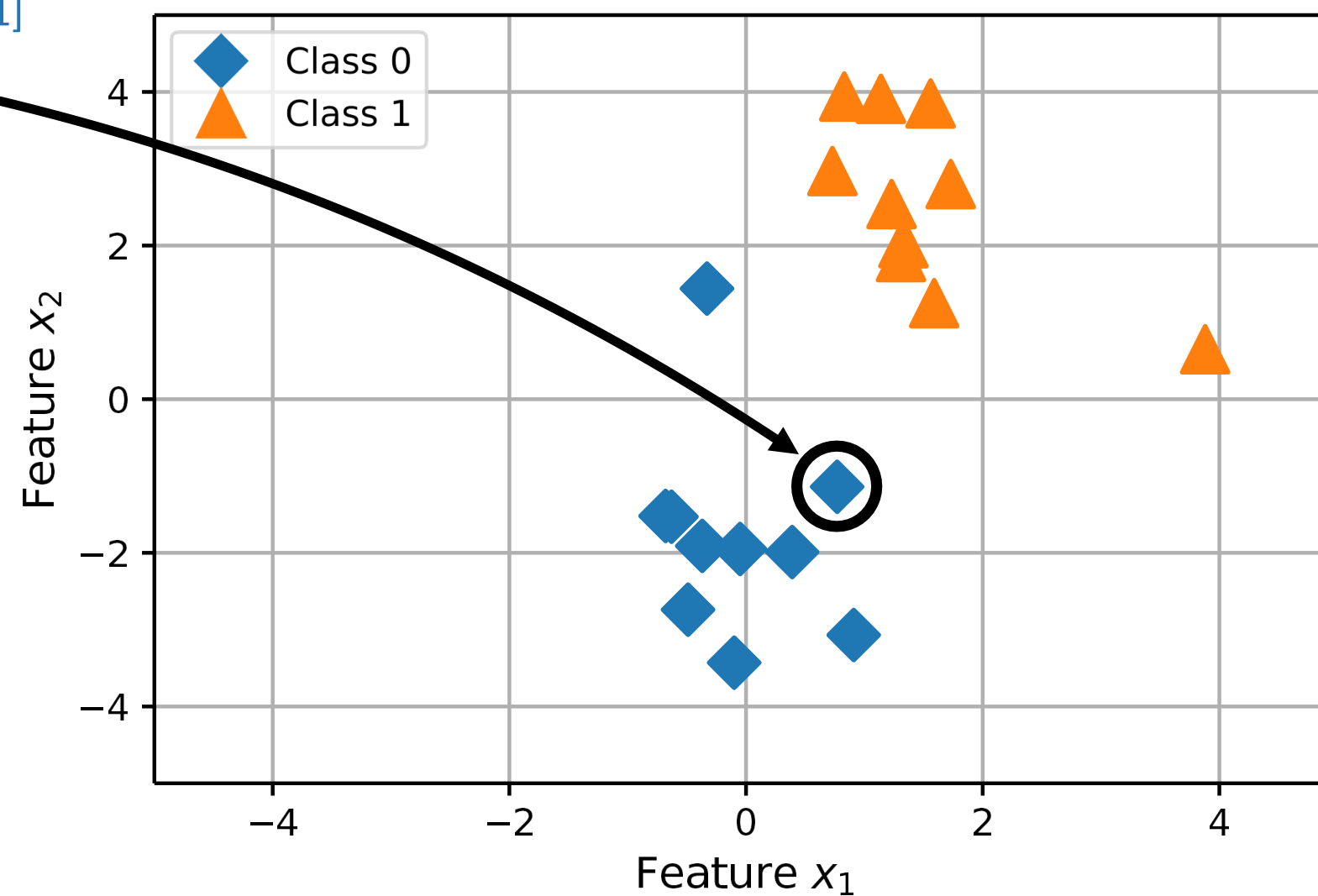


x_1	x_2
0.77	-1.14
-0.33	1.44
0.91	-3.07
-0.37	-1.91
-0.63	-1.53
0.39	-1.99
...	...
1.33	2.03

y
0
0
0
0
0
0
...
1

Training
example
 $x^{[1]}$

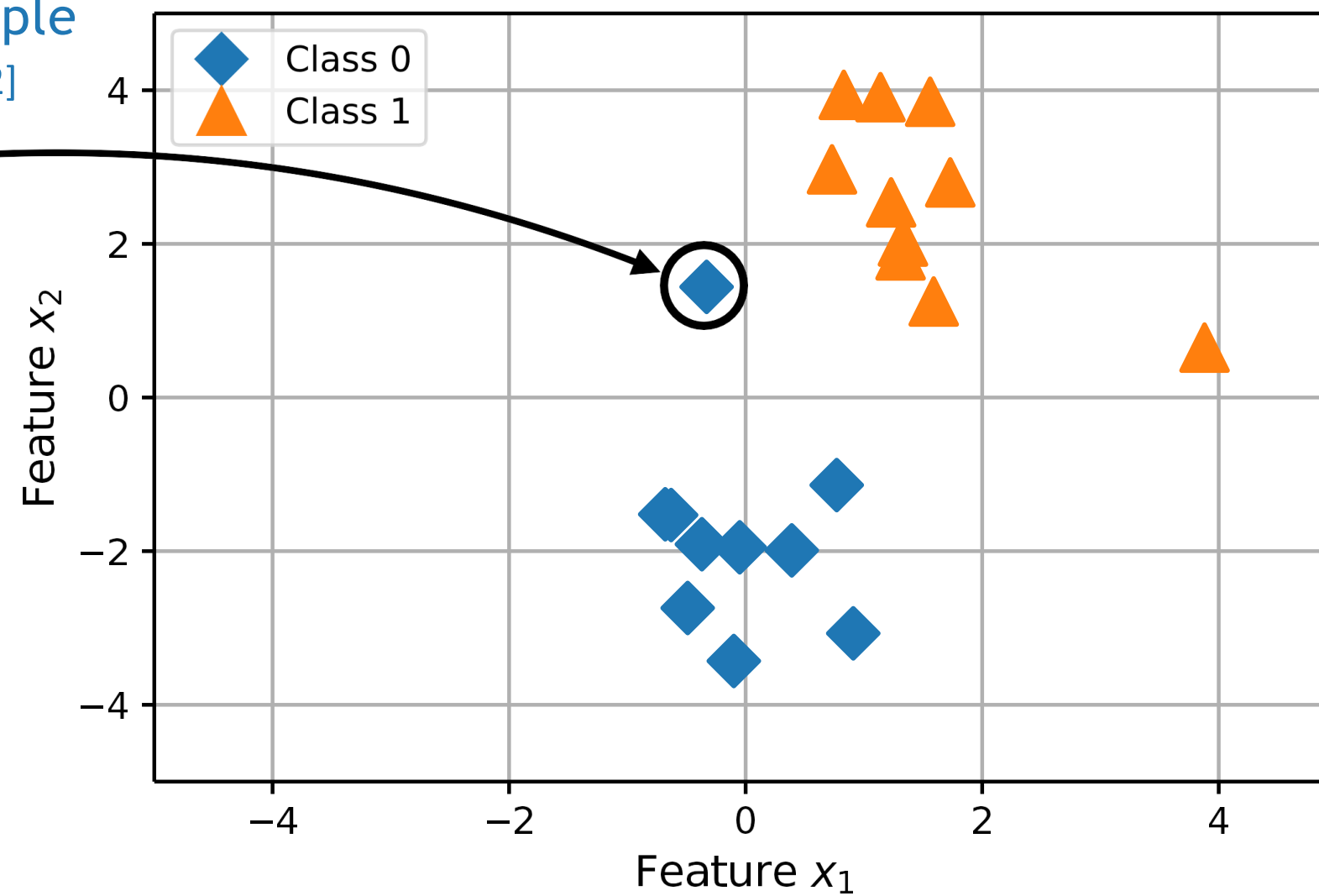
$$x^{[1]} = [x_1^{[1]}, x_2^{[1]}]$$



x_1	x_2
0.77	-1.14
-0.33	1.44
0.91	-3.07
-0.37	-1.91
-0.63	-1.53
0.39	-1.99
...	...
1.33	2.03

y
0
0
0
0
0
0
...
1

Training
example
 $x^{[2]}$

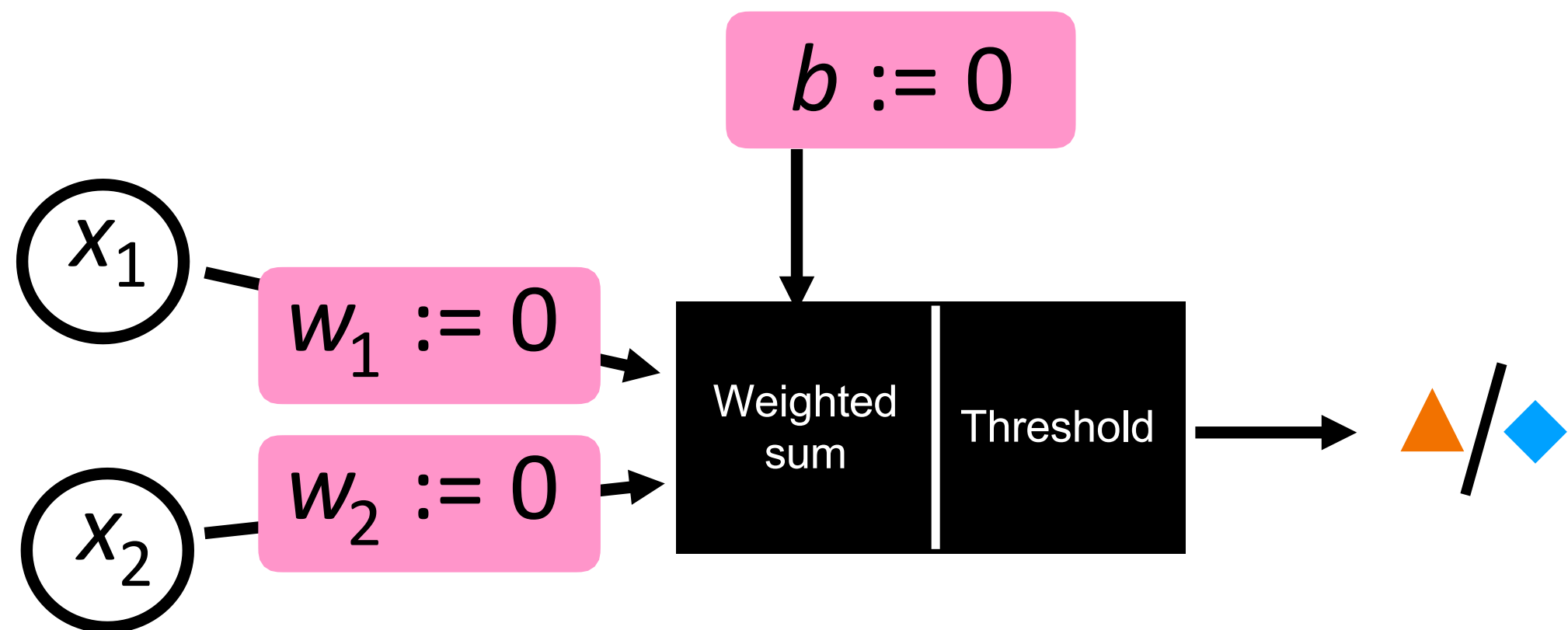


x_1	x_2	y
0.77	-1.14	0
-0.33	1.44	0
0.91	-3.07	0
-0.37	-1.91	0
-0.63	-1.53	0
0.39	-1.99	0
...	...	...
1.33	2.03	1



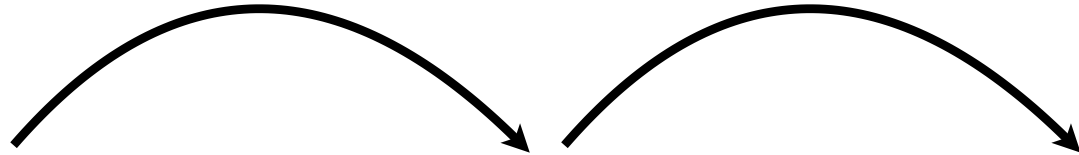
How does the perceptron **learn** the model parameters?

1. Define training set
2. Initialize model weights and bias to zero



How does the perceptron **learn** the model parameters?

1. Define training set
2. Initialize model weights and bias to zero
3. For every training epoch:


$$D = (h\mathbf{x}^{[1]}, y^{[1]}_i, h\mathbf{x}^{[2]}, y^{[2]}_i, \dots, h\mathbf{x}^{[n]}, y^{[n]}_i)$$

How does the perceptron **learn** the model parameters?

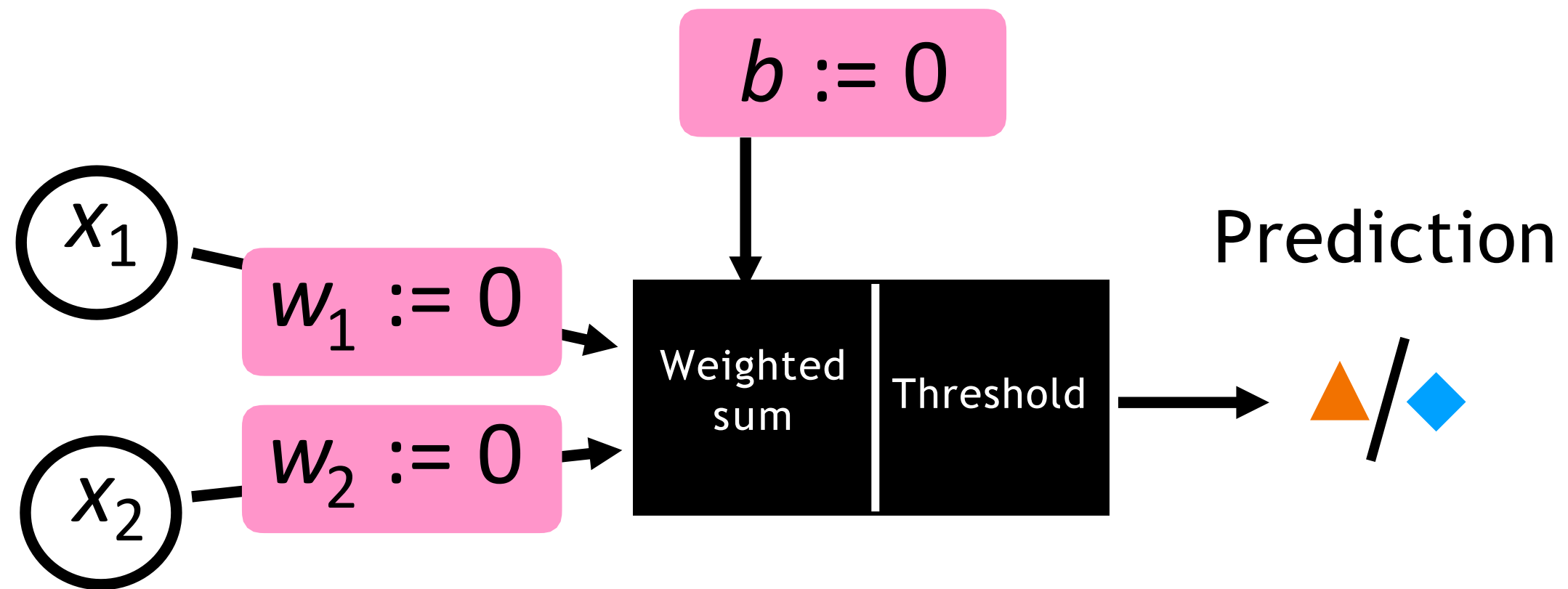
1. Define training set (where 'x' is the input and 'y' is the true label)
2. Initialize model weights and bias to zero
3. For every training epoch:
 - a. For every training example $\langle x^{[i]}, y^{[i]} \rangle \in D$:
 - (i) Make a prediction (predicted label = $\hat{y}$)
 - (ii) Compute the error

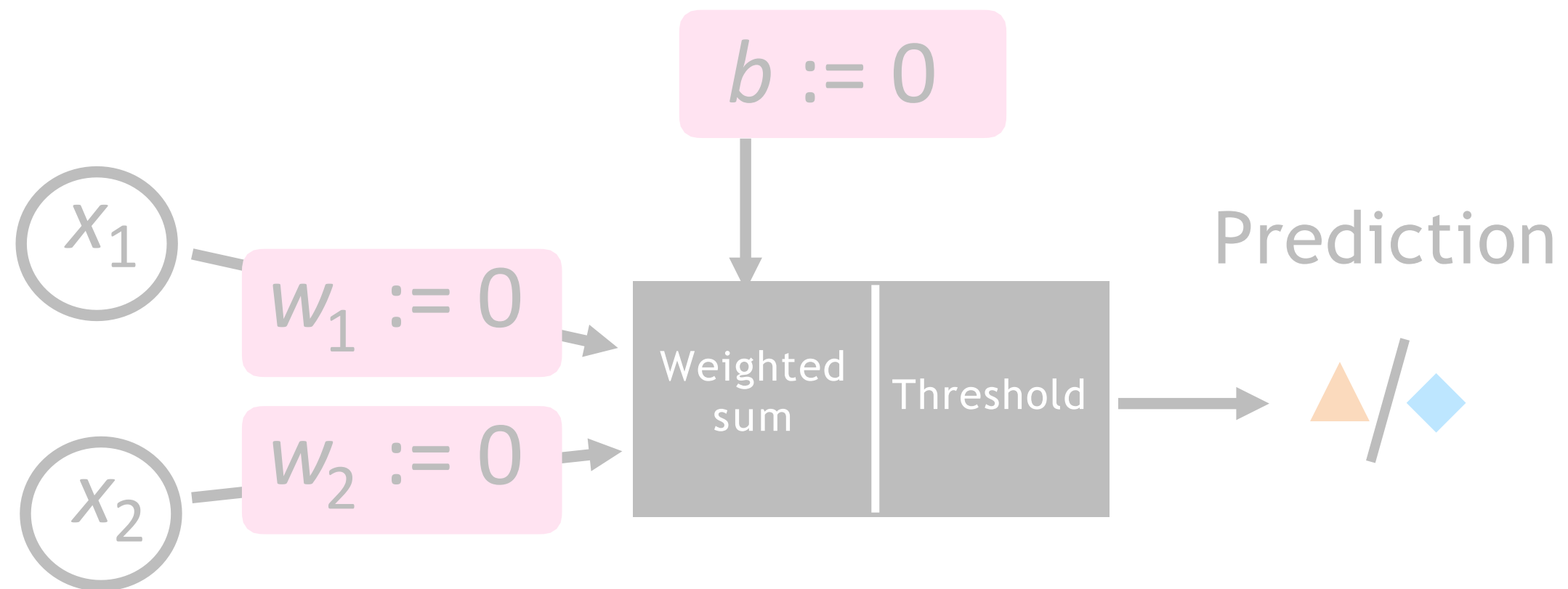
$$error = y - \hat{y}$$

- (iii) Update the weights and bias based on the error

$$\text{Updated weight } (w) = \text{previous weight}(w) + \text{input } (x) * \text{error}$$

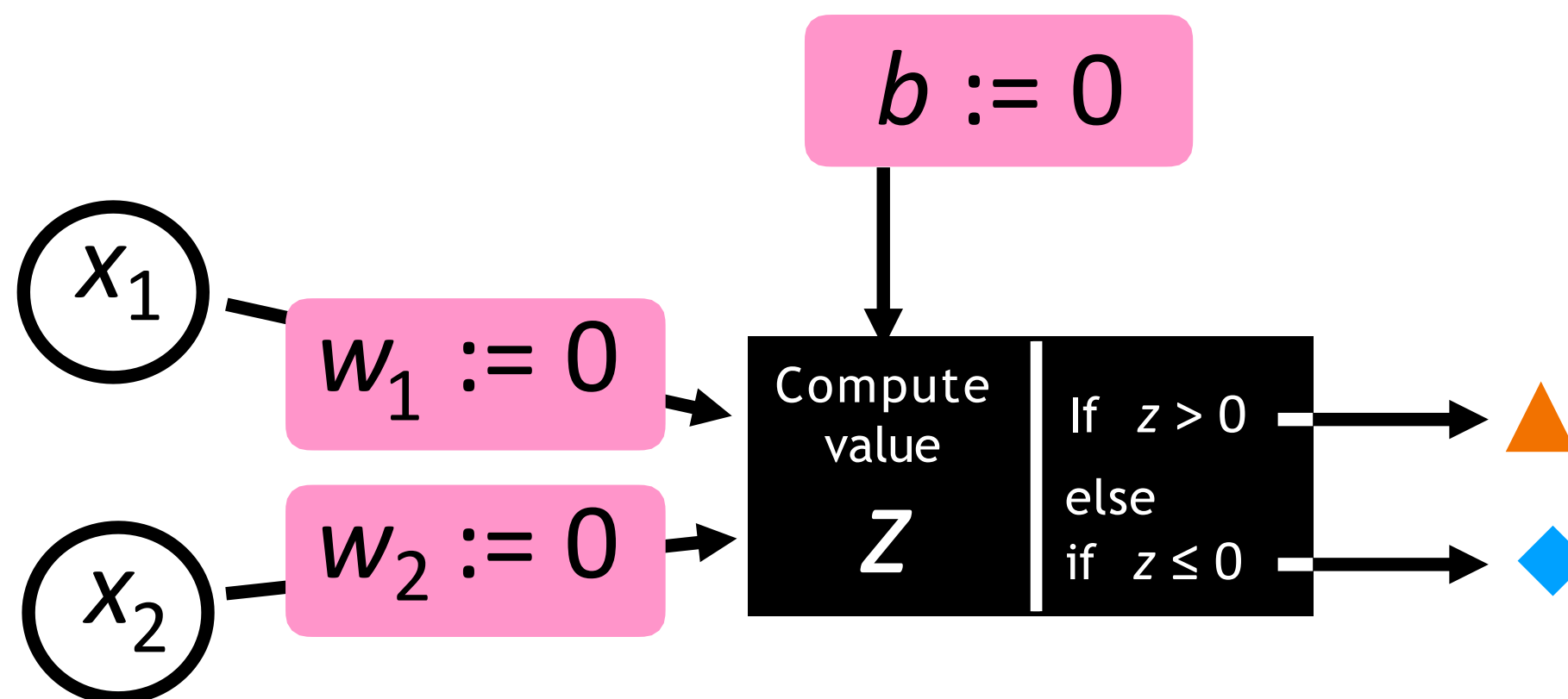
$$\text{Updated bias } (b) = \text{previous bias } (b) + \text{error}$$



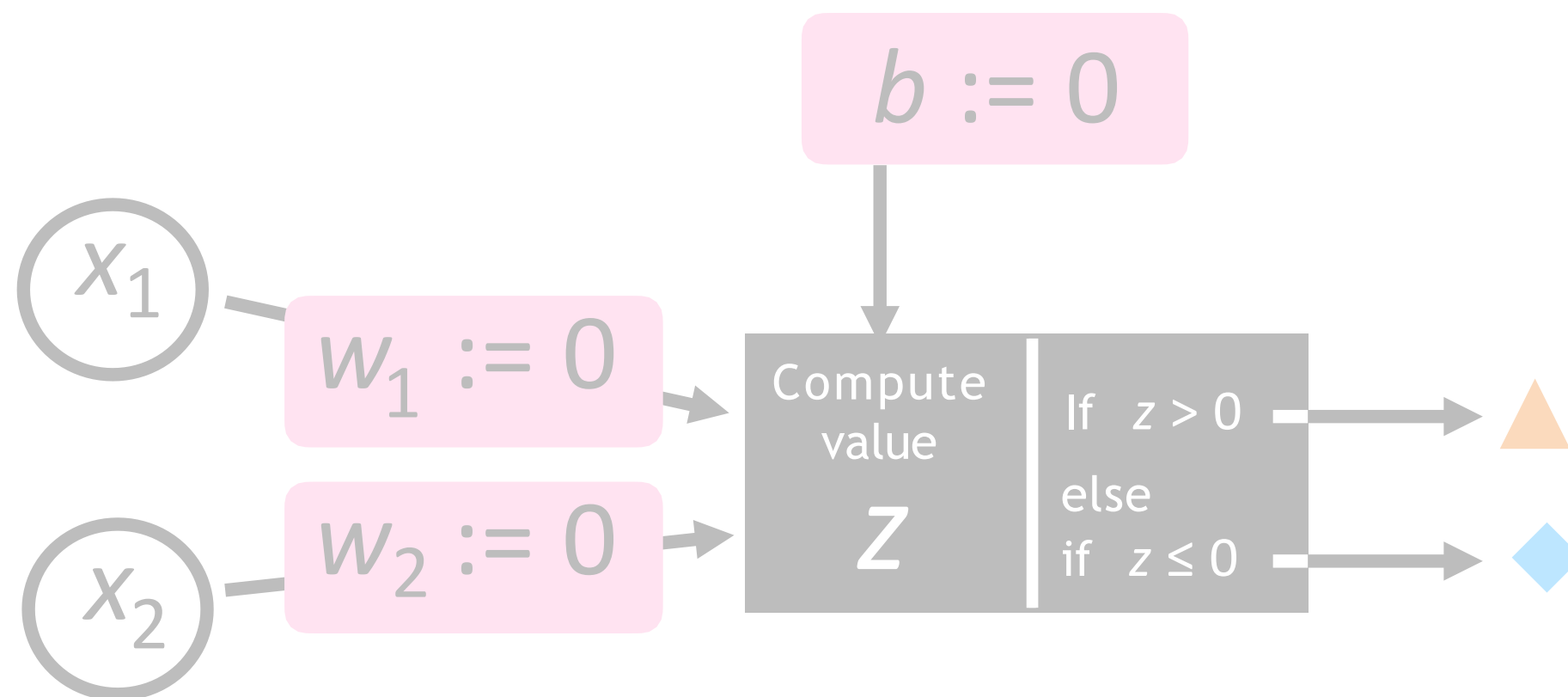


$$\textcircled{x_1} \times w_1 + \textcircled{x_2} \times w_2 + b = z$$

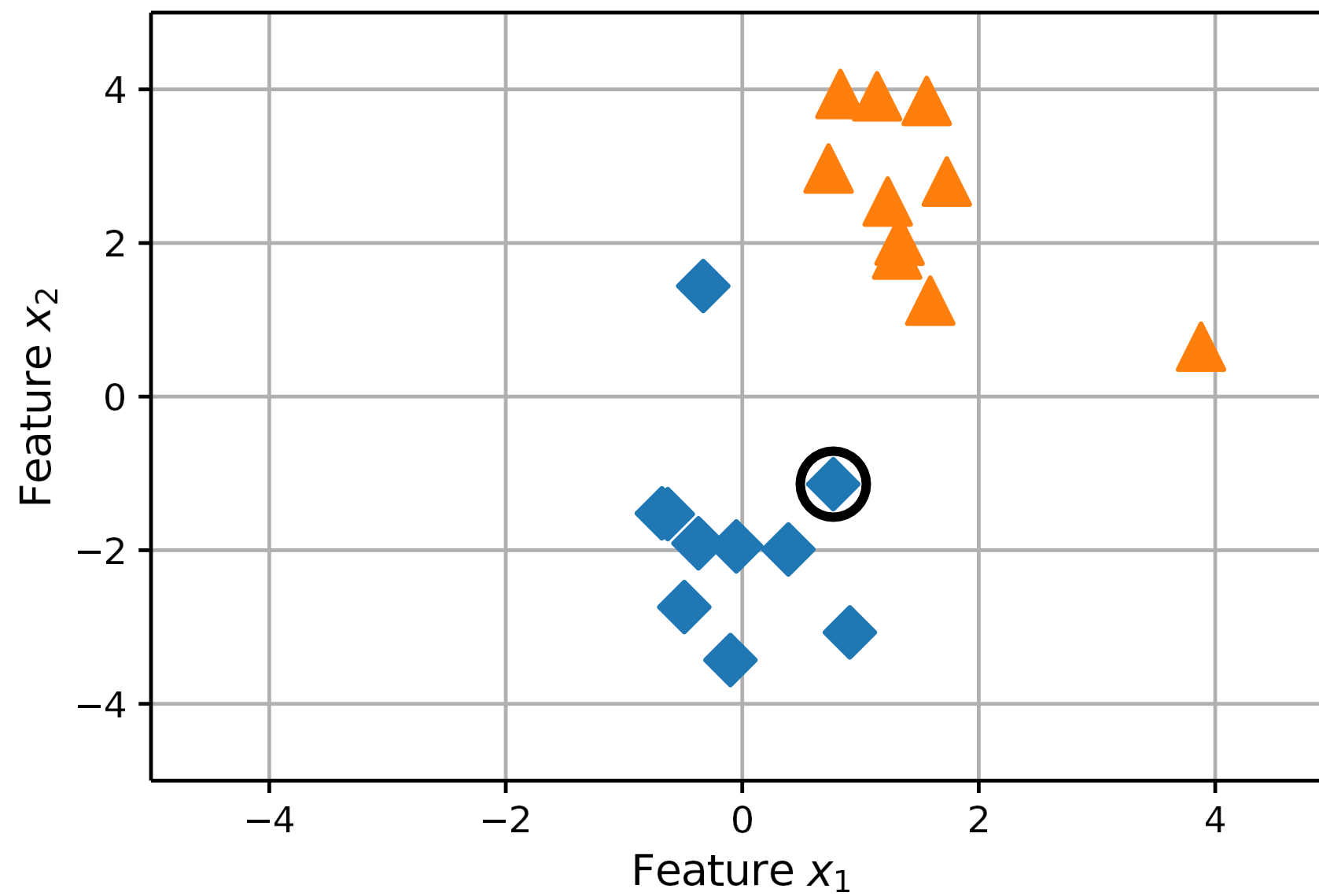
$$(x_1) \times w_1 + (x_2) \times w_2 + b = z$$



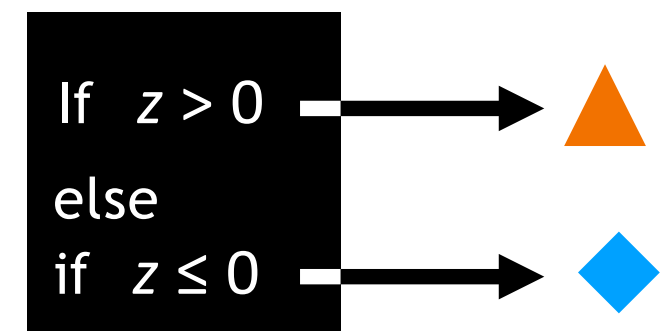
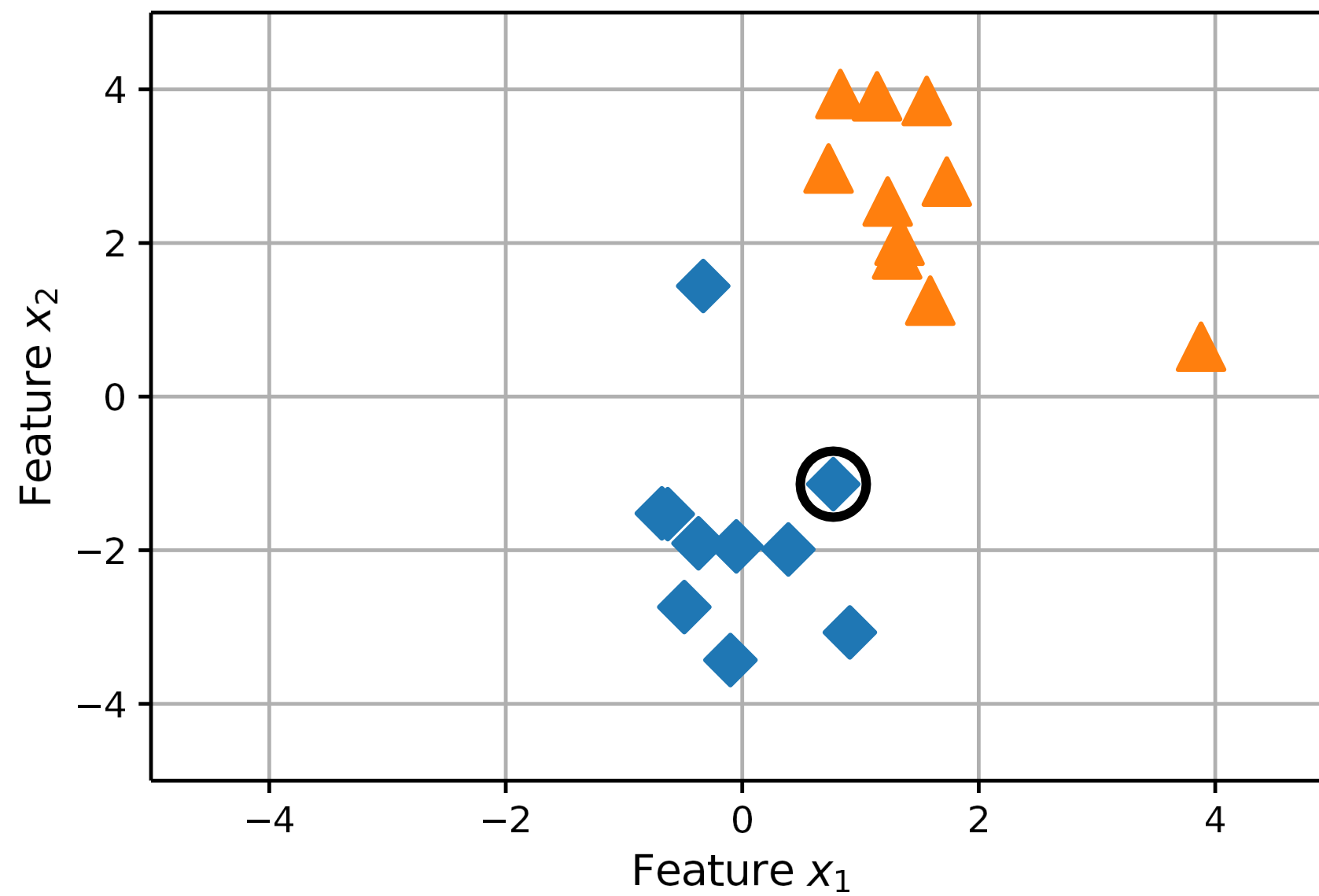
$$\textcircled{x_1} \times \boxed{0} + \textcircled{x_2} \times \boxed{0} + \boxed{0} = 0$$



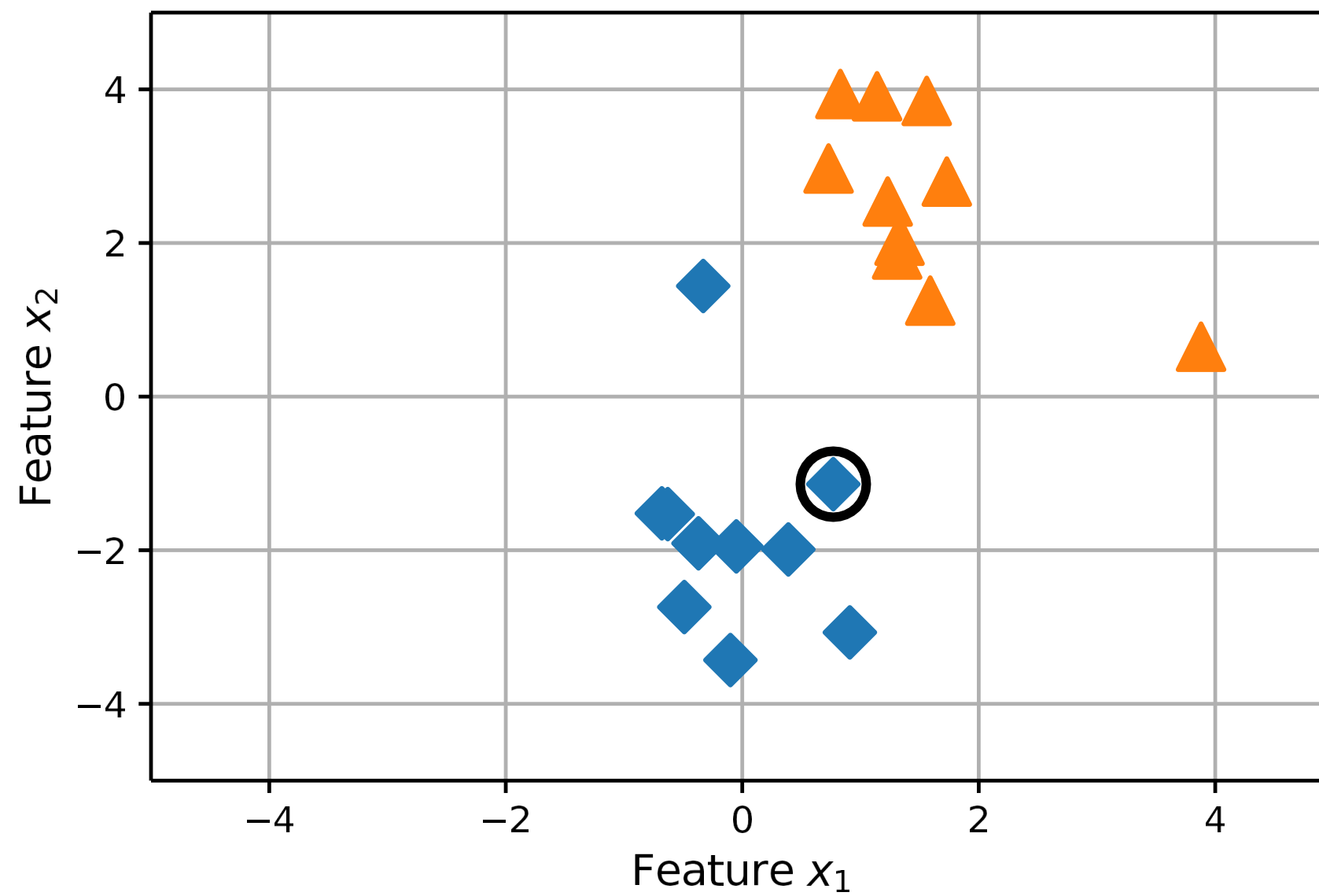
Epoch 1, Iteration 1



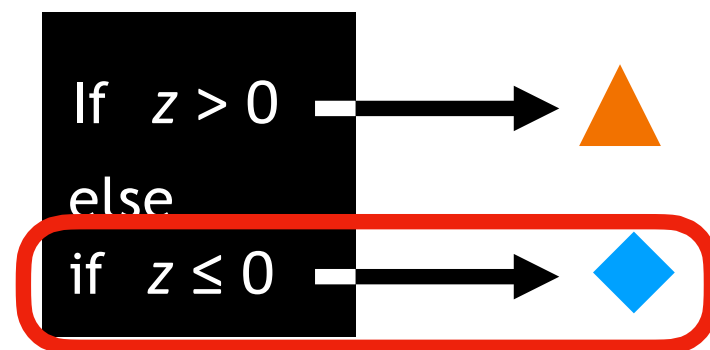
Epoch 1, Iteration 1



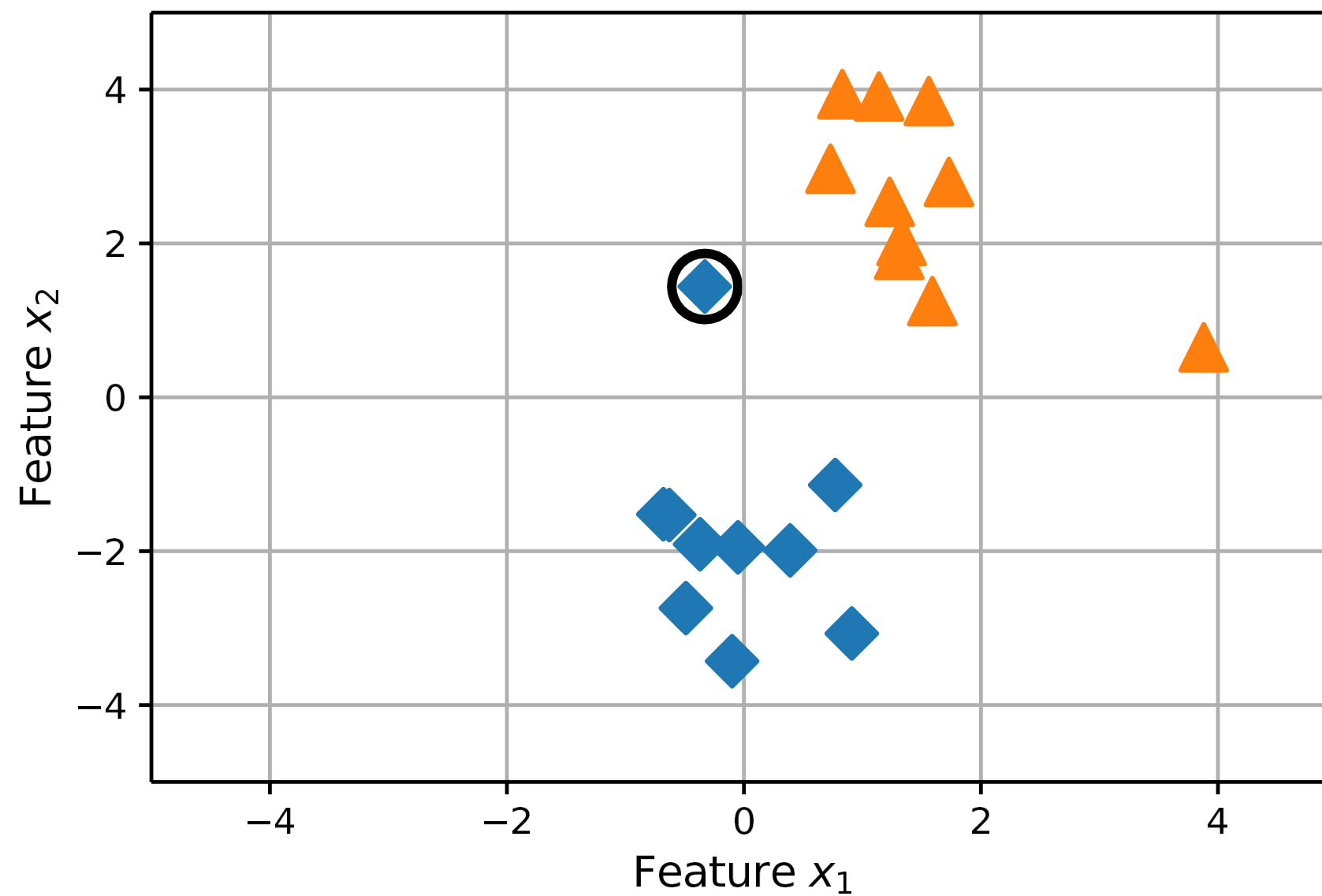
Epoch 1, Iteration 1



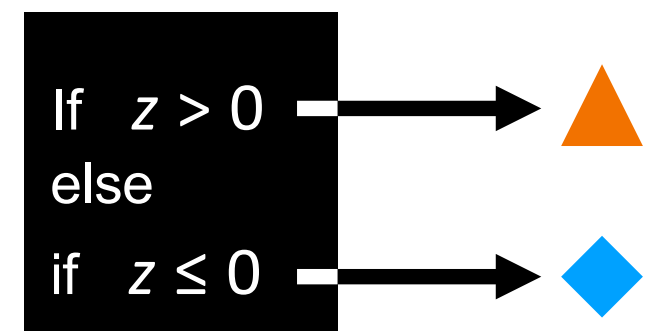
$z = 0$



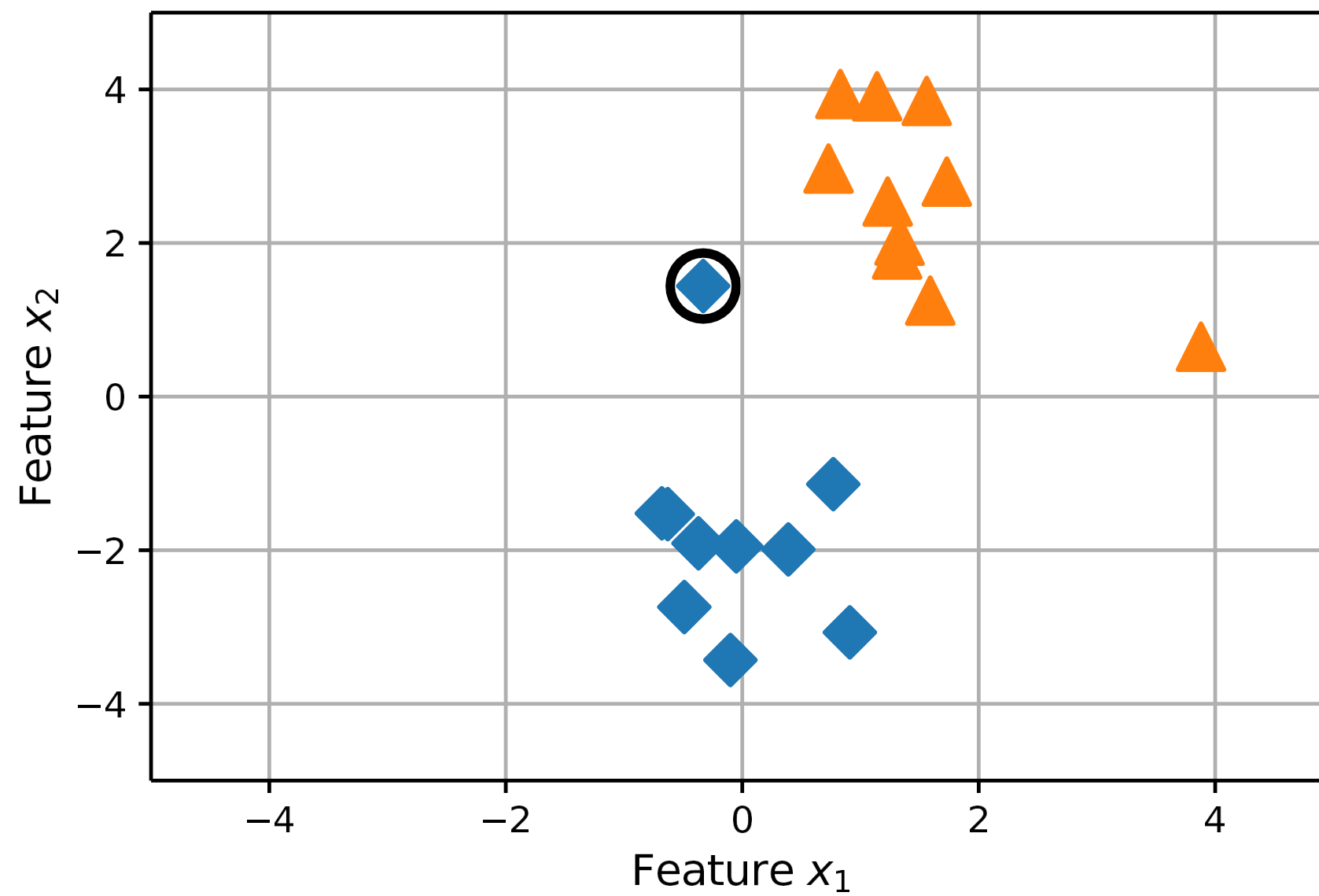
Epoch 1, Iteration 2



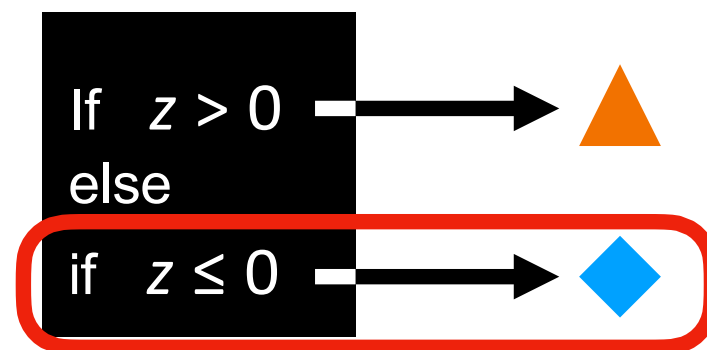
$$z = 0$$



Epoch 1, Iteration 2

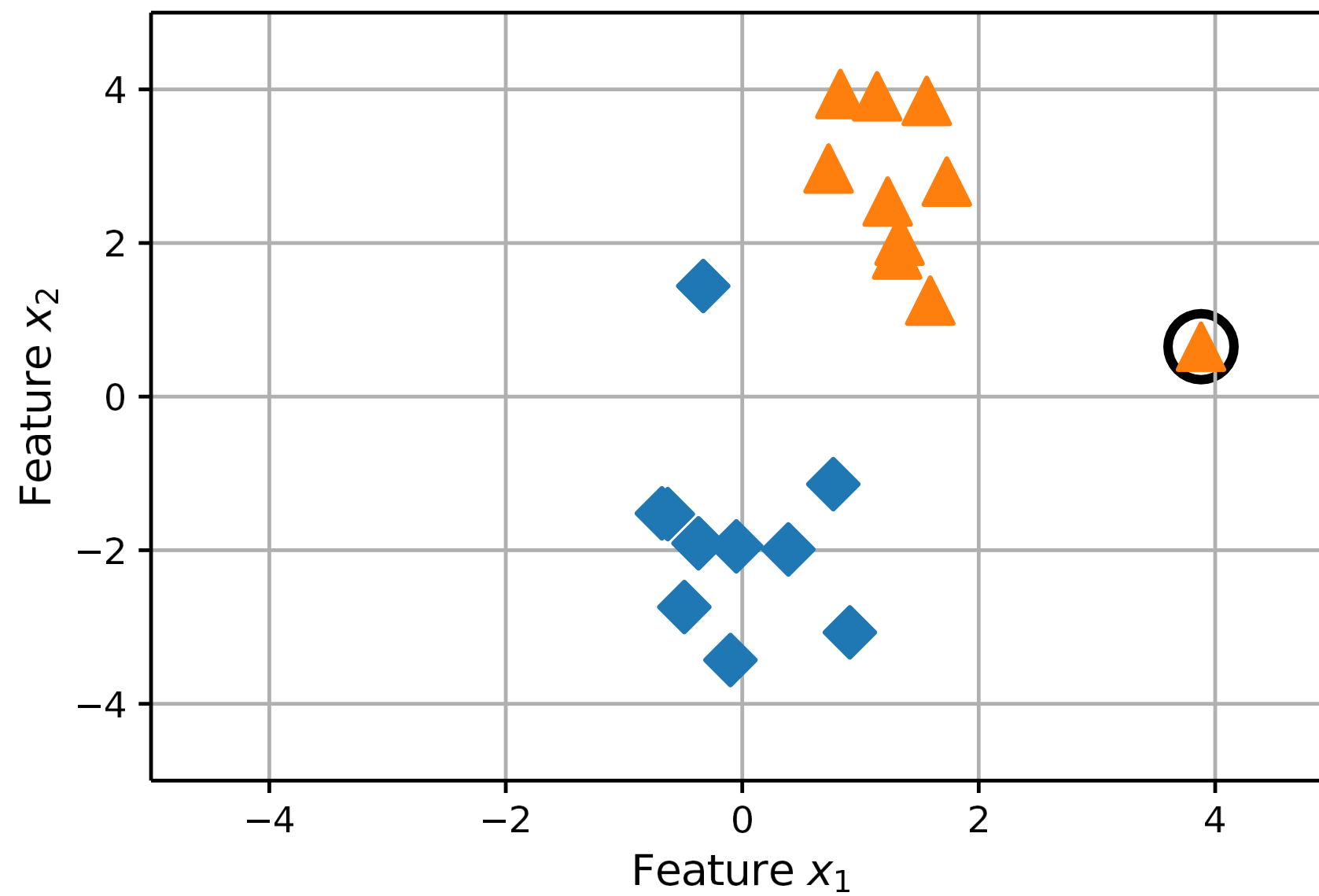


$$z = 0$$

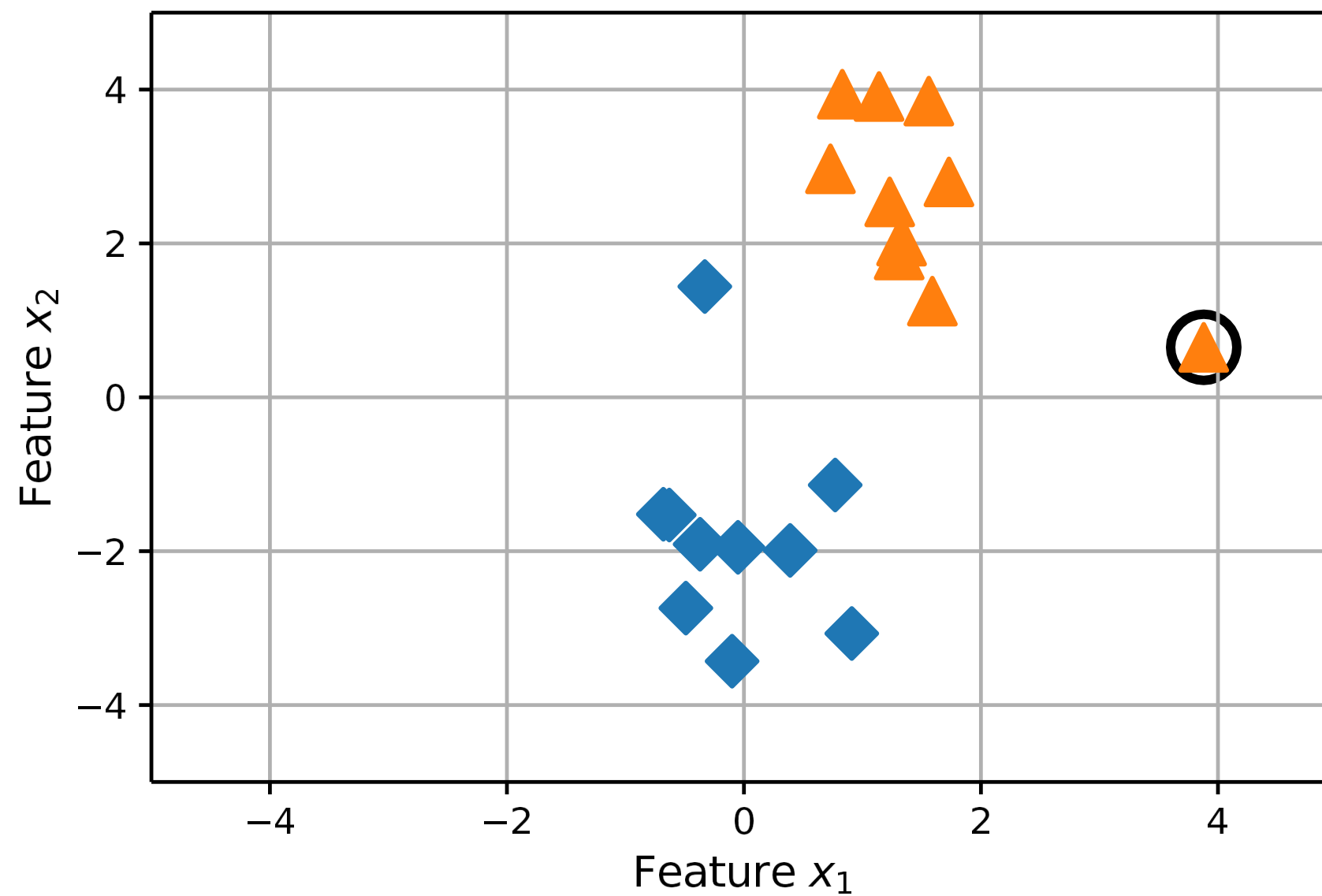


Pro tip: shuffle the dataset

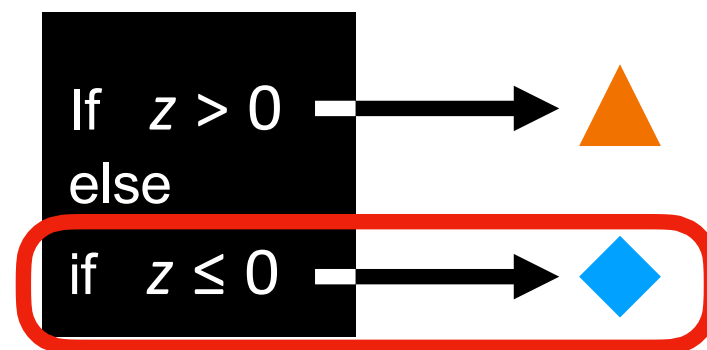
Epoch 1, Iteration 11



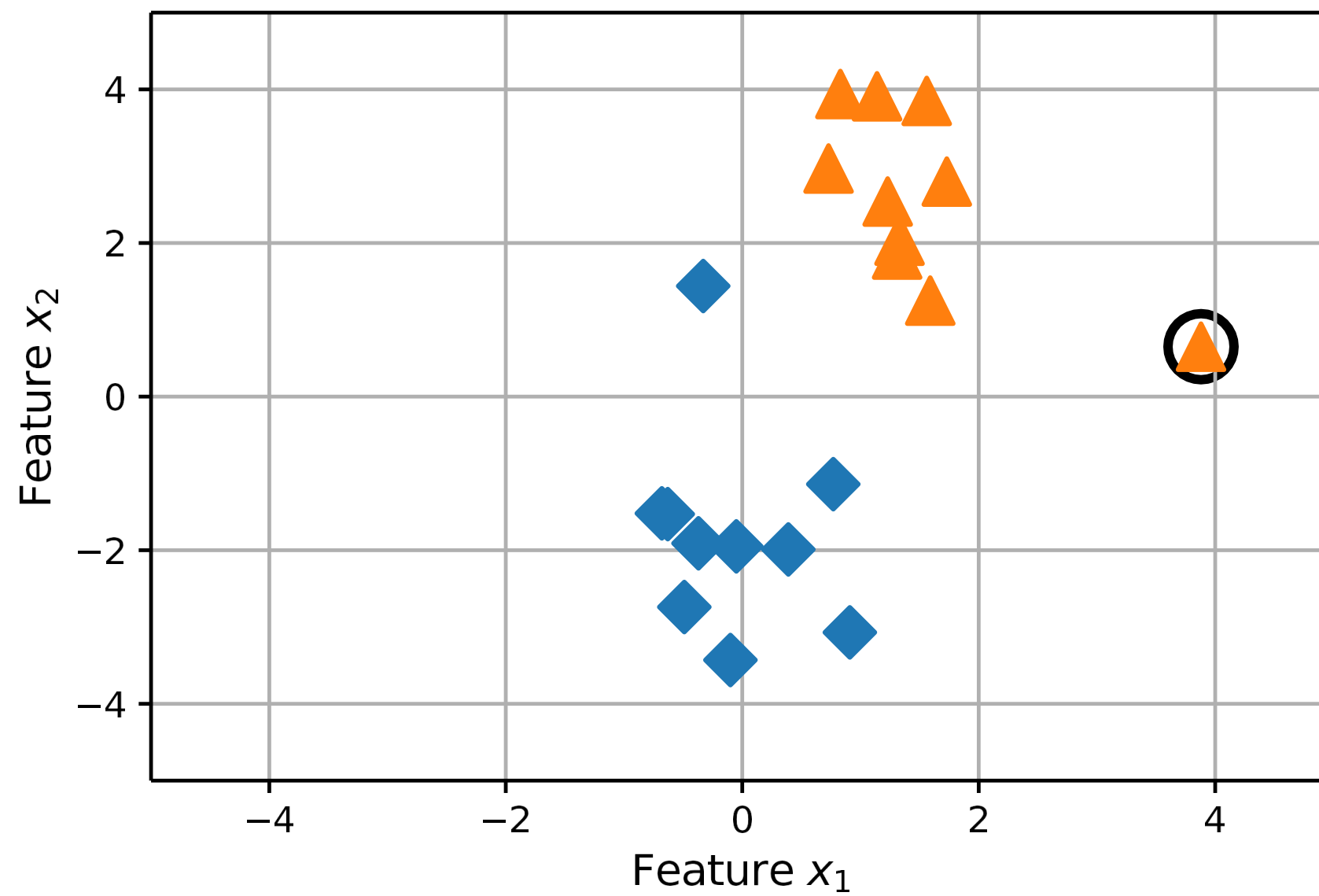
Epoch 1, Iteration 11



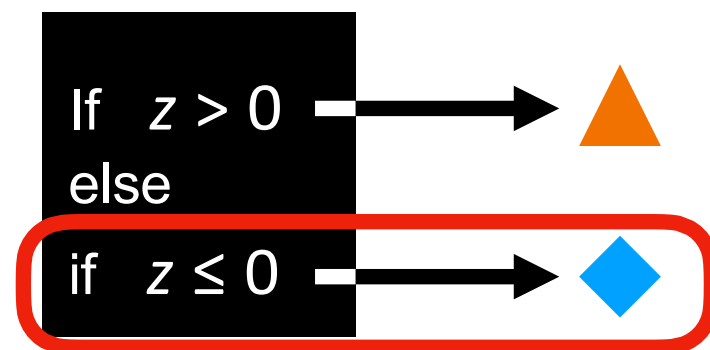
$$z = 0$$

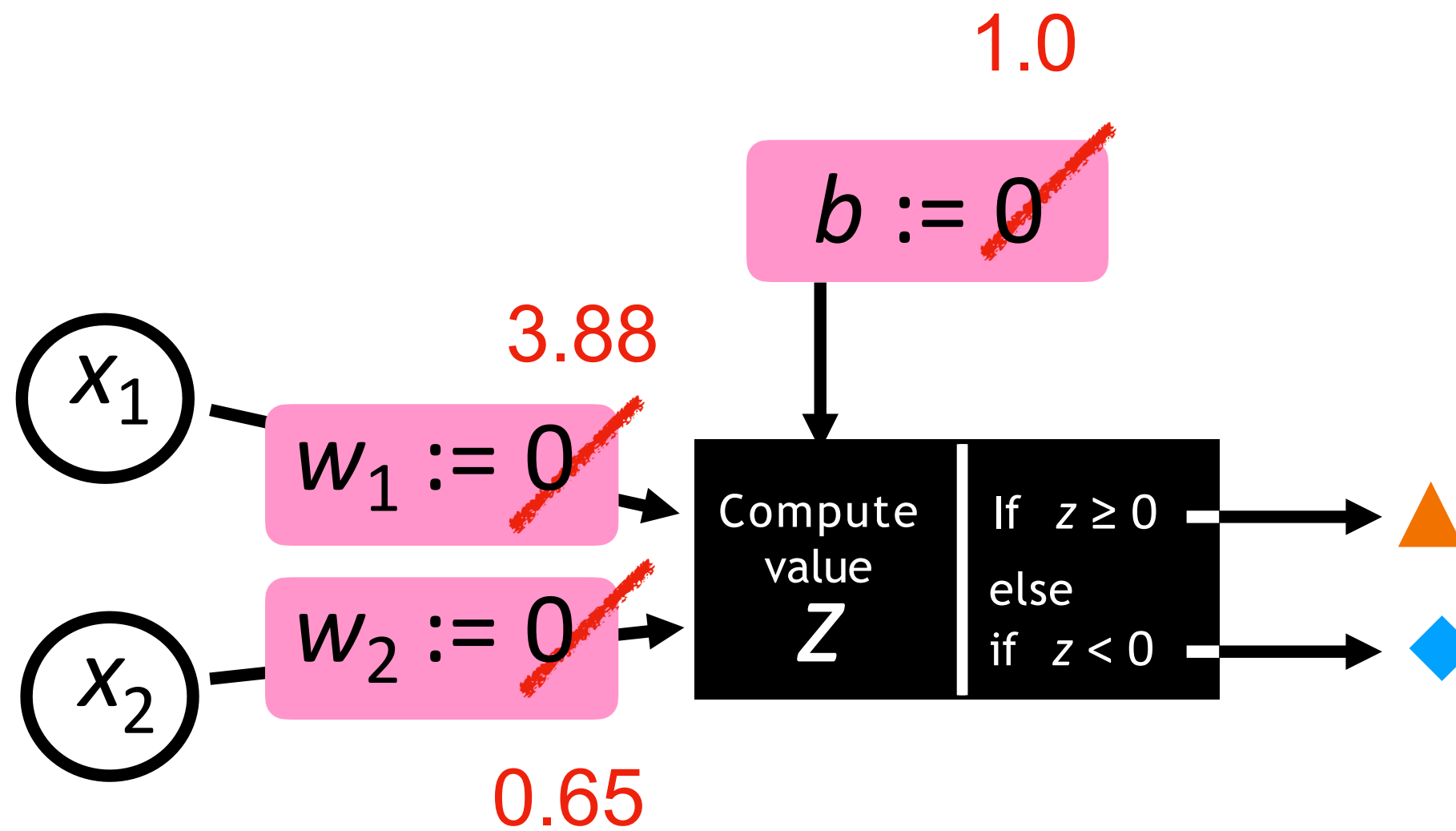


Epoch 1, Iteration 11

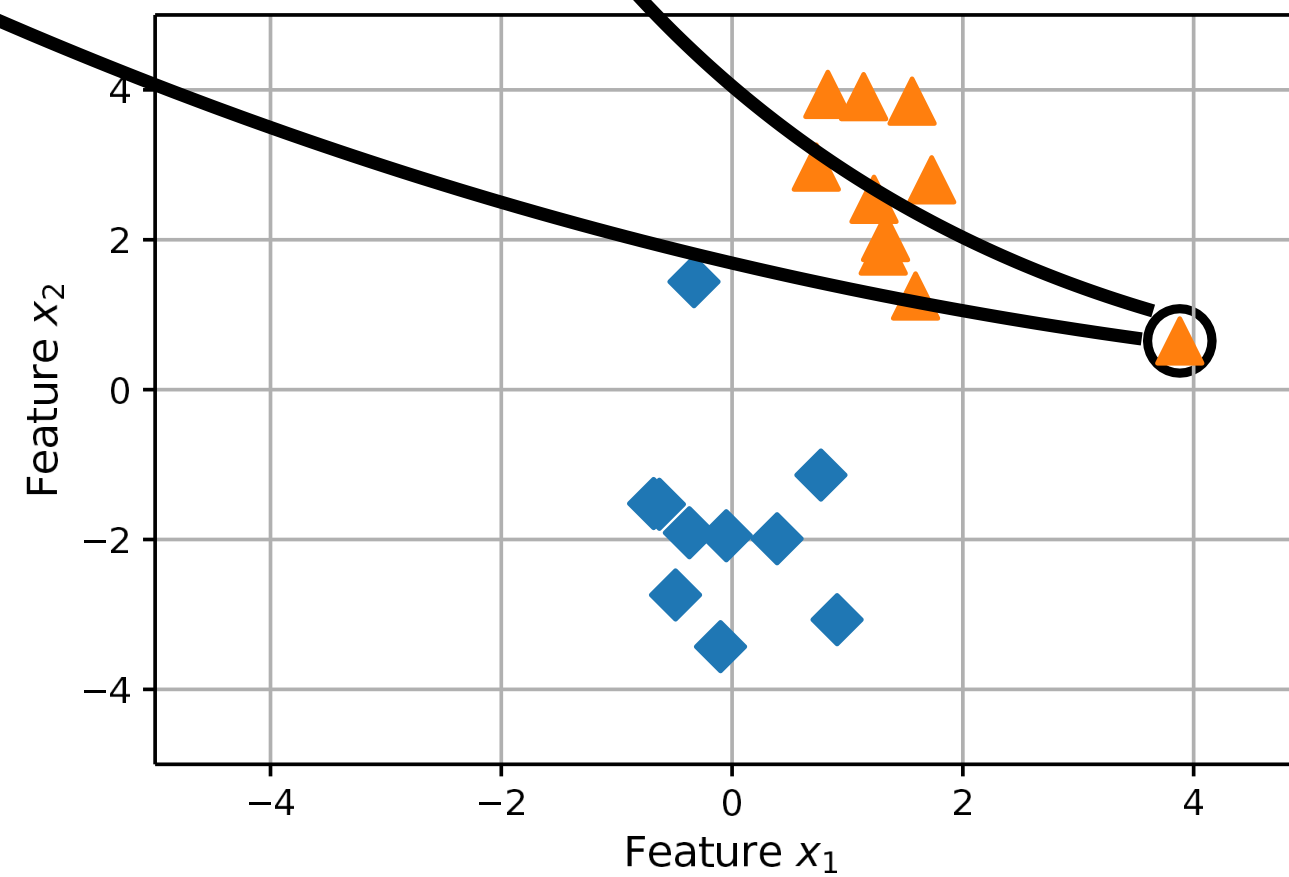


$$z = 0$$

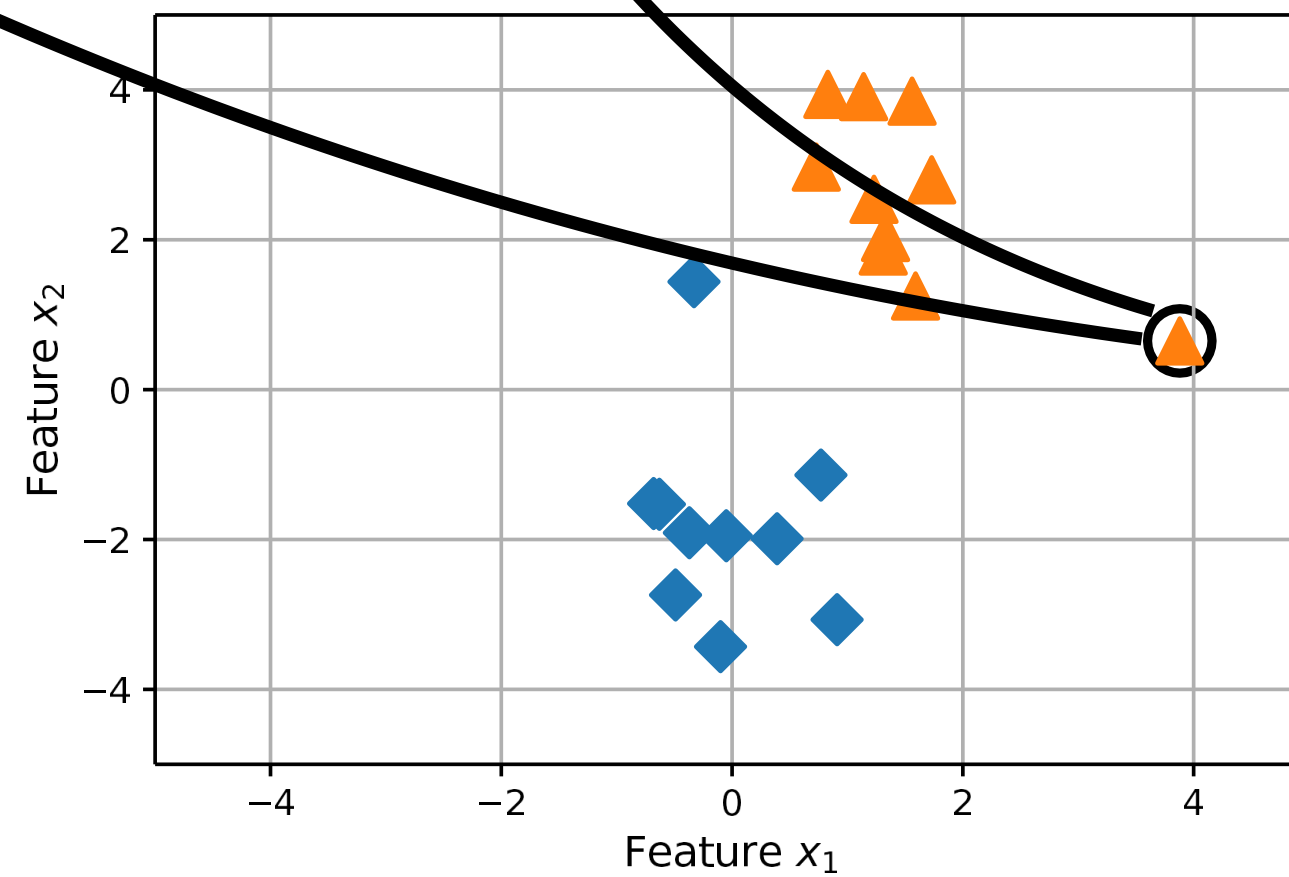




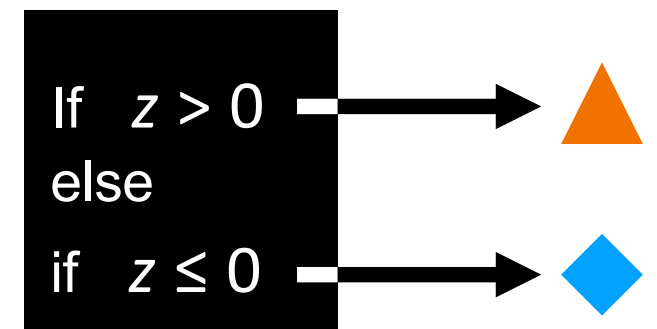
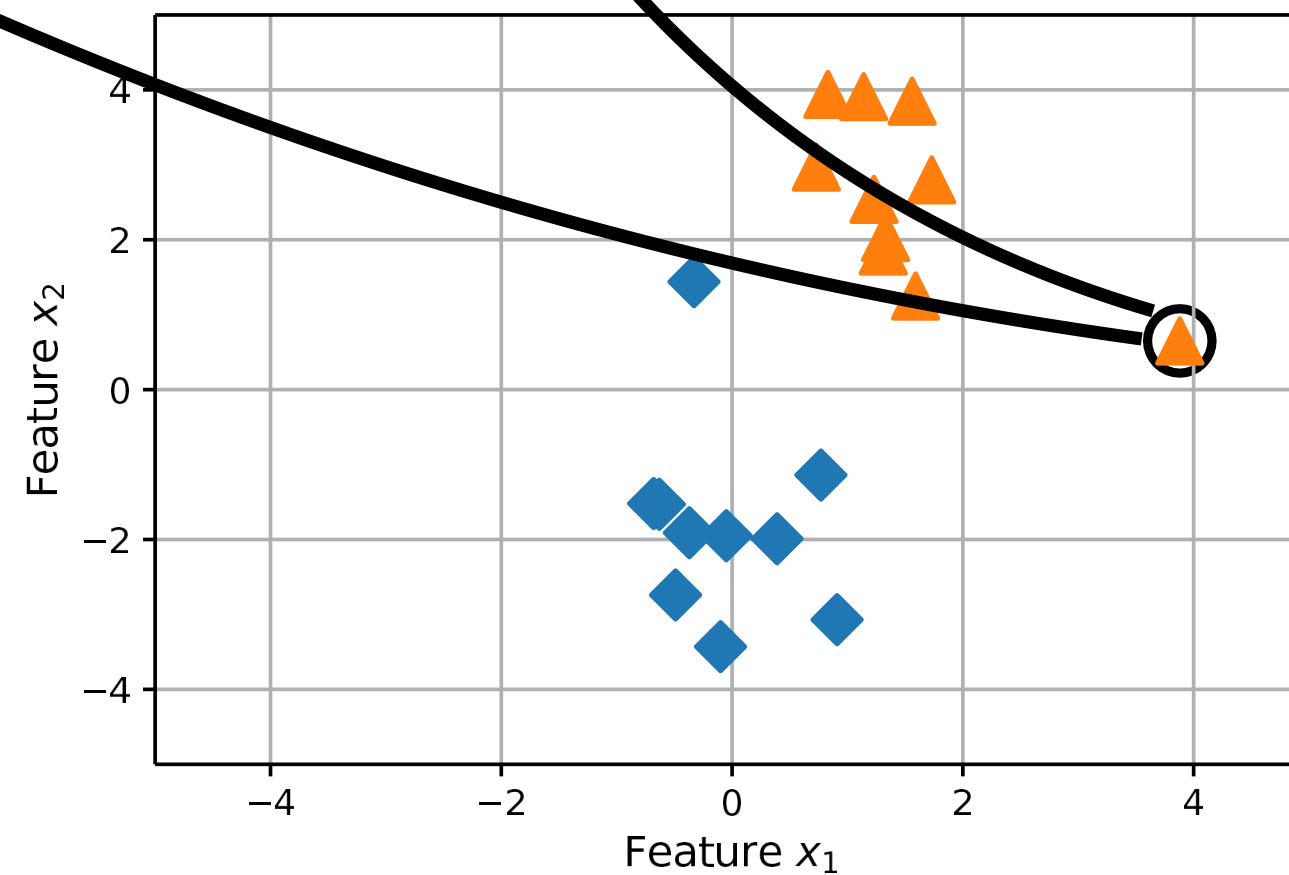
$$x_1 \times 3.88 + x_2 \times 0.65 + 1.0 = z$$



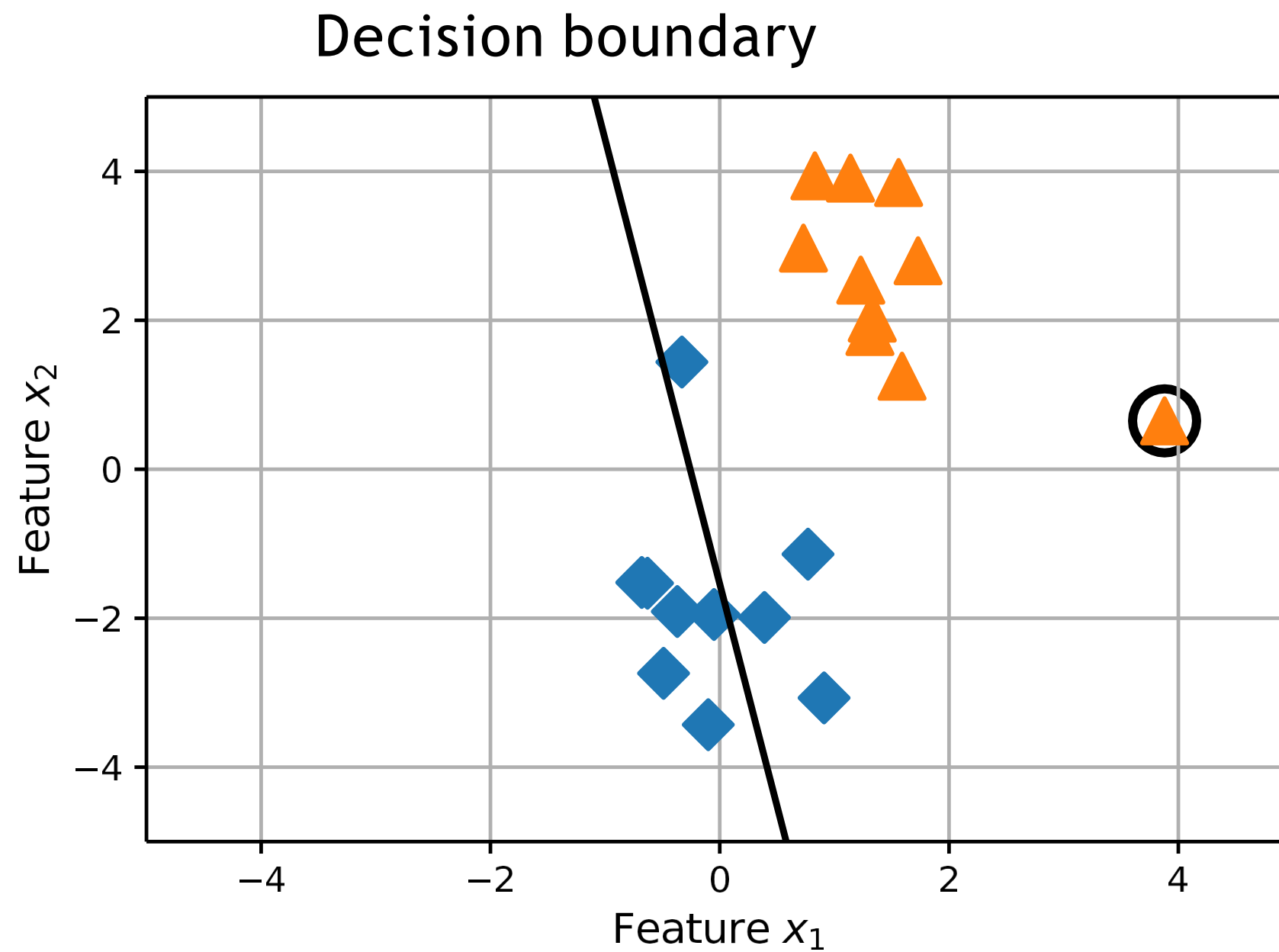
$$\textcircled{3.88} \times \text{3.88} + \textcircled{0.65} \times \text{0.65} + \text{1.0} = z$$



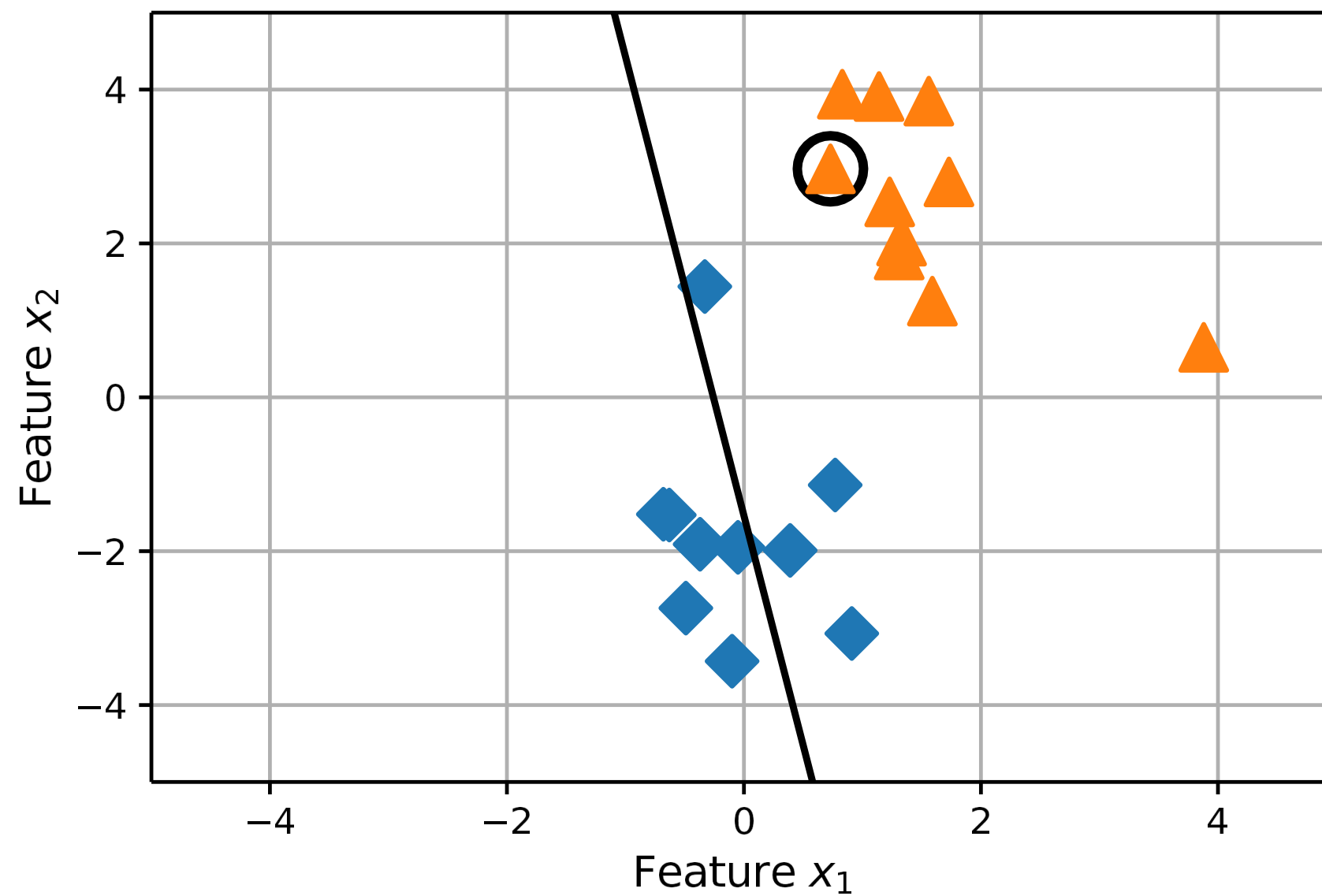
$$(3.88) \times 3.88 + (0.65) \times 0.65 + 1.0 = 15.48$$



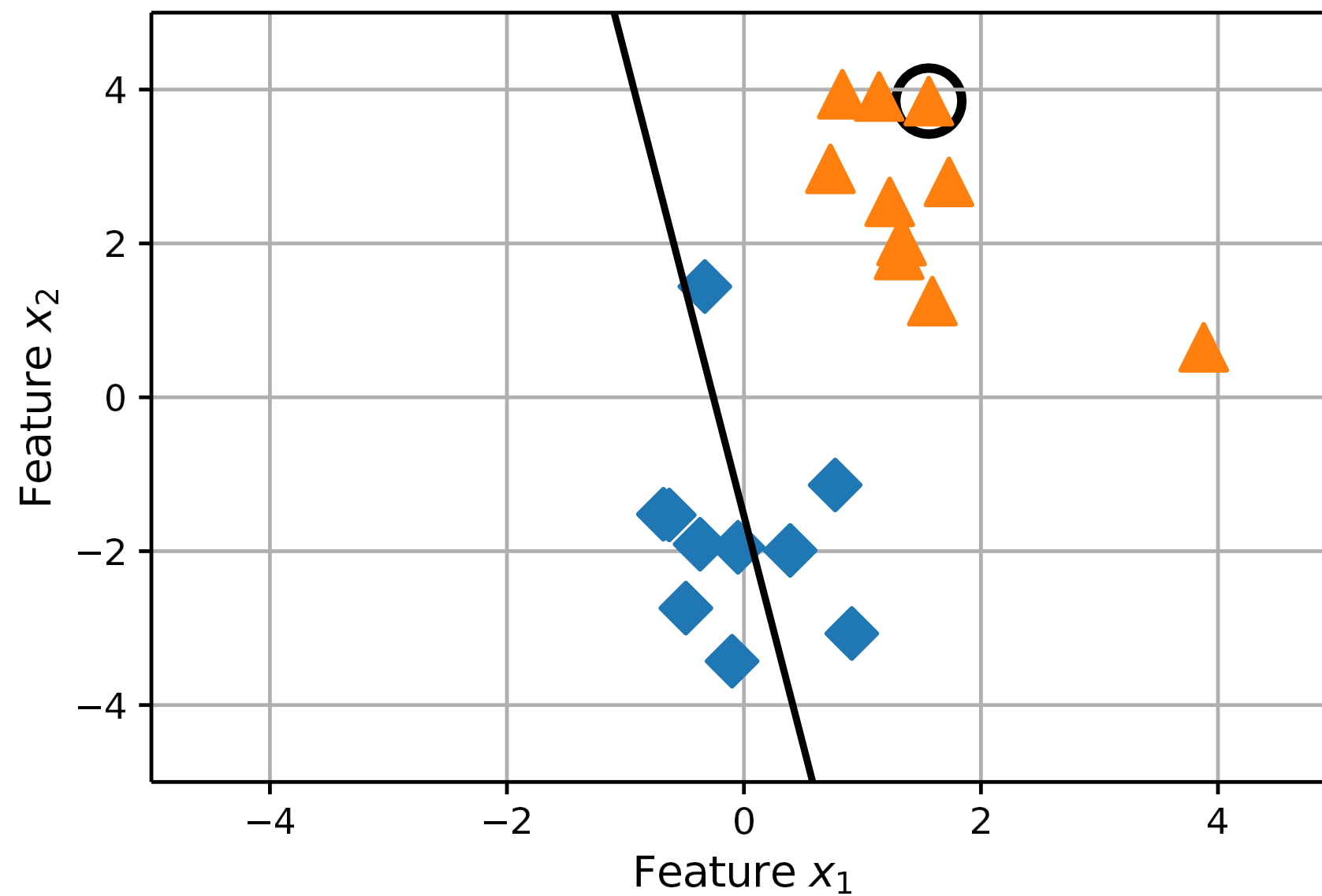
After the Update in Epoch 1, Iteration 11



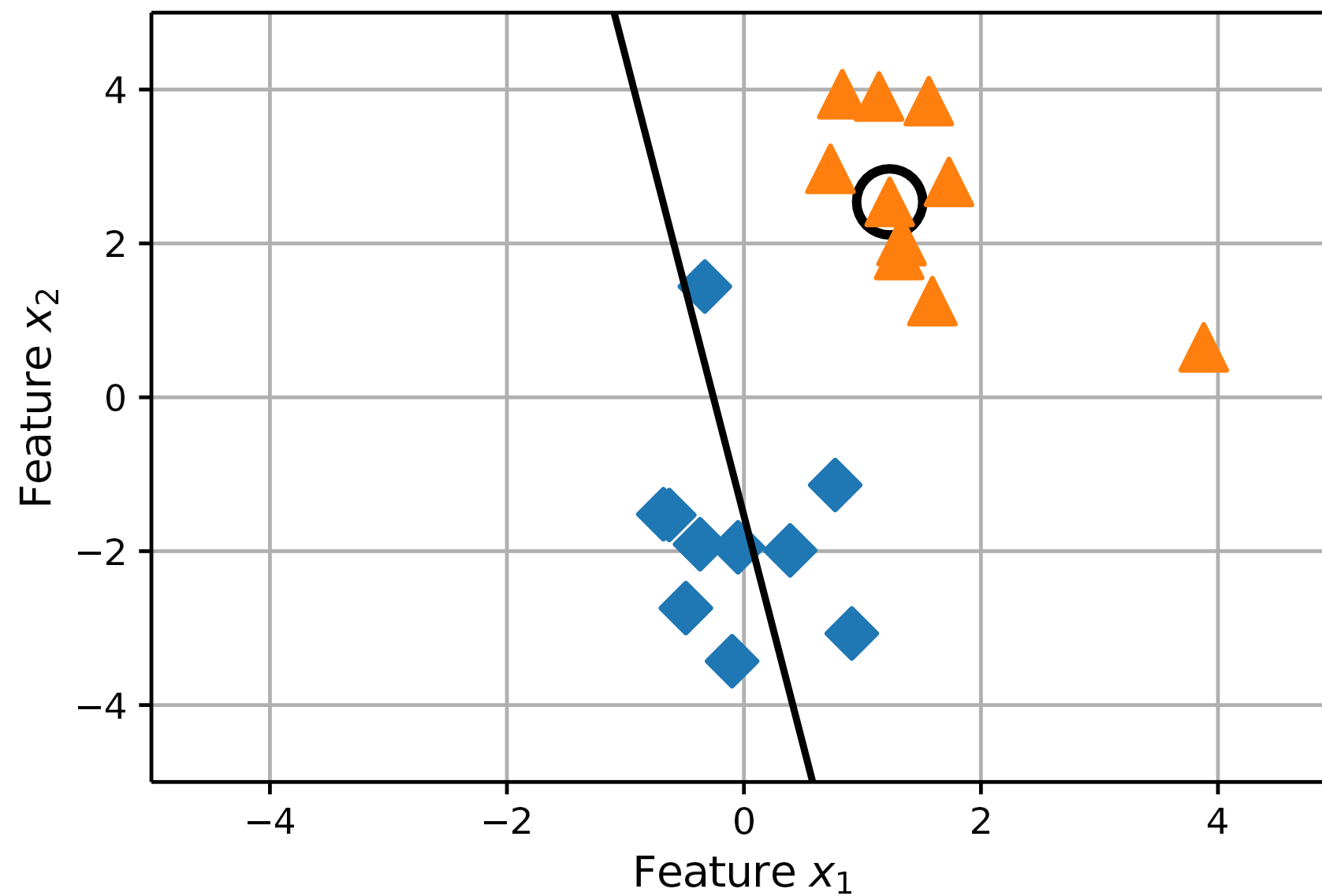
Epoch 1, Iteration 12



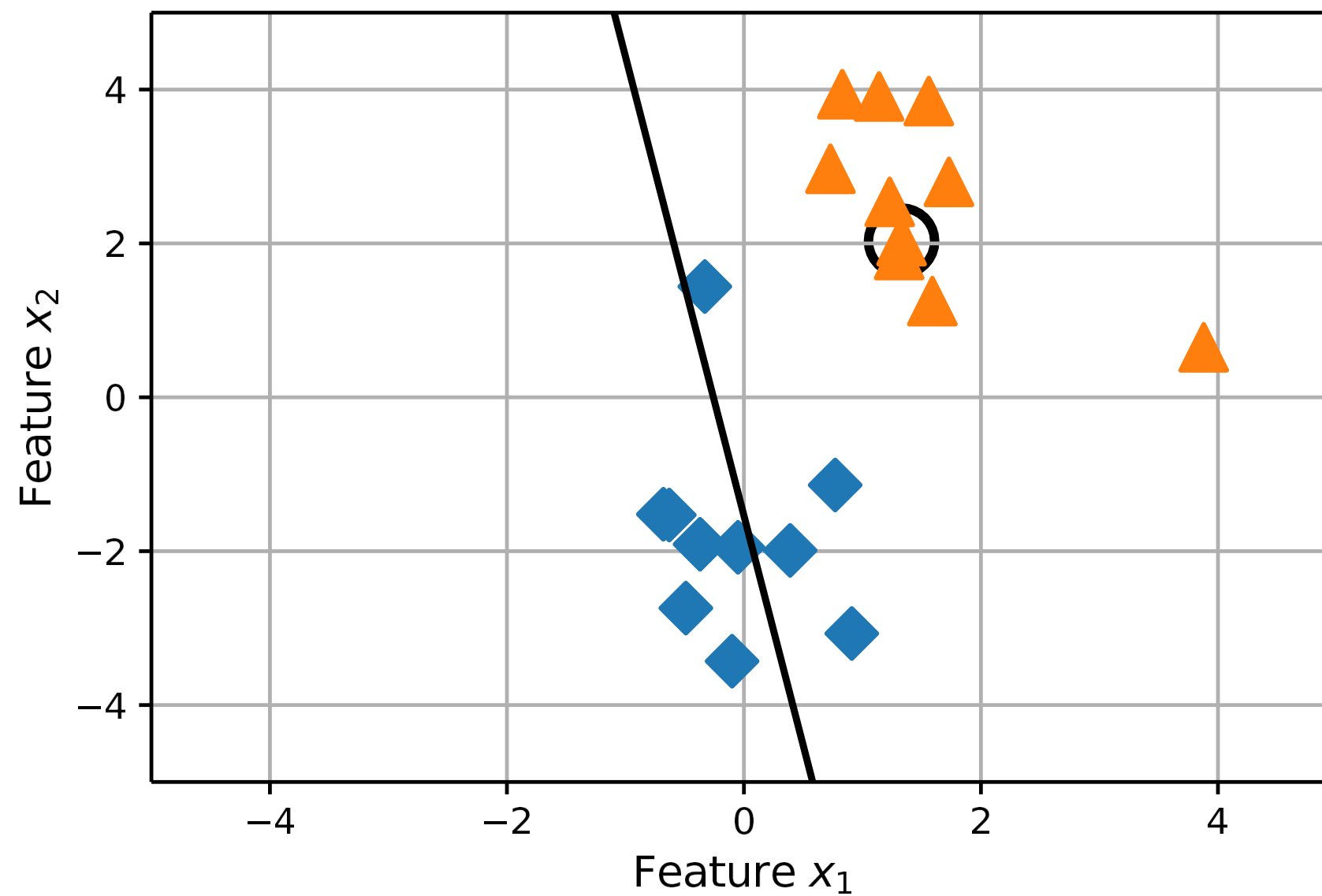
Epoch 1, Iteration 18



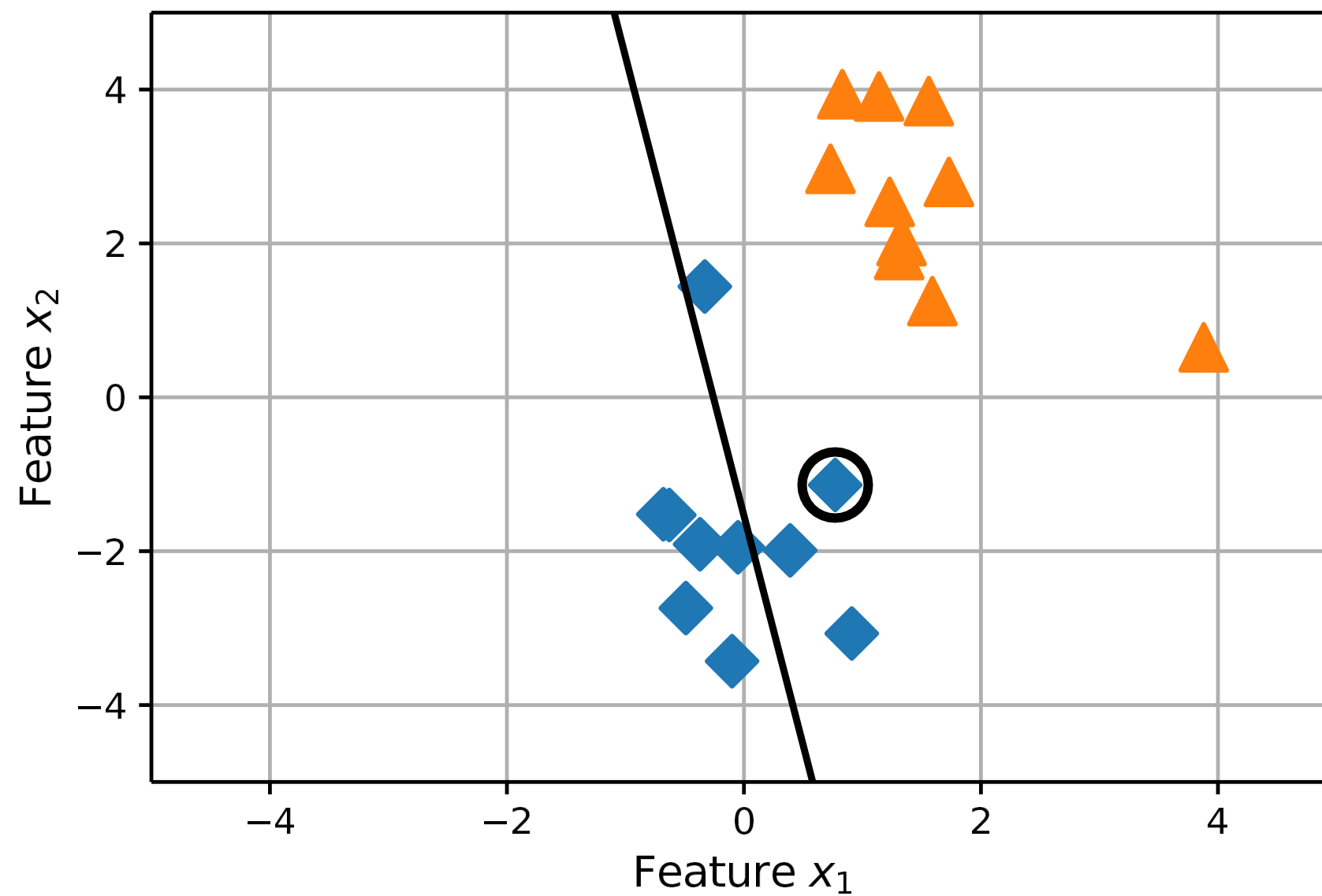
Epoch 1, Iteration 19



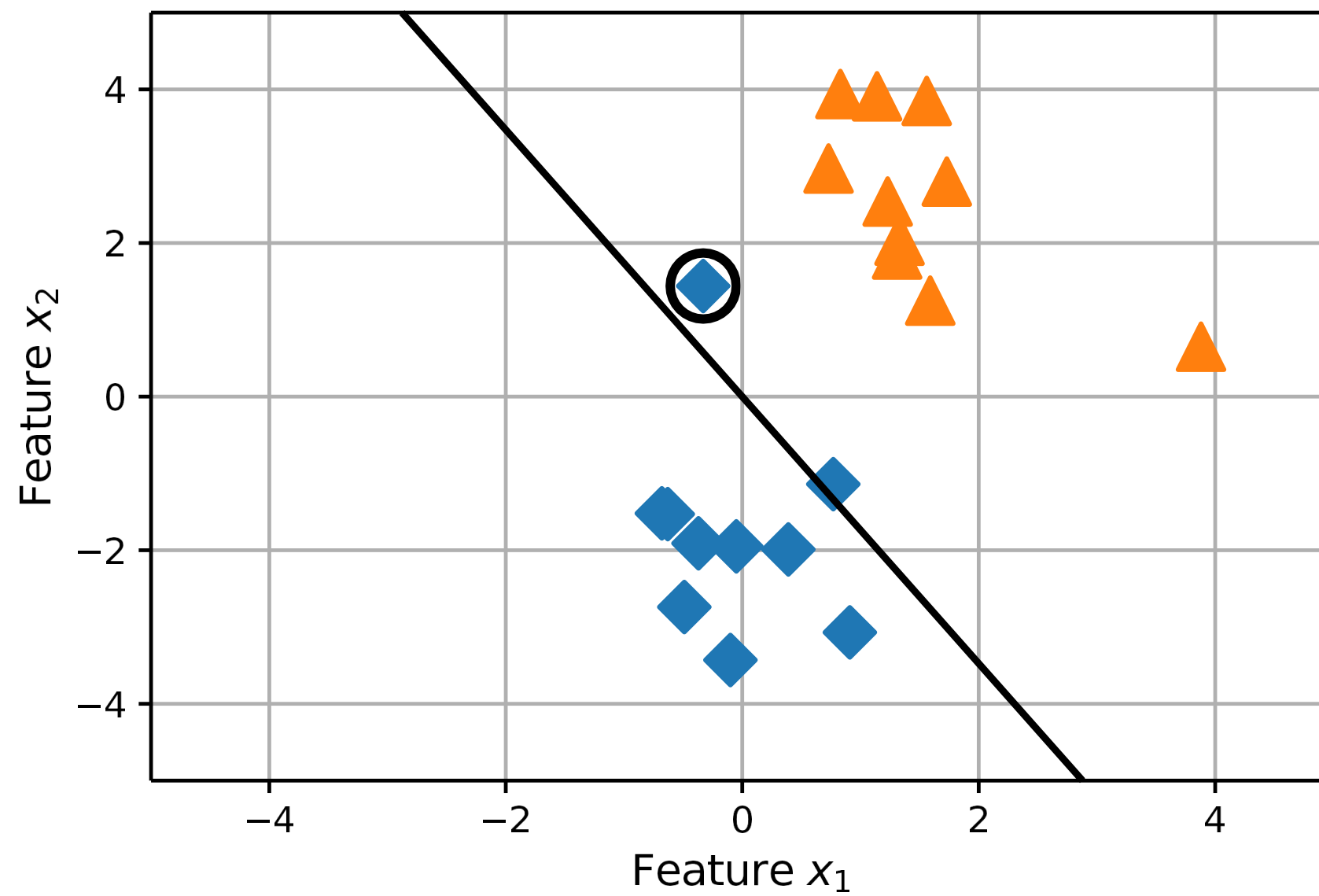
Epoch 1, Iteration 20



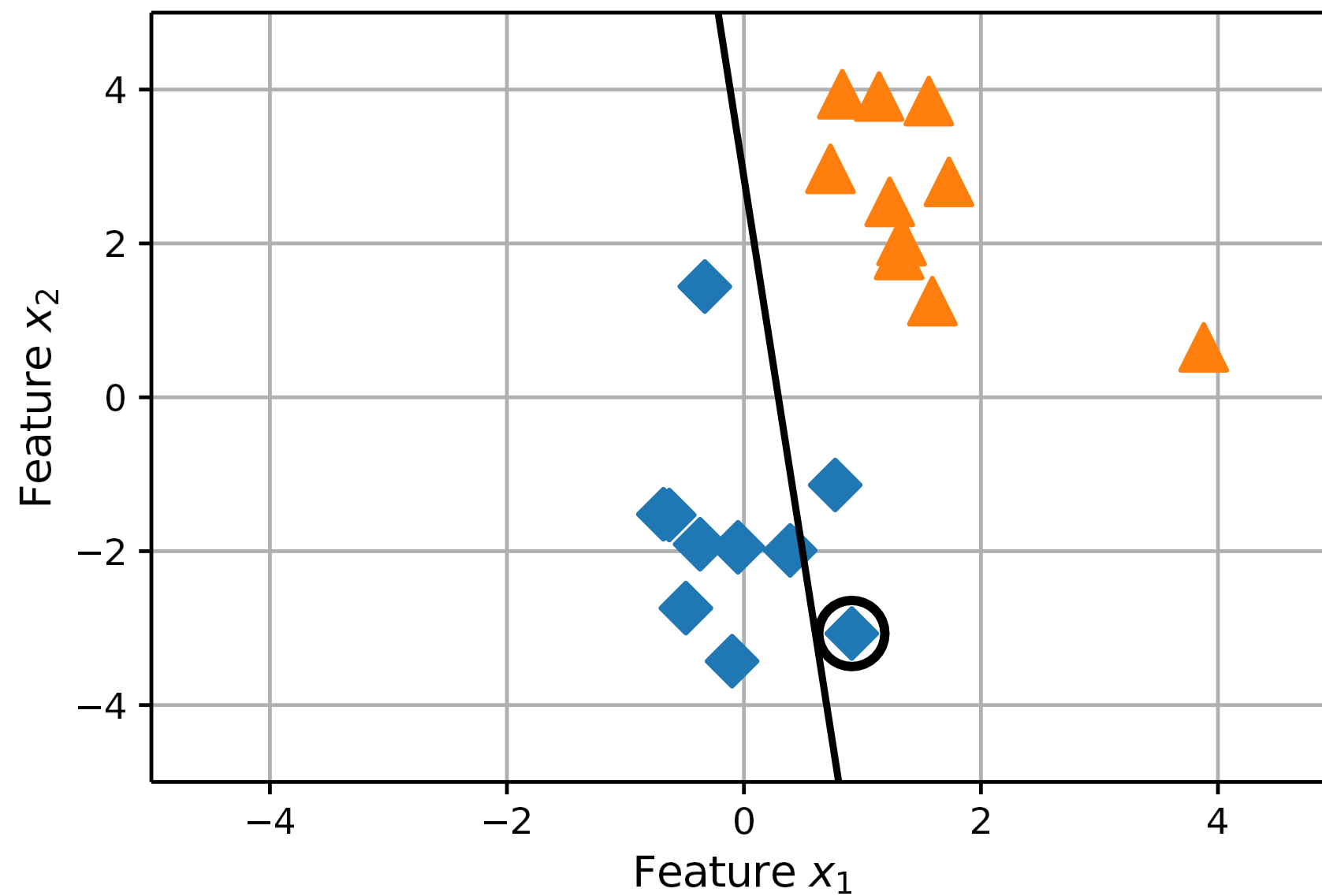
Epoch 2, Iteration 1



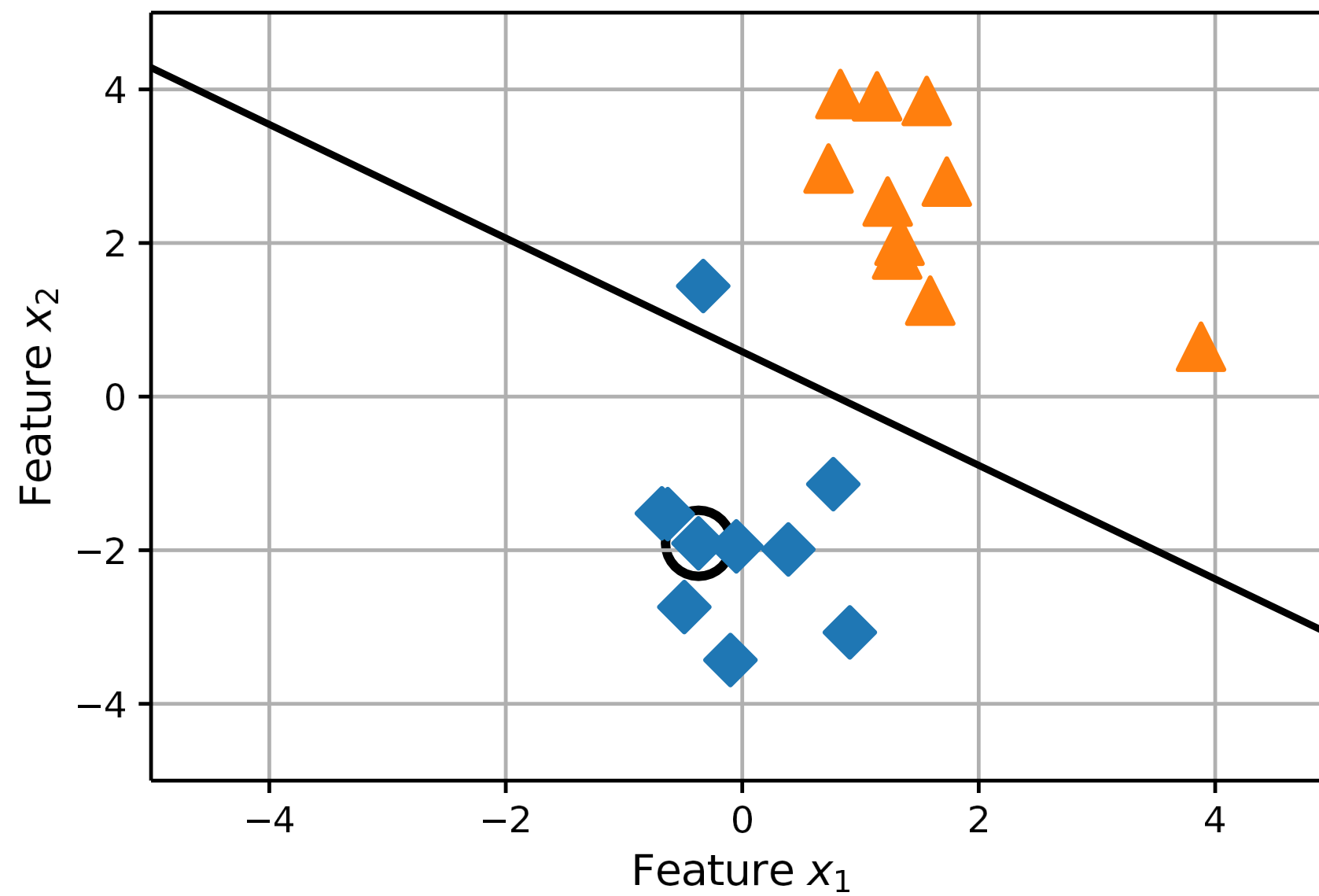
Epoch 2, Iteration 2



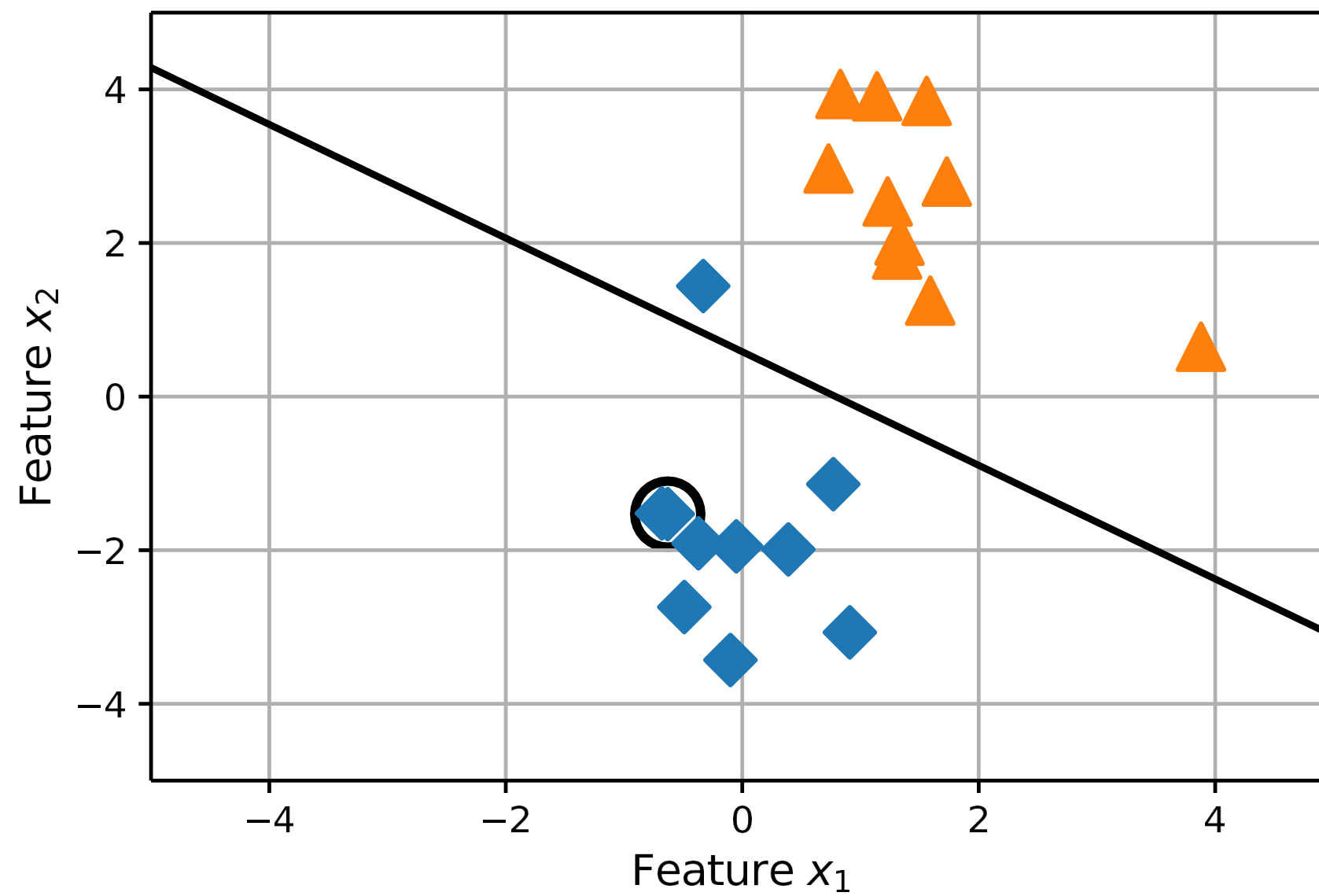
Epoch 2, Iteration 3



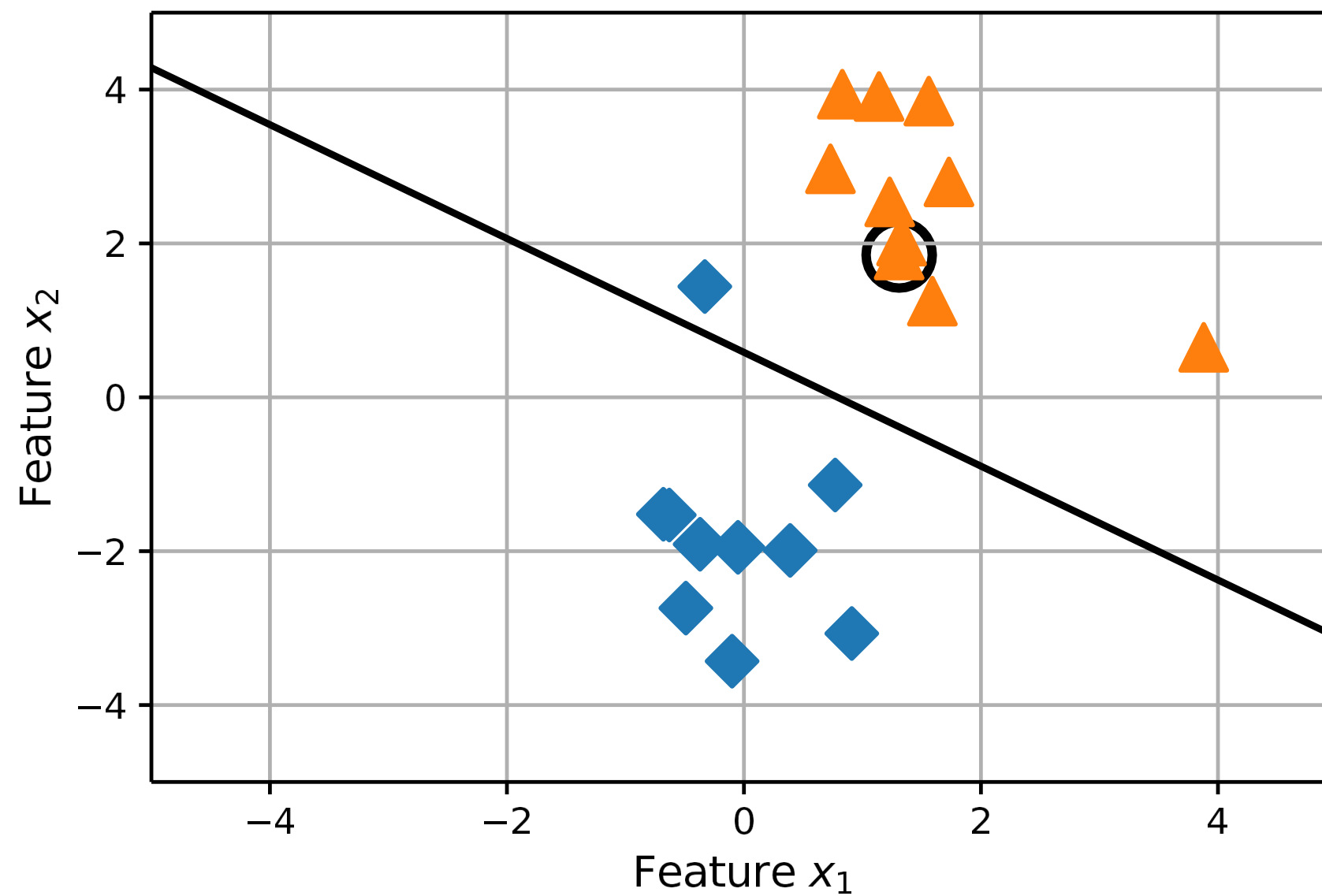
Epoch 2, Iteration 4



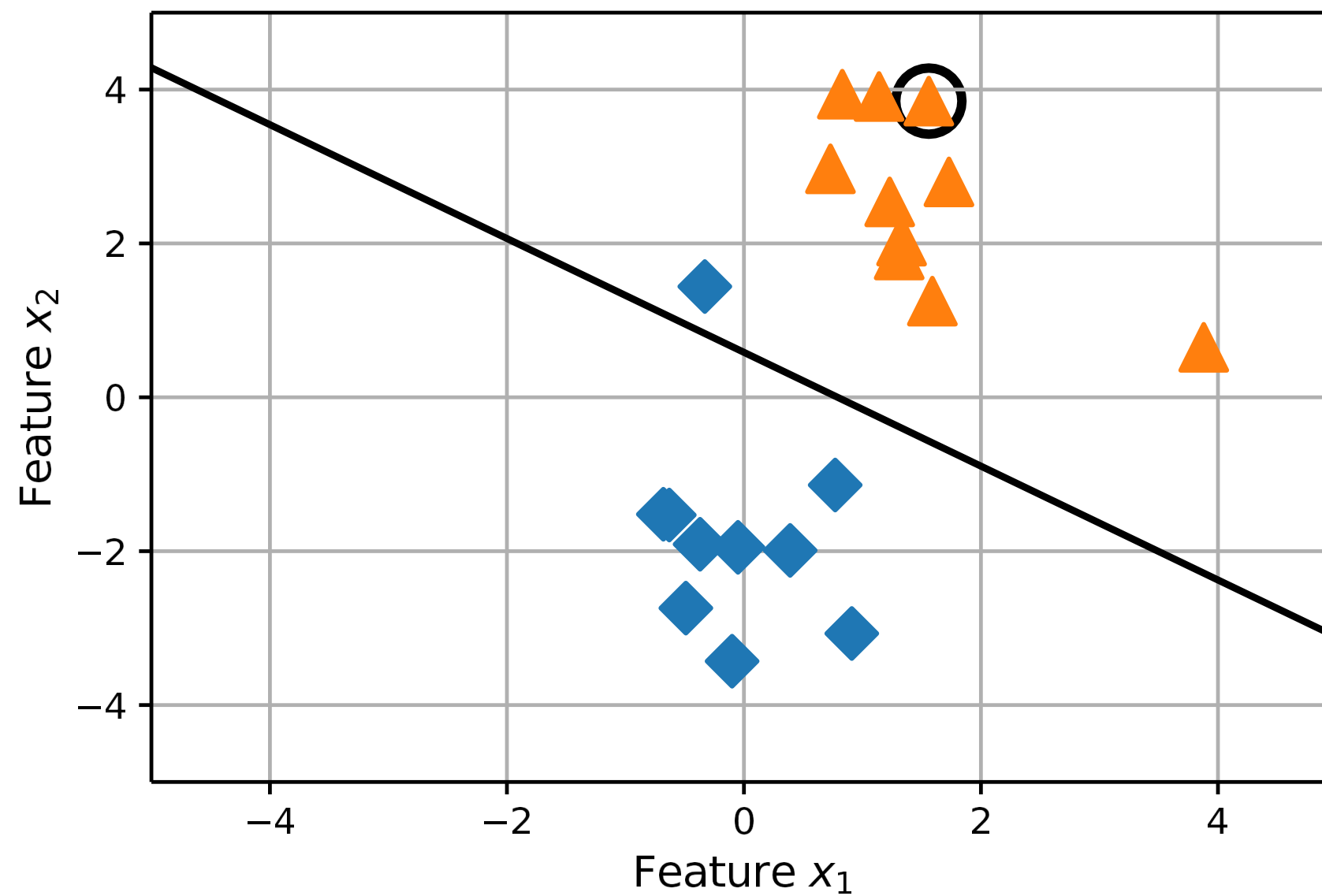
Epoch 2, Iteration 5



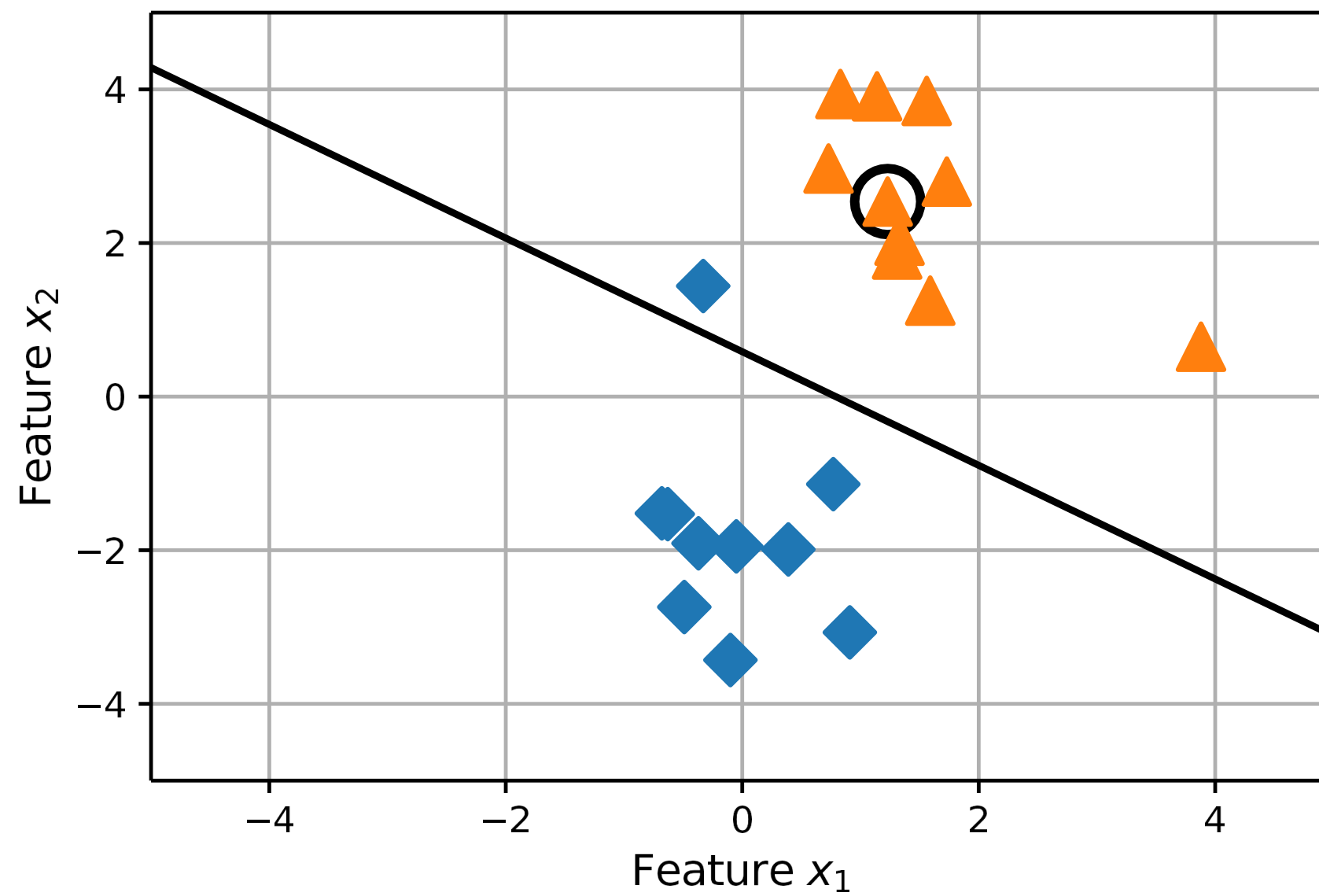
Epoch 2, Iteration 17



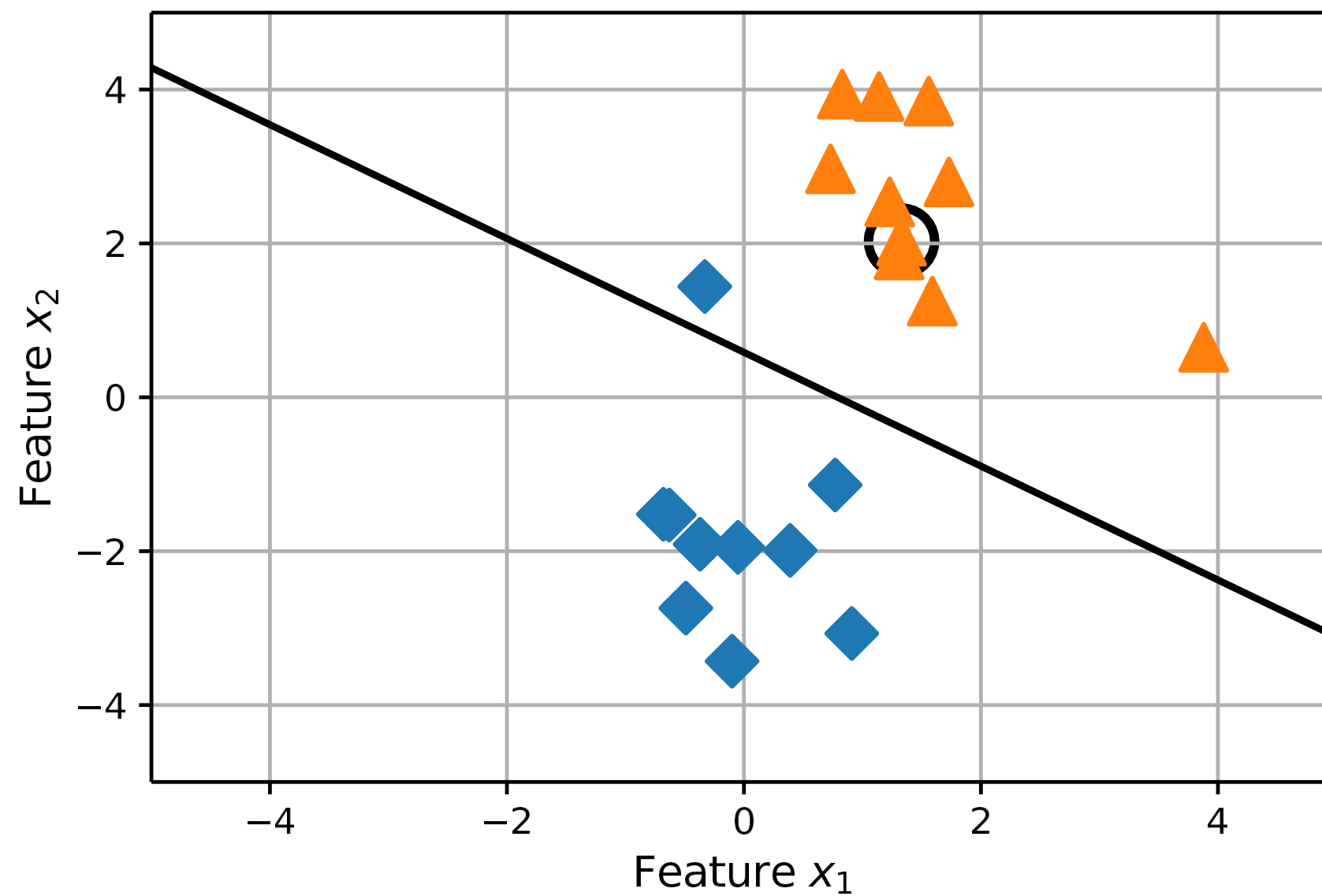
Epoch 2, Iteration 18



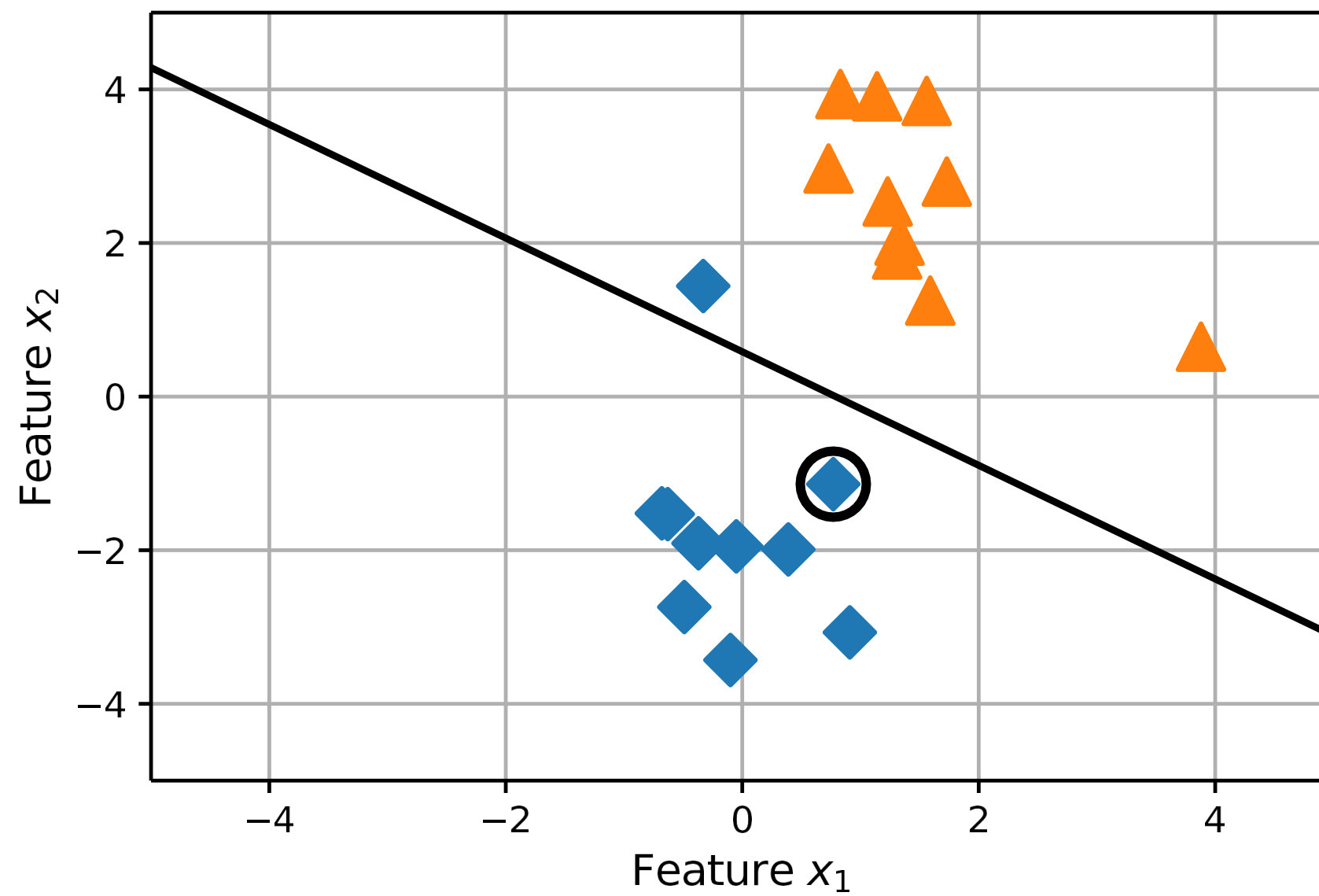
Epoch 2, Iteration 19



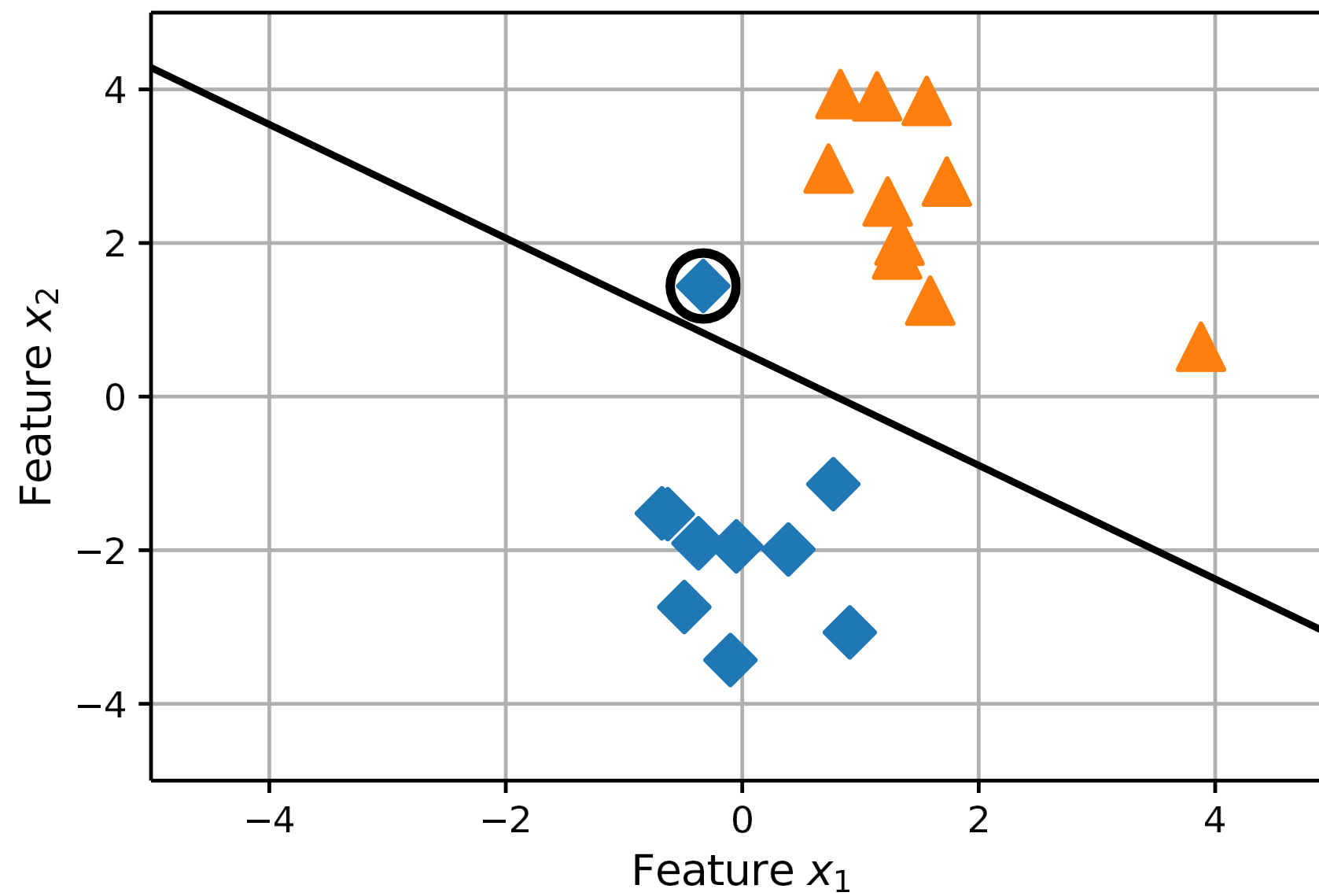
Epoch 2, Iteration 20



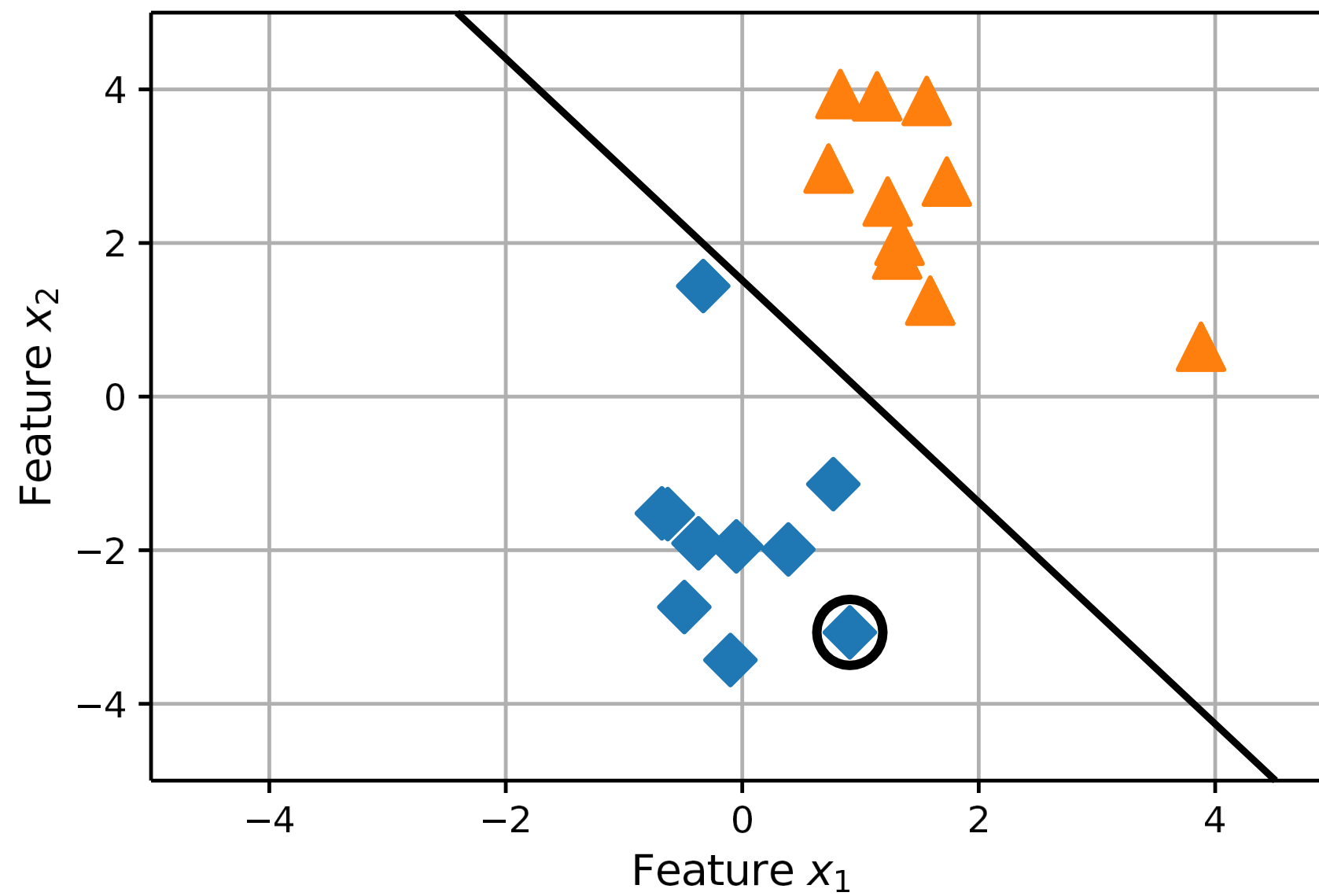
Epoch 3, Iteration 1



Epoch 3, Iteration 2



Epoch 3, Iteration 3



Let us modify this algorithm so that it does not run infinitely.

How does the perceptron **learn** the model parameters?

1. Define training set (where 'x' is the input and 'y' is the true label)
2. Initialize model weights and bias to zero
3. For every training epoch:
 - a. For every training example $\langle x^{[i]}, y^{[i]} \rangle \in D$:
 - (i) Make a prediction (predicted label = $\hat{y}$)
 - (ii) Compute the error

$$error = y - \hat{y}$$

- (iii) Update the weights and bias based on the error

$$\text{Updated weight } (w) = \text{previous weight}(w) + \text{input } (x) * \text{error}$$

$$\text{Updated bias } (b) = \text{previous bias } (b) + \text{error}$$

Iterate until 100% of the examples are correctly classified:

How does the perceptron **learn** the model parameters?

1. Define training set (where 'x' is the input and 'y' is the true label)
2. Initialize model weights and bias to zero
3. For every training epoch:
 - a. For every training example $\langle x^{[i]}, y^{[i]} \rangle \in D$:
 - (i) Make a prediction (predicted label = $\hat{y}$)
 - (ii) Compute the error

$$error = y - \hat{y}$$

- (iii) Update the weights and bias based on the error

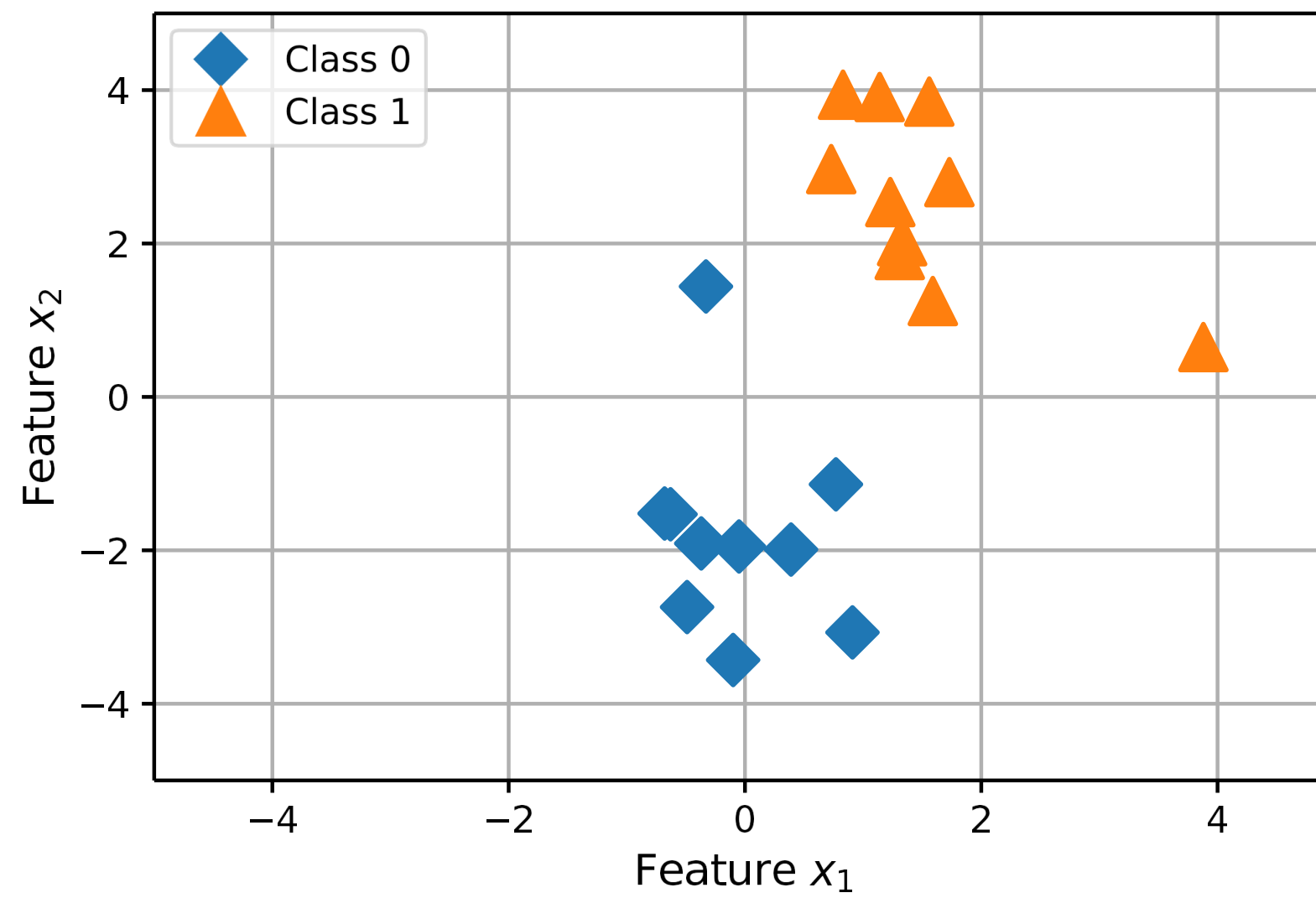
$$\text{Updated weight } (w) = \text{previous weight}(w) + \text{input } (x) * \text{error}$$

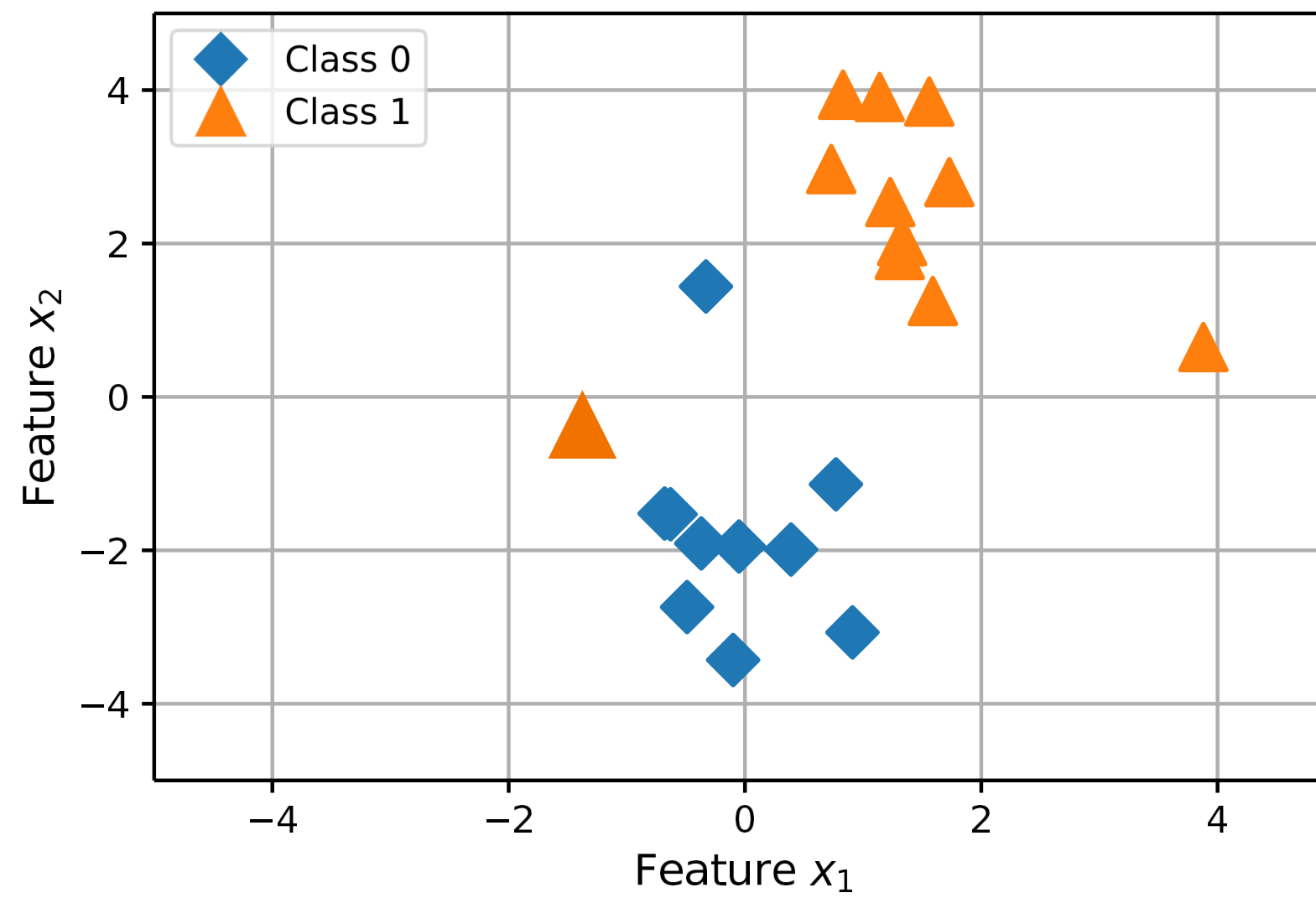
$$\text{Updated bias } (b) = \text{previous bias } (b) + \text{error}$$

Perceptron Hands on Program

- The concept of perceptron is demonstrated in the notebook file
 - [Perceptron-DeepLearning.ipynb](#)

Limitation: Classes Must Be Linearly
Separable

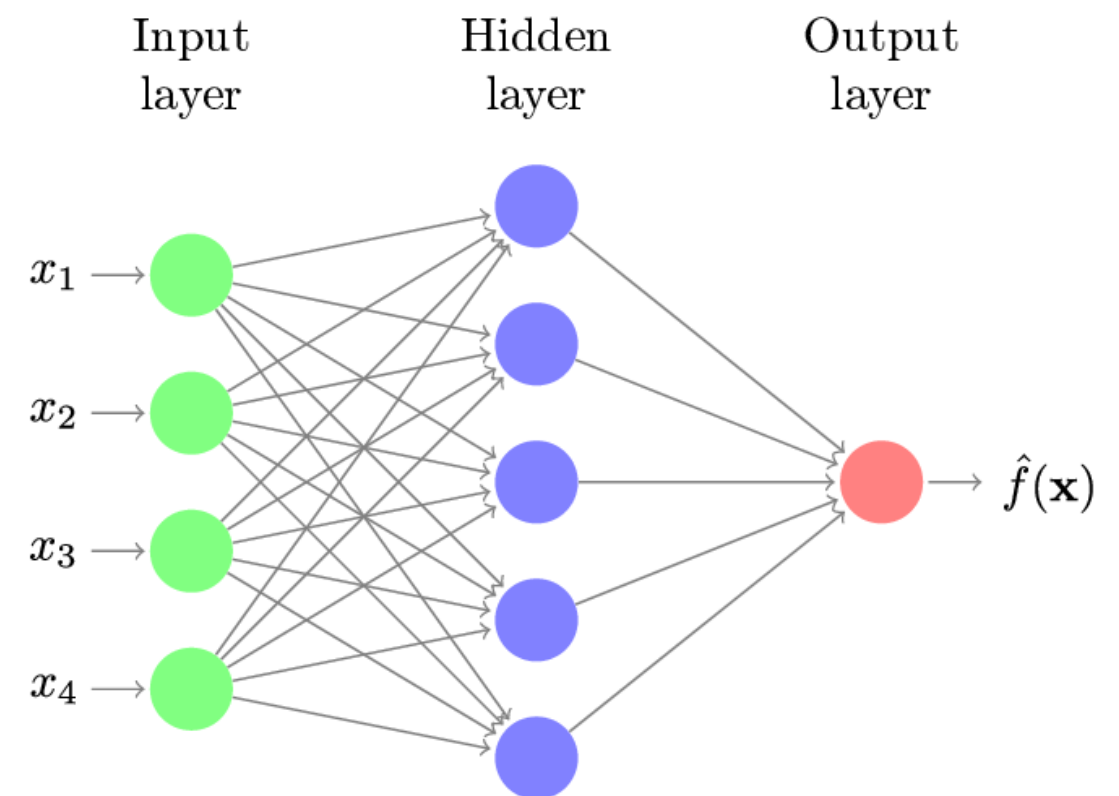
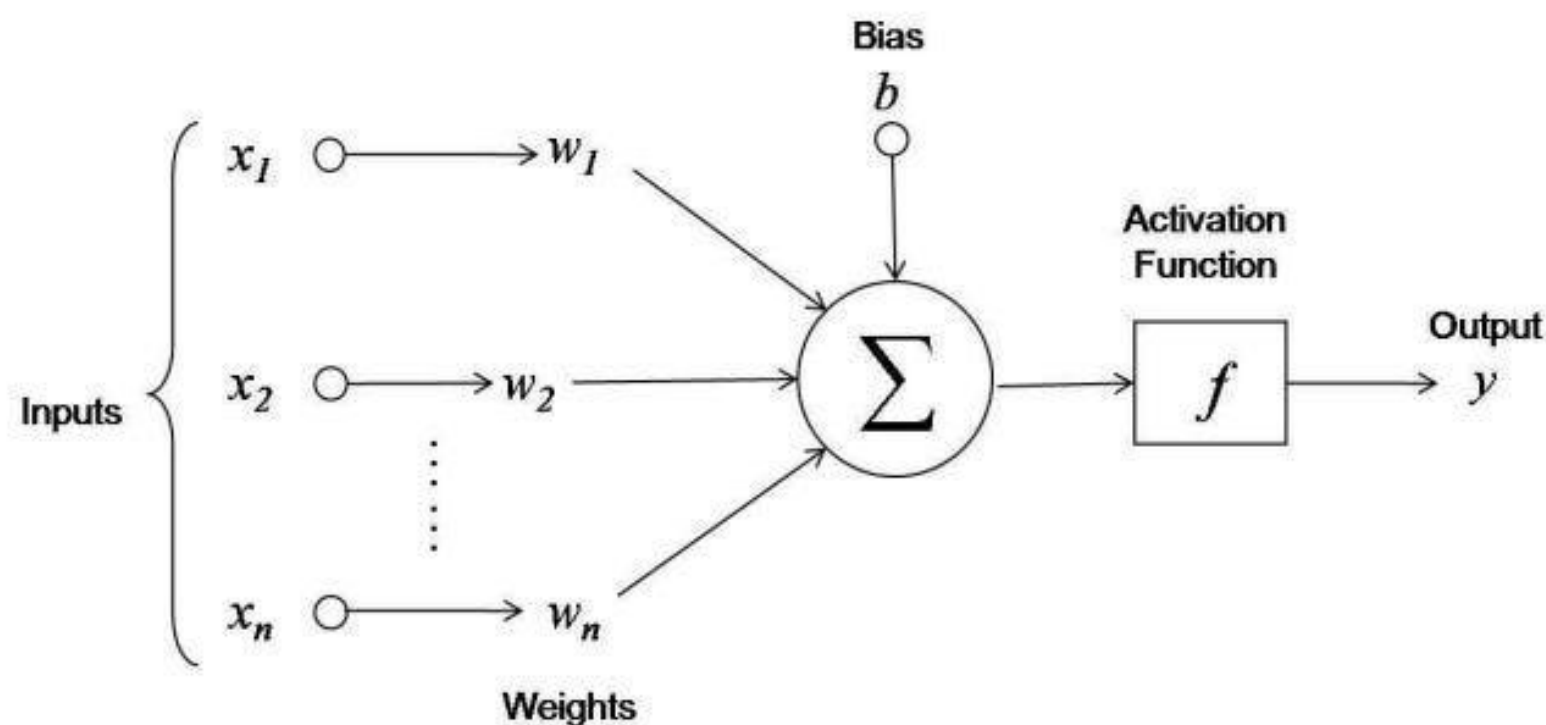




Structure of a Neural Network

Neural Network (Multi-layer Perceptron) basics:

- Input Layer: Receives the raw data.
- Hidden Layers: Intermediate layers where data is processed through weights and biases.
- Output Layer: Produces the final output (e.g., classification, regression).



Structure of a Neural Network

Neurons and Synapses:

- A neuron is the fundamental unit in the network. Each neuron applies a mathematical function (usually linear) to the inputs and passes the result to the next layer.
- Weights: The importance given to each connection between neurons. These values are learned during training.
- Bias: An additional term added to the weighted sum to help the network fit the data better.

Activation Functions

- Sigmoid: Compresses values between 0 and 1. Good for binary classification.

$$\text{Formula: } f(x) = \frac{1}{1+e^{-x}}$$

- ReLU (Rectified Linear Unit): Keeps only positive values and sets negative ones to zero. Most commonly used for hidden layers.

$$\text{Formula: } f(x) = \max(0, x)$$

- Softmax: Used in multi-class classification to convert logits (raw prediction values) into probabilities that sum to 1.

Structure of a Neural Network

- Example: Consider a neural network that classifies images of digits (0-9). The input layer might receive the pixel values, hidden layers transform those values into features, and the output layer predicts the most probable digit.



Image Source: 3blue1brown

Structure of a Neural Network

Plain vanilla
(aka “multilayer perceptron”)

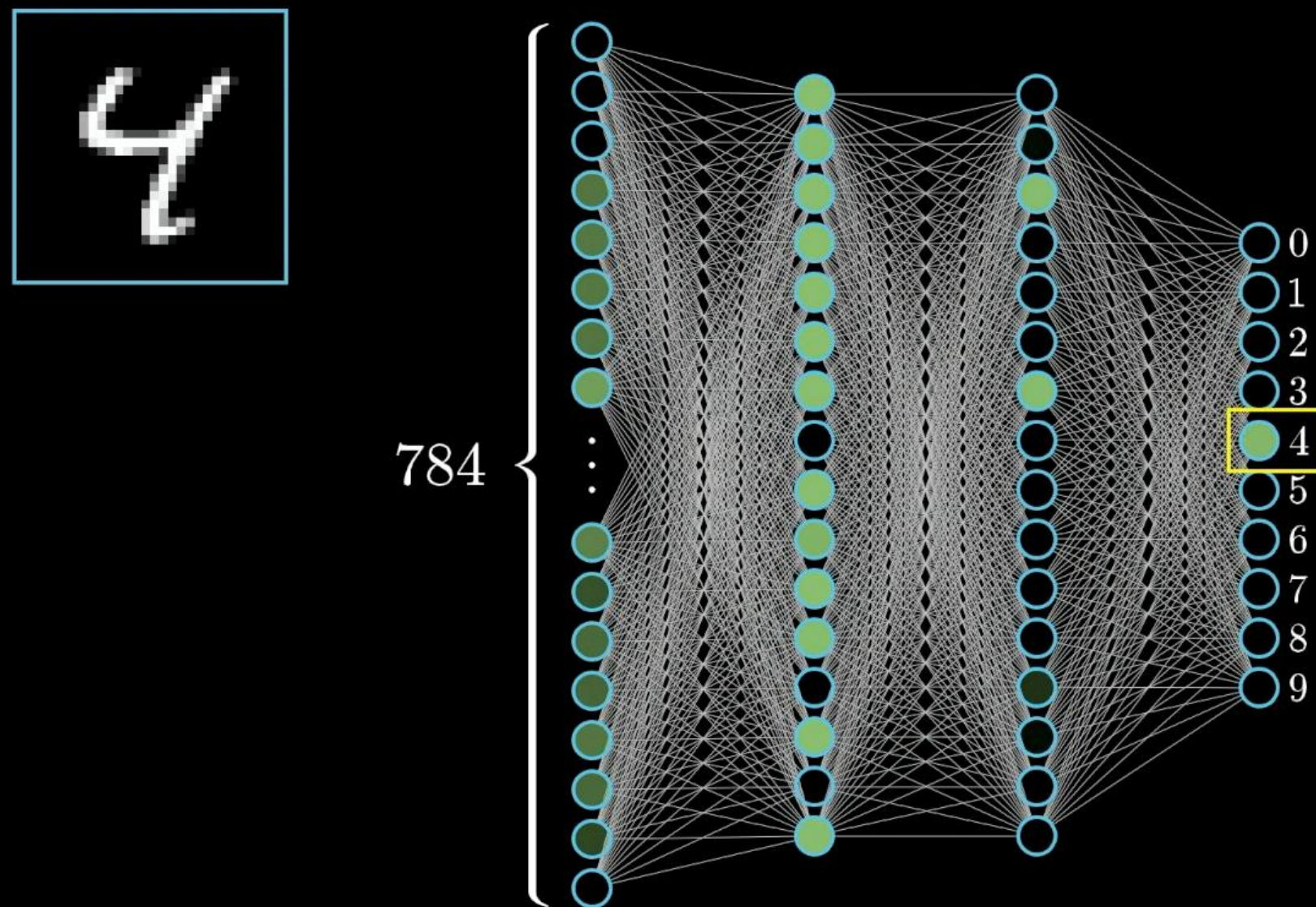


Image Source: 3blue1brown

Neurons

- For now, all I want you to think of when I say "neuron" is "a thing that holds a number." more precisely, a value in the range of 0.0 and 1.0. In essence, neural networks consist of a collection of interconnected neurons.
- You may have an image in your mind of each neuron being lit up when its activation is a high number. This number inside the neuron is called the "**activation**" of that particular neuron.



Image Source: 3blue1brown

Neurons

- These neurons store all the data that travels via our neural network. Therefore, using these neuron values between 0.0 and 1.0, we must represent the inputs and outputs of our network (the images and digit predictions).
- Each pixel in the original image has a value between 0.0 (black) and 1.0 (white).

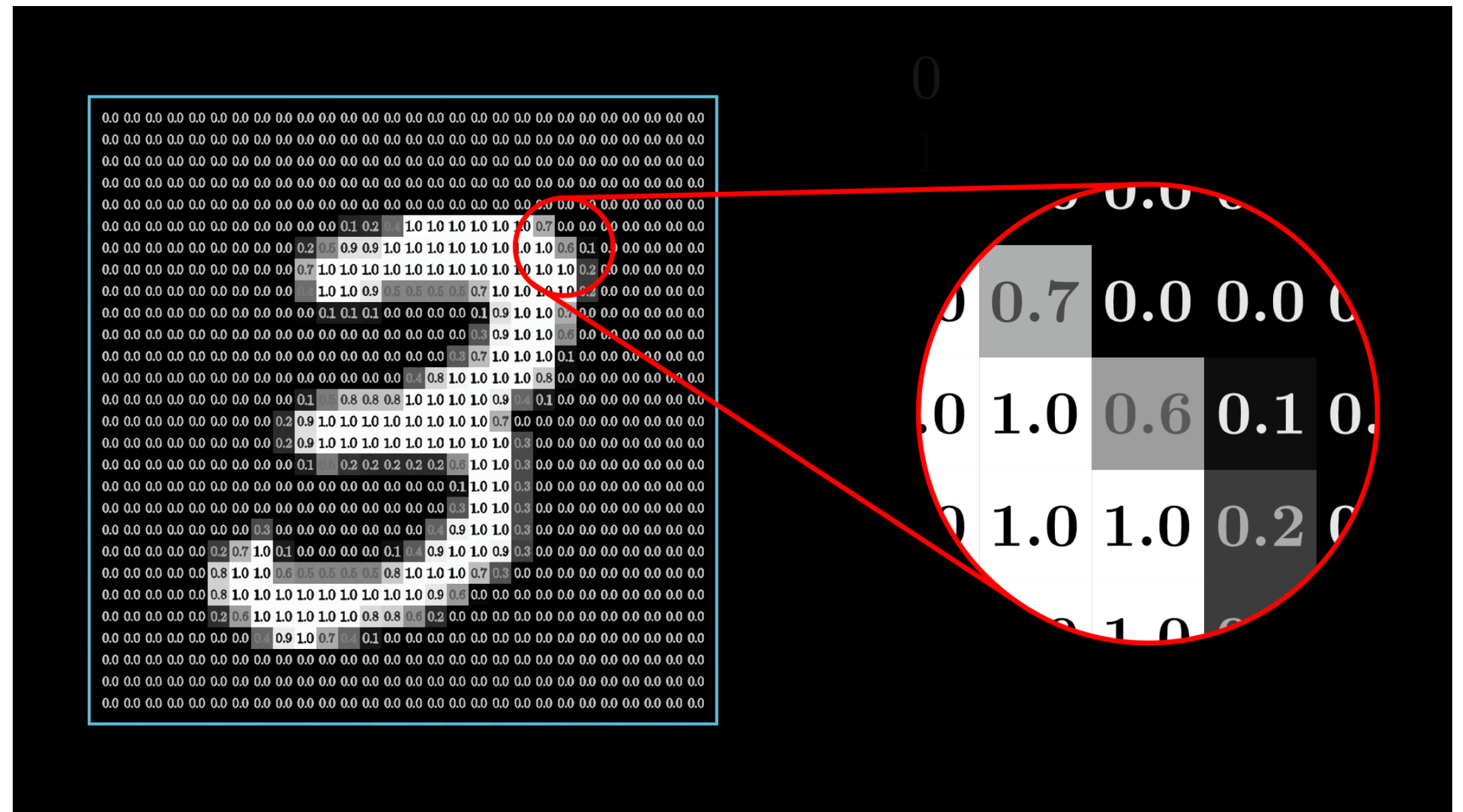
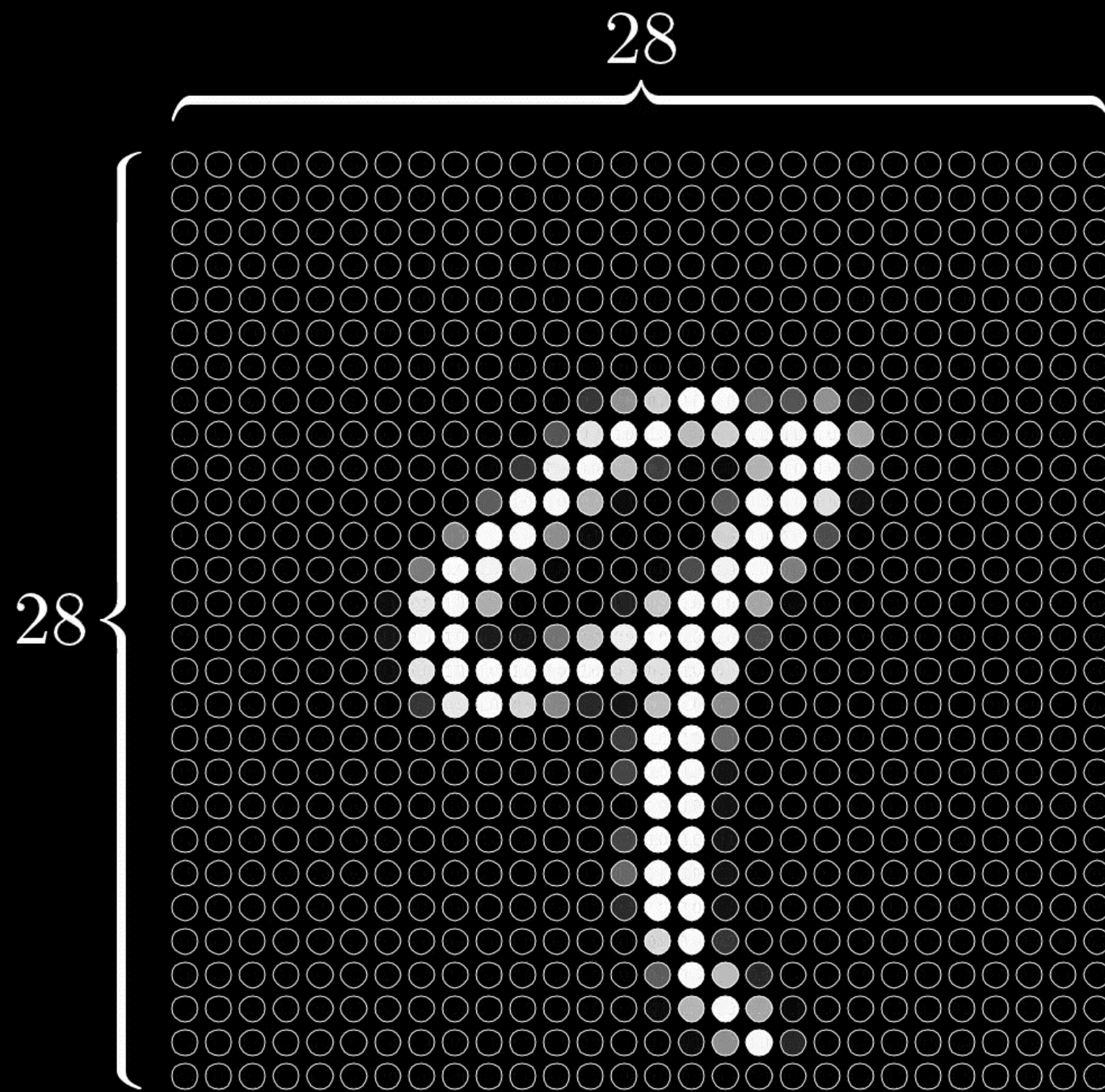


Image Source: 3blue1brown

Neurons



$$28 \times 28 = 784$$

The input layer contains 784 neurons, each of which corresponds to a single pixel in the original image.

Image Source: 3blue1brown

Neurons

- The 10 neurons that make up the final layer of our network will each represent a potential digit. The amount that these neurons fire, which is again a number between 0.0 and 1.0, will indicate how much the system believes a particular digit and an image match.

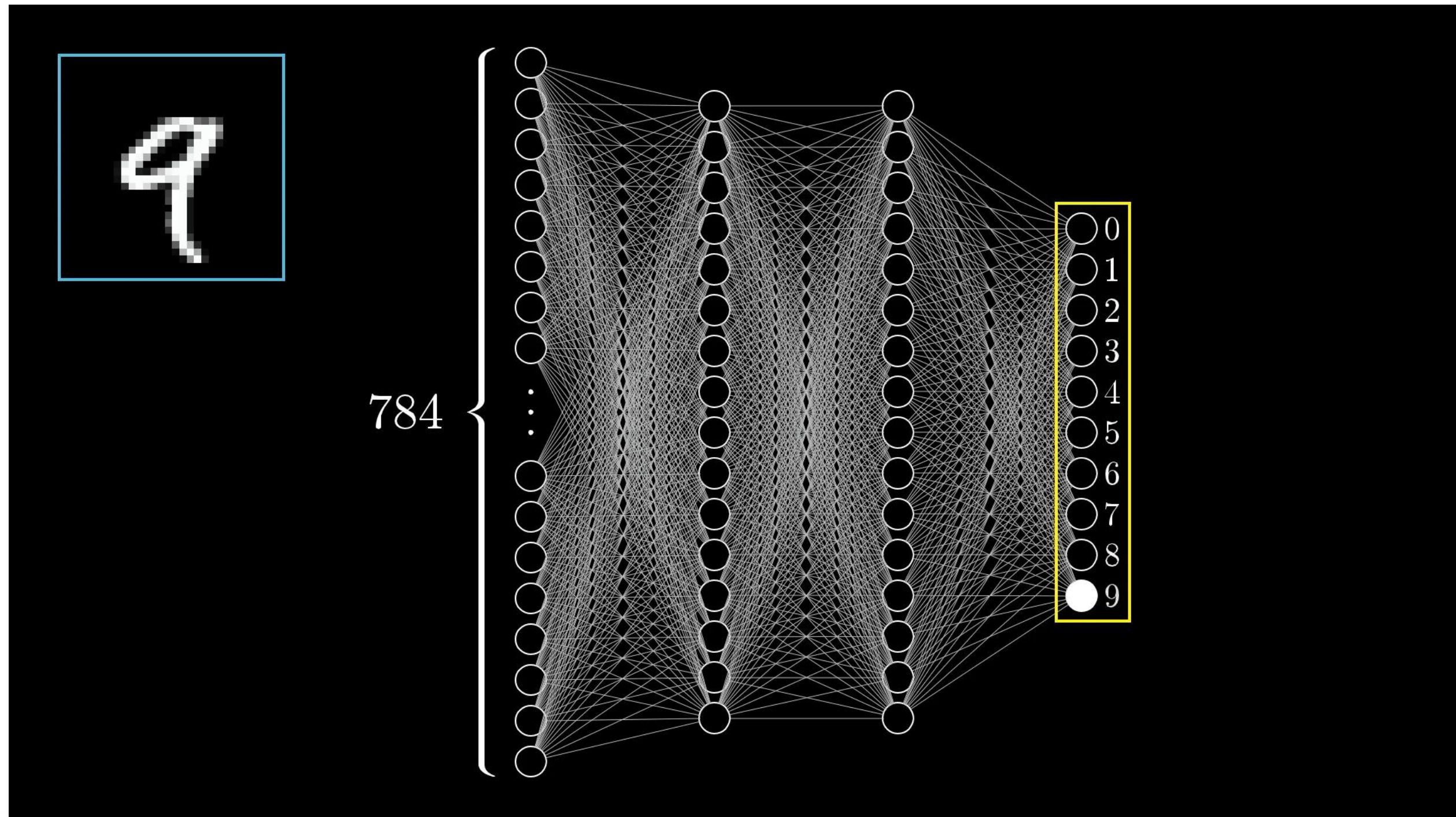


Image Source: 3blue1brown

Neurons

- What kind of digit does this network believe it is looking at based on the output layer of the network shown below? How certain does it feel?

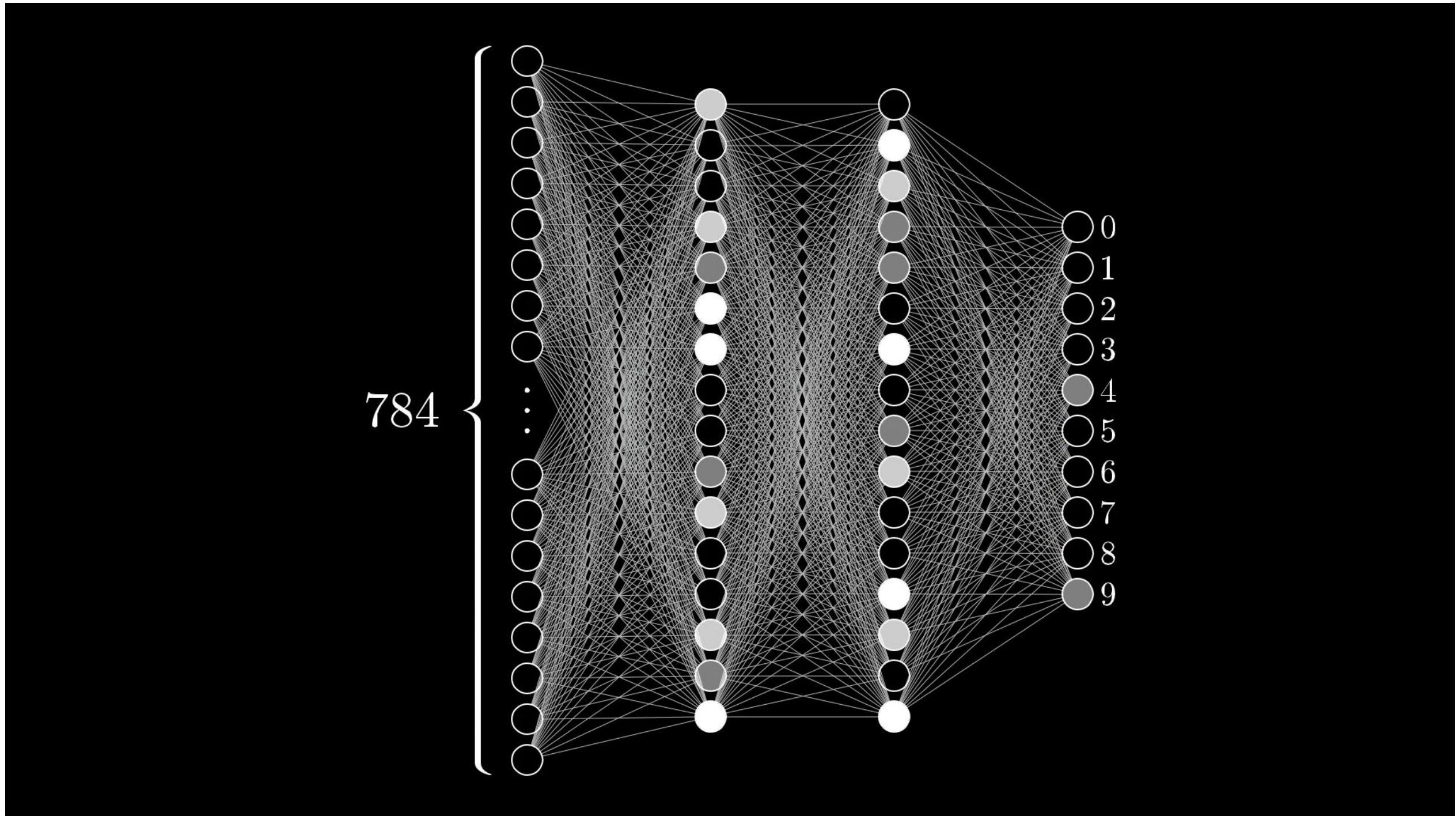


Image Source: 3blue1brown

Neurons

- What kind of digit does this network believe it is looking at based on the output layer of the network shown below? How certain does it feel?
- Answer: It's having trouble deciding whether the input image is a 4 or a 9.

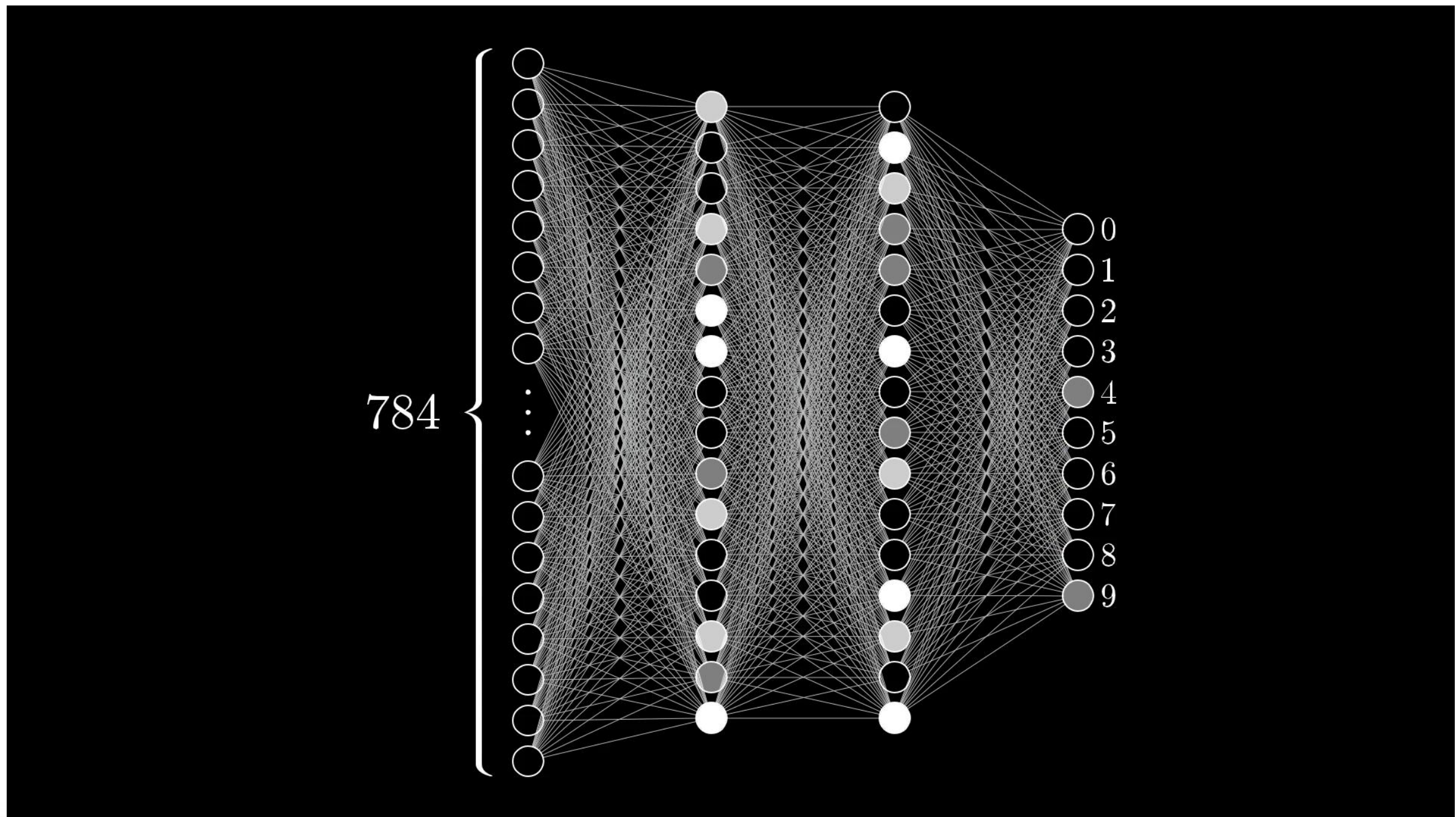
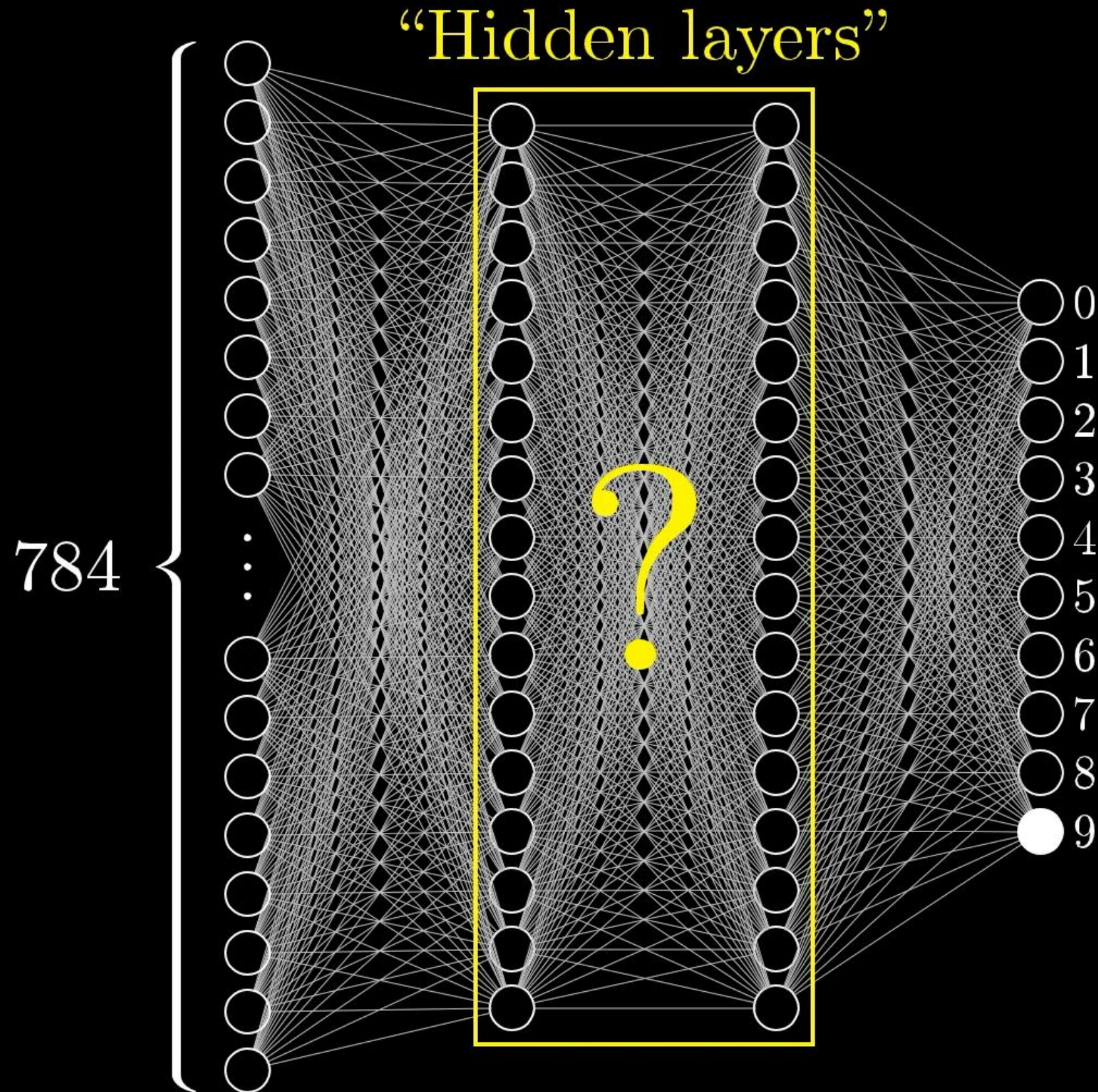
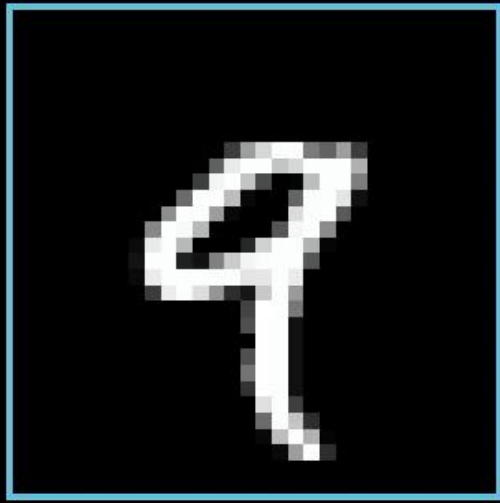


Image Source: 3blue1brown

Hidden Layers



Why Use Layers?

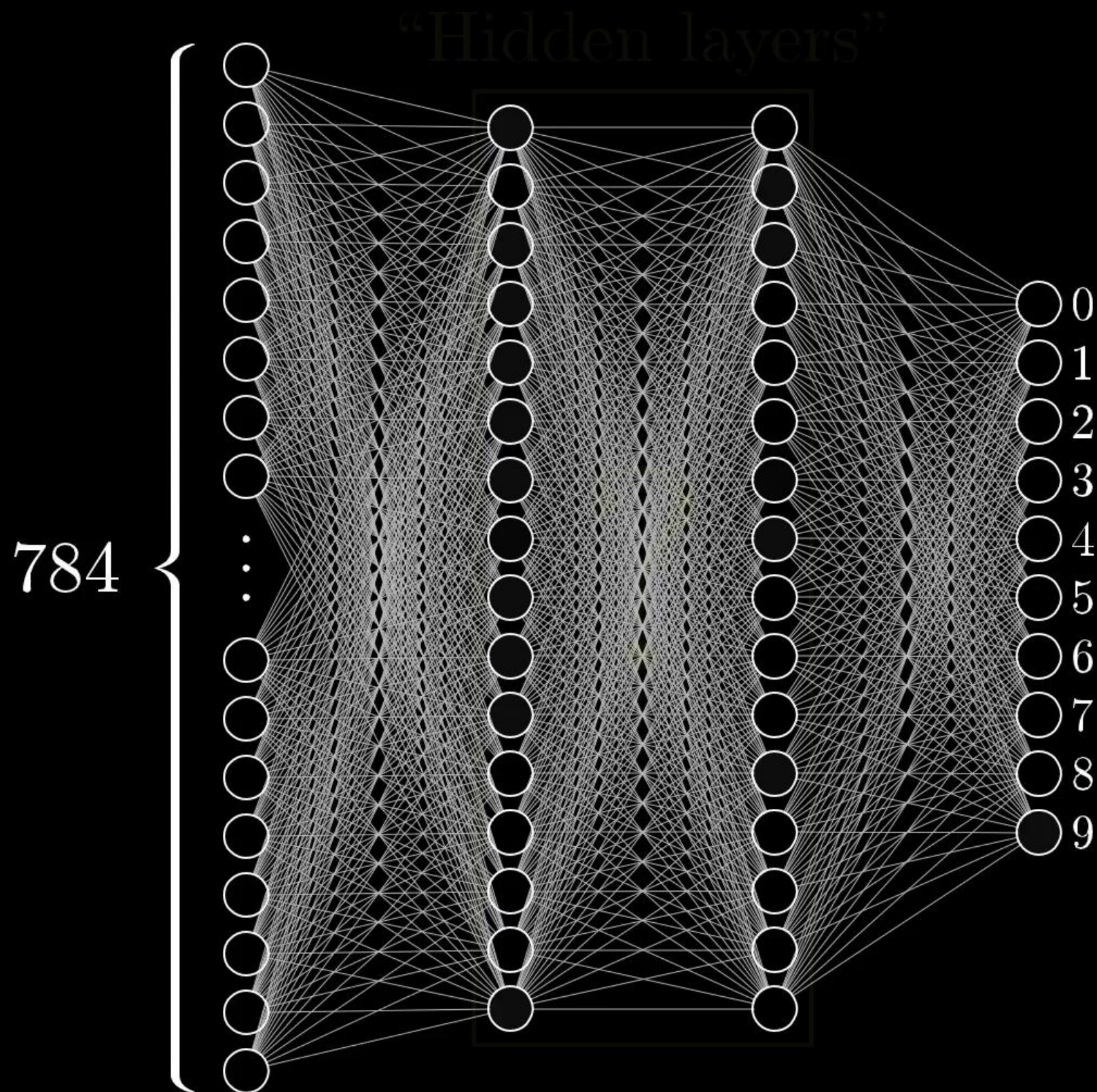


Image Source: 3blue1brown

Why Use Layers?

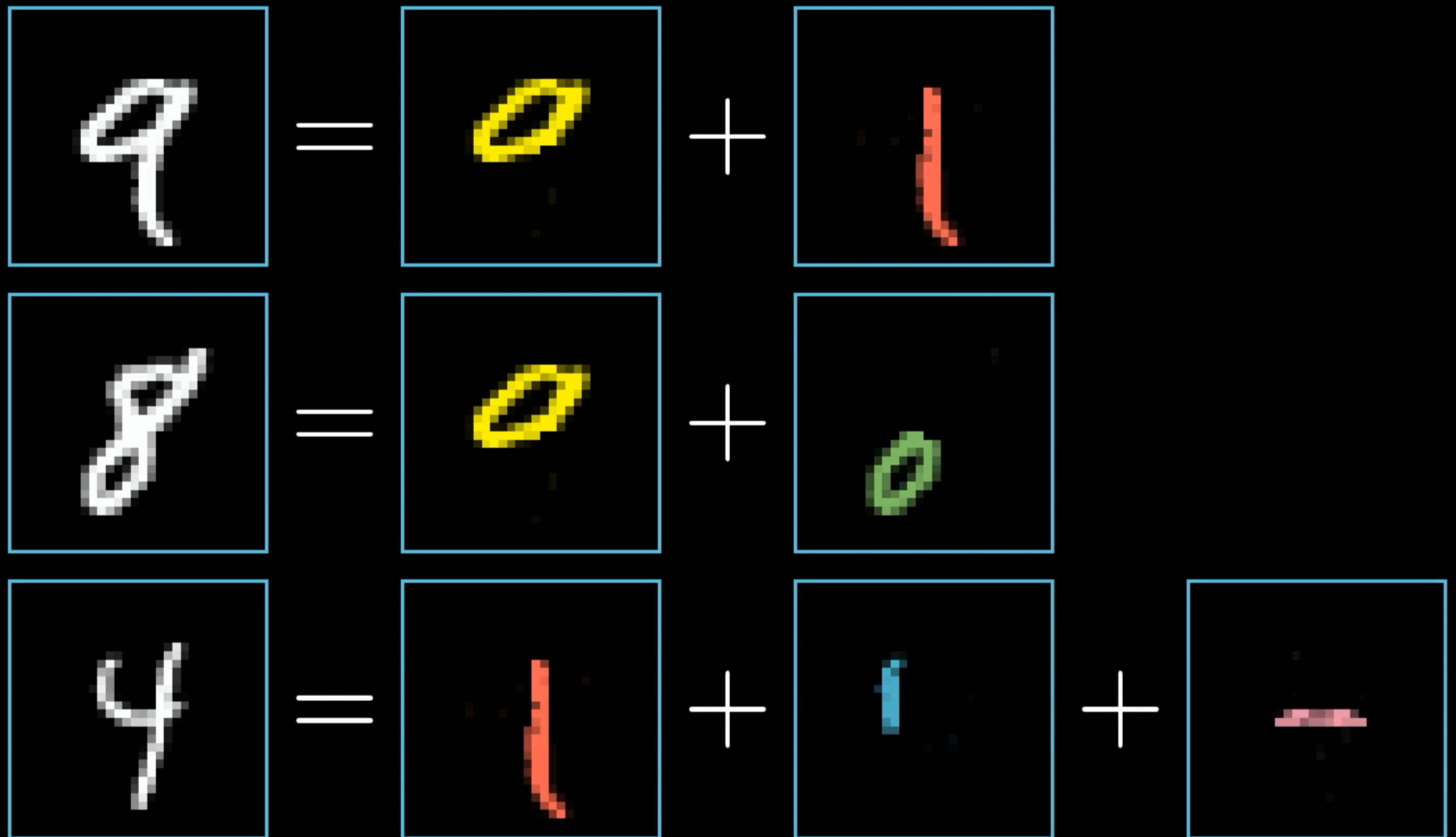


Image Source: 3blue1brown

Why Use Layers?

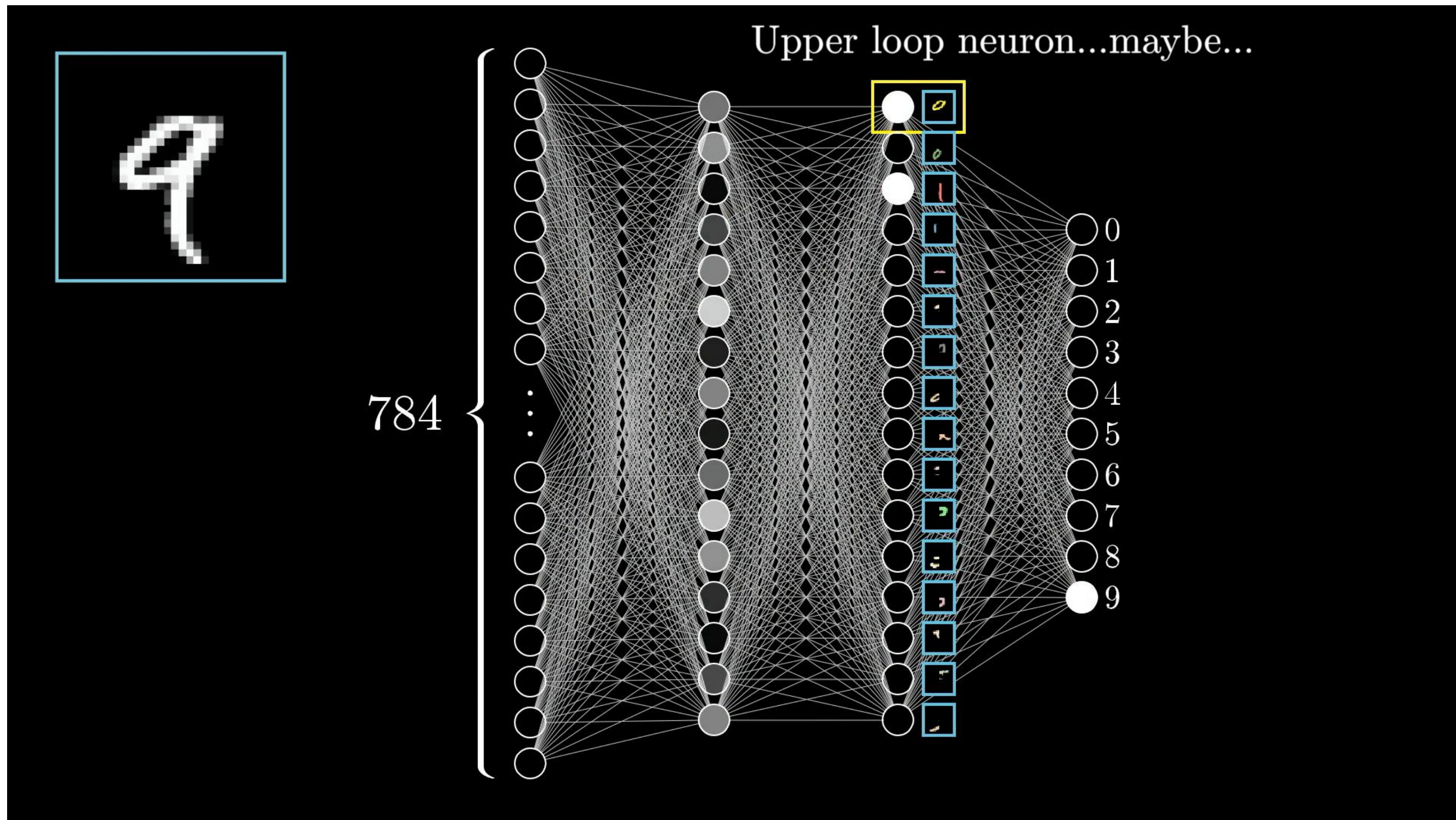


Image Source: 3blue1brown

Why Use Layers?

- A loop can be broken down into several small edges.

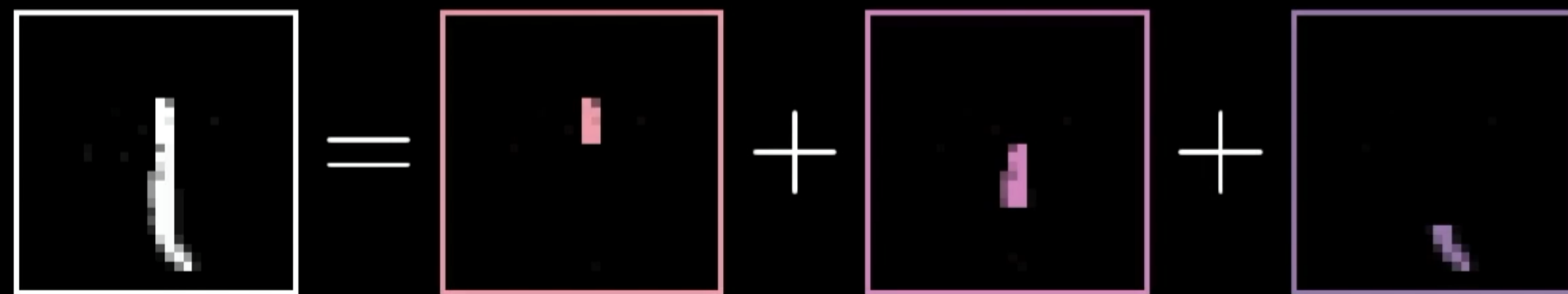
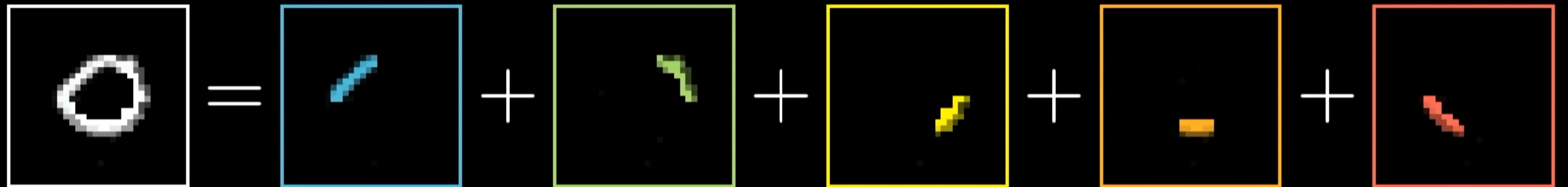


Image Source: 3blue1brown

Does the network actually do this?

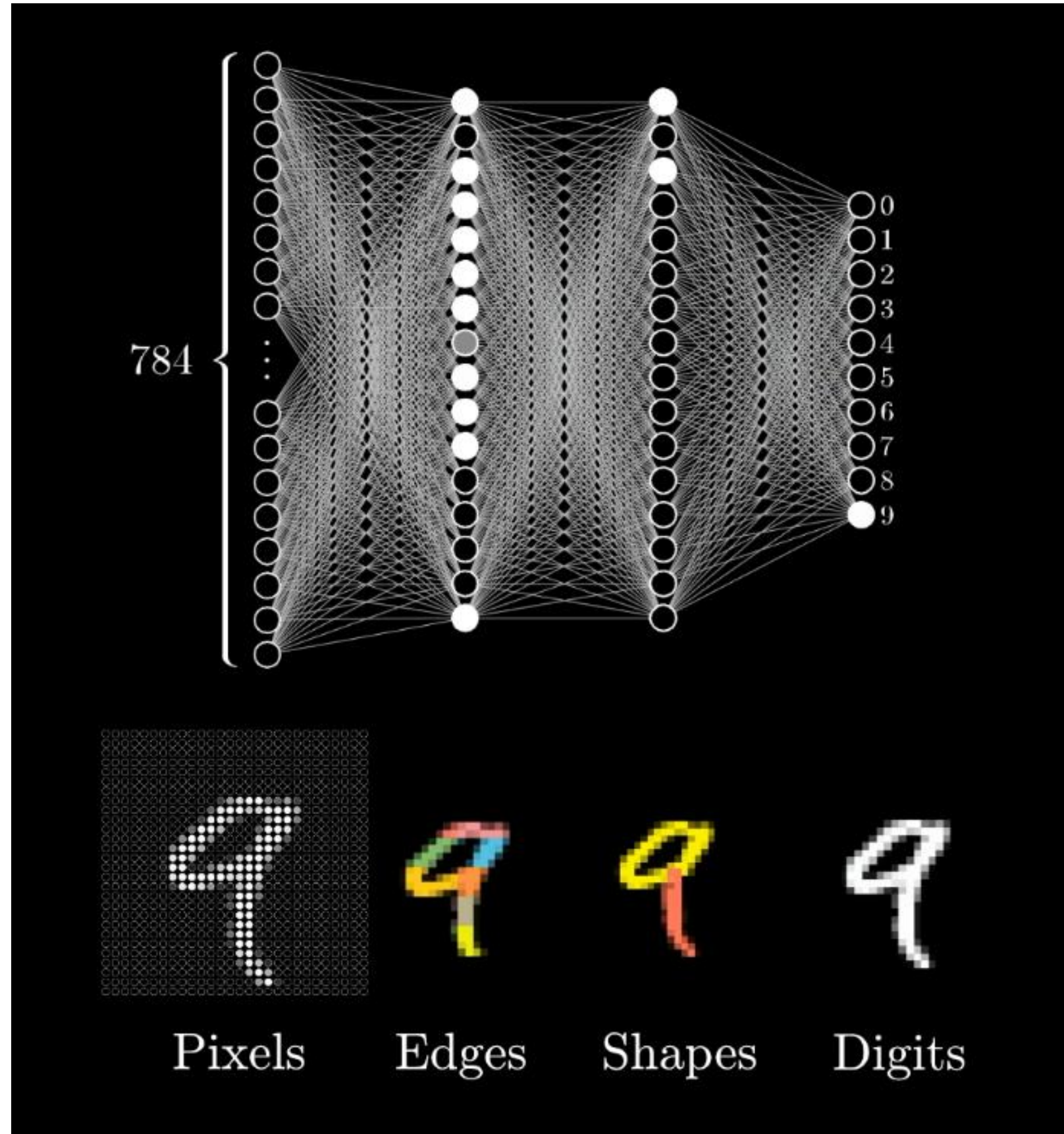


Image Source: 3blue1brown

Layers Break Problems Into Bite-Sized Pieces



Image Source: 3blue1brown

The way information is transmitted from one layer to another

- The objective is to develop a mechanism that is capable of combining pixels into edges, edges into patterns, and patterns into digits
- We desire for a particular neuron in the second layer to detect if the image includes a specific edge.

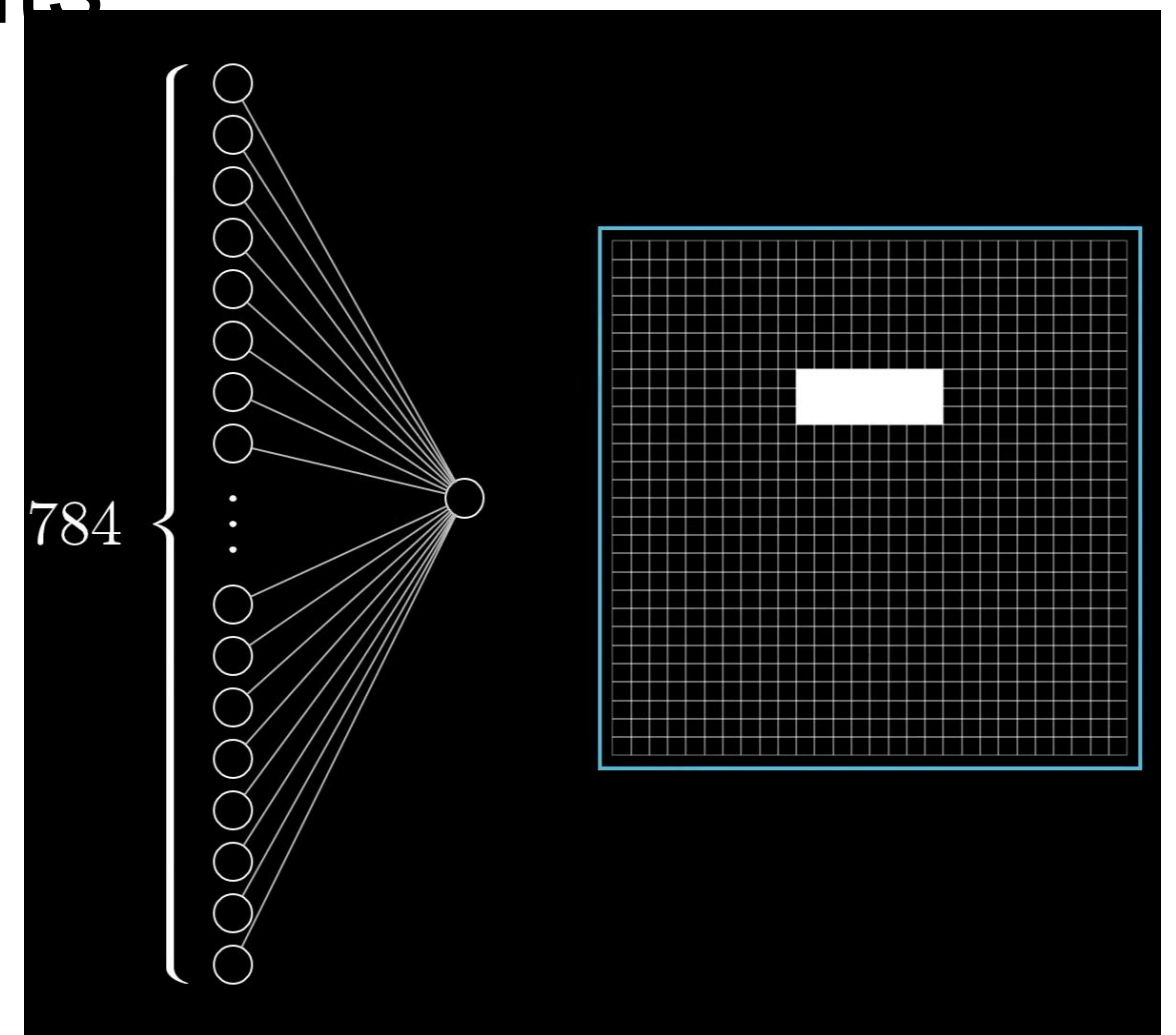


Image Source: 3blue1brown

The way information is transmitted from one layer to another

- Our plan is to give a measure of importance to the connections between our neuron and the neurons in the initial layer. These weights are simply numerical values.
- Each weight represents the level of correlation between a neuron in the first layer and a neuron in the second layer.
- If the neuron is active in the initial layer, a **positive** weight indicates that the following layer's neuron should also be active, while a **negative** weight indicates it should be inactive.

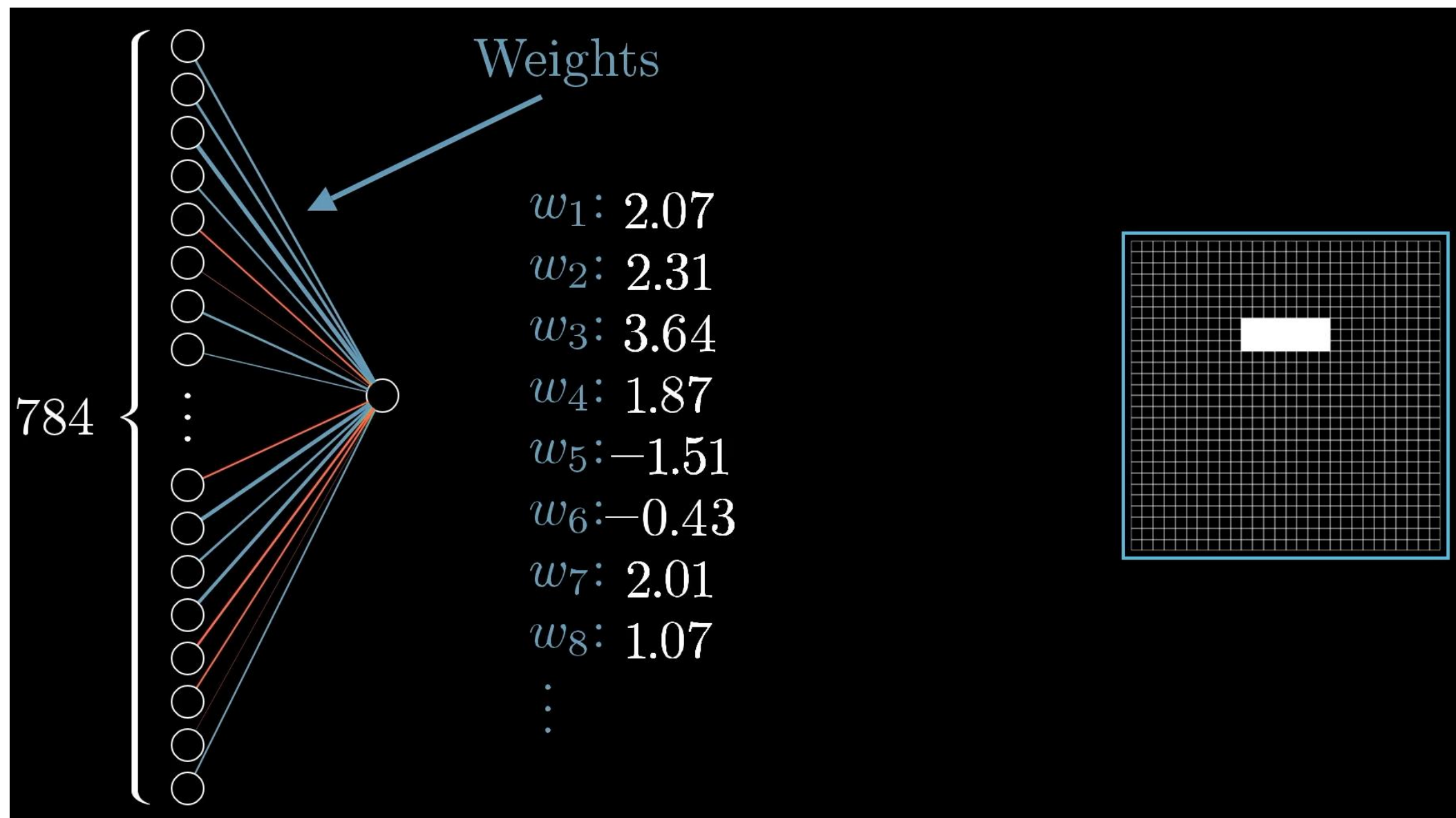


Image Source: 3blue1brown

The way information is transmitted from one layer to another

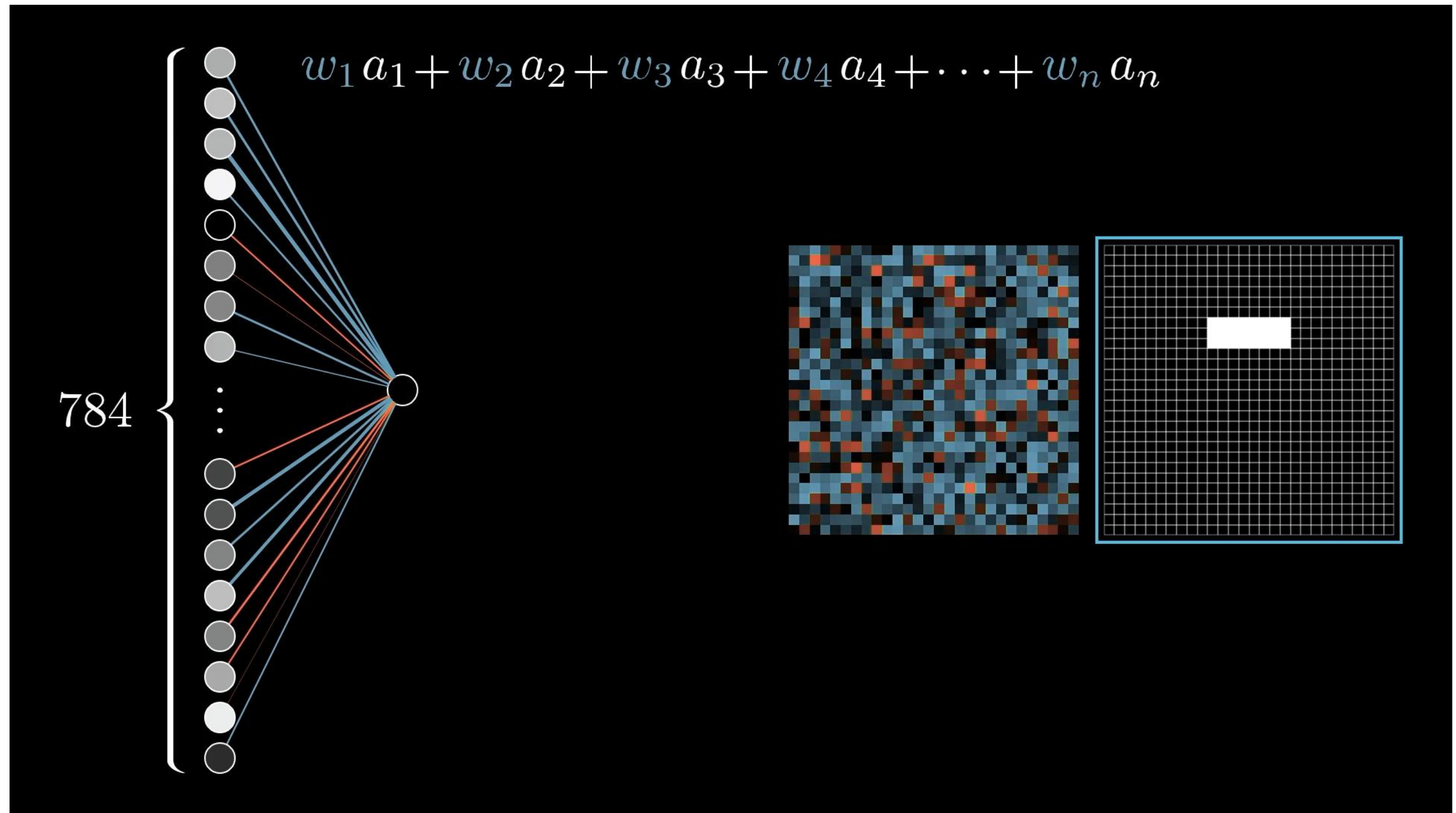
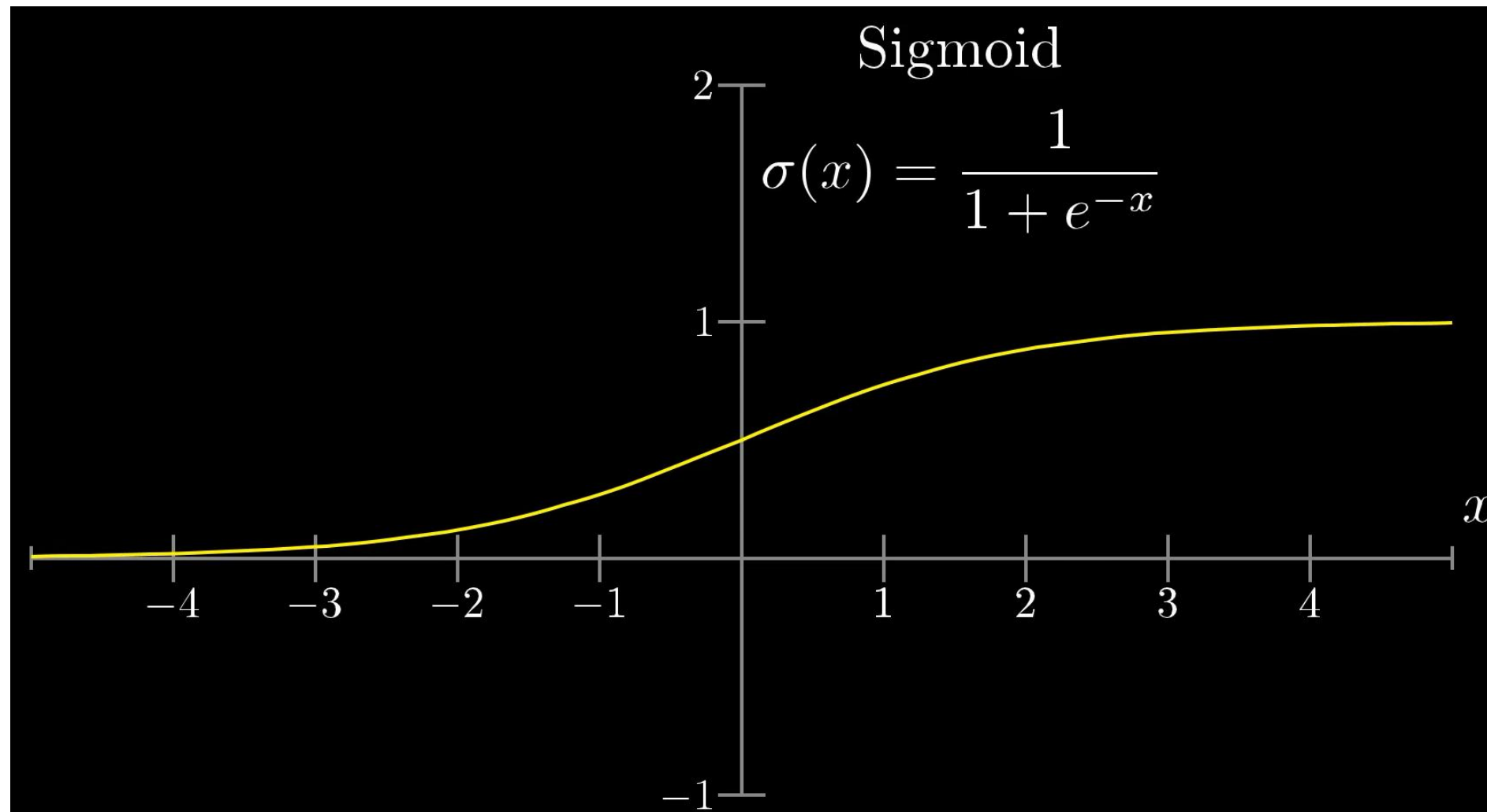


Image Source: 3blue1brown

Sigmoid Squishification



$$\sigma(w_1 a_1 + w_2 a_2 + w_3 a_3 + \cdots + w_n a_n \boxed{-10})$$

“bias”

Only activate meaningfully
when weighted sum > 10

Image Source: 3blue1brown

ReLU

Rectified linear unit

$$\text{ReLU}(a) = \max(0, a)$$

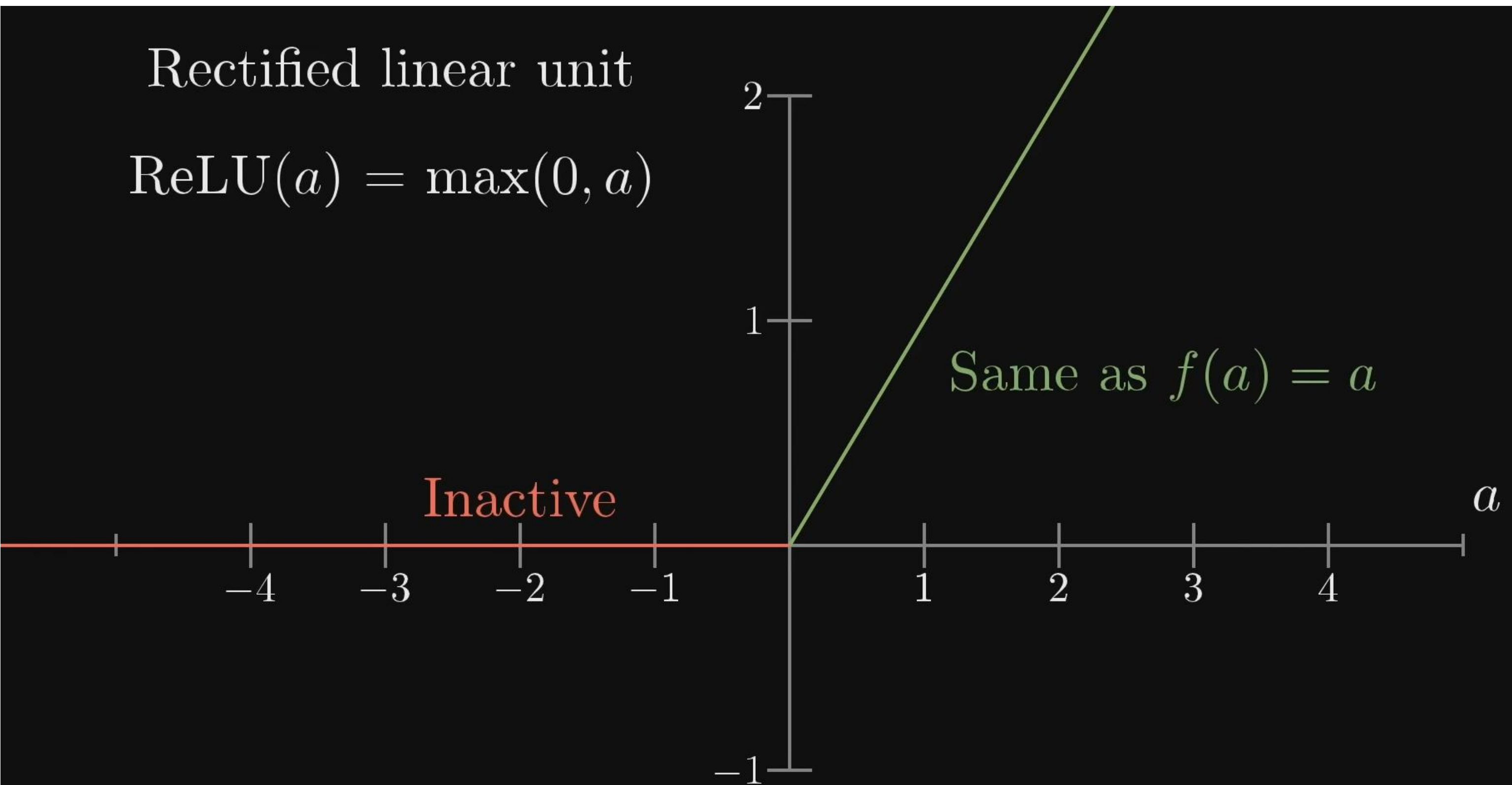


Image Source: 3blue1brown

Complete Neurons

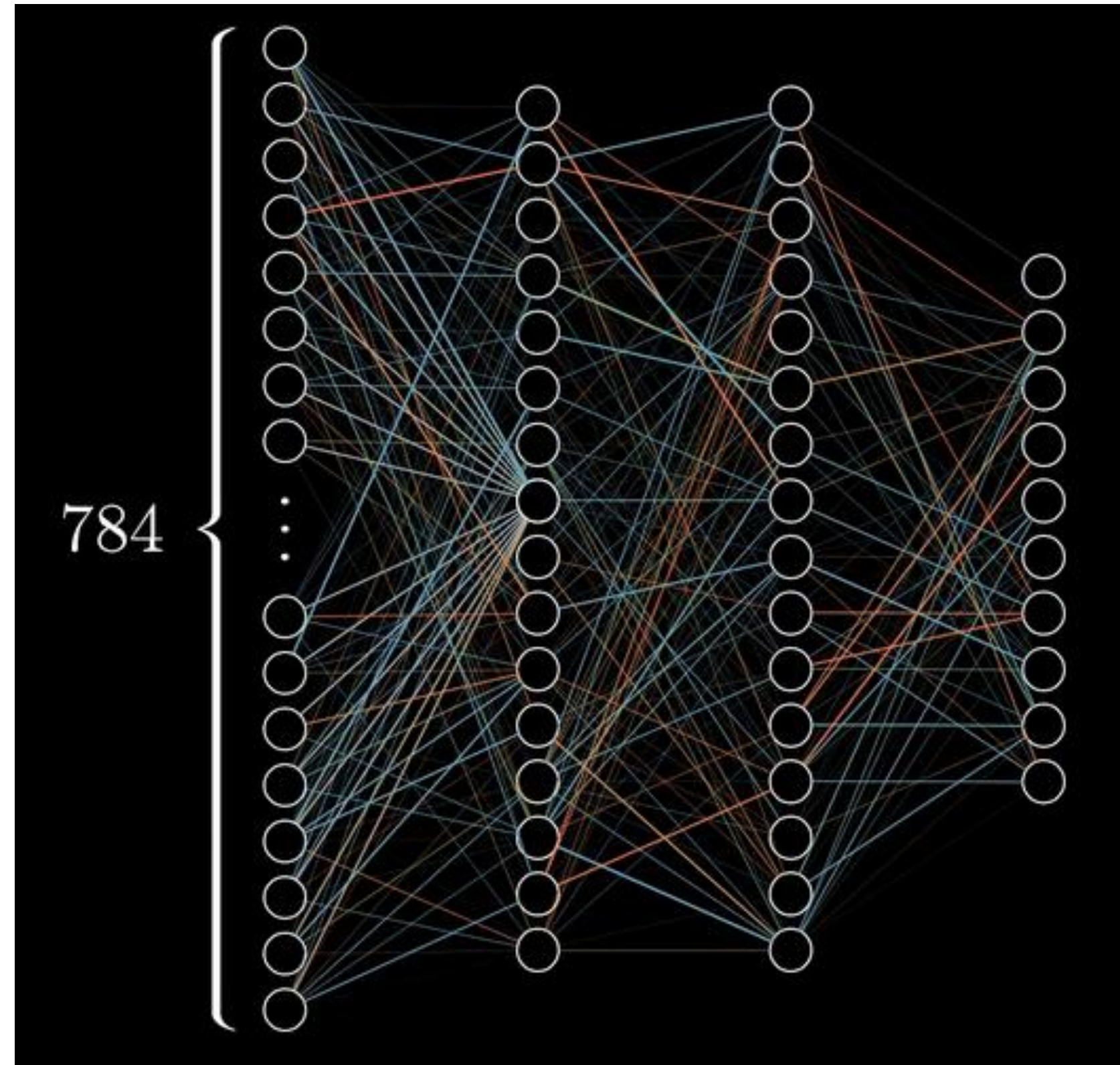


Image Source: 3blue1brown

Feed-Forward Propagation

- Forward propagation is the process where input data passes through the network, layer by layer, to produce an output.
- Steps in Forward Propagation:
 1. **Input Layer:** Raw data is fed into the network.
 2. **Weights and Biases:** Each neuron computes a weighted sum of its inputs, adds a bias, and applies the activation function.
 3. **Hidden Layers:** The output from one layer becomes the input for the next.
 4. **Output Layer:** The final layer produces the prediction.

Feed-Forward Propagation

- Mathematical Representation: For a neuron, the output is given by:

$$z = b + \sum_{i=1}^m x_i \times w_i$$

where:

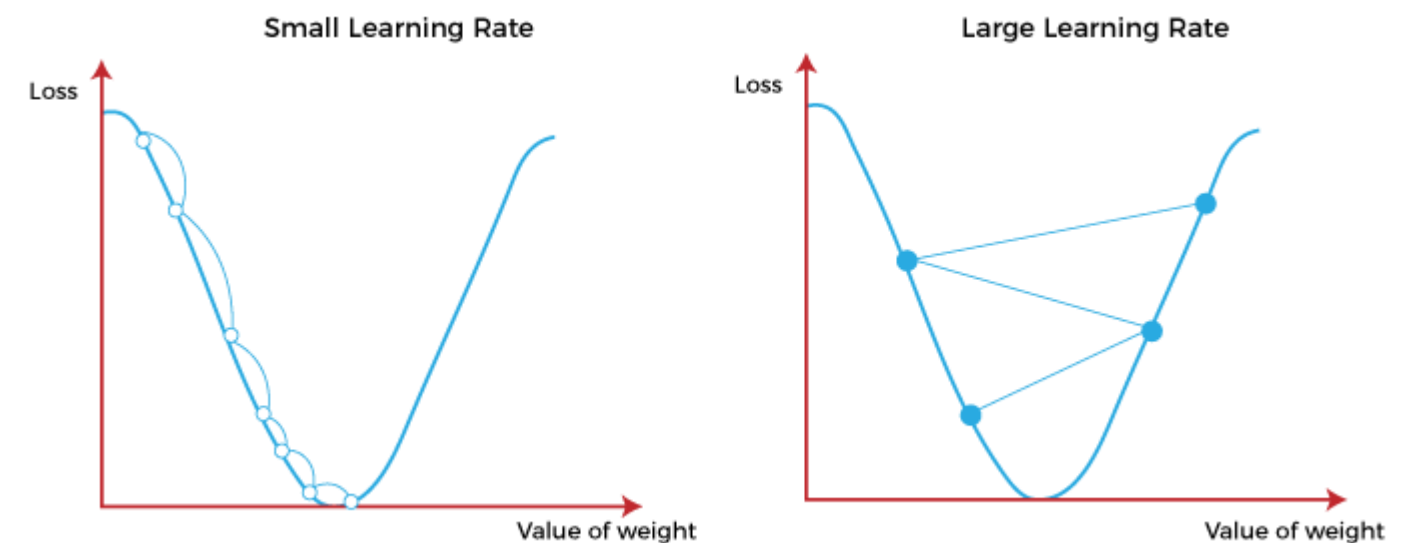
- w_i are the weights,
 - x_i are the input features,
 - b is the bias.
- Example (Digit Classification):
 - Input: An image represented as pixel values (e.g., 28x28 matrix for grayscale).
 - Hidden layers: Identify features like edges, shapes.
 - Output: A probability distribution across 10 digits (0 to 9).

Backpropagation and Learning

- Backpropagation is the algorithm used to update the weights of the neural network based on the error between predicted and actual output.
- Steps in Backpropagation:
 - **Calculate Loss:** Measure the difference between the predicted output and the true label using a loss function (e.g., Mean Squared Error for regression or Cross-Entropy for classification).
 - **Compute Gradients:** Use the chain rule of calculus to compute how much each weight in the network contributed to the error.
 - **Update Weights:** Adjust the weights to reduce the error using a method like **Gradient Descent**.

Backpropagation and Learning

- Gradient Descent:
 - Objective of Gradient Descent is to minimize the loss by updating weights iteratively.
- Formula:
$$w = w - \eta \cdot \nabla L(w),$$
where:
 - η is the learning rate,
 - $\nabla L(w)$ is the gradient of the loss function with respect to the weights.
- Example: In our digit classification model, if the network wrongly predicts a digit, the backpropagation algorithm adjusts the weights to improve future predictions.



Training a Neural Network

Steps in Training:

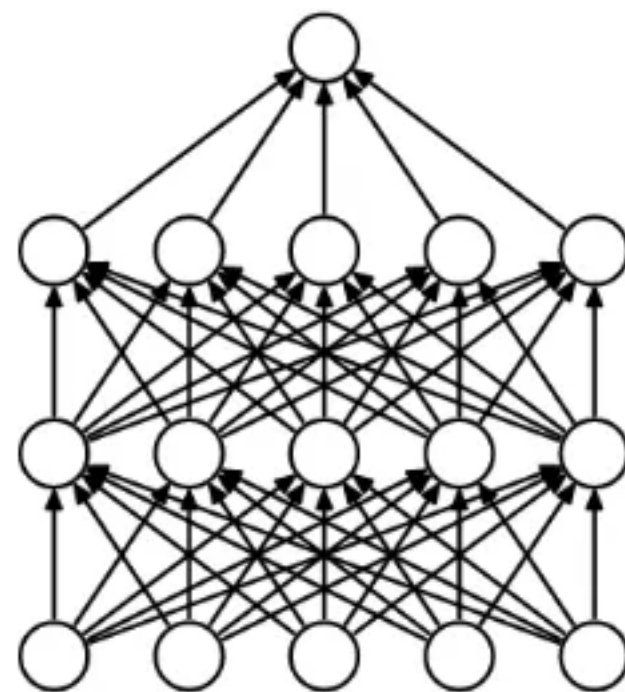
- **Initialize Weights:** Start with small random values for the weights.
- **Forward Propagation:** Pass the input data through the network.
- **Loss Calculation:** Measure the error between predicted and actual output.
- **Backpropagation:** Calculate gradients and adjust the weights.
- **Repeat:** Iterate over many epochs until the model converges.

Important Terms:

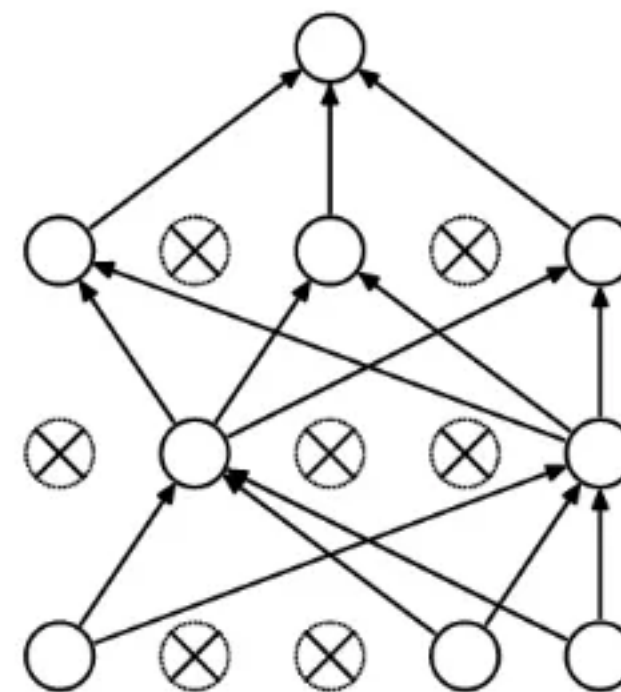
- **Epoch:** One complete pass through the entire training dataset.
- **Batch Size:** The number of samples processed before updating the model's parameters.
- **Learning Rate:** Controls how much the weights are updated after each step.

Dropout layer

- The term “dropout” refers to dropping out the nodes (input and hidden layer) in a neural network (as seen in Figure). All the forward and backwards connections with a dropped node are temporarily removed, thus creating a new network architecture out of the parent network. The nodes are dropped by a dropout probability of p .



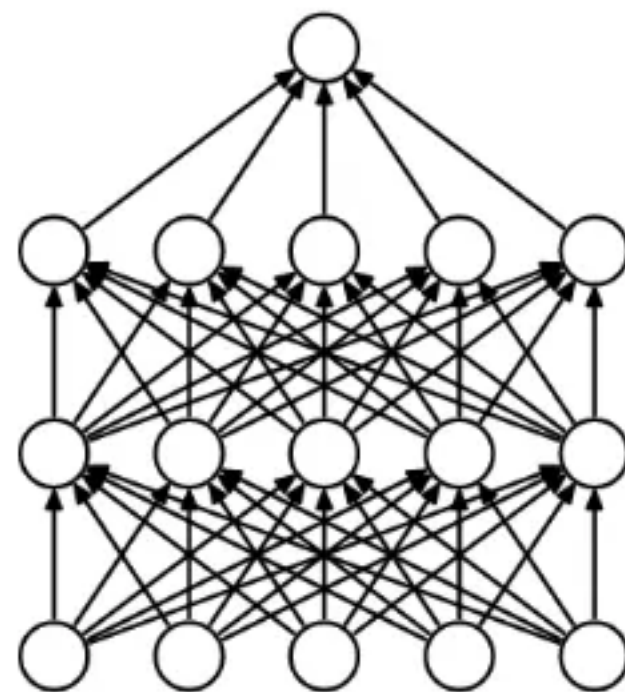
(a) Standard Neural Net



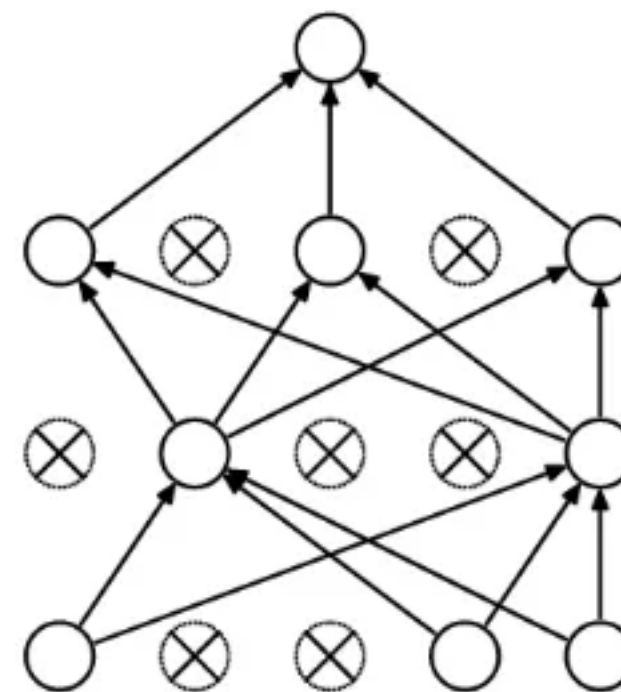
(b) After applying dropout.

Dropout layer

- Let's try to understand with a given input $x: \{1, 2, 3, 4, 5\}$ to the fully connected layer. We have a dropout layer with probability $p = 0.2$ (or keep probability = 0.8). During the forward propagation (training) from the input x , 20% of the nodes would be dropped, i.e. the x could become $\{1, 0, 3, 4, 5\}$ or $\{1, 2, 0, 4, 5\}$ and so on. Similarly, it applied to the hidden layers.
- For instance, if the hidden layers have 1000 neurons (nodes) and a dropout is applied with drop probability = 0.5, then 500 neurons would be randomly dropped in every iteration (batch).



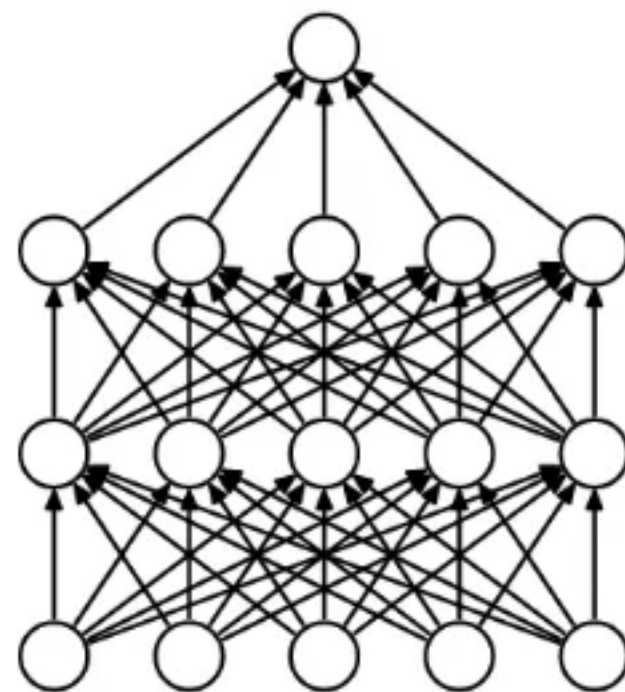
(a) Standard Neural Net



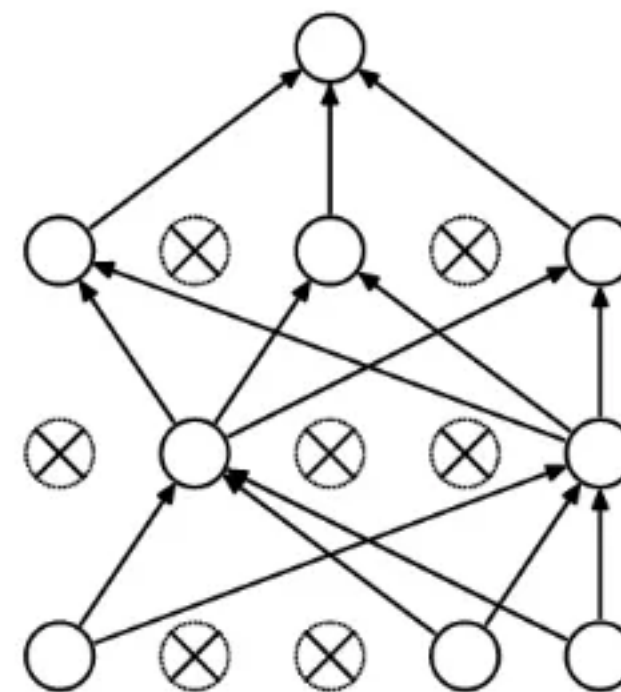
(b) After applying dropout.

Dropout layer

- Generally, for the input layers, the keep probability, i.e. 1- drop probability, is closer to 1, 0.8 being the best as suggested. For the hidden layers, the greater the drop probability more sparse the model, where 0.5 is the most optimized keep probability, that states dropping 50% of the nodes.



(a) Standard Neural Net



(b) After applying dropout.

How does it solve the Overfitting problem?

- In the overfitting problem, the model learns the statistical noise. To be precise, the main motive of training is to decrease the loss function, given all the units (neurons). So in overfitting, a unit may change in a way that fixes up the mistakes of the other units. This leads to complex co-adaptations, which in turn leads to the overfitting problem because this complex co-adaptation fails to generalise on the unseen dataset.
- Now, if we use dropout, it prevents these units to fix up the mistake of other units, thus preventing co-adaptation, as in every iteration the presence of a unit is highly unreliable. So by randomly dropping a few units (nodes), it forces the layers to take more or less responsibility for the input by taking a probabilistic approach.
- This ensures that the model is getting generalised and hence reducing the overfitting problem.

Convolution Neural Networks (CNN)

- CNNs are a specialized type of neural network that are particularly good at processing and analyzing image data.
- They are inspired by the visual cortex of the human brain, which is responsible for recognizing patterns and objects in images.

How do convolutional neural networks work?

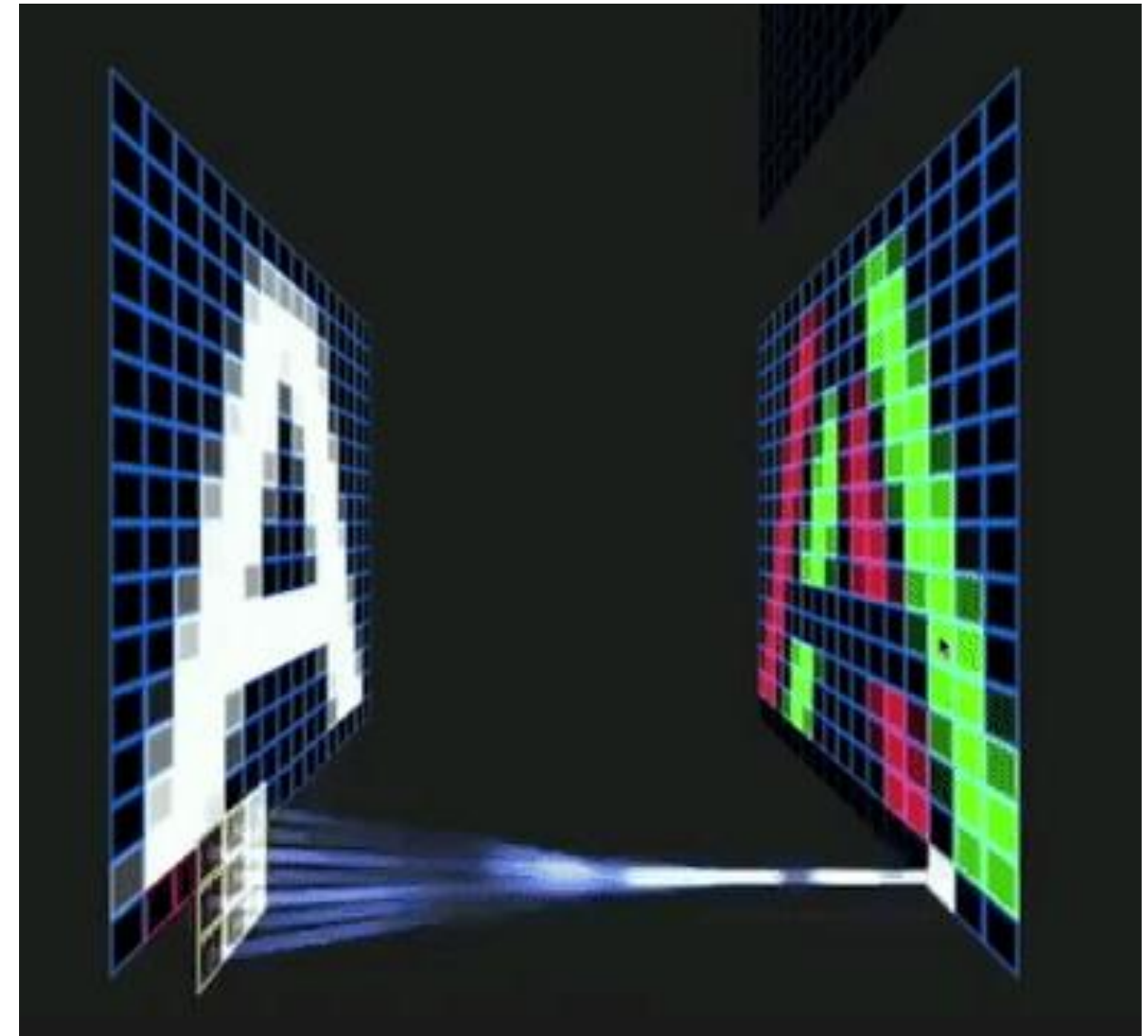
Convolutional neural networks are distinguished from other neural networks by their superior performance with image, speech, or audio signal inputs. They have three main types of layers, which are:

- **1.1 Convolutional layer**
- **1.2 Pooling layer**
- **1.3 Fully-connected (FC) layer**

The convolutional layer is the first layer of a convolutional network. While convolutional layers can be followed by additional convolutional layers or pooling layers, the fully-connected layer is the final layer. With each layer, the CNN increases in its complexity, identifying greater portions of the image. Earlier layers focus on simple features, such as colors and edges. As the image data progresses through the layers of the CNN, it starts to recognize larger elements or shapes of the object until it finally identifies the intended object.

1.1 Convolutional layer

- This is the core operation in a CNN.
- It involves applying a filter, or kernel, to the input image.
- The filter slides across the image, element-wise multiplying the values in the filter with the corresponding values in the image.
- The result is a feature map.

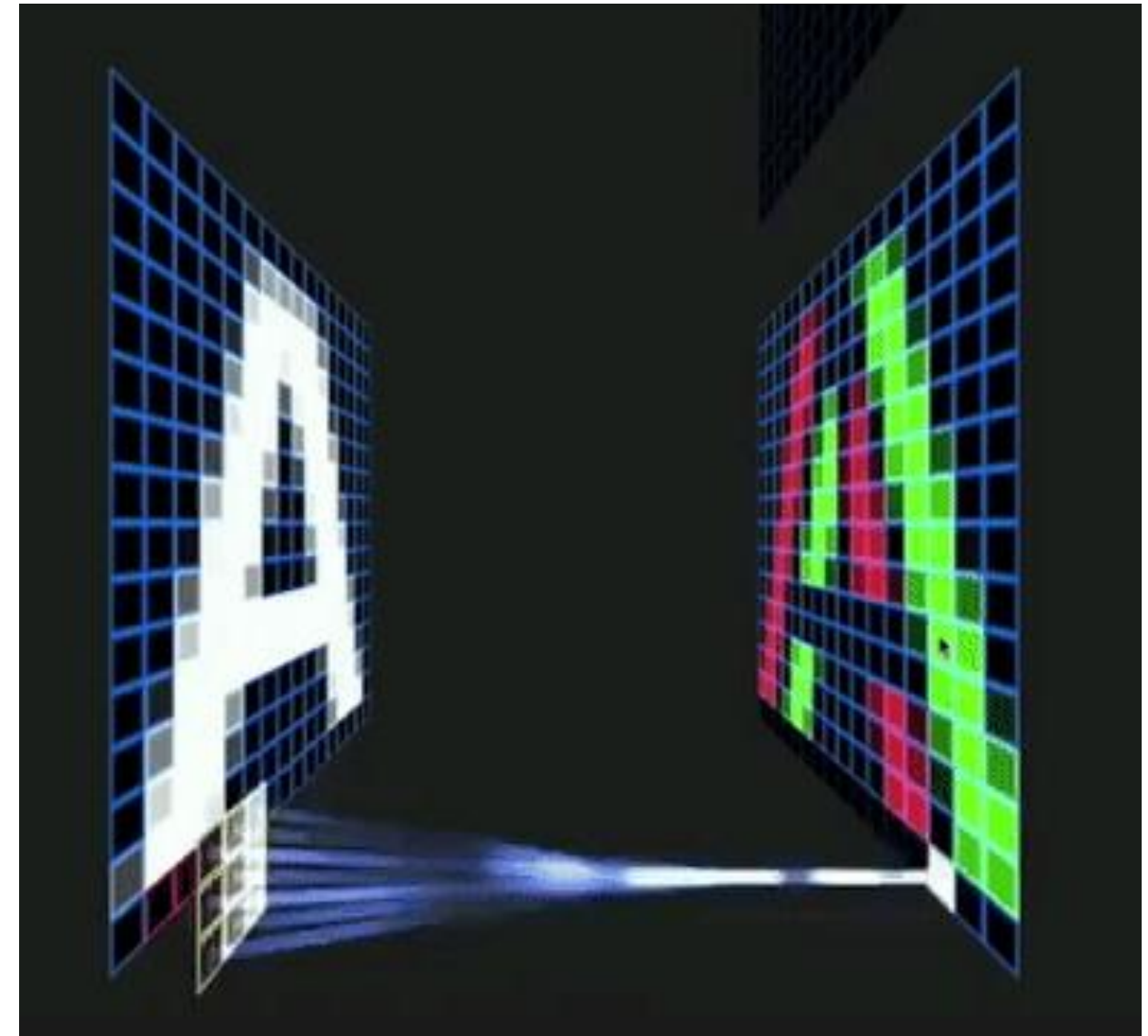


1.1 Convolutional layer

There are three hyperparameters which affect the volume size of the output that need to be set before the training of the neural network begins. These include:

1. The **number of filters** affects the depth of the output. For example, three distinct filters would yield three different feature maps, creating a depth of three.
 2. **Stride** is the distance, or number of pixels, that the kernel moves over the input matrix. While stride values of two or greater is rare, a larger stride yields a smaller output.
 3. **Zero-padding** is usually used when the filters do not fit the input image. This sets all elements that fall outside of the input matrix to zero, producing a larger or equally sized output.
- There are three types of padding:

1. **Valid padding**
2. **Same padding**
3. **Full padding**



$$I = \begin{bmatrix} 1, & 2, & 0, & 1, \\ 3, & 1, & 2, & 2, \\ 0, & 1, & 3, & 1, \\ 2, & 2, & 2, & 0 \end{bmatrix}$$

$$K1 = \begin{bmatrix} 1, & 0, & -1, \\ 1, & 0, & -1, \\ 1, & 0, & -1 \end{bmatrix}$$

$$K2 = \begin{bmatrix} 0, & 1, & 0, \\ 0, & 1, & 0, \\ 0, & 1, & 0 \end{bmatrix}$$

$$H_{out} = (H - k + 2p/s) + 1 = (4 - 3 + 0/1) + 1 = 2, \text{Wo}$$

$$ut = 2$$

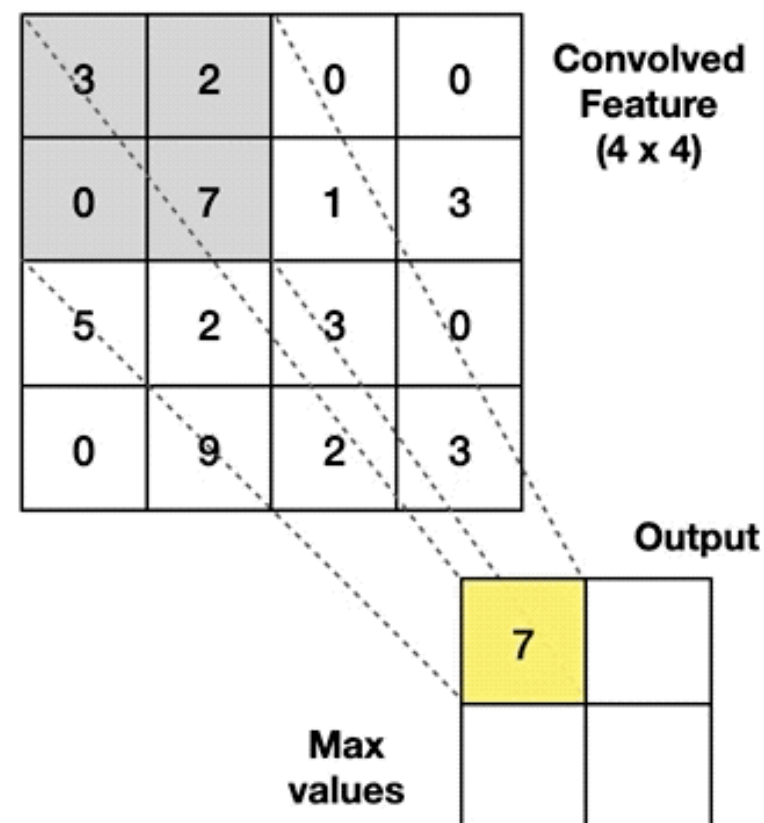
1.2 Pooling layer

- This operation reduces the dimensionality of the feature maps.
- It is typically done to reduce computational cost and prevent overfitting. Common pooling techniques include max pooling and average pooling.

Max Pooling

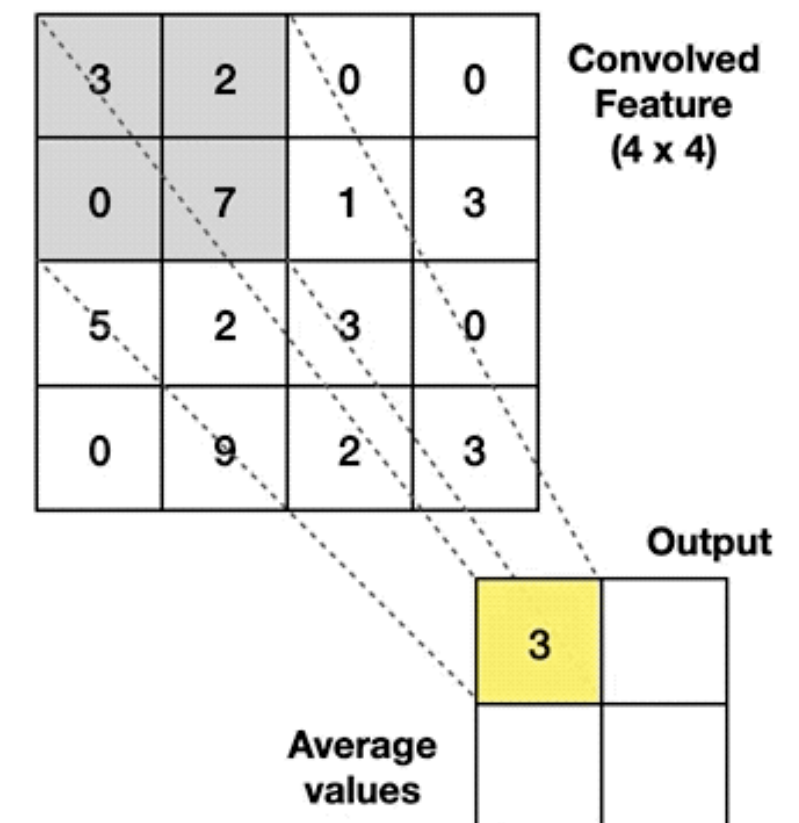
Take the **highest** value from the area covered by the kernel

Example: Kernel of size 2 x 2; stride=(2,2)



Average Pooling

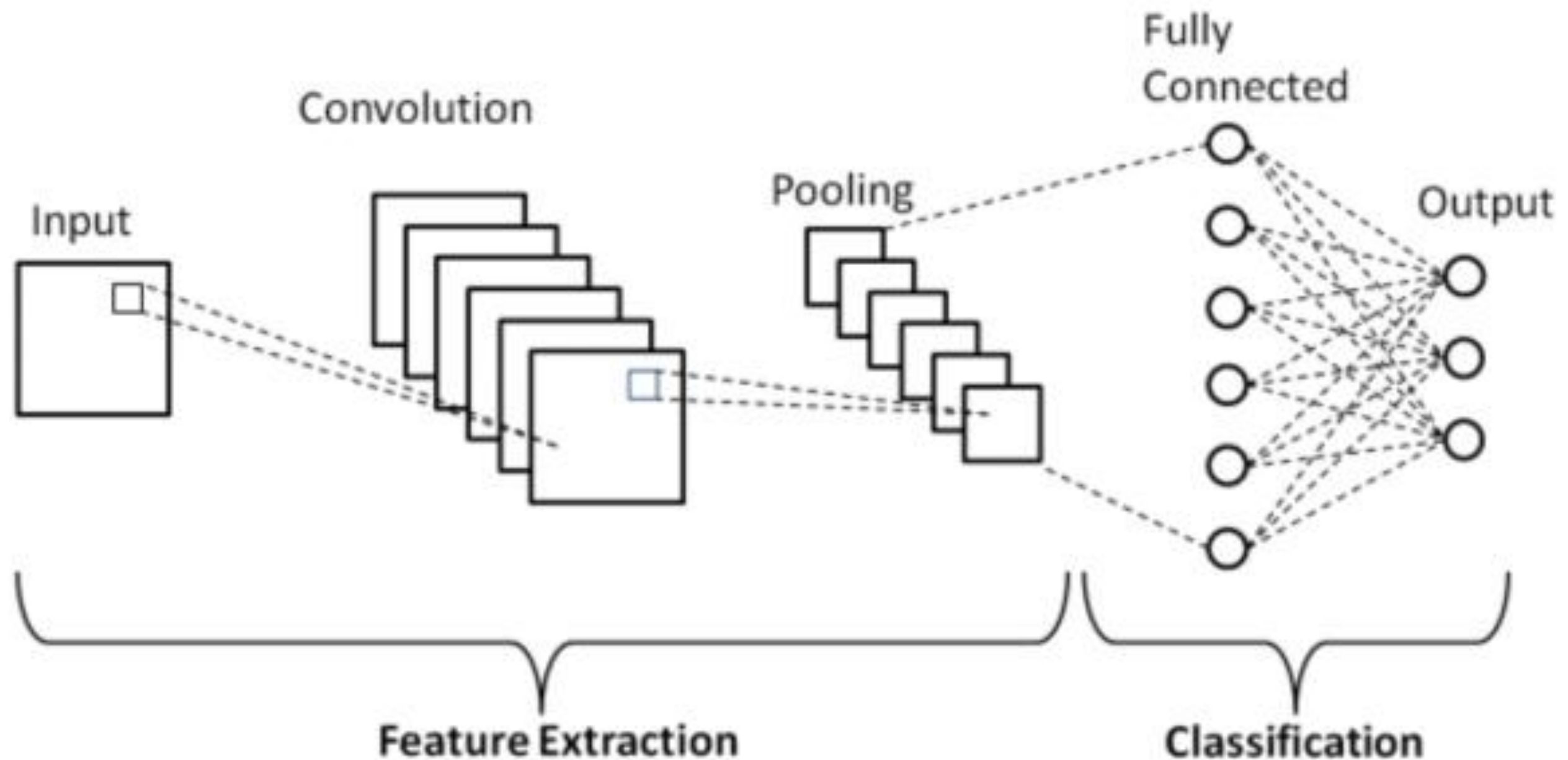
Calculate the **average** value from the area covered by the kernel



1.3 Fully-connected (Dense) layer

- After convolution and pooling, the feature maps are flattened into a one-dimensional vector, which is then fed into a fully connected layer.
- This layer is similar to a traditional neural network and is used to classify the input image.

Convolution Neural Network



Types of Convolution Neural Network

- LeNet-5
- AlexNet
- VGGNet
- GoogLeNet
- ResNet
- ZFNet

Application of Convolution Neural Network

- Image Classification: Identifying objects in images (e.g., cats vs. dogs, cars vs. bikes)
- Object Detection: Locating and identifying objects within an image (e.g., faces, license plates)
- Image Segmentation: Pixel-wise labeling of images (e.g., separating foreground objects from background)
- Medical Image Analysis: Diagnosing diseases based on medical images (e.g., X-rays, MRIs)

Hands on program on MNIST dataset using CNN

- [MNIST-Dataset-DeepLearning.ipynb](#)

Recurrent Neural Networks

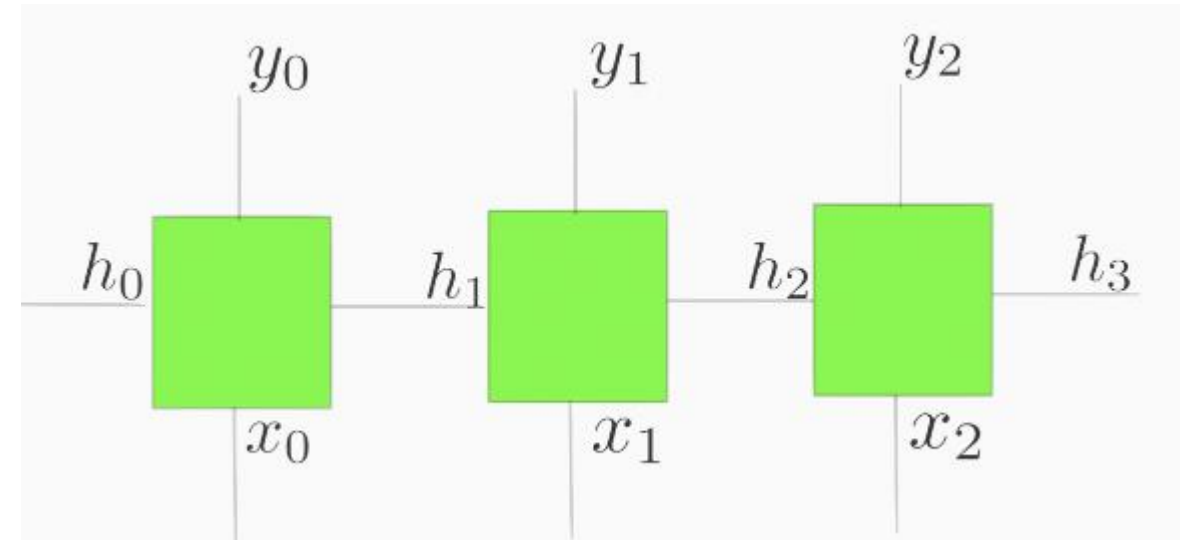
- A recurrent neural network, or RNN, is a deep neural network trained on **sequential or time series data** to create a machine learning model can make sequential predictions or conclusions based on sequential inputs.
- An RNN might be used to predict daily flood levels based on past daily flood, tide and meteorological data.
- But RNNs can also be used to solve ordinal or temporal problems such as language translation, natural language processing (NLP), speech recognition, and image captioning.
- RNNs are incorporated into popular applications such as Siri, voice search, and Google Translate.

How RNNs work

- Like feedforward and convolutional neural networks (CNNs), recurrent neural networks utilize training data to learn.
- They are distinguished by their “memory” as they take information from prior inputs to influence the current input and output.
- While traditional deep neural networks assume that inputs and outputs are independent of each other, the output of recurrent neural networks depend on the prior elements within the sequence.

How RNNs work

- Let's take an idiom, such as “feeling under the weather”, which is commonly used when someone is ill, to aid us in the explanation of RNNs.
- In order for the idiom to make sense, it needs to be expressed in that specific order.
- As a result, recurrent networks need to account for the position of each word in the idiom and they use that information to predict the next word in the sequence.



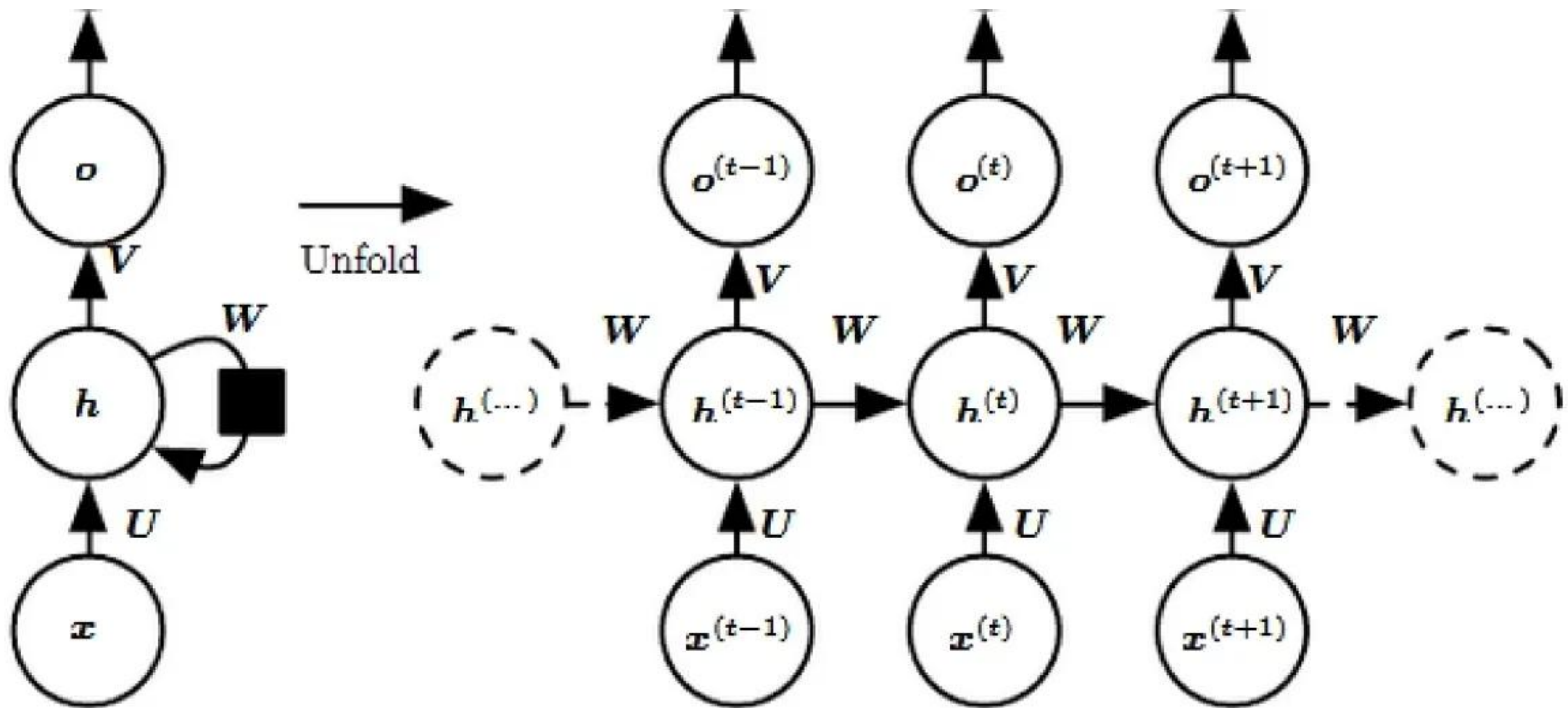
How RNNs work

- Another distinguishing characteristic of recurrent networks is that they share parameters across each layer of the network.
- While feedforward networks have different weights across each node, recurrent neural networks share the same weight parameter within each layer of the network.
- That said, these weights are still adjusted in the through the processes of backpropagation and gradient descent to facilitate reinforcement learning

How RNNs work

- Recurrent neural networks leverage **backpropagation through time** (BPTT) algorithms to determine the gradients, which is slightly different from traditional backpropagation as it is specific to sequence data.
- The principles of BPTT are the same as traditional backpropagation, where the model trains itself by calculating errors from its output layer to its input layer.
- These calculations allow us to adjust and fit the parameters of the model appropriately. BPTT differs from the traditional approach in that BPTT sums errors at each time step whereas feedforward networks do not need to sum errors as they do not share parameters across each layer.

How RNNs work

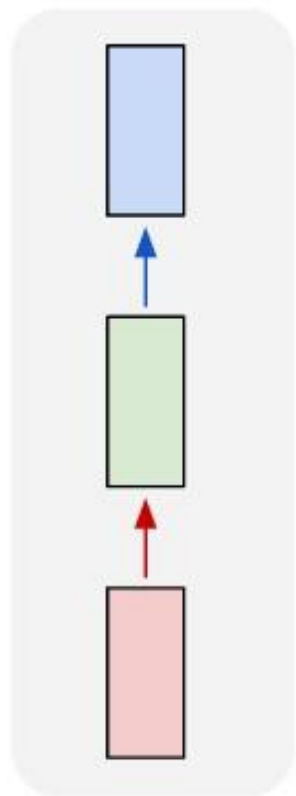


How RNNs work

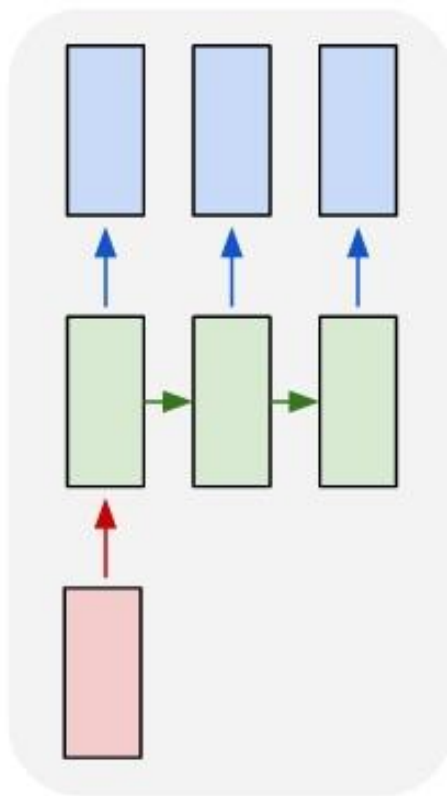
- The bottom layer x is the **input layer**. The model inputs are denoted with $x(t)$ where t is the time step. $x(t)$ can be a word and its place in a sentence or the price of a stock on a specific day.
- The middle layer h consists of multiple **hidden layers** with their own activation functions. $h(t)$ denotes the hidden state of the network at time step t . Hidden states act as “memory” of the model and they are calculated based on the current input $x(t)$ and previous state $h(t-1)$.
- The top layer o is the **output layer**. $o(t)$ represents the output of the model at time step t . The current output is determined by current input, $x(t)$, and the current hidden state, $h(t)$, which depends on previous hidden states. This is the distinguishing feature of RNNs since current output depends on both current input and previous inputs.
- Parameters (U , V , W) represent the weights between inputs, hidden states, and outputs. They control the extent of influence between these.

Types of Recurrent Neural Networks

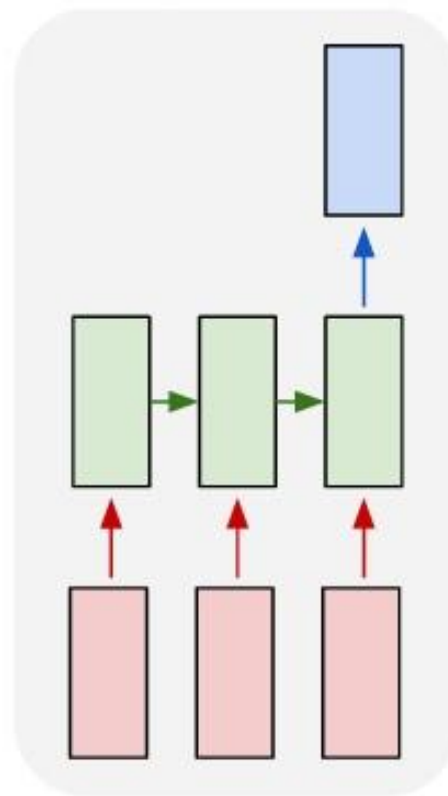
one to one



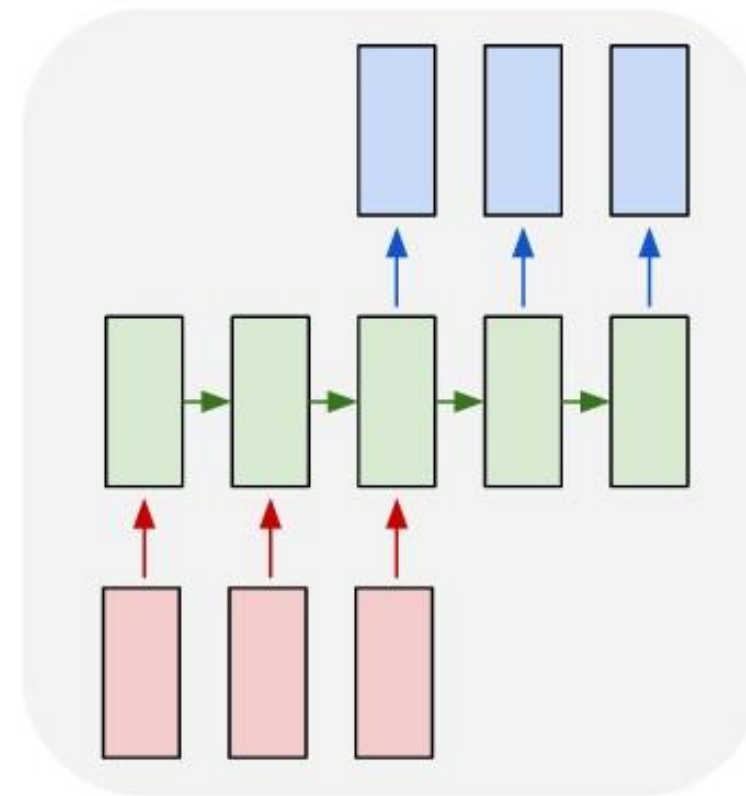
one to many



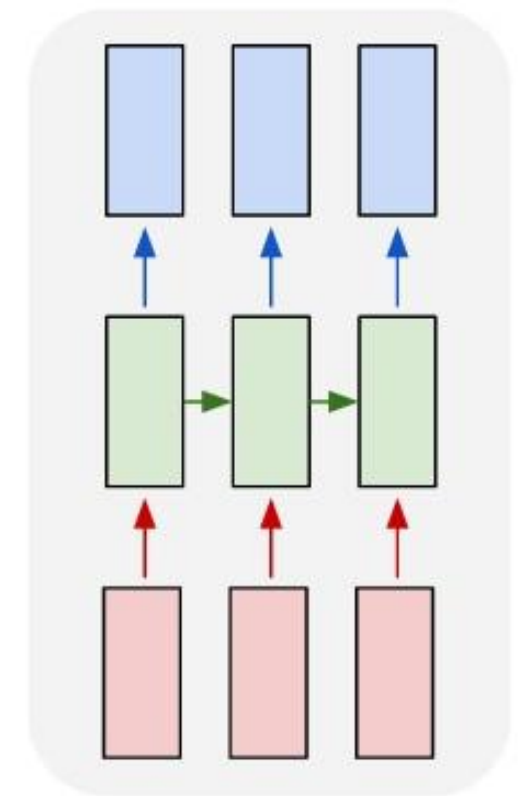
many to one



many to many



many to many



Applications of Recurrent Neural Networks

RNNs and their variants LSTMs and GRUs are used in problems where the input data is sequential by nature.

Applications with sequential data include:

- Time series analysis such as stock price forecasting
- Machine translation
- Speech recognition: Google's voice search uses LSTM.
- Image captioning
- Sentiment analysis

What are the challenges with RNNs?

- RNNs leverage the backpropagation through time (BPTT) algorithm where calculations depend on previous steps. However, if the value of gradient is too small in a step during backpropagation, the value would be even smaller in the next step. This causes gradients to decrease exponentially to a point where the model stops learning.
- This is called the **vanishing gradient problem** and causes RNNs to have a short-term memory: earlier outputs have increasingly small or no effect on the current output. This can be seen in the “What time is it?” problem above where colors for earlier words shrink as the model moves through the sentence.
- The vanishing gradient problem can be remedied by different RNN variants. Two of them are called **Long Short-Term Memory (LSTM)** and **Gated Recurrent Units (GRU)**. These algorithms use mechanisms called “gates” to control how much and which information to retain and forget.

Long Short-Term Memory (LSTM)

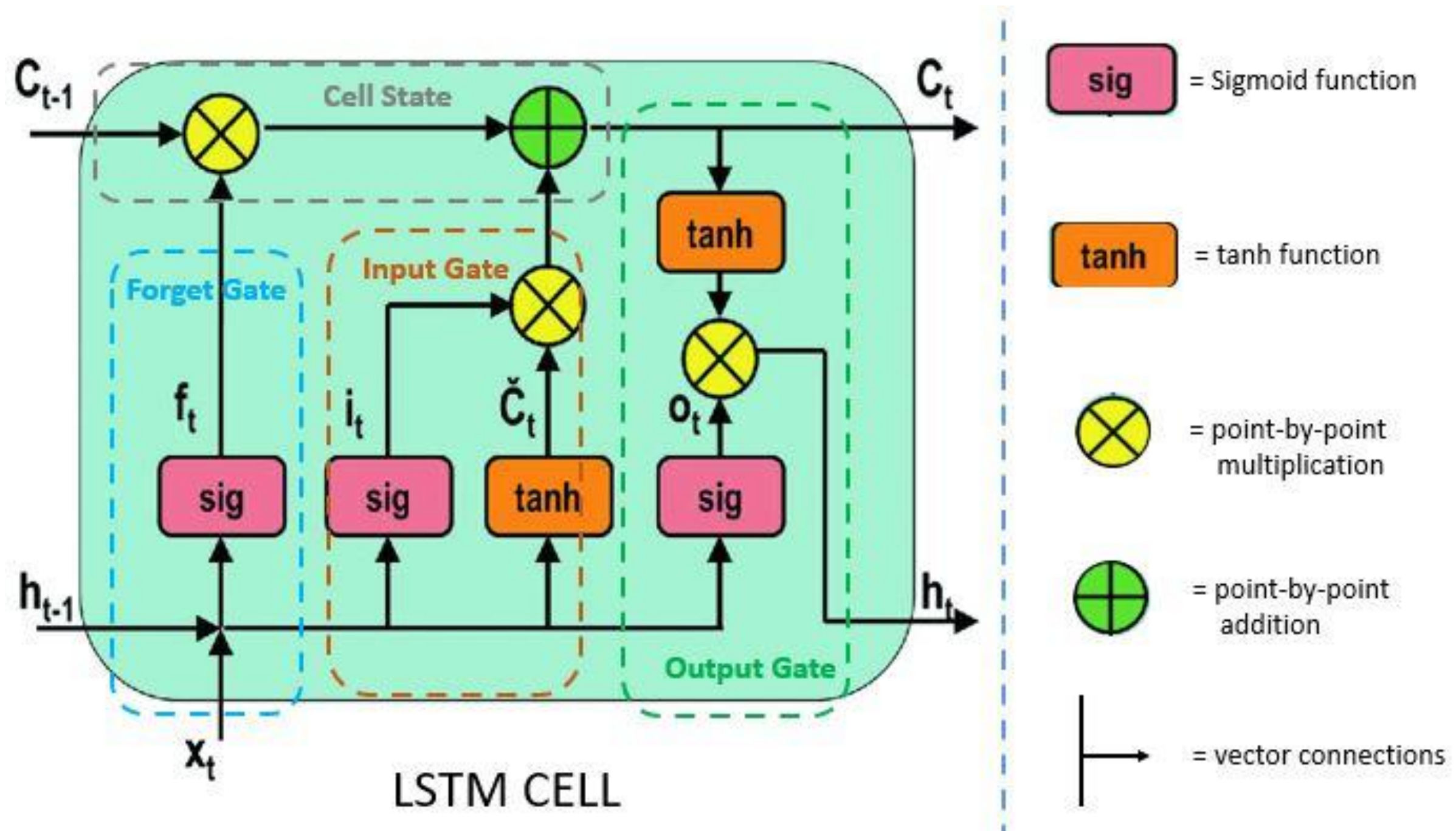
- “LSTMs” – are a special kind of RNN, capable of learning long-term dependencies. They were introduced by Hochreiter & Schmidhuber (1997).
- LSTMs are explicitly designed to avoid the long-term dependency problem. Remembering information for long periods of time is practically their default behavior, not something they struggle to learn!

Long Short-Term Memory (LSTM)

LSTM improves upon RNN in 3 ways:

- It has a separate vector to save long-term dependencies.
- It has multiple operations within a cell.
- It solves the problem of vanishing and exploding gradients.

Long Short-Term Memory (LSTM)



Natural Language Processing (NLP)

- NLP is a field of artificial intelligence that focuses on the interaction between computers and humans through natural language.
- Enable computers to understand, interpret, and generate human language.
- Facilitate communication between humans and machines.
- NLP can be divided into two overlapping subfields:
 - Natural Language Understanding (NLU), which focuses on semantic analysis or determining the intended meaning of text, and
 - Natural Language Generation (NLG), which focuses on text generation by a machine.

Why Does NLP Matter?

- NLP is an integral part of everyday life and becoming more so as language technology is applied to diverse fields like
 - Retailing: for instance, in customer service chatbots, and
 - Medicine: interpreting or summarizing electronic health records.
- Conversational agents such as Amazon's Alexa and Apple's Siri utilize NLP to listen to user queries and find answers.
- The most sophisticated such agents — such as GPT, which was recently opened for commercial applications — can generate sophisticated prose on a wide variety of topics as well as power chatbots that can hold coherent conversations.
- Google uses NLP to improve its search engine results, and social networks like Facebook use it to detect and filter hate speech.

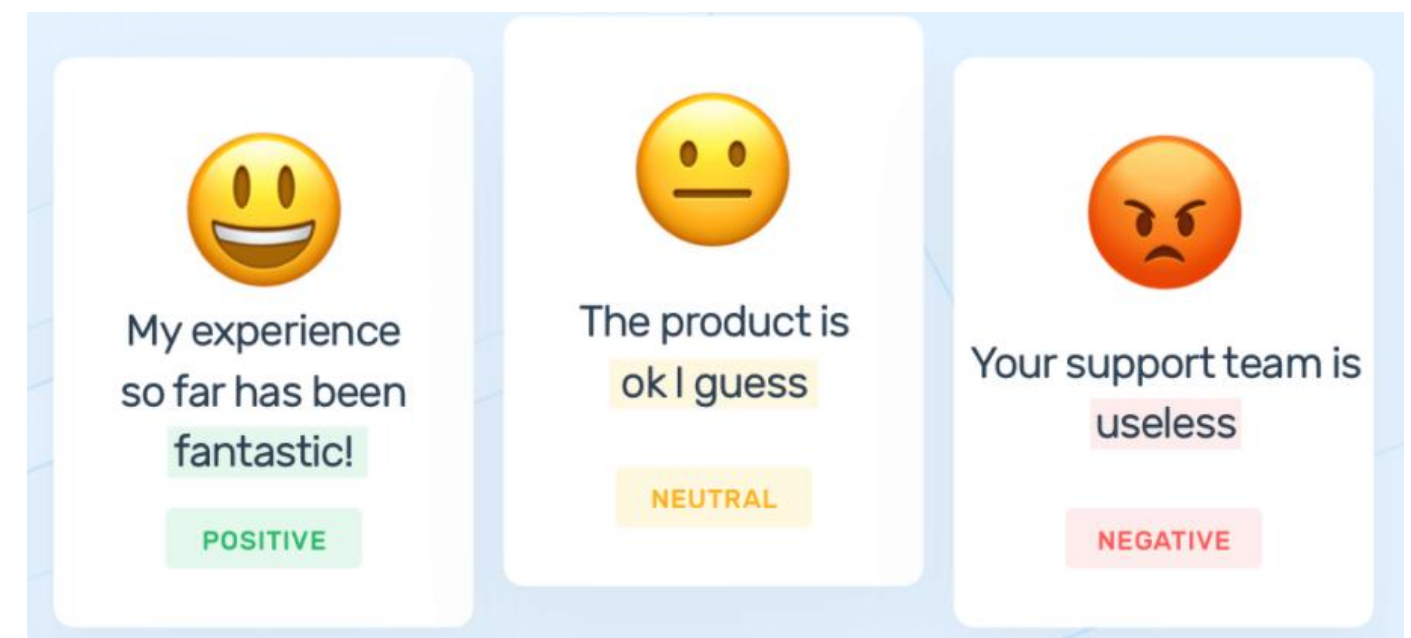
Applications of NLP

1. Sentiment analysis
 - a) Toxicity classification
2. Machine translation
3. Named Entity Recognition
4. Spam Detection
5. Grammatical Error Correction
6. Topic Modeling
7. Text Generation
8. Information retrieval
9. Summarization
10. Question Answering

Applications of NLP

Sentiment analysis

- Sentiment analysis is the process of classifying the emotional intent of text.
- Generally, the input to a sentiment classification model is a piece of text, and the output is the probability that the sentiment expressed is **positive**, **negative**, or **neutral**.
- Typically, this probability is based on either hand-generated features, word n-grams, TF-IDF features, or using deep learning models to capture sequential long- and short-term dependencies.
- Sentiment analysis is used to classify customer reviews on various online platforms as well as for niche applications like identifying signs of mental illness in online comments.



Applications of NLP

Toxicity classification

- Toxicity classification is a branch of sentiment analysis where the aim is not just to classify hostile intent but also to classify categories such as threats, insults, obscenities, and hatred towards certain identities.
- The input to such a model is text, and the output is generally the probability of each class of toxicity.
- Toxicity classification models can be used to moderate and improve online conversations by silencing offensive comments, detecting hate speech, or scanning documents for defamation.

Applications of NLP

Machine translation

- Machine translation automates translation between different languages.
- The input to such a model is text in a specified source language, and the output is the text in a specified target language.
- **Google Translate** is perhaps the most famous mainstream application.
- Such models are used to improve communication between people on social-media platforms such as Facebook or Twitter.
- Effective approaches to machine translation can distinguish between words with similar meanings.
- Some systems also perform language identification; that is, classifying text as being in one language or another.

Applications of NLP

Named Entity Recognition

- Named entity recognition aims to extract entities in a piece of text into predefined categories such as personal names, organizations, locations, and quantities.
- The input to such a model is generally text, and the output is the various named entities along with their start and end positions.
- Named entity recognition is useful in applications such as summarizing news articles and combating disinformation.

Apple^{ORG} today^{DATE} announced the
second^{QUANTITY} generation iPhone SE^{COMM}
a powerful new iPhone^{COMM} featuring
a 4.7-inch^{QUANTITY} Retina HD display.

Applications of NLP

Spam Detection

- Spam detection is a prevalent binary classification problem in NLP, where the purpose is to classify emails as either spam or not.
- Spam detectors take as input an email text along with various other subtexts like title and sender's name.
- They aim to output the probability that the mail is spam.
- Email providers like Gmail use such models to provide a better user experience by detecting unsolicited and unwanted emails and moving them to a designated spam folder.

Applications of NLP

Grammatical Error Correction

- Grammatical error correction models encode grammatical rules to correct the grammar within text.
- This is viewed mainly as a sequence-to-sequence task, where a model is trained on an ungrammatical sentence as input and a correct sentence as output.
- Online grammar checkers like Grammarly and word-processing systems like Microsoft Word use such systems to provide a better writing experience to their customers. Schools also use them to grade student essays.

Applications of NLP

Topic Modeling

- Topic modeling is an unsupervised text mining task that takes a corpus of documents and discovers abstract topics within that corpus.
- The input to a topic model is a collection of documents, and the output is a list of topics that defines words for each topic as well as assignment proportions of each topic in a document.
- Topic modeling is being used commercially to help lawyers find evidence in legal documents.

Applications of NLP

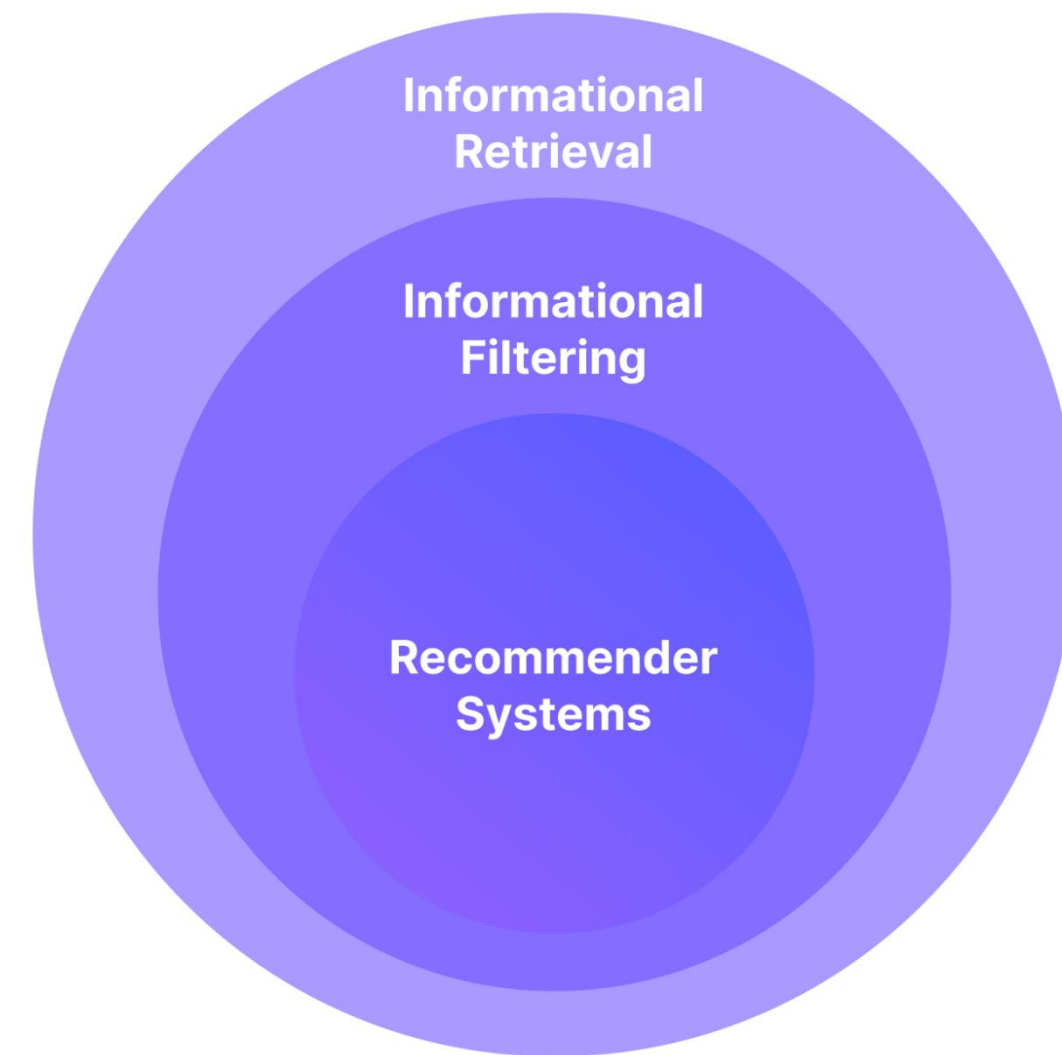
Text Generation

- Text generation, more formally known as natural language generation (NLG), produces text that's similar to human-written text.
- Such models can be fine-tuned to produce text in different genres and formats — including tweets, blogs, and even computer code.
- Text generation has been performed using Markov processes, LSTMs, BERT, GPT-2, Google's LaMDA, and other approaches. It's particularly useful for autocomplete and chatbots.
 - **Autocomplete** predicts what word comes next and autocomplete systems of varying complexity are used in chat applications like WhatsApp. Google uses autocomplete to predict search queries.
 - **Chatbots** automate one side of a conversation while a human conversant generally supplies the other side. They can be divided into the following two categories:
 - Database Query
 - Conversation generation

Applications of NLP

Information retrieval

- Information retrieval finds the documents that are most relevant to a query.
- This is a problem every search and recommendation system faces.
- The goal is not to answer a particular query but to retrieve, from a collection of documents that may be numbered in the millions, a set that is most relevant to the query.
- Document retrieval systems mainly execute two processes: indexing and matching.



Applications of NLP

Summarization

- Summarization is the task of shortening text to highlight the most relevant information.
- Summarization is divided into two method classes:
 - **Extractive summarization** focuses on extracting the most important sentences from a long text and combining these to form a summary.
 - **Abstractive summarization** produces a summary by paraphrasing. This is similar to writing the abstract that includes words and sentences that are not present in the original text.

Applications of NLP

Question Answering

- Question answering deals with answering questions posed by humans in a natural language.
- One of the most notable examples of question answering was Watson, which in 2011 played the television game-show Jeopardy against human champions and won by substantial margins.
- Generally, question-answering tasks come in two flavors:
 - **Multiple choice:** The multiple-choice question problem is composed of a question and a set of possible answers. The learning task is to pick the correct answer.
 - **Open domain:** In open-domain question answering, the model provides answers to questions in natural language without any options provided, often by querying a large number of texts.

Challenges in NLP

- Ambiguity: Natural language is inherently ambiguous due to factors like homonyms, polysemy, and context.
- Complexity:
 - Human language is complex, involving syntax, semantics, pragmatics, and cultural nuances.
- Data Scarcity: Obtaining large, high-quality datasets for training NLP models can be challenging.

Key Concepts in NLP

- 1) Text Preprocessing
 - a. Tokenization
 - b. Stopword Removal
 - c. Stemming and Lemmatization
 - d. Text Normalization
 - e. Sentence Segmentation
- 2) Text Representation (Feature Engineering)
 - a. Bag of Words (BoW)
 - b. Term Frequency-Inverse Document Frequency (TF-IDF)
 - c. Word Embeddings
- 3) Part-of-Speech (PoS) Tagging
- 4) Language Models

Tokenization

- Dividing text into smaller units, such as words or sentences.

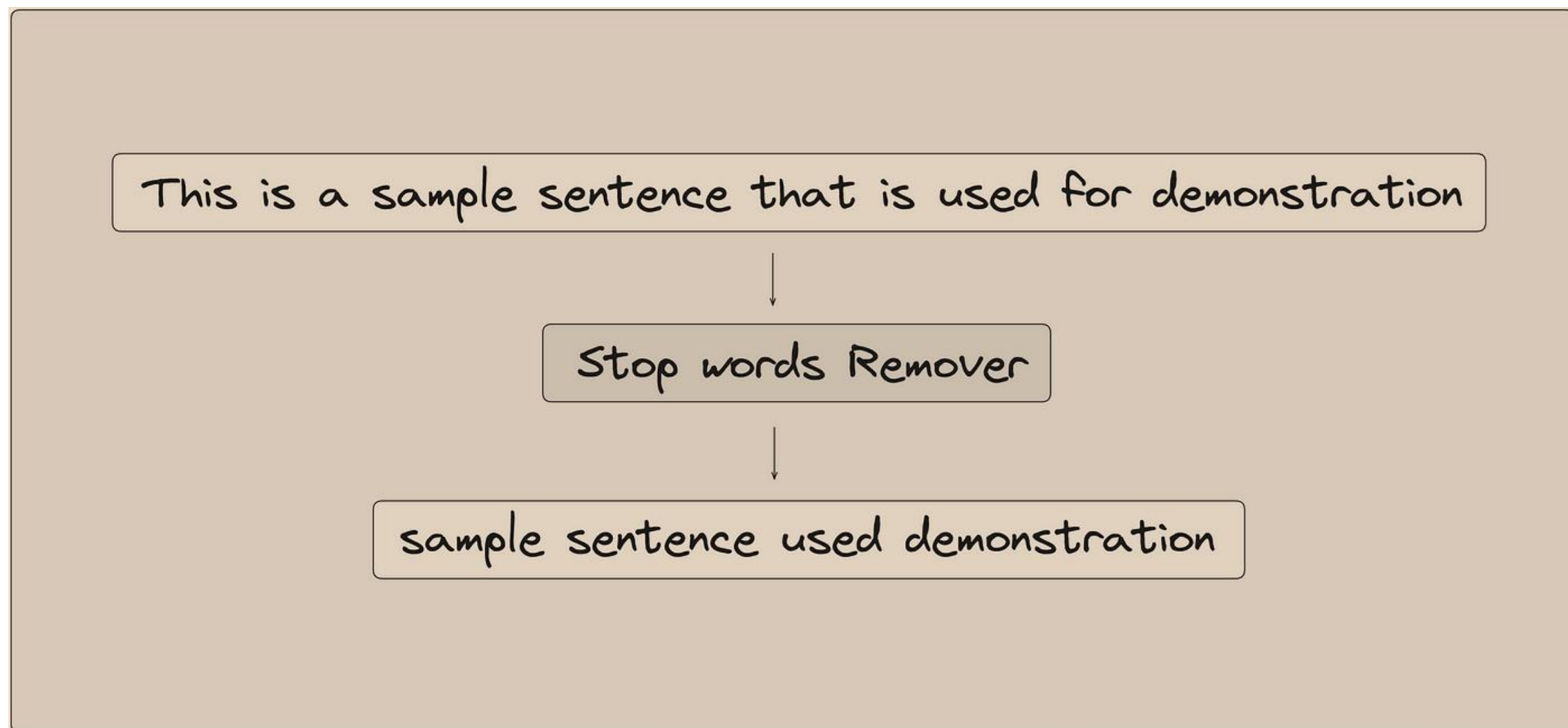
Natural Language Processing



['Natural', 'Language', 'Processing']

Stopword Removal

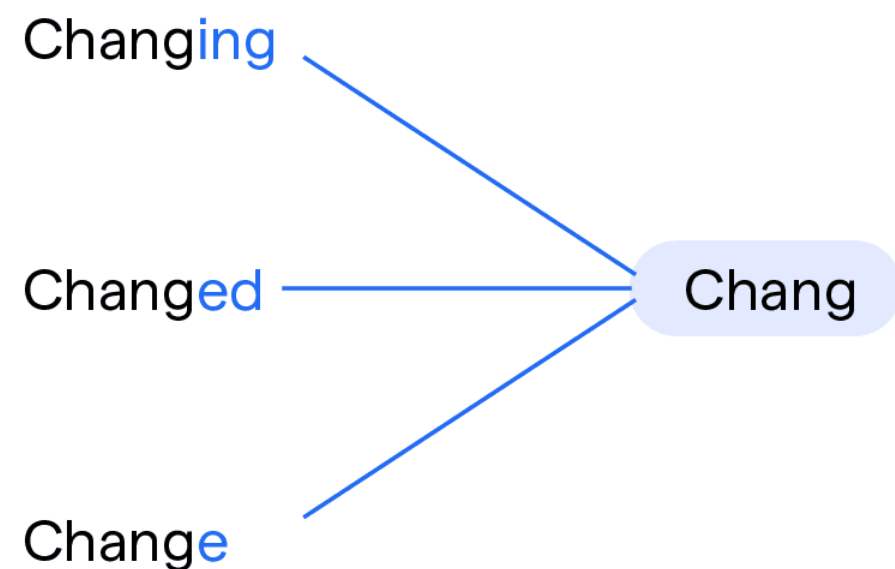
- Removing common words (like “and”, “the”, “is”) that may not carry significant meaning.



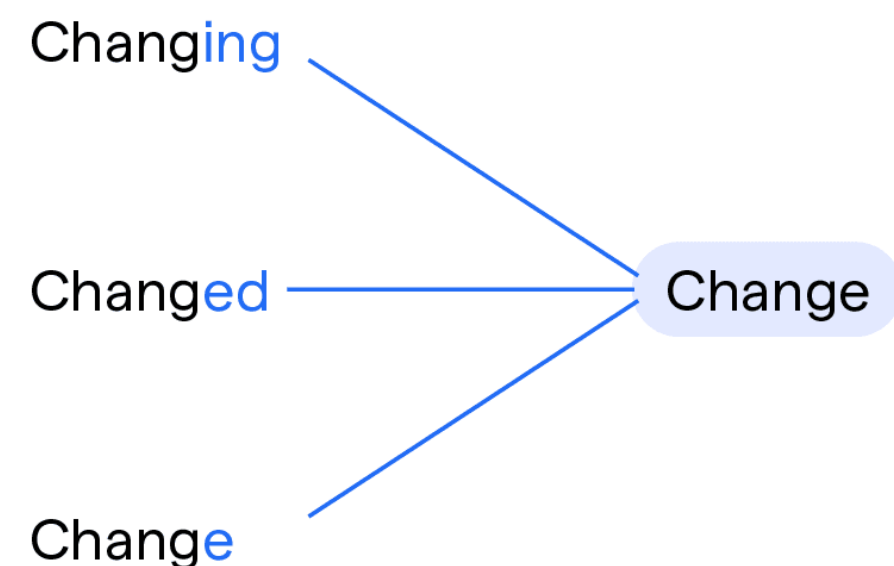
Stemming and Lemmatization

- Reducing words to their base or root forms.

Stemming

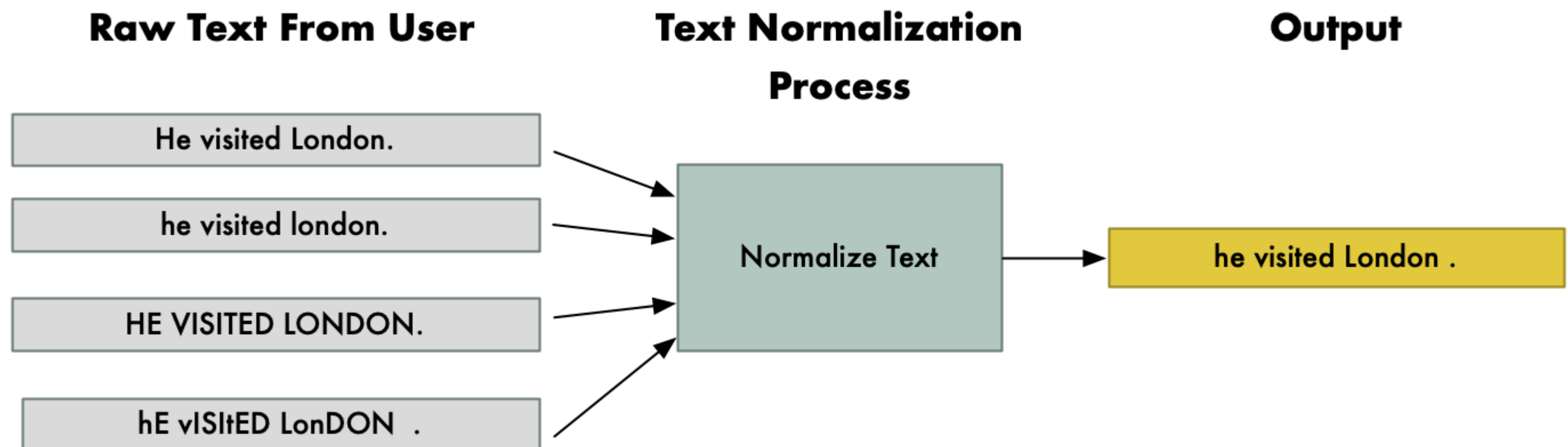


Lemmatization



Text Normalization

- Standardizing text, including case normalization, removing punctuation, and correcting spelling errors.



Sentence Segmentation

- Sentence segmentation breaks a large piece of text into linguistically meaningful sentence units.
- This is obvious in languages like English, where the end of a sentence is marked by a **period (full stop)**, but it is still not trivial.
- A period can be used to mark an abbreviation as well as to terminate a sentence, and in this case, the period should be part of the abbreviation token itself.
- The process becomes even more complex in languages, such as ancient Chinese, that don't have a delimiter that marks the end of a sentence.

Bag of Words (BoW)

- Representing text as a collection of words, ignoring grammar and word order but keeping track of word frequency.
- BoW counts the number of times each word or **n-gram** (combination of n words) appears in a document.

Document D1	<i>The child makes the dog happy</i> the: 2, dog: 1, makes: 1, child: 1, happy: 1
Document D2	<i>The dog makes the child happy</i> the: 2, child: 1, makes: 1, dog: 1, happy: 1



	child	dog	happy	makes	the	BoW Vector representations
D1	1	1	1	1	2	[1,1,1,1,2]
D2	1	1	1	1	2	[1,1,1,1,2]

Term Frequency-Inverse Document Frequency (TF-IDF)

In Bag-of-Words, we count the occurrence of each word or n-gram in a document. In contrast, with TF-IDF, we weight each word by its importance. To evaluate a word's significance, we consider two things:

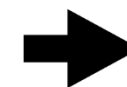
- **Term Frequency:** How important is the word in the document?
- **Inverse Document Frequency:** How important is the term in the whole corpus?

$tf(t, d)$								
	blue	bright	can	see	shining	sky	sun	today
1	1/2	0	0	0	0	1/2	0	0
2	0	1/3	0	0	0	0	1/3	1/3
3	0	1/3	0	0	0	1/3	1/3	0
4	0	1/6	1/6	1/6	1/6	0	1/3	0

X

$idf(t, D)$								
	blue	bright	can	see	shining	sky	sun	today
	0.602	0.125	0.602	0.602	0.602	0.301	0.125	0.602

- TF-IDF: Multiply TF and IDF scores, use to rank importance of words within documents
- Most important word for each document is highlighted



$tfidf(t, d, D) = tf(t, d) \cdot idf(t, D)$								
	blue	bright	can	see	shining	sky	sun	today
1	0.301	0	0	0	0	0.151	0	0
2	0	0.0417	0	0	0	0	0.0417	0.201
3	0	0.0417	0	0	0	0.100	0.0417	0
4	0	0.0209	0.100	0.100	0.100	0	0.0417	0

Word Embeddings

- Using dense vector representations of words where semantically similar words are closer together in the vector space
 - **Word2Vec**: Techniques like Skip-gram and CBOW for generating word embeddings.
 - **GloVe** (Global Vectors): similar to Word2Vec as it also learns word embeddings, but it does so by using matrix factorization techniques rather than neural learning.
 - **FastText**: Extending Word2Vec by considering subword information.

Word2Vec

Word2Vec, introduced in 2013, uses a **vanilla neural network** to learn high-dimensional word embeddings from raw text.

Continuous Bag of Words (CBOW):

- CBOW predicts the target word based on its context (surrounding words). It uses the context words as input to predict the target word. This architecture is efficient for smaller datasets and frequent word

Skip-gram:

- Skip-gram predicts the context words given a target word. It uses the target word as input to predict the context words. Skip-gram performs well with larger datasets and infrequent words.

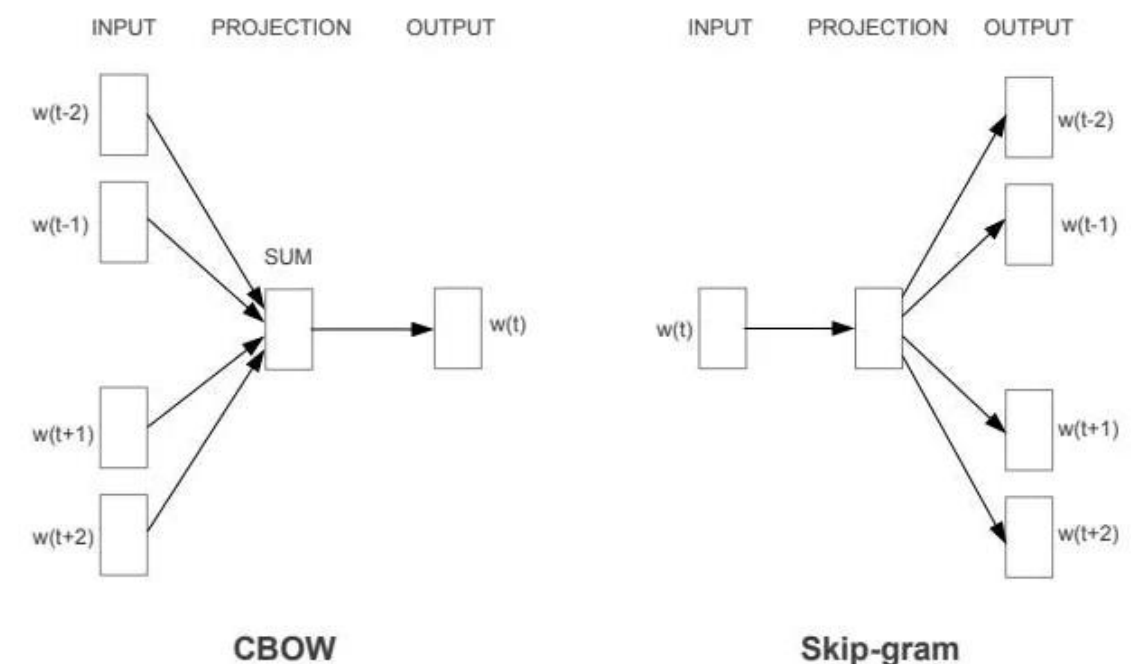
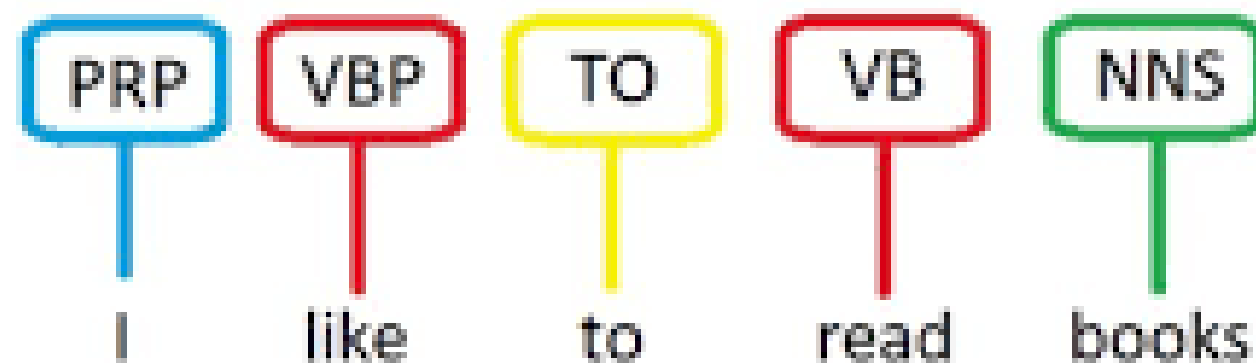


Figure 1: New model architectures. The CBOW architecture predicts the current word based on the context, and the Skip-gram predicts surrounding words given the current word.

Part-of-Speech (POS) Tagging

- Assigning parts of speech to each word in a sentence (e.g., PRP (Personal Pronoun), VBP (Verb non 3rd person), NNS (Noun, plural)).

POS Tagging



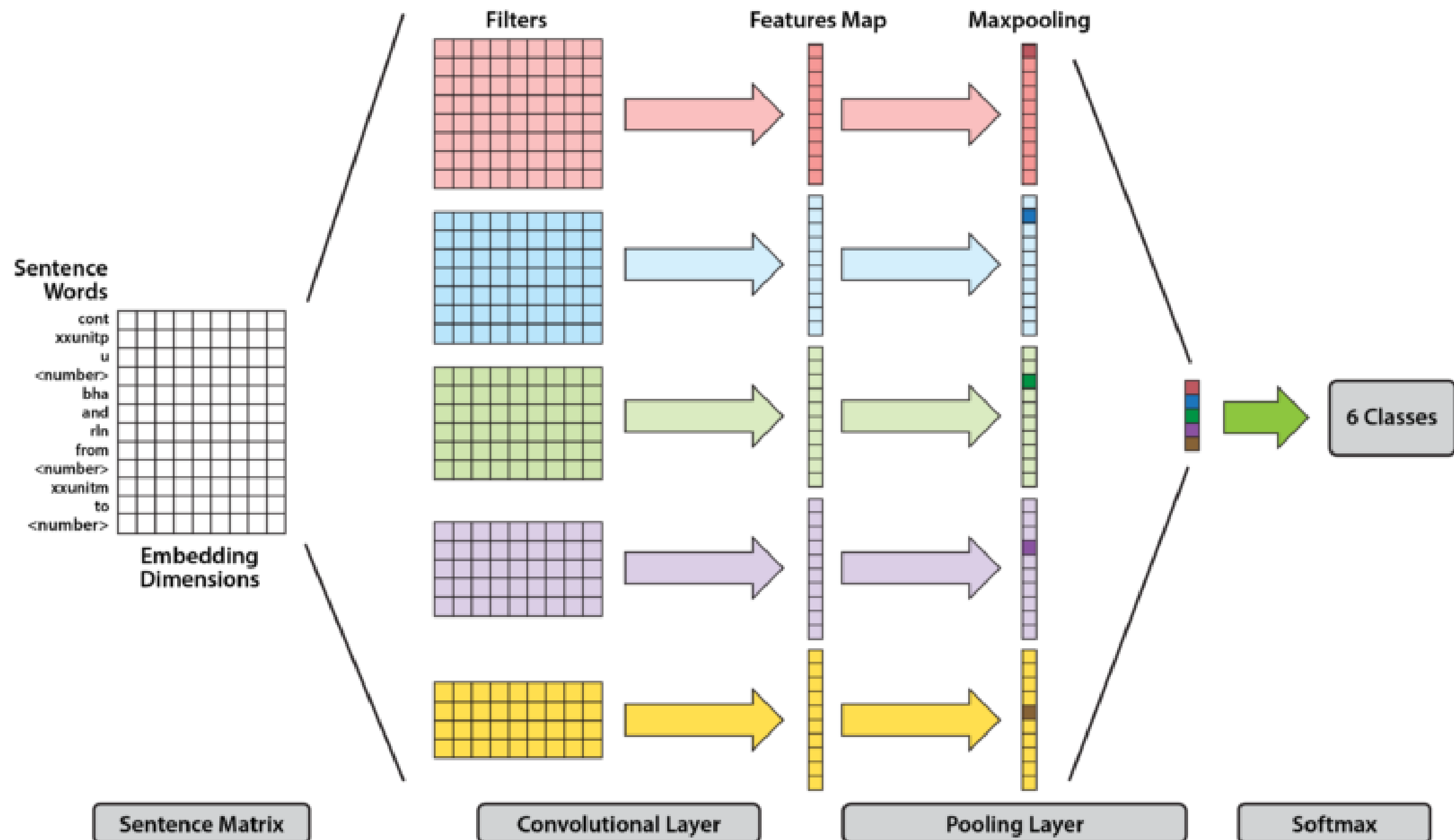
Traditional Machine Learning NLP Techniques

- **Logistic Regression**
 - a supervised classification algorithm that aims to predict the probability that an event will occur based on some input. In NLP, logistic regression models can be applied to solve problems such as sentiment analysis, spam detection, and toxicity classification.
- **Naïve Bayes**
- **Decision Tree**
 - a class of supervised classification models that split the dataset based on different features to maximize information gain in those splits.
- **Latent Dirichlet Allocation (LDA)**
 - used for **topic modeling**. LDA tries to view a document as a collection of topics and a topic as a collection of words.
- **Hidden Markov models**
 - probabilistic models that decide the next state of a system based on the current state. For example, in NLP, we might suggest the next word based on the previous word.

Deep Learning for NLP

- **Recurrent Neural Networks (RNNs)**
 - **Architecture:** Sequential processing of data.
 - **Challenges:** Vanishing and exploding gradients.
 - **Applications:** Language modeling, machine translation.
- **Long Short-Term Memory (LSTM) Networks**
 - **LSTM Architecture:** Gates (input, forget, output) to manage information flow.
 - **Advantages Over RNNs:** Mitigates the vanishing gradient problem.
 - **Use Cases:** Text generation, sentiment analysis.
- **Convolutional Neural Networks (CNNs)**
 - Text classification, named entity recognition
 - 1D CNNs, character-level CNNs

CNN based text classification



Given a sentence, a convolutional neural network uses convolutional layers to refine representations of input words, before combining them to render a classification.

Deep Learning for NLP

Transformer Models

- **Self-Attention Mechanism:** Allows the model to weigh the importance of different words in a sentence.
- **Architecture:** Encoder-decoder structure.
- **Pre-trained Models:** Utilizing pre-trained language models such as BERT, GPT, or transformer-based models that have been trained on large corpora.
- **BERT (Bidirectional Encoder Representations from Transformers):**
 - **Pretraining and Fine-tuning:** Training on large corpora and adapting to specific tasks i.e Transfer Learning.
 - **Applications:** Question answering, named entity recognition.
- **GPT (Generative Pre-trained Transformer):**
 - **Generative Capabilities:** Text generation, completion.
 - **Versions:** From GPT-1 to GPT-4 and beyond.

Important NLP Models

- **Eliza** was developed in the mid-1960s to try to solve the Turing Test; that is, to fool people into thinking they're conversing with another human being rather than a machine.
- **Tay** was a chatbot that Microsoft launched in 2016. It was supposed to tweet like a teen and learn from conversations with real users on Twitter.
- **BERT** and his Muppet friends: Many deep learning models for NLP are named after Muppet characters, including ELMo, BERT, Big BIRD, ERNIE, Kermit, Grover, RoBERTa, and Rosita.
- **Generative Pre-Trained Transformer (GPT)** is a 175 billion parameter model that can write original prose with human-equivalent fluency in response to an input prompt.
- **Language Model for Dialogue Applications (LaMDA)** is a conversational chatbot developed by Google. LaMDA is a transformer-based model trained on dialogue rather than the usual web text.
- **GPT-4 (Generative Pre-trained Transformer 4)**: Released by OpenAI on March 14, 2023, GPT-4 is a multimodal large language model capable of processing both text and images. It was trained to predict the next token and fine-tuned with reinforcement learning from human and AI feedback for alignment and policy compliance. GPT-4 is accessible through ChatGPT Plus and its commercial API.

Important NLP Models

- **Gemini:** Developed by Google DeepMind, Gemini was announced during the Google I/O keynote on May 10, 2023. Designed as a multimodal large language model, Gemini can process various data types simultaneously, including text, images, audio, video, and computer code. It was developed collaboratively by DeepMind and Google Brain.
- **LLaMA (Large Language Model Meta AI):** Introduced by Meta AI in February 2023, LLaMA is a series of large language models with parameters ranging from 7 billion to 65 billion. The 13 billion parameter model outperforms larger models like GPT-3 on most NLP benchmarks. In July 2023, Meta released LLaMA 2, an open-source model available for both research and commercial use.
- **DBRX:** An open-source large language model developed by Mosaic ML and Databricks, DBRX was released on March 27, 2024. It employs a mixture-of-experts transformer architecture with a total of 132 billion parameters, with 36 billion active for each token. DBRX outperforms other open-source models like Meta's LLaMA 2 and xAI's Grok in various benchmarks, including language understanding and programming tasks.
- **HiPool:** Proposed in May 2023, HiPool is a model designed to handle long documents using graph neural networks. It segments sequences into fixed-length chunks to capture sentence-level information and utilizes graphs to model intra- and cross-sentence correlations. HiPool has demonstrated superior performance over hierarchical sequential models, especially with longer sequences.
- **DeepSeek-R1:** Released in November 2024 by the Chinese AI firm DeepSeek, the R1 model has over 671 billion parameters and employs a "mixture of experts" approach, activating only relevant networks for specific prompts. This design significantly reduces processing power requirements. Remarkably, DeepSeek-R1 achieves performance on par with leading models like OpenAI's o1 in reasoning tasks and is open-source, allowing researchers to examine and utilize the model freely.

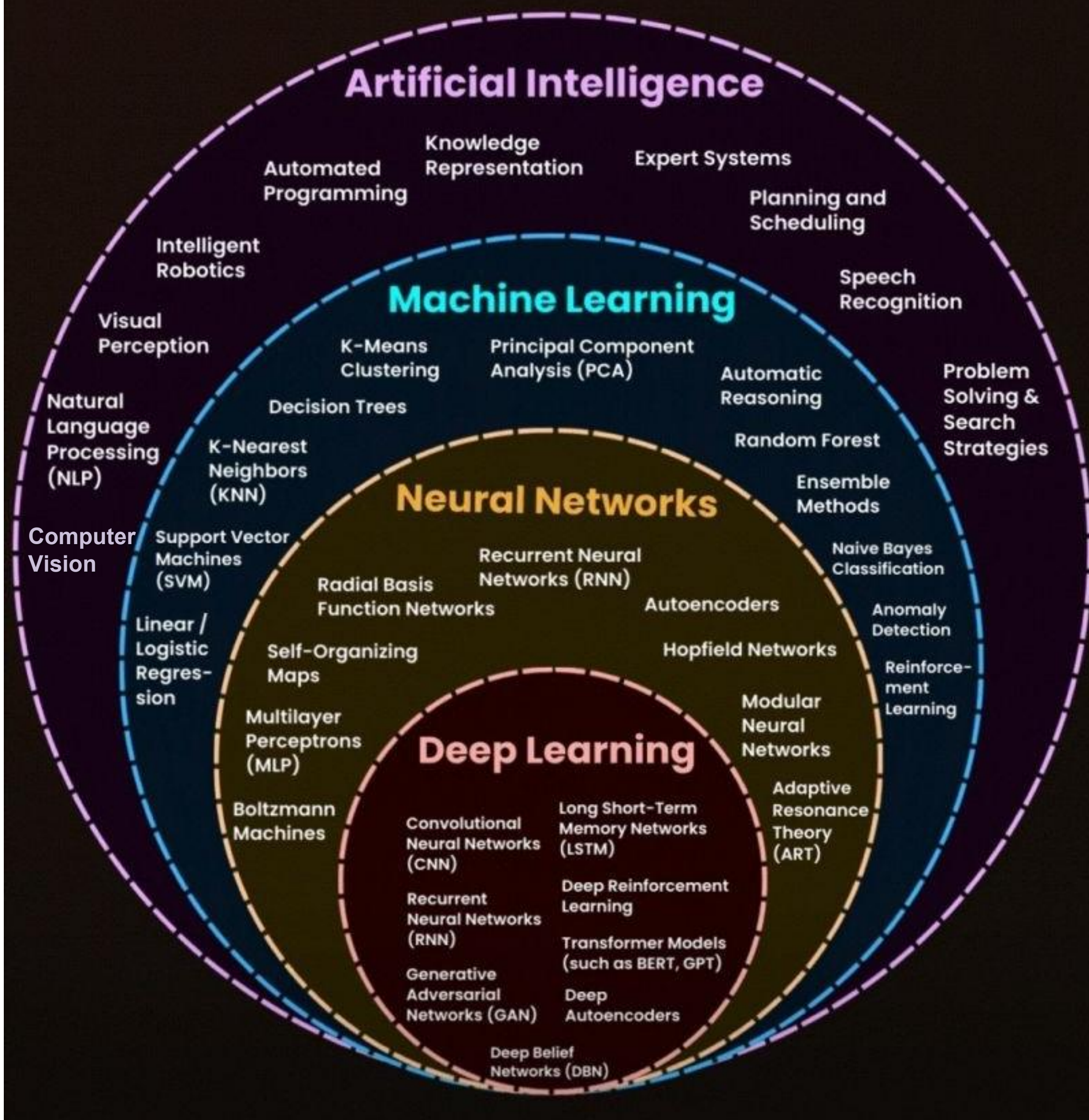
Important NLP Models

- **Qwen (Tongyi Qianwen):** Developed by Alibaba Cloud, Qwen is a family of large language models (LLMs) introduced in 2023. The series includes models of varying sizes, such as Qwen-7B, Qwen-14B, and Qwen-72B, with the numbers indicating the parameter count in billions. Qwen models are transformer-based and have been pretrained on extensive datasets comprising web texts, books, and code. They are designed for a wide range of natural language processing tasks, including text generation, language understanding, and code assistance. In July 2024, Qwen was ranked as the top Chinese language model in certain benchmarks and third globally, following models from Anthropic and OpenAI. The Qwen series also encompasses specialized models like Qwen-Chat for conversational applications and Qwen2.5-VL, a vision-language model capable of processing both textual and visual inputs. These models are open-sourced and available on platforms like GitHub and Hugging Face.
- **Grok:** Grok is a generative AI chatbot developed by xAI, a company founded by Elon Musk. Launched in 2023, Grok is based on a large language model designed to perform next-token prediction and has been fine-tuned using extensive feedback from both humans and earlier models. The initial version, Grok-1, featured a 314 billion parameter Mixture-of-Experts architecture. Subsequent iterations, such as Grok-2 and Grok-3, have introduced enhanced reasoning capabilities and increased computational power. Grok-3, unveiled in December 2024, operates in multiple modes, including "Think" for step-by-step reasoning and "Big Brain" for complex tasks requiring more computational resources. Early reviews suggest that Grok-3 is competitive with leading models from OpenAI and Google. Grok is accessible through the Grok app with an X Premium Plus subscription.

Python libraries for NLP

- Natural Language Toolkit (NLTK)
- spaCy
- TensorFlow, PyTorch and Keras(Deep Learning framework)
- Hugging Face
- Gensim

Mid Term



Computer Vision (CV)

- Computer Vision (CV) is a field of Artificial Intelligence (AI) that deals with computational methods to help computers understand and interpret the content of digital images and videos.
- Hence, CV aims to make computers see and understand visual data input from cameras or sensors.
- Computer vision tasks seek to enable computer systems to automatically see, identify, and understand the visual world, simulating human vision using computational methods.

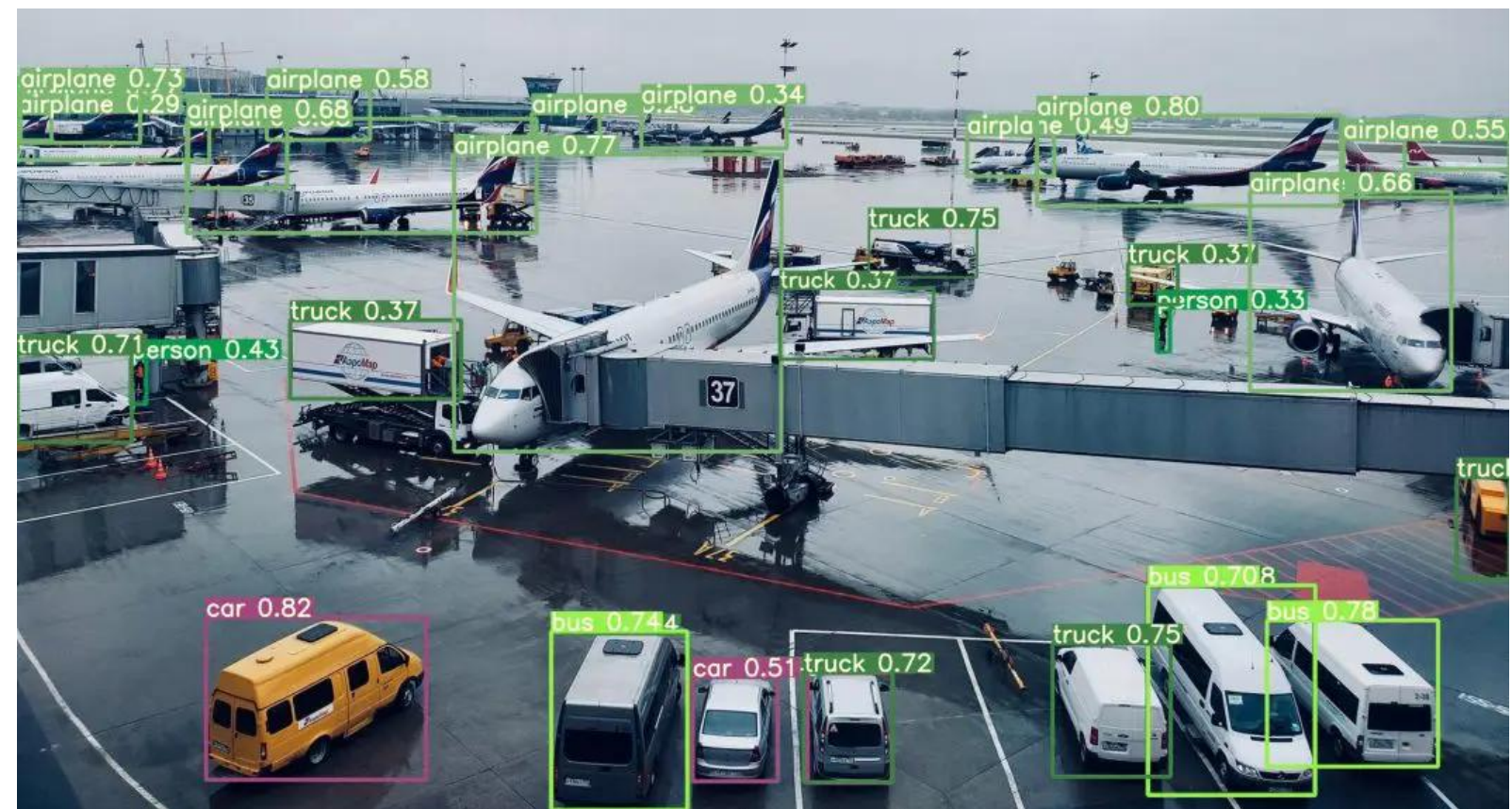


Image source: viso.ai

Human Vision vs. Computer Vision

- Computer vision aims to artificially imitate human vision by enabling computers to perceive visual stimuli meaningfully. It is, therefore, also called **machine perception**, or **machine vision**.
- While the problem of “vision” is trivially solved by humans (even by children), computational vision remains one of the most challenging fields in computer science, especially due to the enormous complexity of the varying physical world.



Image source: viso.ai

Human Vision vs. Computer Vision

- Human sight is based on a lifetime of learning with context to train how to identify specific objects or recognize human faces or individuals in visual scenes.
- Hence, modern artificial vision technology uses machine learning and deep learning methods to train machines how to recognize objects, faces, or people in visual scenes.
- As a result, computer vision systems use **image processing algorithms** to allow computers to find, classify, and analyze objects and their surroundings from data provided by a camera.

What Is the Value of Computer Vision?

- Computer vision systems can perform product inspection, infrastructure monitoring, or analyze thousands of products or processes in real-time, for defection detection.
- Due to its speed, objectivity, continuity, accuracy, and scalability, computer vision systems can quickly surpass human capabilities.
- The latest deep learning models achieve above human-level accuracy and performance in real-world image recognition tasks such as
 - **Facial recognition,**
 - **Object recognition,**
 - **Position estimation,**
 - **Optical character recognition,**
 - **Motion analysis,**
 - **Pattern recognition,**
 - **Object detection,**
 - **Image classification.**



Where is CV applications used?

- Computer vision applications are used in a wide range of industries, ranging from **security** and **medical imaging** to **manufacturing, automotive, agriculture, construction, smart city, transportation**, and many more.
- As AI technology advances and becomes more flexible and scalable, significantly more use cases become possible and economically viable.

A Practical Example of Computer Vision

- Computer vision machine learning requires a massive amount of data to train a deep learning algorithm that can accurately recognize images.
- For example, to train a computer to recognize a helmet, it needs to be fed large quantities of helmet images with people wearing helmets in different scenes to learn the characteristics of a helmet.
- Next, the trained algorithm can be applied to newly generated images, for example, videos of surveillance cameras, to recognize a helmet. This is, for example, used in computer vision applications for equipment inspection to reduce accidents in construction or manufacturing.



Computational Vision Inspired by the Human Brain

- Computer vision works by recognizing images or “seeing” images similar to humans, using learned features with a confidence score.
- Therefore, neural networks essentially simulate human decision-making, or neuron activation mechanisms, and deep learning trains the machine to do what the human brain does naturally.
- The characteristic layered structure of deep neural networks is the foundation of Artificial Neural Networks (ANN). Each layer adds to the knowledge of the previous layer.

Human-level Performance of Computer Vision

- Deep learning tasks are computationally heavy and expensive, depending on significant computing resources, and require massive datasets to train models on.
- Compared to traditional image processing, deep learning algorithms enable machines to learn by themselves, without a developer programming it to recognize an image based on pre-determined features. As a result, deep learning methods achieve very high accuracy.
- Today, deep learning enables machines to achieve human-level performance in image recognition tasks.
- For example, in deep face recognition, AI models achieve a detection accuracy (e.g., Google FaceNet achieved 99.63%) that is higher than the accuracy humans can achieve (97.53%).

Human-level Performance of Computer Vision

- Computational vision with deep learning has also achieved human performance in classifying skin cancer with a level of competence comparable to dermatologist experts.

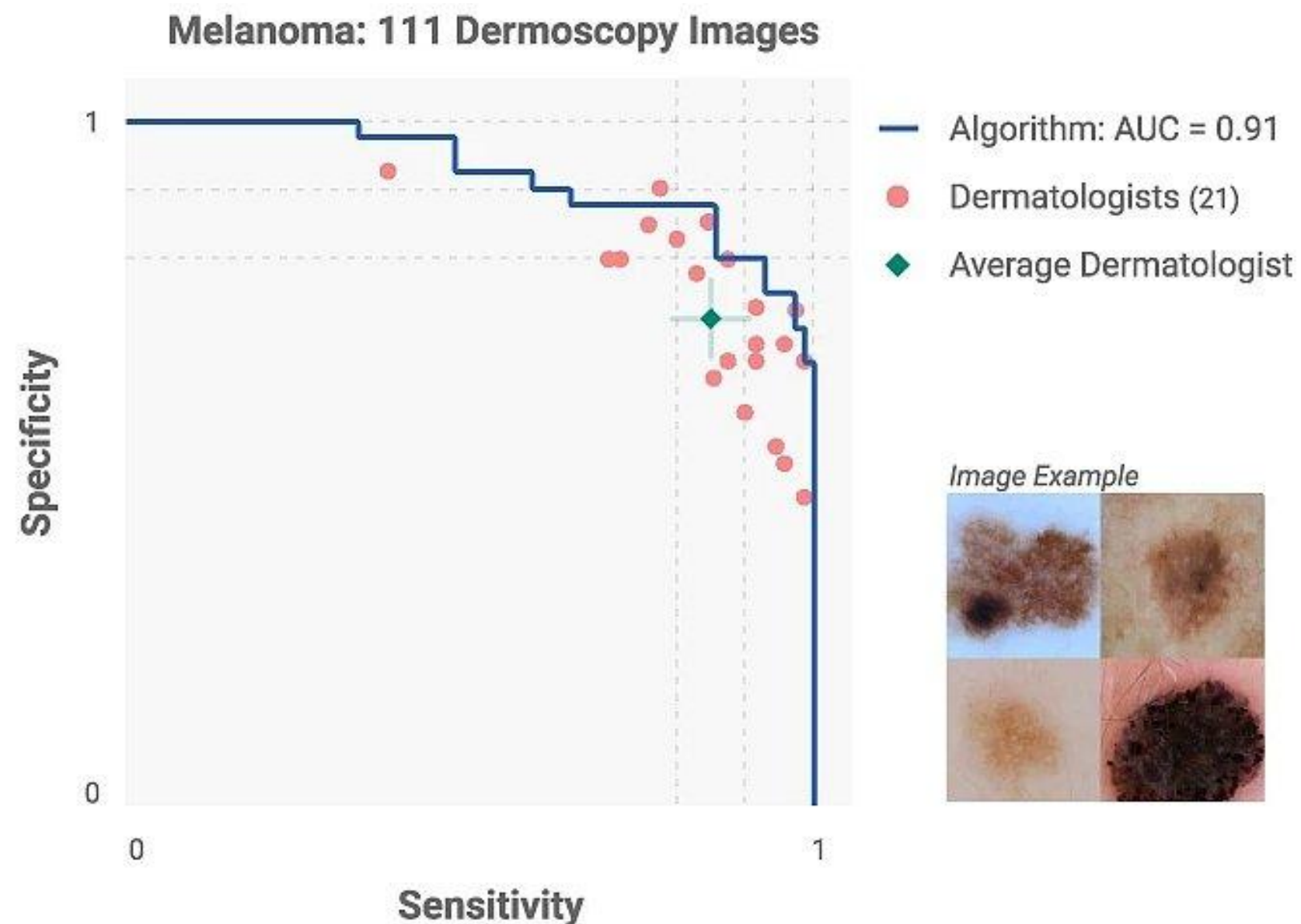


Image source: <https://www.nature.com/articles/s41746-020-00376-2>

How Does Computer Vision AI Work?

- Modern computer vision systems combine image processing with machine learning and deep learning techniques.
- Hence, developers combine different software (etc., OpenCV or OpenVINO) and AI algorithms to create a multi-step process, a computer vision pipeline.
- The organization and setup of a computer vision system vary based on the application and use case.

How Does Computer Vision AI Work?

- All computer vision systems contain the same typical functions:
 - Step #1: Image acquisition. The digital image of a camera or image sensor provides the image data or video. Technically, any 2D or 3D camera or sensor can be used to provide image frames.
 - Step #2: Pre-processing. The raw image input of cameras needs to be preprocessed to optimize the performance of the subsequent computer vision tasks. Pre-processing includes noise reduction, contrast enhancement, re-scaling, or image cropping.

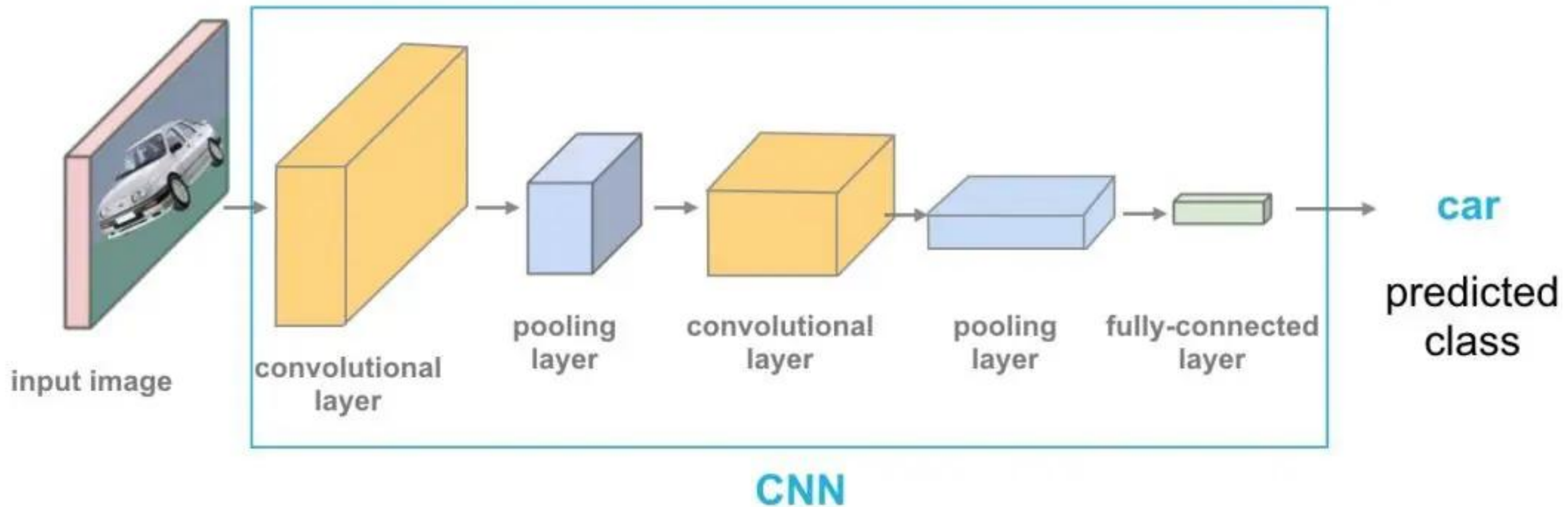
How Does Computer Vision AI Work?

- All computer vision systems contain the same typical functions:
 - Step #3: Computer vision algorithm. The image processing algorithm, most popularly a deep learning model (DL model), performs image recognition, object detection, image segmentation, and image classification on every image or video frame.
 - Step #4: Automation logic. The AI algorithm output information needs to be processed with conditional rules based on the use case. This part performs automation based on information gained from the computer vision task. For example, pass or fail for automatic inspection applications, match or no-match in recognition systems, and flag for human review in insurance, surveillance and security, military, or medical recognition applications.

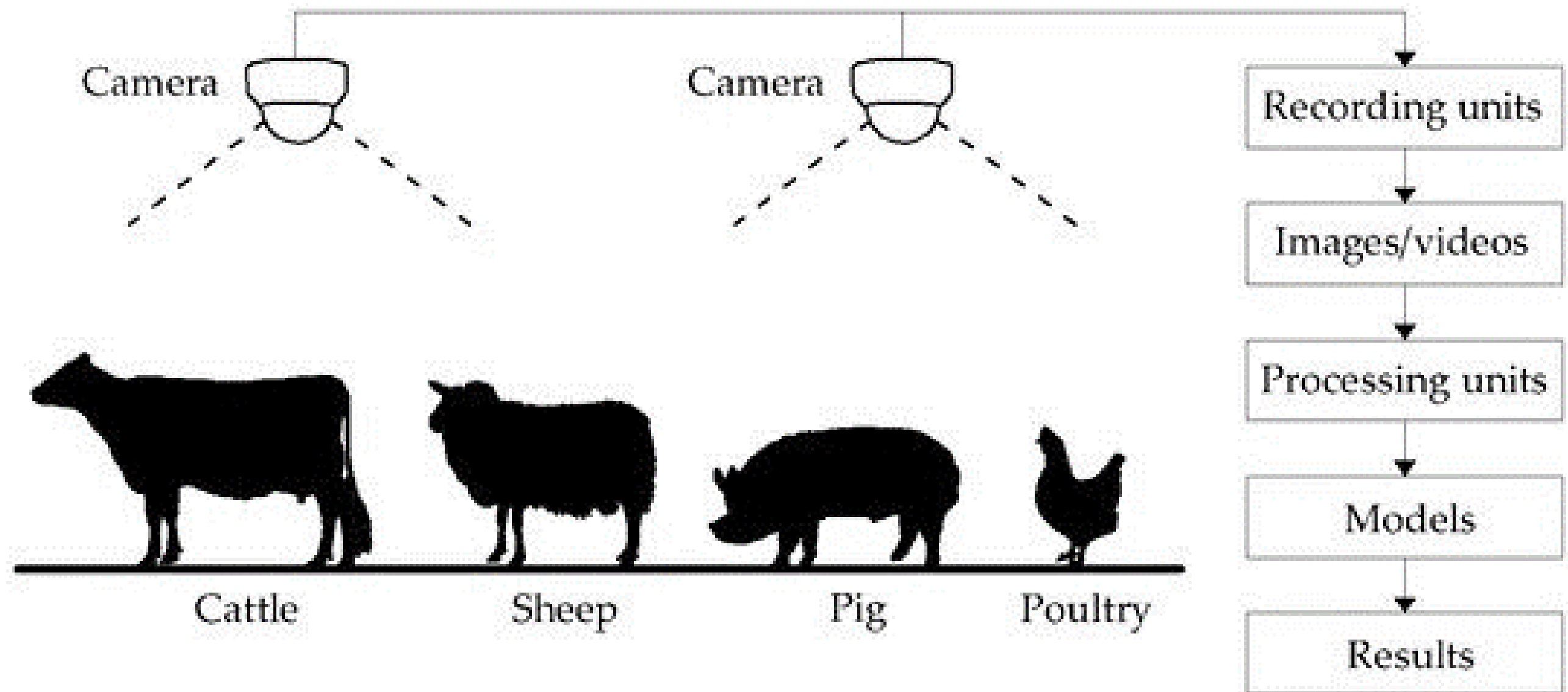
How Does Computer Vision AI Work?

- To train an algorithm for computer vision, state-of-the-art technologies leverage deep learning, a subset of machine learning. Many high-performing methods in modern computer vision software are based on a **Convolutional Neural Network (CNN)**.
- Such layered neural networks enable a computer to learn about the context of visual data from images. If enough data is available, the computer learns how to tell one image from another. As image data is fed through the model, the computer applies a CNN to “look” at the data.
- The CNN helps a machine learning/deep learning model to understand images by breaking them down into pixels that were given labels to train specific features, so-called image annotation. The AI model uses the labels to perform convolutions and make predictions about what it is “seeing” and checks the accuracy of the predictions iteratively until the predictions meet the expectation (start to come true).

How Does Computer Vision AI Work?



How Does Computer Vision AI Work?



Example of a computer vision system in agriculture for smart animal farming

The Best CV Deep Learning Models

In computer vision, specifically, real-time object detection, there are single-stage and multi-stage algorithm families.

- Single-stage algorithms aim for real-time processing and the highest computational efficiency. The most popular algorithms include SSD, RetinaNet, YOLOv3, YOLOR, YOLOv5, YOLOv7, and YOLOv8.
- Multi-stage algorithms perform multiple steps and achieve the highest accuracy but are rather heavy and resource-intensive. The widely used multi-stage algorithms include Recurrent Convolutional Neural Networks (R-CNN) such as Mask-RCNN, Fast RCNN, and Faster RCNN.



OSM Mapping Example with Mask R-CNN

Applications of Computer Vision

Manufacturing

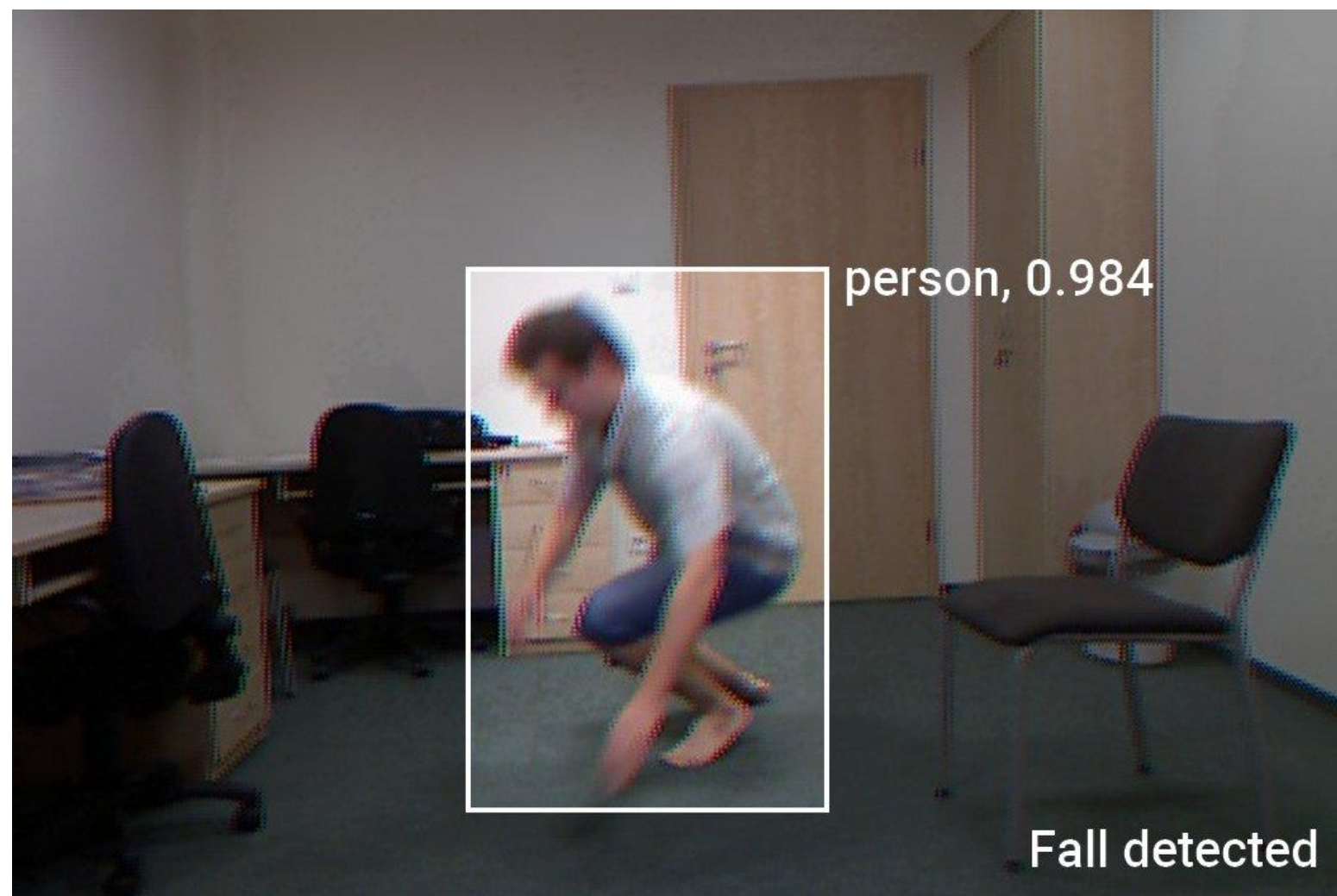
- Industrial computer vision is used in manufacturing industries on the production line for automated product inspection, object counting, process automation, and to increase workforce safety with PPE detection and mask detection.



Applications of Computer Vision

Healthcare

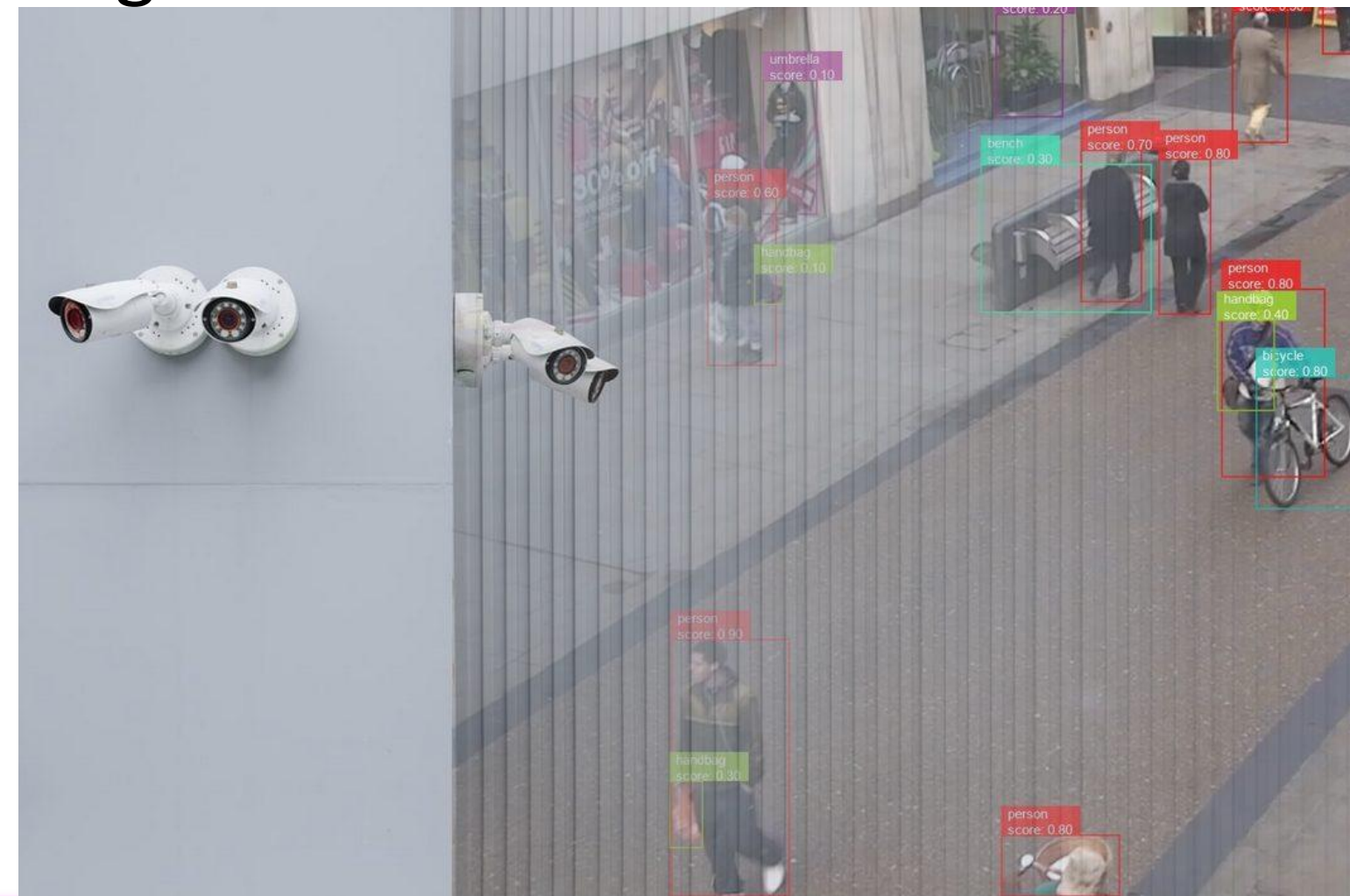
- Among applications of computer vision in healthcare, a prominent example is automated human fall detection to create a fall risk score and trigger alerts.



Applications of Computer Vision

Security

- In video surveillance and security, person detection is performed for intelligent perimeter monitoring. Another popular use case is deep face detection and facial recognition with above-human-level accuracy.



Applications of Computer Vision

Agriculture

- Use cases of computational vision in agriculture and farming include automated animal monitoring to detect animal welfare, and early detect diseases and anomalies.



3 erosomyia_sp

3 erosomyia_sp	93%
2 erosomyia_sp	4%
2 procontarinia_rubus	1%

3 erosomyia_sp

3 erosomyia_sp	97%
2 erosomyia_sp	2%
3 apoderus_javanicus	0%

2 ischnaspis_longirostris

2 ischnaspis_longirostris	99%
1 ischnaspis_longirostris	0%
3 erosomyia_sp	0%

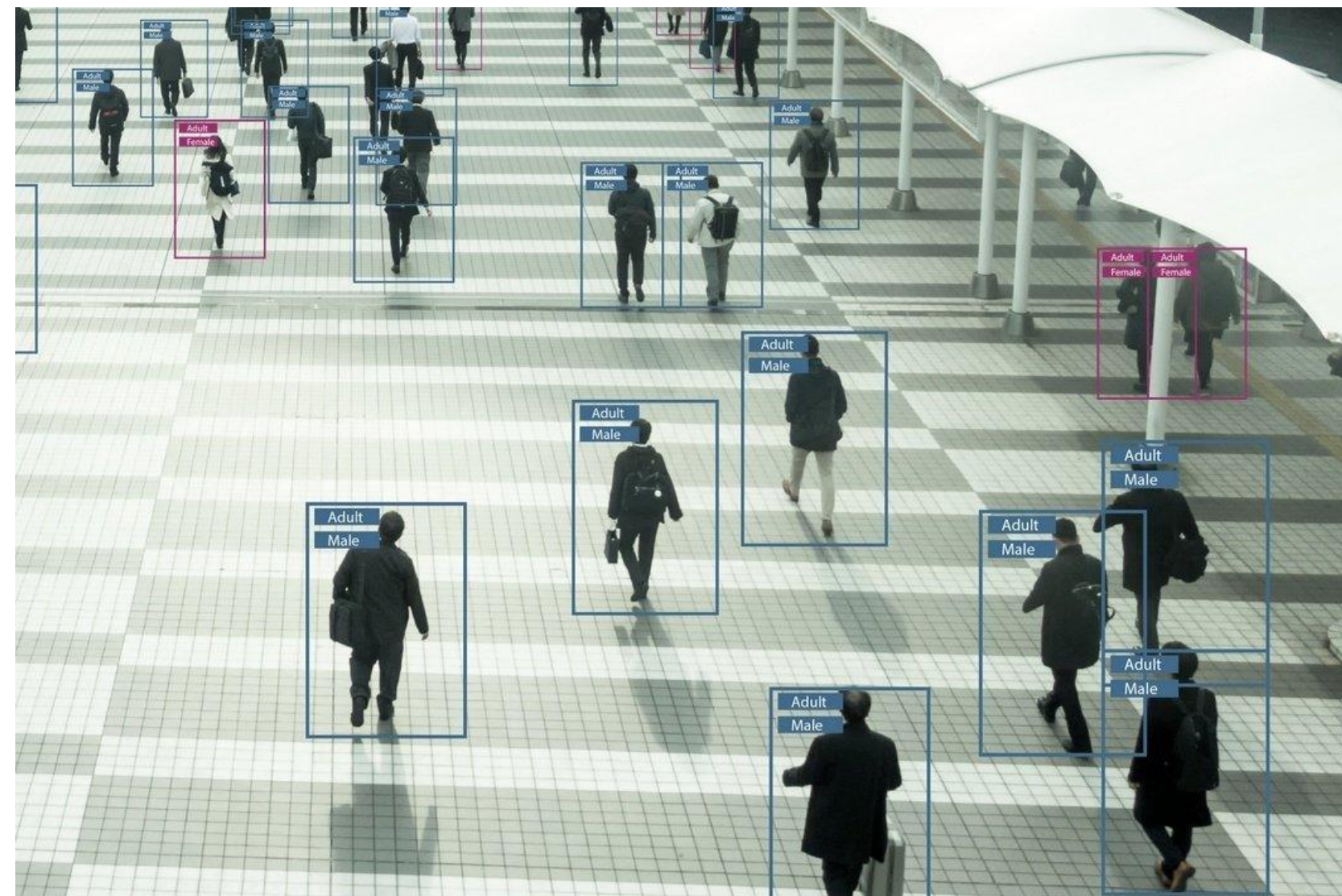
1 apoderus_javanicus

1 apoderus_javanicus	99%
2 apoderus_javanicus	1%
3 apoderus_javanicus	0%

Applications of Computer Vision

Smart Cities

- Used as a key strategy in smart cities for crowd analysis, weapon detection, traffic analysis, vehicle counting, self-driving cars/autonomous vehicles, and infrastructure inspection.



Applications of Computer Vision

Retail

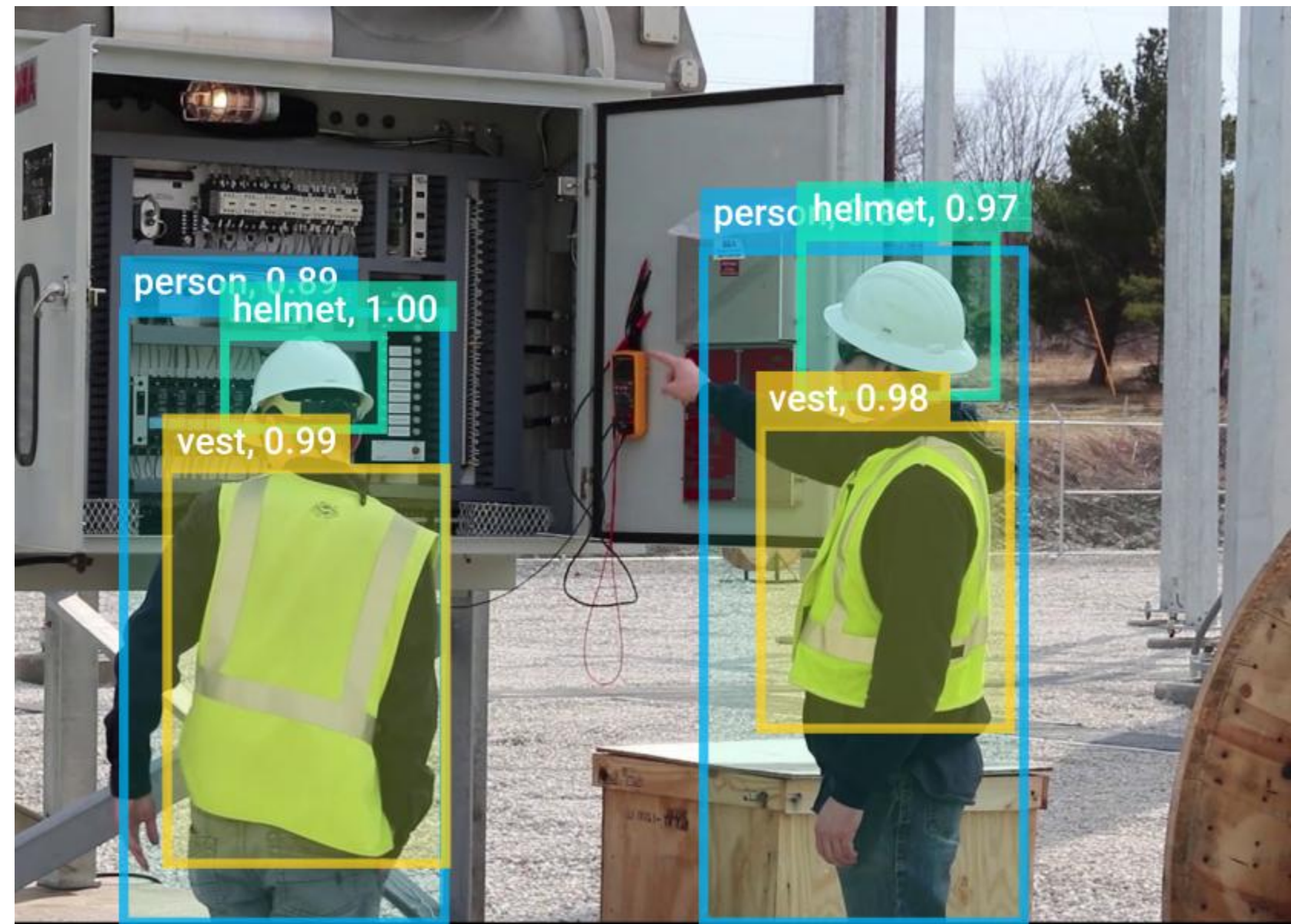
- For example, video surveillance cameras in retail stores track movement patterns of customers and perform people counting or footfall analysis to identify bottlenecks, customer attention, and waiting times.



Applications of Computer Vision

Insurance

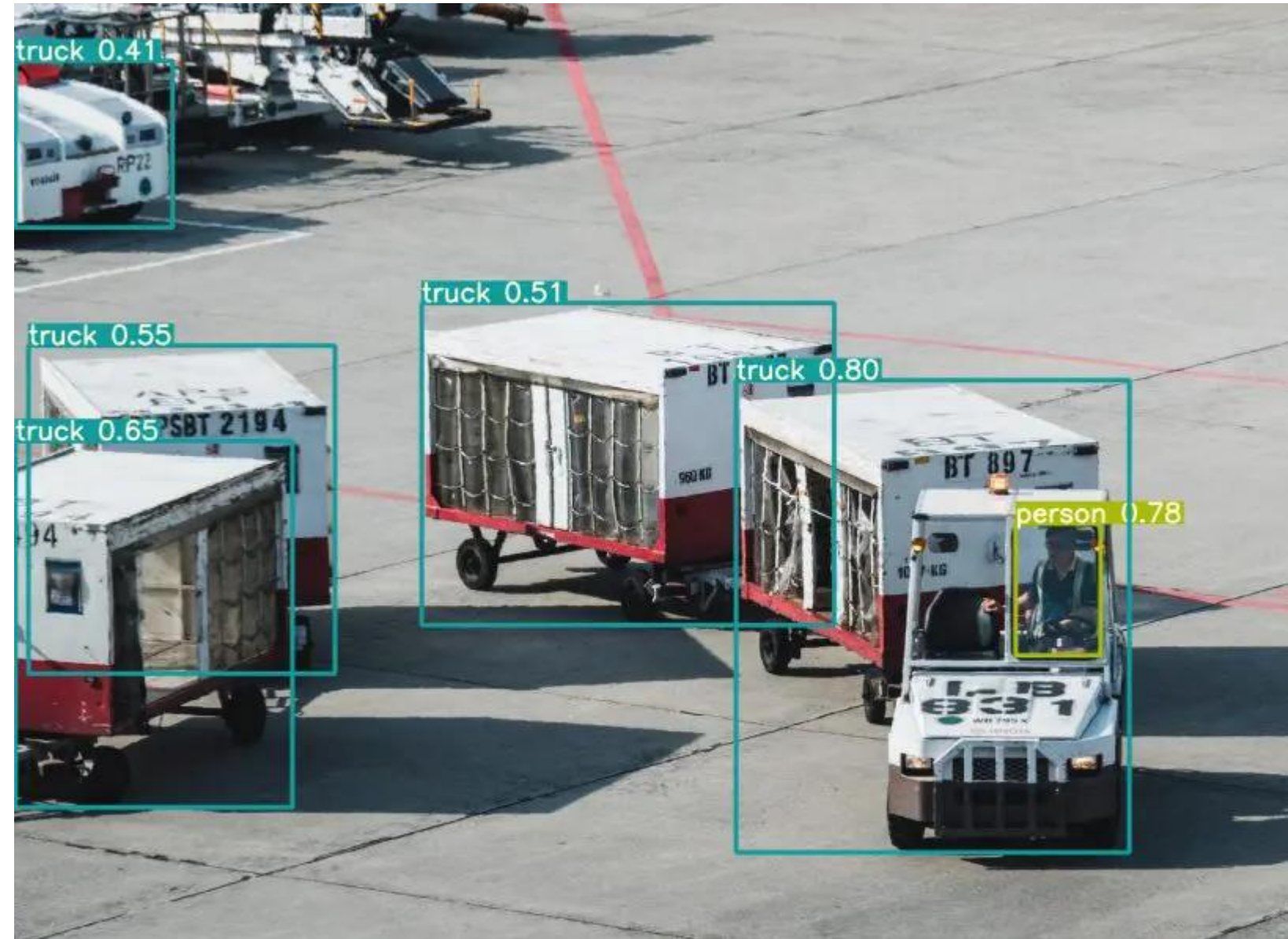
- Computer vision in Insurance leverages AI vision for automated risk management and assessment, claims management, visual inspection, and forward-looking analytics.



Applications of Computer Vision

Logistics

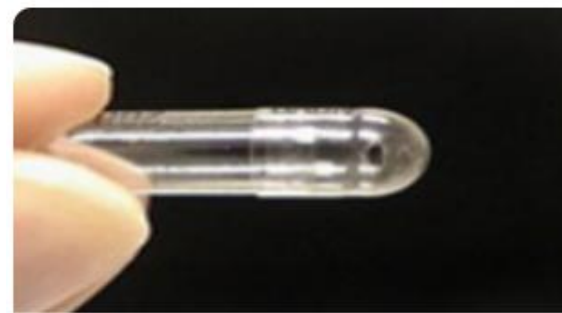
- AI vision in Logistics applies deep learning to implement AI-triggered automation and save costs by reducing human errors, predictive maintenance, and accelerating operations throughout the supply chain.



Applications of Computer Vision

Pharmaceutical

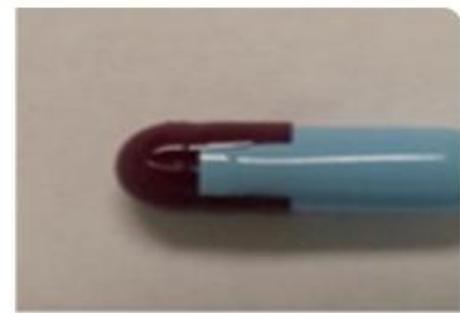
- Computer vision in pharmaceutical industries is used for packaging and blister detection, capsule recognition, and visual inspection for equipment cleaning



Oil Holes



Cracked



Telescope
d



Short body



Dents



Double
caps

Applications of Computer Vision

Augmented Reality and Virtual Reality

- Computer vision for augmented and virtual reality creates immersive experiences by integrating real-world or virtual environment perception, thus allowing users to interact with virtual surroundings in real-time.



Some concepts in CV

Image Classification

- Image classification forms the fundamental building block of Computer Vision. CV engineers often start with training a Neural Network to identify different objects in an image (Object Detection).
- Existing architectures for image classification include [ResNet-50](#), [ResNet-100](#), [ImageNet](#), [AlexNet](#), [VggNet](#), and more.
- Existing architectures for image classification include ResNet-50, ResNet-100, ImageNet, AlexNet, VggNet, and more.

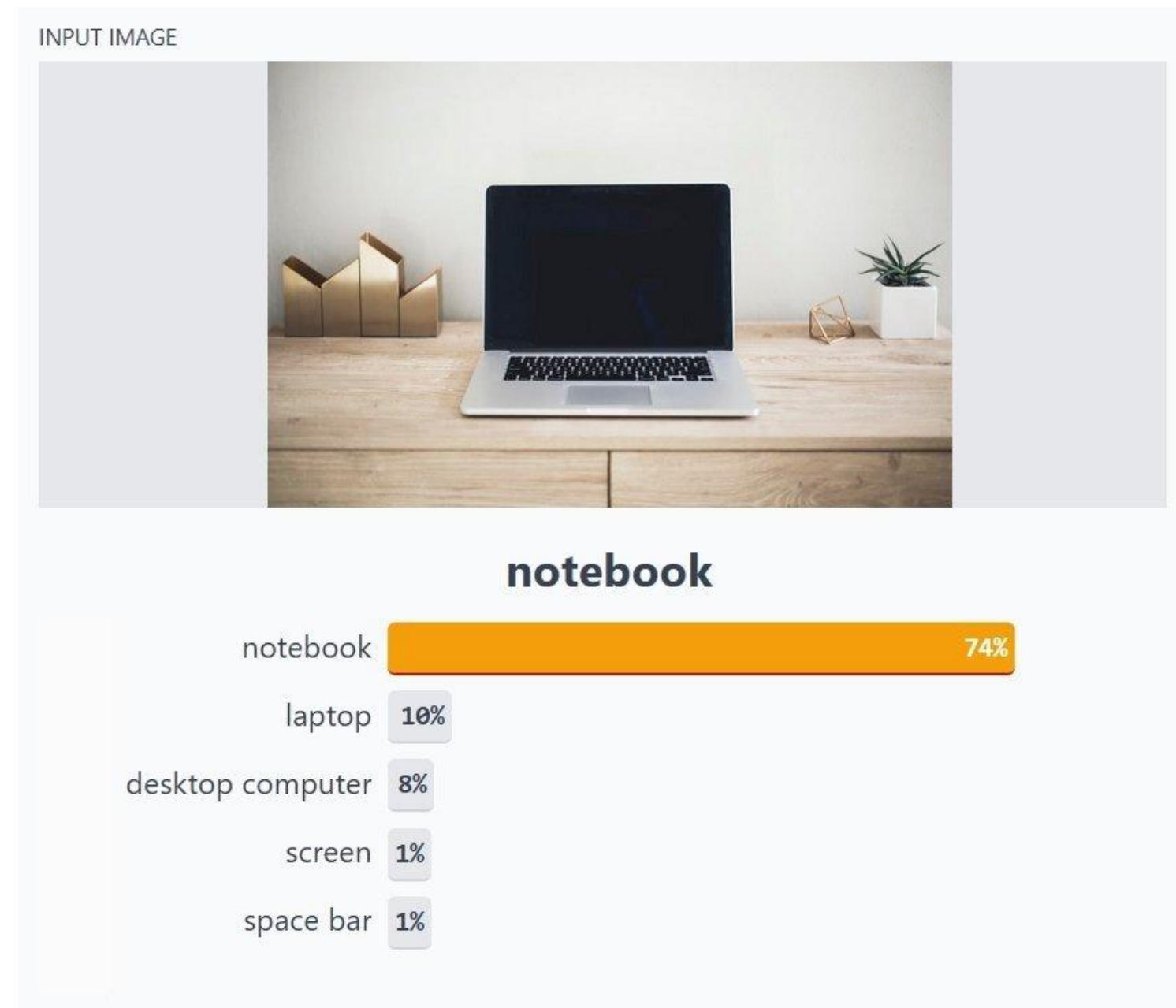
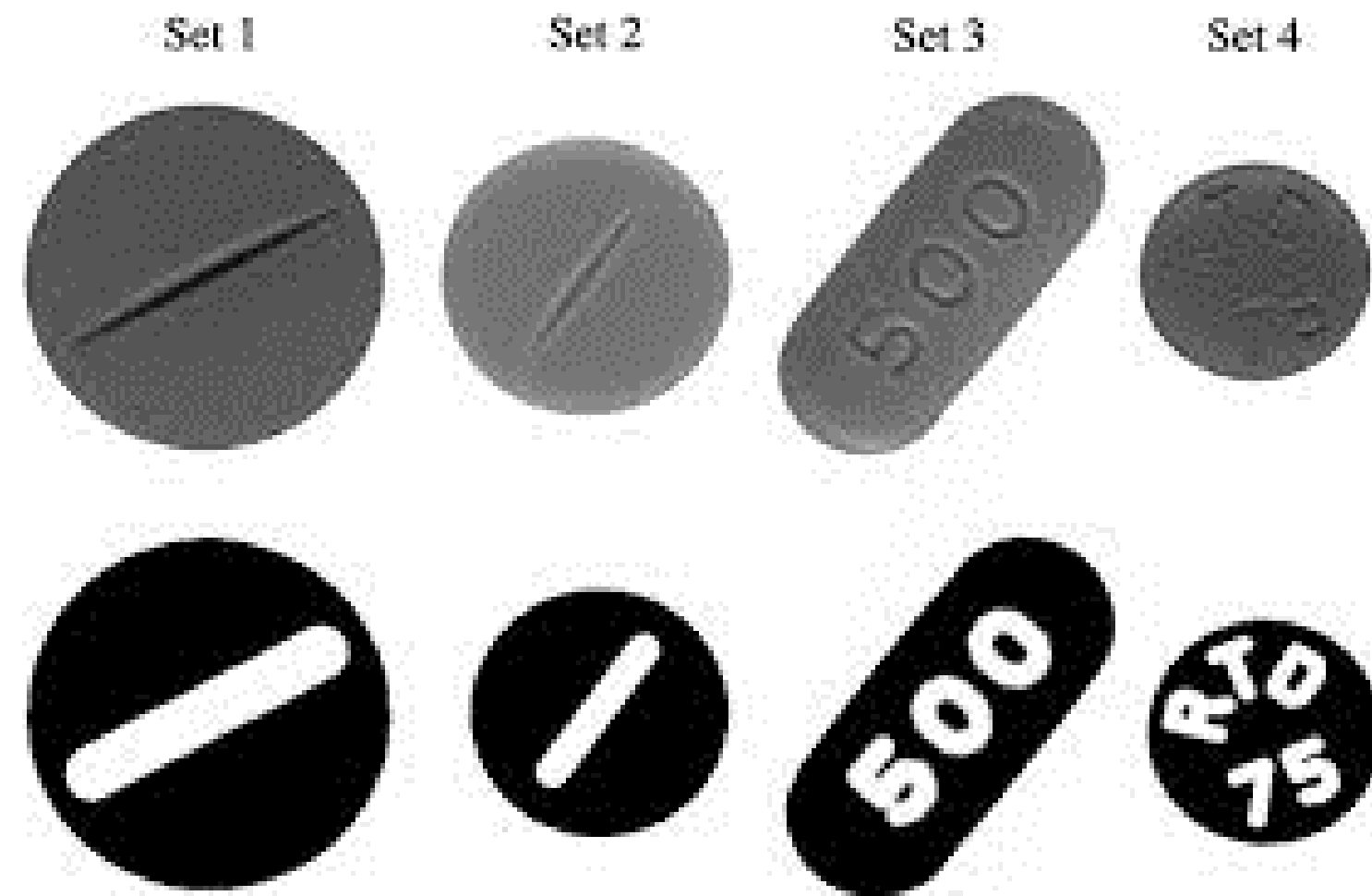


Image Processing

- Image processing is a key aspect of AI vision systems since it involves transforming images to extract certain information or optimize it for subsequent tasks in a CV system.
- Basic image processing techniques include smoothing, sharpening, contrasting, de-noising, or colorization.



Optical Character Recognition

- Optical character recognition or optical character reader (OCR) is a technique that converts any kind of written or printed text from an image into a machine-readable format.
- Existing architectures for OCR extractions include EasyOCR, Python-tesseract, or Keras-OCR. These machine learning software tools are popularly used for Number Plate Recognition as an example.

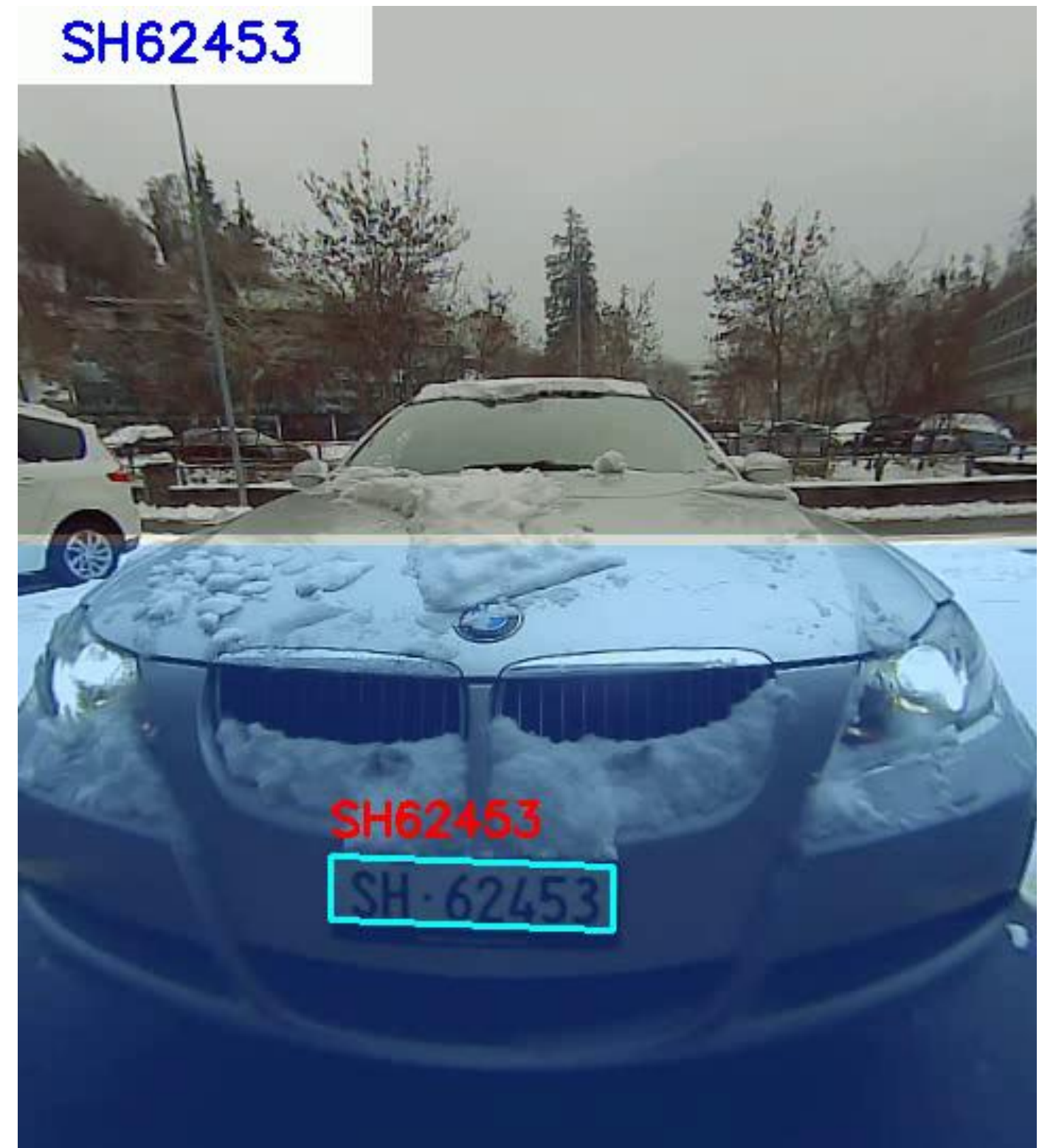


Image Segmentation

- While image classification aims to identify the labels of different objects in an image, instance segmentation tries to find the exact boundary of the objects in the image.
- Example: Image segmentation with YOLOv8 applied for pothole detection.



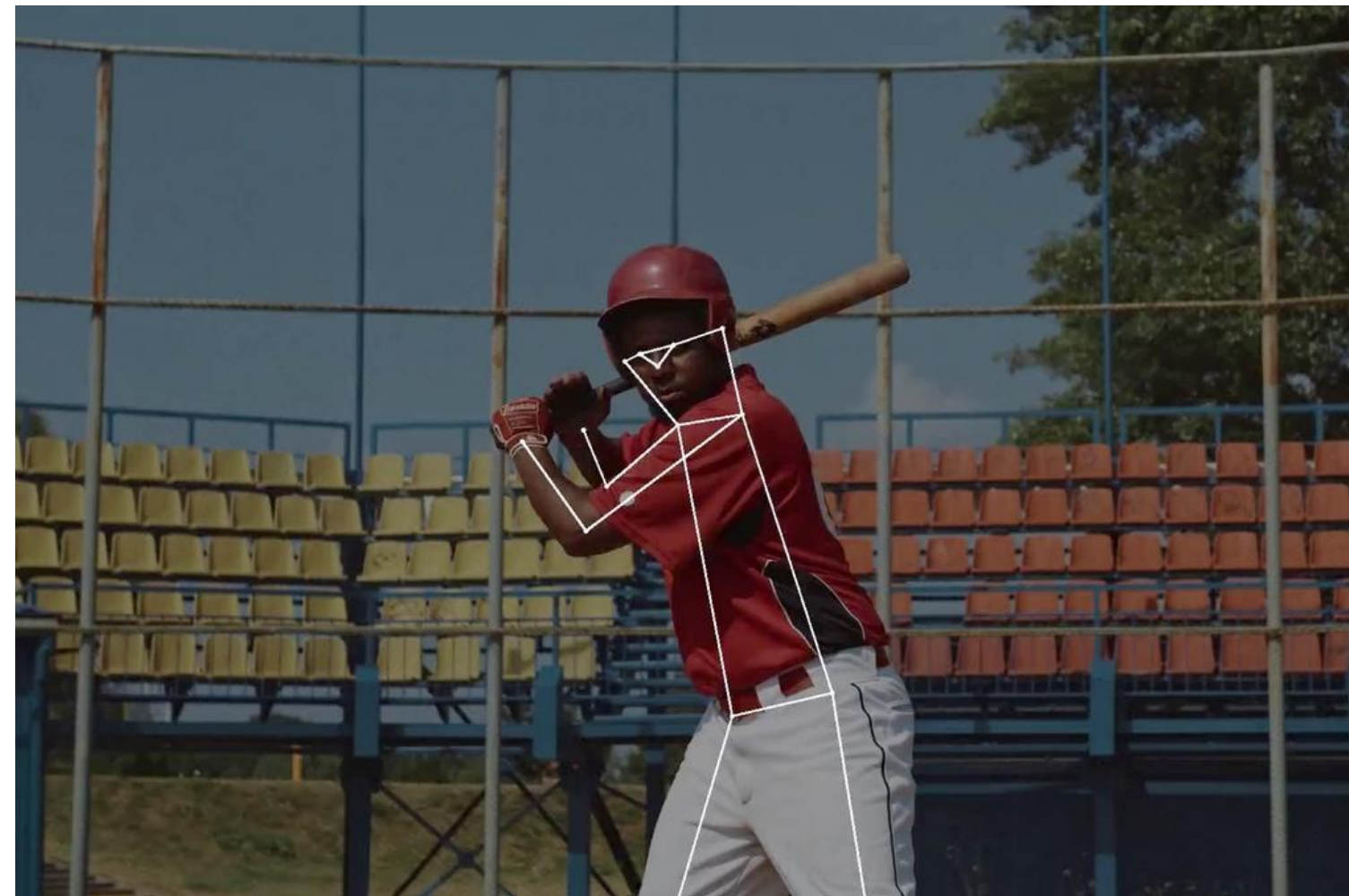
Object Detection with Vision AI

- Object detection focuses on detecting an object in an image and then tracking the object through a series of frames.
- Object Detection is often applied to video streams, whereby the user tracks multiple objects simultaneously with unique identities.
- Popular architectures of object detection include the AI vision algorithms YOLO, R-CNN, or MobileNet.



Pose Estimation

- Pose Estimation makes computers understand the human pose.
- Popular architectures around Pose Estimation include **OpenPose**, **PoseNet**, **DensePose**, or **MeTRAbs**.
- These are useful for real-world problems including crime detection via poses or ergonomic assessments to improve organizational health.



End of Chapter 7